IACE PLATFORM · ENGINEERING

How this repository actually works

Every important flow traced from the click to the row, with each path a link into the checkout. Written from the code as it stands, not from the documents that describe it.

Commit ab5d593 on staging
Built 14 October 2026, regenerated from the code by 23 parallel readers
Paths clickable, and checked against disk at build time

Contents

Before you start 3

Part I — The live sitting **5**

The sitting, from the bell to the claim 5

Marking the paper 14

The score card, the key and the per-question report 19

Rank, percentile, the board and the rollups 23

Part II — Making a paper **28**

Building a test, and freezing it 28

The question bank 36

The authoring workshop 42

Importing in bulk 49

Part III — Who may do what **55**

Signing in 55

What a student may reach 60

The roster, the admins and the catalog 65

Events and their candidates 72

Watching a live test, and intervening 76

Reports 82

Part IV — The clients **87**

The student portal 87

The admin portal 93

The mobile client 97

What the three clients share 102

Part V — The platform underneath **108**

The machinery every module leans on 108

Telling a student something happened 114

A student's own account 120

Part VI — The data model **125**

Every table, and what the database refuses 125

Part VII — What it runs on **132**

The boxes, the pipeline, and where the docs drift 132

Part VIII — What is wrong with it **136**

The ledger 136

Before you start

How to read this

This document is a map of the code, written by reading the code. It is not a summary of the other documents in `docs/`. Those were treated as claims to be checked, and where one disagreed with the repository the document was corrected in the same pass rather than smoothed over here.

Every path in here is a link. A monospaced path like `apps/api/src/attempts/submit.service.ts` opens that file in your checkout, and a directory opens the folder. The build refuses to turn a path into a link unless it is on disk at build time, so a path that renders as plain text is one the build could not find.

The flow traces are the point. Each one starts at a real entry: an HTTP method and route, a queue job, a cron tick, or a click in a component. It does not stop until the value is durable: a column written, a Redis key set with its TTL, a job enqueued, a response shape returned. Guards, transaction boundaries and every branch that changes the outcome are steps in their own right.

> **Note** Where something could not be verified, it says so. "Read, not measured" means the figure was derived from the source rather than observed on a running system.

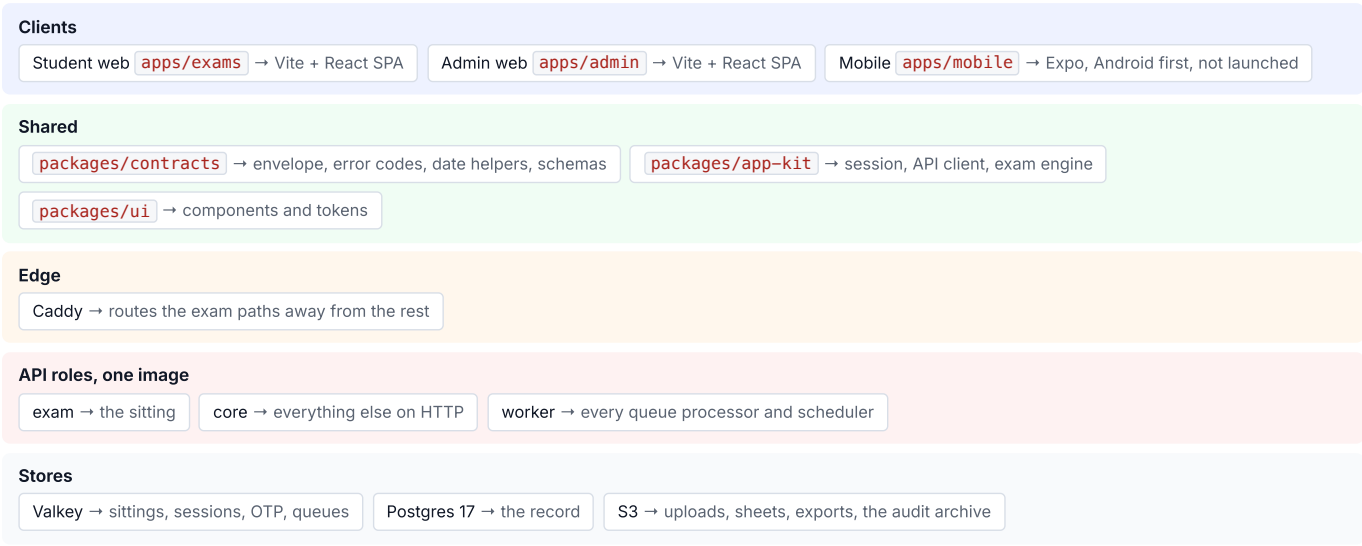
What changed since the last edition

The previous edition was built on 1 October 2026 from `7bee05e`. This one is 351 commits and about 64,000 added lines later. The changes that reshape whole chapters:

- **A student has no PIN.** They sign in with a code sent by SMS every time, and the first time is the signup. WhatsApp is wired but switched off. An admin can issue a desk code to a student whose SMS never arrives.
- **Reports** are a new module: 53 code-owned reports on one route, printed, exported by a worker, and five of them self-served to students.
- **The question bank holds four kinds of question.** Typing passages and descriptive answers joined choice and typed-text questions. A language subject is written in its own language.
- **A broadcast is one row the bell resolves**, not a copy per student. The outbox and its relay are gone; a drop or bonus re-marks through `Test.paperRevision` and the sweep.
- **A sitting stays with the sign-in that opened it.** A second device is held off with a hold screen, a reload resumes the same sitting, and forward-only is enforced on the server.
- **Exports run on a ninth queue**, and the audit log is a super admin's alone.
- **Staging runs on AWS** and is switched off between sessions. Production does not exist yet.

What this is a map of

THE WHOLE SYSTEM, ONE PICTURE



One NestJS codebase runs as three containers from one image, differing only in the `API_ROLE` environment variable. Caddy decides which container a request reaches by matching the exam role's routes exactly. That split is why a results rush cannot slow a hall that is still writing.

The repository, counted

What	How much
HTTP routes	225 across 36 controller files: 176 admin (21 of them imports), 39 student, 7 auth, 3 health and metrics
Prisma models / enums	45 / 36
Migrations	128, 6,333 lines of SQL
Things Prisma cannot express	14 triggers, 16 functions, 1 view, 8 partial uniques, 8 CHECKs, 9 GIN indexes
Queues	9 (`apps/api/src/queue/queues.ts`)

What	How much
TypeScript	~199,000 lines across 1,253 tracked files
Test files	347, of which 87 run against Postgres

The shape of those numbers matters more than the numbers. Admin routes outnumber student routes four and a half to one, because the student surface is small and hot while the admin surface is broad and cold. The SQL is large for a Prisma project because everything that has to be true no matter which code path runs is a trigger, not a service method. A sat paper that cannot move and an option list that cannot drift from its question are two examples.

What a reader should distrust

- `apps/api/src/events` is not an event bus. It is an admin entity: an exam event with a candidate roster. The domain-event catalog lives in `apps/api/src/common/events`.
- `AttemptSheet.answers` has no question ids in it. It is one slot per paper row, by position.
- A comment naming a gate is not a gate. Every chapter ends with the comments found lying about the code they sit on.

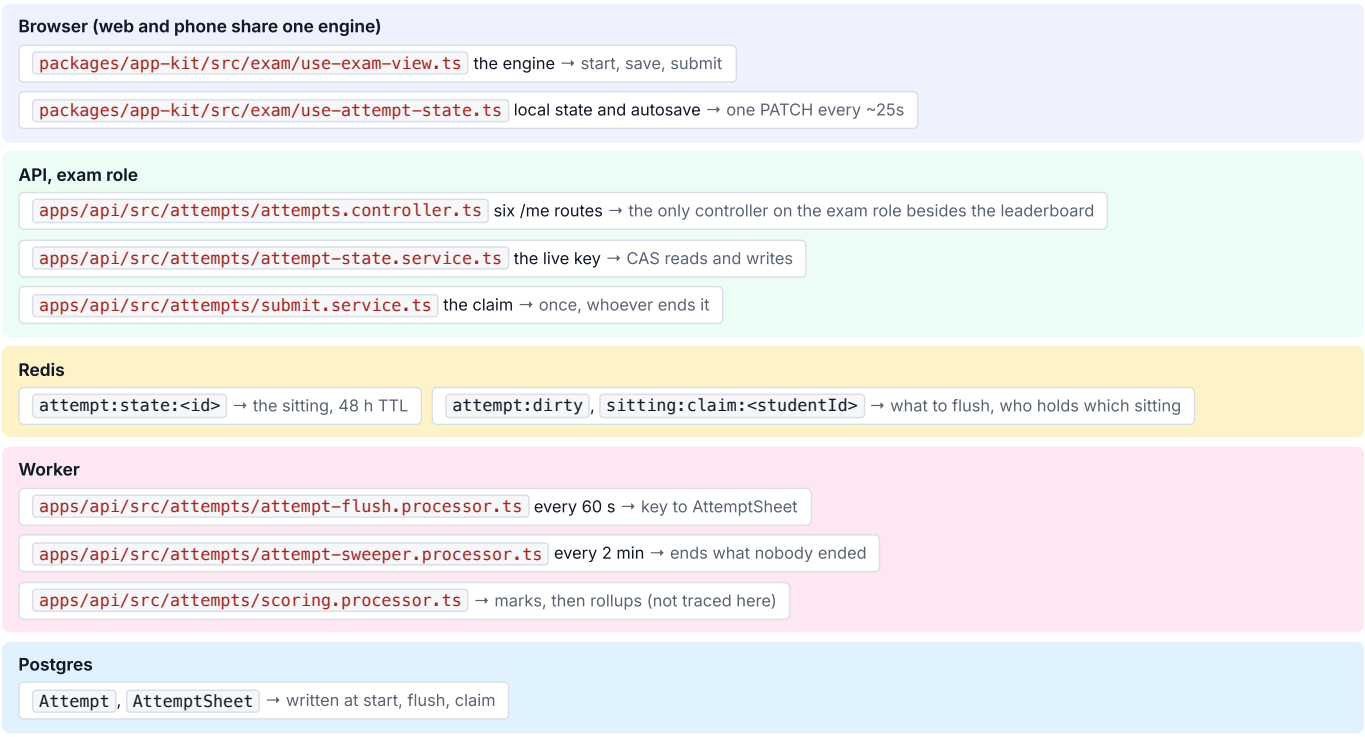
Part I — The live sitting

The sitting, from the bell to the claim

Everything in this chapter happens between a student pressing Begin and that student's answers becoming a marked sheet. It is the only part of the system with a hard scaling constraint: three thousand people sit the same paper in the same minute, at six to eight thousand under stress. Every decision below was made against that number.

The shape to hold in your head: the hot path does not touch Postgres. A live sitting is **one Redis key**, rewritten whole under a compare-and-swap. The browser counts down to a deadline the server granted. Marks are computed by a worker after the fact. Postgres is written three times in a sitting: at the start, on a one-minute flush, and at the submit.

ONE SITTING, END TO END



Where it lives

Piece	Path	Role
Controller	`apps/api/src/attempts/attempts.controller.ts`	Six `/me` routes: start, brief, shared paper, own paper, state read, state save, submit. On `API_ROLES.EXAM` (`apps/api/src/attempts/attempts.module.ts:49`)
Start and resume	`apps/api/src/attempts/attempts.service.ts`	The only writer of a new `Attempt` row
Live key	`apps/api/src/attempts/attempt-state.service.ts`	`open`, `resume`, `save`, `current`, `take`, the CAS `patch`
Pure rules	`apps/api/src/attempts/attempt-state.ts`	`applyBatch`, `isStale`, `withinReach`, `heldElsewhere`, pause credit, `packHeld`
End	`apps/api/src/attempts/submit.service.ts`	Ends a sitting exactly once, student or sweeper
Sheet writer	`apps/api/src/attempts/attempt-sheet.service.ts`	The one writer of `AttemptSheet.answers` and `Attempt.sectionState`
Positional codec	`apps/api/src/attempts/answer-sheet.ts`	Slot encoding shared by the key and the sheet
Scoring ask	`apps/api/src/attempts/scoring-queue.ts`	The keyed BullMQ add
Client engine	`packages/app-kit/src/exam/`	`use-exam-view.ts`, `use-attempt-state.ts`, `start-sitting.ts`, `pending-answers.ts`, `use-countdown.ts`
Client policy	`packages/app-kit/src/autosave-policy.ts`	Intervals, timeouts, retry schedule
Contracts	`packages/contracts/src/attempts.ts`	Schemas, `ME_ATTEMPT_ROUTES`, clock helpers, `PRESENT_GRACE_SEC`
Web screen	`apps/exams/src/features/exam/exam.tsx`	Starts the sitting, hands the engine the paper, draws stand-down and ended screens
Phone screen	`apps/mobile/app/exam/[testId].tsx`	The same engine, a different shell

What it is for

A candidate's hundred-question sitting must not touch Postgres per keystroke. The module keeps the whole live sitting in one Redis key, writes Postgres three times, and hands scoring to a worker. The server owns `startedAt` and `endsAt`; the client only counts down to the `endsAt` it was told. Every write of the live key is a compare-and-swap, so no concurrent save lands on a state it did not read.

The engine is shared. `useExamView` serves the web screen and the phone screen; skins (default, Railways, Banking, SSC) are templates that draw an `ExamView` and decide nothing about what is saved. A skin cannot forget forward-only or a section lock because the engine holds the rule once.

Flow: starting a sitting

POST /ME/TESTS/:TESTID/ATTEMPT — PRESSING BEGIN

1. The exam screen mounts and asks to start. The address names a sitting only on a reload or Continue here (`?resume=`); once the start answers, the page replaces its address with the attempt id, so a plain reload is a reclaim, never a second start
   `apps/exams/src/features/exam/use-sitting-address.ts:17`
   → one query, `staleTime: Infinity` , `gcTime: 0`
2. Body is `{languages?, tab, resume?, holdsPaper}` . `holdsPaper` is true when the instructions screen already fetched the shared paper, so the server skips building it. Retried three times, and only for a hang or a fault; a refusal is an answer
   `packages/app-kit/src/exam/start-sitting.ts:79`
   → `StartAttemptInput`
3. Request budget on the exam role: 8 s, then `SERVICE_UNAVAILABLE` . The connection also carries a `statement_timeout`
   `apps/api/src/common/request-budget.ts:53`
   → refusal the client may retry
4. `JwtAuthGuard` verifies the token and that the Redis session still exists; `@Actors(STUDENT)` turns an admin token away
   `apps/api/src/auth/guards/jwt-auth.guard.ts:33`
   → `request.user` with `sessionId`
5. `@SittingRateLimit()` counts hits per student, default 60 a minute (`RATE_LIMIT_SITTING_PER_MIN`). Autosave, heartbeat and submit share this bucket
   `apps/api/src/config/env.schema.ts:107`
   → throttle counter in Redis
6. Handler passes the student, the test, the body and the sign-in (`sessionId`) to `start`
   `apps/api/src/attempts/attempts.controller.ts:66`
   → —
7. One wave of three independent reads: every sitting this student has on this test (newest first), the test with its blueprint and sections, and `assertFree`
   `apps/api/src/attempts/attempts.service.ts:85`
   → rows in memory
8. `assertFree` : reads `sitting:claim:<studentId>` , then that sitting's key. If the student's OTHER sign-in holds it and was heard from inside the last 60 s, `SITTING_HELD_ELSEWHERE` (409), whichever test was asked for
   `apps/api/src/attempts/attempt-state.service.ts:140`
   → refusal; no row written
9. Branch, resume: a row with status `IN_PROGRESS` exists and the body named nothing or named it. The access gate is skipped: it asks whether a sitting may begin, and this one has
   `apps/api/src/attempts/attempts.service.ts:95`
   → see the resume flow
10. Branch, a reclaim of a sitting that is no longer live: body named an attempt, none matches
    `apps/api/src/attempts/attempts.service.ts:103`
    → `SITTING_ENDED` (409); the web screen redirects to the submitted screen
11. Access gate, deliberately after the resume check. A test that does not exist reads the same as one the student cannot reach
    `apps/api/src/access/access-resolver.service.ts:157`
    → `FORBIDDEN` unless the catalog says this student may start now
12. Test must be `ACTIVE` and finalized
    `apps/api/src/attempts/attempt-rules.ts:13`
    → `CONFLICT` with a message naming which
13. Slot arithmetic: `attemptNo` is the count of non-live sittings plus one (voided ones count: it is a unique key, not a tally). The new sitting is `isGraded` only if no earlier sitting holds the ranked slot and the paper carries marks
    `apps/api/src/attempts/attempt-rules.ts:36`
    → `{attemptNo, ranksAgain}`
14. Paper length is read before the transaction from the per-process paper cache, so 6,000 starts do not each hold a transaction open for a count
    `apps/api/src/attempts/attempt-sheet.service.ts:21`
    → number of paper rows
15. Transaction opens with `TX_LIMITS.SHORT`
    `apps/api/src/attempts/attempts.service.ts:178`
    → —
16. `Attempt` INSERT: `startedAt = now` , `endsAt = startedAt + scopedDurationSec` , `shuffleSeed = randomInt(2**31)` , `languages` from the offer and the pick. The duration is the test's sections narrowed to its scope, never the whole configuration's
    `apps/api/src/attempts/attempts.service.ts:155`
    → `Attempt` row, status `IN_PROGRESS`
17. `AttemptSheet` INSERT: one untouched slot per paper row
    `apps/api/src/attempts/attempt-sheet.service.ts:26`
    → `answers` = blank array
18. Blueprint lock: `UPDATE BaseConfig SET locked = true WHERE locked = false` , same transaction. Lock on first attempt
    `apps/api/src/attempts/attempts.service.ts:182`
    → `BaseConfig.locked = true` ; commit

19. Branch, two starts raced and `@@unique([testId, studentId, attemptNo])` rejected one: the loser re-reads the winner, opens its key, and answers `startedByThisCall: false`. `CONFLICT` only if the winner already ended
`apps/api/src/attempts/attempts.service.ts:119`
→ no second row

20. `open` seeds `attempt:state:<id>` through the CAS: `revision 0`, empty `answers` and `sections`, `lastSeenAt`, `forwardOnly` (FORWARD_ONLY papers only), and the tab and sign-in that opened it
`apps/api/src/attempts/attempt-state.service.ts:90`
→ Redis key, EX 172800 (48 h)

21. `takeClaim`: one `SET sitting:claim:<studentId> <attemptId> EX 172800 GET`. If the previous holder is a different sitting, its key's `tab` is patched to `null`, which stands it down (its next save gets `SITTING_SET_ASIDE`). If that patch throws, the old claim is put back
`apps/api/src/attempts/attempt-state.service.ts:148`
→ claim key; the earlier sitting set aside

22. Paper attached unless the screen said it holds one. A build failure is logged and answered as `paper: null`; the screen fetches it
`apps/api/src/attempts/attempts.controller.ts:73`
→ `ExamPaper` or null

23. Response is `StartedAttempt` plus `serverNow`, HTTP 200 (not 201). A second start of the same test is not a second row: it resumes, `startedByThisCall: false`, same `startedAt`
`apps/api/src/attempts/attempts.controller.ts:79`
→ `{success, data, meta.requestId}`

> **Note** The claim is per student, not per test. Opening any sitting stands down whichever sitting that student last claimed, which is why a student cannot answer two tests at once. A student on a different sign-in (web against the phone app) is refused instead of standing the other down, for as long as that sign-in is being heard from. See the stand-down section.

Flow: autosaving an answer

The screen draws local state. It never waits on the server. A save that fails never takes an answer off the screen; what a save could not deliver stays queued.

PATCH /ME/ATTEMPTS/:ID/STATE — THE AUTOSAVE

1. A tap records locally: the change goes into the pending queue (`sessionStorage` on web, the phone's store on mobile, so it survives a reload) and into the drawn state at once
`packages/app-kit/src/exam/use-attempt-state.ts:391`
→ pending map, drawn answers

2. Time on the open question is bookkeeping this hook keeps: leaving a question banks its seconds, capped at `QUESTION_TIME_MAX_SEC` (a day)
`packages/app-kit/src/exam/use-attempt-state.ts:402`
→ `timeSpentSec` in the change

3. The timer fires at 25 s plus or minus 5 s of jitter, redrawn each time so sittings that began together drift apart. Twenty unsent answers flush early
`packages/app-kit/src/autosave-policy.ts:26`
→ one save

4. Idle, in touch (the server answered within the last 60 s) and nothing in the air: the tick still sends, an empty batch. This is the heartbeat; it is what makes silence mean absence. Past the grace a silent screen stops beating until the student acts or reloads
`packages/app-kit/src/exam/use-attempt-state.ts:368`
→ empty batch

5. `save`: bumps the local revision, takes the unsent batch (answers capped at `SAVE_BATCH_MAX` = 200, plus every section touched), and sends `{revision, answers[], sections?, tab}` with a 15 s abort
`packages/app-kit/src/exam/use-attempt-state.ts:277`
→ PATCH body

6. Same guard stack as the start and the same sitting rate-limit bucket. No `@HttpCode`, so 200
`apps/api/src/attempts/attempts.controller.ts:122`
→ —

7. `save` runs inside a `rebuilt` patch of the key
`apps/api/src/attempts/attempt-state.service.ts:163`
→ —

8. CAS loop, up to `PATCH_TRIES` = 5: GET the raw bytes, apply the change, write through a Lua script that lands only if the key still holds exactly those bytes, re-applying the TTL. Five lost races: `CONFLICT` "being written to right now"
`apps/api/src/attempts/attempt-state.service.ts:268`
→ `replaceJson`

9. Key missing: rebuilt from Postgres once (one `Attempt` read with `sheet.answers`, `sectionState` and the navigation flags). Another student's id is `NOT_FOUND`; a row that is not `IN_PROGRESS` is `SITTING_ENDED`
`apps/api/src/attempts/attempt-state.service.ts:372`
→ seed state; `lastSeenAt` is the sheet's `updatedAt`

10. A key put back from Postgres is checked against the row once more AFTER the write. If an end claimed the row in between, the key this call wrote is removed (only while it still holds those bytes) and the save answers `SITTING_ENDED`. If the row's deadline moved meanwhile, the key follows it
`apps/api/src/attempts/attempt-state.service.ts:307`
→ no zombie key behind an ended sitting

11. Judged inside the swap, against the same read: ownership (`NOT_FOUND`), time (`CONFLICT` past `endsAt` + 30 s), and the tab. A tab that is not the holder gets `SITTING_TAKEN_OVER` (another tab or device holds this sitting) or `SITTING_SET_ASIDE` (`tab` is `null`: the student opened another test)
`apps/api/src/attempts/attempt-state.service.ts:463`
→ 409 with a code the screen reads

12. Staleness: `batch.revision <= held.revision` means a newer save already landed; the batch is not applied and the ack says `applied: false`. The tab still counts as present
`apps/api/src/attempts/attempt-state.ts:188`
→ ack `applied`

13. Forward-only narrowing: on a FORWARD_ONLY paper, changes to seats behind the furthest seat reached in that section are filtered out of the batch. Seats come from the per-process paper cache, so a forward-only save still reads no Postgres once warm
`apps/api/src/attempts/attempt-state.ts:193`
→ batch minus the closed seats

14. Merge per question: state re-derived server-side from the answer and the mark (the screen's state is a guess), `timeSpentSec = max(held, sent)`, `answeredAt` kept from the first time it was given, `firstActionAt` the earliest of the two. Section `openedAt` is stamped by the server the first time it sees the section, never by the screen
`apps/api/src/attempts/attempt-state.ts:213`
→ `revision = batch.revision`, `lastSeenAt = now`

15. Packed by position before the write: `[stateIndex, optionId or null, timeSpentSec, firstActionSec, answeredSec, typedAnswer?]`, times as whole seconds after `startedAt`. Measured in the code comment at 10.3 KB a sitting against 24.6 KB for named fields
`apps/api/src/attempts/attempt-state.ts:79`
→ `attempt:state:<id>`, EX 172800

16. Marked for the flusher AFTER the write, unless the batch was a heartbeat: `SADD attempt:dirty <id>`
`apps/api/src/attempts/attempt-state.service.ts:185`
→ member of `attempt:dirty`

17. Ack `{revision, applied, endsAt, serverNow}`. The client re-seeds its counter from `saved.revision`, re-anchors its clock from `endsAt` and `serverNow` (this is how an extension reaches the screen), and drops the acknowledged copies from its queue only if `applied`
`packages/app-kit/src/exam/use-attempt-state.ts:292`
→ clock re-anchored; pending queue shortened

Nothing is written to Postgres on this path. Postgres catches up on a timer.

EVERY 60 S — THE FLUSH

1. A BullMQ job scheduler (fixed scheduler id, so a redeploy does not stack a second) fires on the worker
`apps/api/src/attempts/attempts.module.ts:113`
→ job on `attempt-flush`

2. The pass reads the dirty set and works it in lanes of eight (`FLUSH_LANES`), one pass owning the whole set
`apps/api/src/attempts/attempt-flush.processor.ts:28`
→ ids

3. Each lane is read with one MGET, so the bytes read are the bytes later compared
`apps/api/src/attempts/attempt-state.service.ts:333`
→ `FlushRead[]`

4. Each sitting is written whole, gated on the sitting still being `IN_PROGRESS`: `UPDATE AttemptSheet SET answers = …, updatedAt = now()`, and the section clocks into `Attempt.sectionState`
`apps/api/src/attempts/attempt-sheet.service.ts:36`
→ `AttemptSheet.answers`, `Attempt.sectionState`

5. A sitting is unmarked only where its key still holds the bytes that were written (one Lua script); a save since keeps its own mark. A failed write stays marked
`apps/api/src/attempts/attempt-state.service.ts:343`
→ `attempt:dirty` shrinks

A crash costs the durable copy at most a minute, never the answers: the key holds them.

Flow: the clocks

There are two kinds of paper. A composite paper has one clock. A sectional paper (`timerTemplate` other than `COMPOSITE_FREE`) gives each section its own clock and opens them one at a time. Both clocks are counted on the client against instants the server stamped.

ONE PAPER CLOCK — WHAT THE COUNTDOWN COUNTS TO

1. The paper (or a later save) carries `endsAt` and `serverNow`. The engine records `Date.now()` the moment the payload lands as `arrivedAt`
`packages/app-kit/src/exam/start-sitting.ts:101`
→ `ExamClock`

2. Seconds left are `(endsAt - serverNow) - (deviceNow - arrivedAt)`. The device clock only measures how long the page has been open, so a wrong device clock gains nothing
`packages/contracts/src/attempts.ts:288`
→ integer seconds

3. One interval for the life of the screen reads the latest clock through a ref (a re-keyed interval used to skip a second at every autosave)
`packages/app-kit/src/exam/use-countdown.ts:23`
→ `left`

4. Every save answer replaces the clock with the server's, so an admin's extension or a pause credit lands without a reload
`packages/app-kit/src/exam/use-attempt-state.ts:295`
→ new `ExamClock`

5. At zero, `lapse` records the clock the paper ran out on, locks the paper (no answer, no move, no section change) whether or not the hand-in lands, and calls `end`
`packages/app-kit/src/exam/use-exam-view.ts:252`
→ `ranOut`, `lapsed`

6. If a later save answers with time left on a paper that had run out, the lock lifts and the failed hand-in is forgotten
`packages/app-kit/src/exam/use-exam-view.ts:261`
→ paper open again

A SECTION CLOCK — ENTERING, RUNNING, CLOSING

1. On a sectional paper the first section's clock is stamped only when the state read has settled and shows it was never opened. The screen is inert until then (a reload could otherwise reopen a closed section)
   `packages/app-kit/src/exam/use-exam-view.ts:160`
   → `enterSection(first, durationSec)`
2. Entering a section is told, not asked: the section goes into the pending batch as `{remainingSec, closed: false}` and a flush is forced
   `packages/app-kit/src/exam/use-exam-view.ts:283`
   → queued section
3. On the server, `sectionsAfter` stamps `openedAt` the first time it sees the section, and clears it when the section is closed. The screen's own `remainingSec` is not read
   `apps/api/src/attempts/attempt-state.ts:235`
   → `sections[id].openedAt`
4. Remaining seconds are `durationSec - (serverNow - openedAt)` from two server instants. A section with no duration has no clock of its own and falls through to the paper's
   `packages/contracts/src/attempts.ts:295`
   → `sectionSec`
5. The section timer is re-anchored to the device clock whenever its allowed seconds change, so its own elapsed time is never subtracted twice
   `packages/app-kit/src/exam/use-countdown.ts:54`
   → `left`
6. At zero the section closes by the same path as a manual hand-in: `closeSection(id, 0)`, then the next open section opens, or the paper goes in if none remains
   `packages/app-kit/src/exam/use-exam-view.ts:295`
   → `sections[id].closed = true`
7. `openSections` allows exactly one section at a time (the first not closed), so earlier sections cannot be reopened and later ones cannot be jumped to
   `packages/contracts/src/attempts.ts:474`
   → `reachable`
8. A candidate can hand a section in early (`submitSection`, null on a one-clock paper and on the last open section, where handing it in is handing the paper in)
   `packages/app-kit/src/exam/use-exam-view.ts:295`
   → same close path
9. Durable: the flush writes the section clocks into `Attempt.sectionState`, and a key rebuilt from Postgres reads them back, so a lost key does not hand every section its full time again
   `apps/api/src/attempts/attempt-sheet.service.ts:48`
   → `Attempt.sectionState` jsonb

> **Watch** Section closing is a screen rule, like forward-only used to be. The server stamps `openedAt` and stores `closed`, but nothing on the save path refuses an answer to a section whose time has run out or that was closed. The paper-level deadline is the only clock the server enforces (`attempt-state.ts:253`).

Forward-only

On a `FORWARD_ONLY` paper a seat left is closed for good. Three layers keep it:

- **Palette and Next.** `canOpen` allows only the current or a later seat; Next stops wrapping from the last seat to the first (`packages/app-kit/src/exam/use-exam-view.ts:279`). Marking for review is switched off (`use-exam-view.ts:307`), because a flag asks for a second look the paper will never grant.
- **Landing.** A reload lands on the furthest seat reached, not on seat one (`use-exam-view.ts:175`).
- **The server.** `withinReach` drops any change to a seat behind the furthest one reached in that section (`apps/api/src/attempts/attempt-state.ts:193`), so a client that ignores the screen rule still cannot rewrite an earlier answer. The seat order is rebuilt from `shuffleSeed` (stored on the key as `forwardOnly`) and the cached paper.

Flow: submitting

The order is the design. Answers are written to Postgres BEFORE the claim, so a write that throws leaves the sitting open with its state intact. The state is taken behind the claim and written last. Only then is the sitting queued, under its own id.

POST /ME/ATTEMPTS/:ID/SUBMIT — HANDING IN

1. The student confirms (or a clock expires). `bankOpen` banks the seconds on the question still on screen
   `packages/app-kit/src/exam/use-exam-view.ts:198`
   → open question's time banked
2. `finish` waits at most 5 s (`FINISH_WAIT_MS`) for a save in the air, stops all saving, and sends a backlog larger than one batch as ordinary saves. The remaining unsent batch rides the submit
   `packages/app-kit/src/exam/use-attempt-state.ts:326`
   → `LastBatch` or null
3. The call is `{tab, last?}`, aborted at 10 s (`SUBMIT_TIMEOUT_MS`), retried by the engine itself on 1 s, 2 s, 4 s, and only for a hang, a 503 or a throttle. It does not use the query client's retry, which holds while a tab is hidden, and runs with `networkMode: 'always'` so a browser reporting offline cannot park it
   `packages/app-kit/src/exam/use-exam-view.ts:92`
   → up to four tries inside the 30 s grace
4. Same guards and the sitting rate-limit bucket; `@HttpCode(OK)`
   `apps/api/src/attempts/attempts.controller.ts:110`
   → —
5. Row read; another student's id is `NOT_FOUND` and counts a refused submit
   `apps/api/src/attempts/submit.service.ts:55`
   → `AttemptRow`

6. Live key READ (never taken) and the tab judged: a tab that no longer holds the sitting is refused, so a stood-down tab cannot end a sitting being answered elsewhere
   `apps/api/src/attempts/submit.service.ts:62`
   → `SITTING_TAKEN_OVER` or `SITTING_SET_ASIDE`
7. The riding batch is saved as the newest: `state.save(…, {last: true})` forces `revision = max(sent, held + 1)` so a reloaded screen's low counter cannot make its final batch stale. Skipped if the sitting is not `IN_PROGRESS` or the 30 s grace has passed (the row's deadline if the key is gone). If this save throws and the sitting is still open, the submit fails; if another call ended it, `end` reports that outcome
   `apps/api/src/attempts/submit.service.ts:88`
   → state with the last answers
8. Answers written BEFORE the claim: the key is read again and written to the sheet gated on `IN_PROGRESS`. A throw here leaves the sitting open
   `apps/api/src/attempts/submit.service.ts:115`
   → `AttemptSheet.answers`
9. The one gate: `UPDATE Attempt SET status = 'SUBMITTED', submittedAt = now() WHERE id = ? AND status = 'IN_PROGRESS'` (via `updateMany`). Count 0 means somebody else won
   `apps/api/src/attempts/submit.service.ts:150`
   → `status = SUBMITTED`, `submittedAt`
10. Loser path: re-reads the row and answers `{status, submittedAt, submittedByThisCall: false, answeredCount}`, HTTP 200. This is what makes a resend after a 10 s give-up, a second tab and the sweeper converge on one claim and one answer
    `apps/api/src/attempts/submit.service.ts:185`
    → idempotent response
11. Winner: `take` is one `GETDEL` on the key plus `SREM attempt:dirty`. Anything arriving after finds no key and is refused by `durableState` because the row is no longer `IN_PROGRESS`
    `apps/api/src/attempts/attempt-state.service.ts:200`
    → key gone
12. Last write, ungated: the taken state is written even if unchanged, because a stale flush may have landed between steps 8 and 12
    `apps/api/src/attempts/submit.service.ts:124`
    → `AttemptSheet.answers`, `Attempt.sectionState`
13. Scoring enqueued AFTER the last write, `addBulk` onto the `scoring` queue, `jobId = scoring-<attemptId>`, `removeOnFail: true`. A duplicate add is a no-op. A queue failure is caught and logged and never fails a submit that committed; the sweeper asks again
    `apps/api/src/attempts/scoring-queue.ts:37`
    → BullMQ job
14. Count taken off the sheet just written (no read-back when a sheet was written)
    `apps/api/src/attempts/submit.service.ts:140`
    → `answeredCount`
15. Response `{attemptId, status: SUBMITTED, submittedAt, submittedByThisCall: true, answeredCount}` wrapped in the envelope. The client clears its queue, marks the catalog stale without refetching, exits fullscreen and navigates to the submitted screen with `replace`
    `packages/app-kit/src/exam/use-exam-view.ts:227`
    → local queue cleared; route replaced

From here, `ScoringProcessor` (queue `scoring`, concurrency 8, 5 attempts, 5 s backoff, `apps/api/src/queue/queues.ts:23`) writes the score fields and `AttemptSheet.verdicts`, then rollups. That is another chapter.

> **Note** `answeredCount` is counted off what was written. The client compares it with its own tally (`keptSummary`, `use-exam-view.ts:64`): if they differ (an answer given after the sitting ended elsewhere, or dropped as late) the submitted screen drops the device's tally and shows only the server's result.

Flow: resuming (reload mid-exam)

POST /ME/TESTS/:TESTID/ATTEMPT — THE RELOADED SCREEN

1. The screen mounts and starts again with the same tab id and `?resume=<attemptId>`. Tab id is minted per browser tab in `sessionStorage` (per device on the phone) and survives a reload
   `apps/exams/src/lib/tab-id.ts:6`
   → `tab`
2. The live row is found; the gate at step 11 of the start flow is skipped
   `apps/api/src/attempts/attempts.service.ts:95`
   → `AttemptRow`
3. `resume` reads the key, or rebuilds it from `AttemptSheet` if it is gone
   `apps/api/src/attempts/attempt-state.service.ts:113`
   → `HeldState`
4. Branch, abandoned: away longer than 48 h (`PAUSE_LIMIT_SEC`) or the 48 h credit cap already spent. No credit is given; the deadline stands and the sweeper will end it
   `apps/api/src/attempts/attempt-state.ts:296`
   → `creditedMs: 0`
5. Clock credit: `now - lastSeenAt` less a 60 s present-grace, capped at what is left under the cap. Every open section's `openedAt` is pushed by the same amount
   `apps/api/src/attempts/attempt-state.ts:264`
   → `creditMs`
6. One CAS swap writes the new `endsAt`, section clocks, `lastSeenAt = now`, this tab and sign-in, and the accumulated `creditedMs`. Computing credit inside the swap means two reloads cannot each bank the same silence. A different sign-in still being heard from is refused inside the swap
   `apps/api/src/attempts/attempt-state.service.ts:118`
   → `attempt:state:<id>`

7. The claim is re-taken, standing down whatever tab held the student's previous sitting
`apps/api/src/attempts/attempt-state.service.ts:134`
→ `sitting:claim:<studentId>`

8. The only Postgres write on the resume path: `UPDATE Attempt SET endsAt = endsAt + credit` (added, never written back, so an extension that landed since still stands), only when credit is nonzero. If the row came back later than the key, the key follows it
`apps/api/src/attempts/attempts.service.ts:130`
→ `Attempt.endsAt`

9. Response with `startedByThisCall: false` and the real `startedAt` / `endsAt`
`apps/api/src/attempts/attempts.service.ts:100`
→ `StartedAttempt`

10. A screen that just started the sitting skips the state read entirely (it would return revision 0 and nothing). A reloaded one asks `GET /me/attempts/:id/state`, which returns `{attemptId, revision, answers, sections, endsAt, serverNow}` from the key; the client merges it UNDER anything answered locally and seeds its counter to `max(own, held)`
`packages/app-kit/src/exam/use-attempt-state.ts:225`
→ local state seeded

The pause design: a sitting is **put down**, not stopped. Falling silent longer than the grace moves the deadline out by the gap, so a laptop that sleeps, a dropped network or a genuine pause costs one minute. The Pause button (`apps/exams/src/features/exam/exam.tsx:197`) saves what is unsent, refuses to leave if the server did not take it, then leaves the paper; it adds no state and no endpoint.

Multi-tab and multi-device stand-down

Three fields on the key decide who may write: `tab` (a string holds it, `null` was stood down, absent is a key put back cold), `session` (the sign-in the tab belongs to), and `lastSeenAt` (when the server last heard from it).

Situation	What the server does	What the student sees
Second tab, same sign-in, same sitting	Takes the sitting at once. The old tab's next save gets `SITTING_TAKEN_OVER`	Old tab: "continued in another tab or on another device", with Continue here
Student opens a different test (same sign-in)	`takeClaim` patches the first sitting's `tab` to `null`; its next save gets `SITTING_SET_ASIDE`	Old tab: "Another of your tests is open"
Web and the phone app (two sign-ins), holder heard from inside 60 s	Start or resume is refused with `SITTING_HELD_ELSEWHERE` before any row is written, for ANY test	Hold screen with Try again
Same, holder silent for 60 s or more	The other sign-in takes the sitting	The first one is stood down on its next save
Key rebuilt from Postgres	Held by nobody; the first tab back adopts it	none

A stood-down tab stops saving and keeps its unsent queue; "Continue here" reloads with `?resume=<attemptId>` and resends that queue as it stands (`apps/exams/src/features/exam/exam.tsx:209`). Submit from a stood-down tab is refused (submit step 6). If the sitting ended while a tab was stood down, the tab says "This test has ended" and drops its queue.

> **Note** There is no per-question version. A tab continued after a stand-down resends its whole queue, which can overwrite an answer given on the other device in between. A client-side per-question compare was tried and reverted (`29269a8`): a stored queue can reach the server before the seed judges it, and a tab's own applied-but-unacknowledged save was being dropped. The fix needs a server-side version per question. Known limit, also in docs/02 section 7.

The sweeper

EVERY 2 MIN — ENDING WHAT NOBODY ENDED

1. Scheduler fires on the worker
`apps/api/src/attempts/attempts.module.ts:117`
→ job on `attempt-sweep`

2. `endStranded` reads every `IN_PROGRESS` sitting whose `endsAt` is older than now minus the 30 s save grace, a page of 200 at a time, keyset-paged on `(endsAt, id)`
`apps/api/src/attempts/attempt-sweeper.processor.ts:55`
→ page of candidates

3. A live key is a paper put down, so having no key is nearly the whole decision: the sweeper ends a candidate whose key is gone, or whose key says abandoned (silent over 48 h, or credit spent). A key still owed a flush waits a sweep
`apps/api/src/attempts/attempt-sweeper.processor.ts:75`
→ candidates to end

4. Each is ended in lanes of eight through `SubmitService.expire` with the deadline it was listed by in the claim's own `WHERE`: time given since the list (an extension or a credit) matches no row and leaves the sitting open
`apps/api/src/attempts/submit.service.ts:106`
→ the same claim, take, write, queue as a student's submit

5. Then: rollup counting pass, a re-ask for sittings `SUBMITTED` and unscored past the queue grace, and a re-ask for sittings marked against an older paper revision
`apps/api/src/attempts/attempt-sweeper.processor.ts:41`
→ scoring jobs, keyed by id

Two things follow from step 3. A time-up with the tab still open ends by the client's submit, not by the sweeper. A candidate who closes the tab at time-up has not ended their sitting; it waits, resumable, until the pause limit.

The data it touches

Column or key	Holds	Written by
`Attempt.id`, `testId`, `studentId`	the sitting, the paper, the candidate	start
`Attempt.attemptNo`	ended count + 1, voided included	start
`Attempt.isGraded`	the one ranked sitting	start (`slotsAfter`)
`Attempt.status`	`IN_PROGRESS` to `SUBMITTED`	start (default); `claim()`
`Attempt.startedAt`, `endsAt`	server's start and deadline	start; resume credit; support extend (adds)
`Attempt.submittedAt`	the claim's own instant	`claim()`
`Attempt.shuffleSeed`, `languages`	what this sitting sees and in which languages	start
`Attempt.sectionState`	jsonb `{[sectionId]: {remainingSec, closed, openedAt}}`	`attempt-sheet.service.ts:48`
`AttemptSheet.answers`	positional array, one slot per `PaperQuestion.order`	start (blank); flusher; submit
`AttemptSheet.updatedAt`	last durable write; the rebuild's `lastSeenAt`	same writes
`BaseConfig.locked`	blueprint frozen on first start	start transaction
Redis `attempt:state:<attemptId>`	the whole live sitting, 48 h TTL re-applied on every write	open, save, resume, take (removes)
Redis `attempt:dirty`	set of sittings written since their last flush	save adds; flush and take remove
Redis `sitting:claim:<studentId>`	the one sitting this student is answering, 48 h	start and resume
Redis `ratelimit:sitting:student:<id>`	hit counter, 60 s window	throttler

What must already be true

A `Student` the token resolves to; a live Redis session for that token; a `Test` that is `ACTIVE` and finalized; `assertCanStart` satisfied by the catalog (enrolment, branch reach, schedule window); `PaperQuestion` rows for the test; Redis reachable (a save with Redis down has no path: the rate limiter fails open, the state CAS does not); the API role including `exam` for the controller to register and `worker` for the flush and sweep schedulers.

Which screen calls it

Route	Client	Screen
`POST /me/tests/:id/attempt`	`packages/app-kit/src/exam/start-sitting.ts:79`	web `/exam/:testId`, phone `apps/mobile/app/exam/[testId].tsx`
`GET /me/tests/:id/brief`	`packages/contracts/src/attempts.ts` (`ME_ATTEMPT_ROUTES.brief`)	instructions screen
`GET /me/tests/:id/paper`	same	instructions screen: the paper is held before the bell
`GET /me/attempts/:id/paper`	`packages/app-kit/src/exam/start-sitting.ts:101`	fallback when the start carried no paper
`PATCH /me/attempts/:id/state`	`packages/app-kit/src/exam/use-attempt-state.ts:287` (and `:483` on keepalive when the page closes)	the sitting
`GET /me/attempts/:id/state`	`packages/app-kit/src/exam/use-attempt-state.ts:225`	the sitting, reload only
`POST /me/attempts/:id/submit`	`packages/app-kit/src/exam/use-exam-view.ts:207`	the sitting

Leaving: on web, `useLeaveGuard` raises the browser's own prompt when anything is unsent and, on leaving anyway, `leave()` sends everything on a `keepalive` request (it cannot refresh an expired token). On the phone, backgrounding calls `leave()` because the OS may end the app.

Is it optimal

- **Start:** two parallel reads and an `assertFree` (two Redis reads) in one wave, then a three-statement transaction and one to three Redis commands. The shared paper held by the instructions screen is not built a second time (`holdsPaper`). Read, not measured.
- **Resume:** the same reads plus one UPDATE only if the deadline moved.
- **Autosave:** zero Postgres queries; GET, a Lua CAS and one SADD (a forward-only save also touches the process-local paper cache). A heartbeat skips the SADD. Rebuild from Postgres is one query and only runs when the key is gone.
- **State GET:** one Redis GET, and no request at all for a screen that just started the sitting.
- **Submit:** two Postgres reads, two raw UPDATEs plus one `updateMany`, a Redis GETDEL, SREM and one queue add. The two sheet writes are deliberate (steps 8 and 12), not a bug.
- **Payload:** a save carries changed answers only, at most 200; the held state is about 10 KB a sitting.

What it leans on

`ResponseInterceptor` and `AllExceptionsFilter` (no controller builds an envelope); `JwtAuthGuard`, `ActorGuard` and the Redis-backed throttler; `RequestBudgetInterceptor` and `statement_timeout` on the exam role's connection so a client retry does not land on a container still doing the first copy (`apps/api/src/common/request-budget.ts:29`); `packages/contracts` for the schemas both sides validate against; `redisKeys` so no feature

collides on a prefix; `RedisService.replaceJson`, `takeJson` and `removeIfUnchanged`, three Lua scripts that make the CAS and the flush-settle atomic; `PaperSheetService`'s per-process paper cache; `AccessResolverService` for the start gate; BullMQ scoring for everything after the bell.

Why it is shaped this way

- **One key per sitting, rewritten whole, over a row per answer.** A per-answer table at 6K sittings of 100 questions is 600K rows on the hot path; this is one GET and one Lua SET. The cost is that a save is O(sitting) in bytes, which is why `packHeld` is positional.
- **A monotonic revision, with no per-question version.** A stale batch is dropped whole. The alternative is what `29269a8` says is needed; not built.
- **Claim by UPDATE over a transaction or a lock.** `UPDATE … WHERE status = 'IN_PROGRESS'` is one statement and needs no advisory lock; the loser reads the winner's outcome. The cost is that the answers are written twice (submit steps 8 and 12).
- **Keyed BullMQ job over an idempotency table.** `jobId = scoring-<attemptId>` makes duplicate asks free; `removeOnFail: true` stops a retained failure from silently swallowing a re-add.
- `endsAt` **in Postgres as well as Redis.** The sweeper reads the row, so a pause credit must reach both. Every mover adds to the row in one statement and the key takes the later of the two deadlines, so two movers landing together both count (`a09d0d5`).
- **Silence means absence.** An idle screen used to send nothing, which made a student reading a long passage look like a dropped device. The heartbeat (`19f35d0`) is why one sign-in can safely refuse another.

What changed since the last edition

- **Sign-in-aware holding** (`19f35d0`, `e288ffc`): `SITTING_HELD_ELSEWHERE`, the `session` field on the key, the idle heartbeat, `PRESENT_GRACE_SEC` moved to contracts, and `assertFree` in the start's first wave.
- **Section clocks are durable** (`35c05d4`): `Attempt.sectionState` is written beside the flush and read on rebuild.
- **Five races closed** (`a09d0d5`): extend, void and force-submit, the deadline movers, the live-key rebuild, and the claim (`SET … GET` in one command).
- **Hand-in hardened** (`cf73044`): a paper out of time locks whether or not the hand-in lands, the retries run inside the hand-in under `networkMode: 'always'`, the last closed section hands the skin no question, and any resumed paper is inert until the state settles (previously only sectional ones).
- **Reload reclaims** (`17c1a20`): the address names the attempt after the first start.
- **Clock tick** (`d295f67`): one interval for the screen's life.
- **Hand a section in early** (`13f638c`) and **Pause** on every skin and in the app.
- **Typing in a sitting**: a typed answer rides the ordinary autosave, and the engine will not clear it with a click (`use-exam-view.ts:317`).
- **A sitting ended elsewhere says so**: `SITTING_ENDED` on a save, heartbeat or state read stops the screen for good and drops its queue (`c2aaa4d`, `6dbb39e`).
- **A shared bell in the load tier** (`862a06c`): the k6 script can hold every candidate to one instant.

What is wrong with it

- **Correctness.** Section clocks and closes are enforced by the screen only: `sectionsAfter` stamps and stores them (`apps/api/src/attempts/attempt-state.ts:235`) but `answered` (`apps/api/src/attempts/attempt-state.service.ts:463`) never refuses an answer to a closed or expired section. Only the paper's deadline is judged on the server.
- **Correctness.** A continued stand-down overwrites with the whole stale queue, with no per-question version (`packages/app-kit/src/exam/use-attempt-state.ts:184`); known and documented, listed so it is not mistaken for fixed.
- **Stale.** `docs/02-domain-rules.md` section 7 says forward-only "is a rule the SCREEN keeps … and the server takes answers the same way either side of it". The server does filter: `withinReach` drops changes behind the furthest seat (`apps/api/src/attempts/attempt-state.ts:193`, applied at `attempt-state.service.ts:244`).
- **Dead.** `SectionProgress.remainingSec` is sent by the client and stored in the key and in `Attempt.sectionState`, but nothing reads it back: the clock is `openedAt` against `durationSec` (`packages/contracts/src/attempts.ts:168`, `:295`).
- **Dead.** `standDown` on `AttemptStateHandle` is exposed (`packages/app-kit/src/exam/use-attempt-state.ts:87`) and called only by a test; the screens reach a stand-down through `refused`.

Marking the paper

Where it lives

File	Role
`apps/api/src/queue/queues.ts`	The one table of queue names, concurrency, attempts, backoff, retention, and every job id.
`apps/api/src/queue/queue.module.ts`	`BullModule.forRootAsync` on its own ioredis connection (`maxRetriesPerRequest: null`); registers every queue with `jobOptionsFor(name)`.
`apps/api/src/queue/reporting-worker-host.ts`	Base class carrying the two `@OnWorkerEvent` listeners (`failed`, `error`) so no processor restates them.
`apps/api/src/attempts/scoring-queue.ts`	The producer: `queue()` for a first score, `rescore()` for a correction.
`apps/api/src/attempts/scoring.processor.ts`	The worker: read, score, one transaction, announce, recount.
`apps/api/src/attempts/score-paper.ts`	Pure evaluation, plus the packed `sectionScores` codec.
`apps/api/src/attempts/answer-sheet.ts`	The positional sheet encode/decode, `SLOT_STATES`, `verdictsOf`.
`apps/api/src/attempts/paper-sheet.service.ts`	Per-process LRU of a frozen paper's terms, keyed `testId:paperRevision`.
`apps/api/src/attempts/attempt-sweeper.processor.ts`	The repair: ends stranded sittings, re-asks for unscored ones and for ones marked against an older paper.
`apps/api/src/tests/paper.service.ts:606`	`setQuestionStatus`, the drop/bonus write that starts a re-mark.

What it is for

Marking is kept off the request path. A hall of 6-8K students hands in within minutes; scoring inline would put the answer key, a per-question write and a rollup fold on the HTTP handler. A submit claims the row and enqueues an id; a pool of 8 workers evaluates and writes the durable columns.

The same processor is the re-mark mechanism. A dropped or bonus question does not enqueue anything per student: it bumps `Test.paperRevision`, and the sweeper re-asks for every sitting stamped behind that revision. There is no outbox and no relay.

The processor is the only writer of `Attempt.score`, `correctCount`, `wrongCount`, `unattemptedCount`, `sectionScores`, `timeTakenSec`, `scoredRevision`, `evaluatedAt` and of `AttemptSheet.verdicts`. It also runs the student's own rollup fold in the same transaction (`StudentStat`, `StudentSubjectStat`).

Flow: the scoring job

QUEUE JOB `SCORING` — MARKING ONE SITTING

1. Submit (or the sweeper's end of a stranded sitting) claims the row `IN_PROGRESS` to `SUBMITTED` with a conditional `updateMany`, then takes the last Redis state and writes the final sheet
   `apps/api/src/attempts/submit.service.ts:117`
   → `submittedAt` stamped, sheet final
2. The scoring job is enqueued after the last sheet write, so the scorer reads the sheet as it will stay
   `apps/api/src/attempts/submit.service.ts:128`
   → `queue([attempt])`
3. `addBulk` with name `scoring`, data `{attemptId, testId}` (ids only, never the sheet), `jobId` `scoring-<attemptId>`, `removeOnFail: true`
   `apps/api/src/attempts/scoring-queue.ts:37`
   → one BullMQ job per sitting; a second add under the id is swallowed
4. An enqueue that throws is caught and logged; the submit still succeeds and the sweeper picks the sitting up
   `apps/api/src/attempts/submit.service.ts:166`
   → submit response unaffected
5. The worker takes it: concurrency 8, 5 attempts, exponential backoff from 5 s, all read from the policy table
   `apps/api/src/queue/queues.ts:23`
   → `@Processor` at scoring.processor.ts:59
6. The processor is registered only when `API_ROLE` is `worker` or `all`
   `apps/api/src/attempts/attempts.module.ts:86`
   → no scoring on an `api`-only container
7. `process()` calls `score(attemptId)`; nothing branches on the job name, so a first score and a re-score are the same code
   `apps/api/src/attempts/scoring.processor.ts:75`
   → -
8. One `findUnique` with `SCORING_SELECT`: status, `isGraded`, `startedAt`, `submittedAt`, `test.paperRevision`, `test.scope`, `sheet.answers`. A missing row warns and returns null
   `apps/api/src/attempts/scoring.processor.ts:81`
   → job succeeds with nothing written
9. Status gate: only `SUBMITTED` and `EVALUATED` score. `IN_PROGRESS` or `VOIDED` returns null and the job succeeds
   `apps/api/src/attempts/scoring.processor.ts:89`
   → no marks for a voided sitting
10. `termsOf(testId, paperRevision)` returns the paper's rows in `order`, with marks, negative marks, status, correct option ids and answer key; the promise sits in a 64-entry LRU, so a hit costs no query
    `apps/api/src/attempts/paper-sheet.service.ts:160`
    → cache key `testId:paperRevision`
11. `scorableOf` joins sheet to terms by slot: slot N of the sheet is paper row N
    `apps/api/src/attempts/scoring.processor.ts:221`
    → one `ScorableQuestion` per paper row

12. `scorePaper` is pure: verdict, award, section tally (see the table below)
`apps/api/src/attempts/score-paper.ts:84`
→ `PaperScore`

13. `persist` opens `prisma.$transaction(..., TX_LIMITS.SHORT)`, 5 s to start and 10 s to finish
`apps/api/src/attempts/scoring.processor.ts:119`
→ one transaction for everything below

14. `mark` runs ONE raw statement: CTE `held` does `SELECT ... FOR UPDATE` and captures `evaluatedAt IS NULL AS first` and the old `score`; CTE `marked` updates `Attempt`; CTE `sheet` updates `AttemptSheet.verdicts` only if `marked` returned a row
`apps/api/src/attempts/scoring.processor.ts:152`
→ status `EVALUATED`, `evaluatedAt = COALESCE(old, now)`, score, three counts, packed `sectionScores`, `timeTakenSec`, `scoredRevision`

15. The `UPDATE` applies only `WHERE status = ANY(SCORABLE) AND scoredRevision <= paperRevision`. Zero rows (voided under the scorer, or overtaken by newer marks) returns null and the transaction writes and counts nothing
`apps/api/src/attempts/scoring.processor.ts:169`
→ `{applied: false}`; `score()` returns null at :96

16. `sheets !== 1` throws, which rolls the marks back: marks without verdicts is the one state worse than neither
`apps/api/src/attempts/scoring.processor.ts:185`
→ transaction aborted, BullMQ retries

17. First evaluation: `foldStudentSitting` adds the sitting to `StudentStat` then `StudentSubjectStat` inside the same transaction
`apps/api/src/attempts/rollup.service.ts:323`
→ student aggregates commit with the marks

18. First evaluation: `announce` writes a `RESULT_READY` notification, `dedupeKey` `result:<attemptId>`, via `createMany({skipDuplicates: true})`
`apps/api/src/attempts/scoring.processor.ts:190`
→ one `Notification` row, same transaction

19. Re-score instead: if the locked row's previous score differs from the new total, `announceCorrection` writes `RESULT_UPDATED` keyed on the NEW total (`result-updated:<attemptId>:<score>`); an unchanged total says nothing
`apps/api/src/attempts/scoring.processor.ts:137`
→ at most one correction row per distinct total

20. After commit, a first evaluation observes `scoring_duration_seconds` (submit to now)
`apps/api/src/attempts/scoring.processor.ts:99`
→ histogram sample

21. After commit, a re-score that moved marks calls `recount`: `rollup.rebuild(testId)` and `rollup.rebuildStudent(studentId)`, each a keyed ROLLUP job delayed 1 minute so a burst collapses to one. A failure here is logged, never failing a committed score
`apps/api/src/attempts/scoring.processor.ts:108`
→ two delayed jobs in the `rollup` queue

The decision at step 14 is the reason the row is locked: two scorers racing on one sitting (a retry beside a sweeper re-ask) serialize on `FOR UPDATE`, and only one of them sees `first = true`. A scorer that read the sitting unmarked but lost the first evaluation is treated as a re-score, so it neither double-announces nor double-folds (`fix(scoring) 06fa237`).

How a paper is marked

All rules live in `apps/api/src/attempts/score-paper.ts`, a pure function so they read as a table.

Question kind	Verdict (`isCorrect`)	Where
Option question	`correctOptionIds.includes(selectedOptionId)`; null when nothing was selected or the row has no key	score-paper.ts:119
`TEXT_FIELD`	Case- and whitespace-folded string compare, or for `ANSWER_MODE.NUMERIC` `abs(given − wanted) <= tolerance + 1e−9`. An empty answer, missing key, or key with no string answers is null, never wrong	score-paper.ts:127
`DESCRIPTIVE`	Blank is null; written is true where `scoreWriting` earned anything, false where it earned nothing	score-paper.ts:169
`TYPING`	Null, and not counted as unattempted (`isCounted`); the typing result is read off what was typed by the report	score-paper.ts:116

The kinds marked against a key on the question (`TEXT_FIELD`, `DESCRIPTIVE`) are the only ones that carry `answerKey` into the scorer (`KEYED_TYPES`, scoring.processor.ts:242).

Marks awarded (`awardFor`, score-paper.ts:149):

Paper-row status	Marks
`BONUS`	`marks` to everyone, answered or not
`DROPPED`	`marks` to anyone who attempted it (the negative is given back), 0 to anyone who left it
`ACTIVE`, verdict null	0
`ACTIVE`, `DESCRIPTIVE`	`marks` times the share `scoreWriting` gives, rounded to hundredths; no negative
`ACTIVE`, other	`marks` if right, `−negativeMarks` if wrong

`isCorrect` stays the key's verdict whatever the status; a drop or a bonus moves the marks, never the verdict. Totals and section tallies are summed in whole hundredths and divided once (`toHundredths`, :78), because marks are `Decimal(6,2)` and binary floats drift.

Flow: dropping or bonusing a question, and the re-mark

PATCH /ADMIN/TESTS/:ID/PAPER/:ROWID/STATUS — DROP OR BONUS A QUESTION

1. Route guarded by `@RequiresFeature(TEST_MANAGEMENT, WRITE)` and audited as test UPDATE; body parsed by `setPaperQuestionStatusSchema` (`status` of `ACTIVE` , `DROPPED` , `BONUS` , and a required `reason`)
   `apps/api/src/tests/tests.controller.ts:122`
   → validated body
2. Refused with CONFLICT unless the test has been finalized (offered): before that a question is edited, never withdrawn
   `apps/api/src/tests/paper.service.ts:612`
   → `NOT_FROZEN_MESSAGE`
3. `requireRow` loads the row and confirms it belongs to this test, else NOT_FOUND
   `apps/api/src/tests/paper.service.ts:615`
   → row's current status
4. Transaction opens with `beginPaperEdit` , which locks the `Test` row `FOR UPDATE` (the app's lock order is Test before Question)
   `apps/api/src/tests/paper.service.ts:619`
   → test lock held
5. `updateMany` where `testId` , the row's `questionId` and `status != new status` . The write is the gate: two admins clicking cannot both win. Every paper row of that question moves
   `apps/api/src/tests/paper.service.ts:621`
   → `PaperQuestion.status` changed (the sat guards allow a status-only change)
6. Zero rows changed returns 0 and stops: no revision bump, no audit entry
   `apps/api/src/tests/paper.service.ts:625`
   → nothing
7. Otherwise `Test.paperRevision` is incremented in the same transaction. That is the whole re-score request
   `apps/api/src/tests/paper.service.ts:627`
   → `Test.paperRevision` + 1
8. Audit entry against the ROW, with `from` / `to` status and the reason; a log line says the sittings are re-scored by the sweep
   `apps/api/src/tests/paper.service.ts:633`
   → `AuditLog` row
9. The sweeper's `askForRescores` (every 2 minutes) pages sittings with `scoredRevision < paperRevision` , then enqueues `rescore` jobs with `jobId` `scoring-rescore-<attemptId>-<revision>`
   `apps/api/src/attempts/attempt-sweeper.processor.ts:106`
   → one job per sitting per revision
10. The scoring job from the previous flow runs again; step 10 there misses the LRU (new revision, new key) and reads the paper afresh; step 14 stamps `scoredRevision = paperRevision`
    `apps/api/src/attempts/scoring.processor.ts:166`
    → new marks, new verdicts, new stamp
11. Sittings whose total moved get a `RESULT_UPDATED` notification and a delayed rebuild of the test and student rollups; ranks and percentiles follow because they are live counts
    `apps/api/src/attempts/scoring.processor.ts:137`
    → corrected result visible

The admin screen is `apps/admin/src/features/tests/paper-disposition.tsx:113` . Its confirm dialog re-reads the sitting count as it opens (`consequenceOf` , :71) and says every sitting that served the question is scored again; the count is every sitting on the test, so it is a ceiling.

> **Note** A drop does not re-mark instantly. The latency is up to one sweep (2 minutes) plus queue time, then the rollup rebuild's one-minute debounce. A student who reads their card in between sees the old marks.

Flow: the sweep that re-enqueues lost work

CRON ATTEMPT-SWEEP — EVERY 2 MINUTES

1. A job scheduler `attempt-sweep` is upserted at boot, only on a container that serves the worker role; `keepScheduled` registers it again on every Redis reconnect
   `apps/api/src/attempts/attempts.module.ts:110`
   → repeating job, `every: ATTEMPT_SWEEP_EVERY_MS`
2. Concurrency is 1 for this queue: two sweeps over the same set would fight, and no distributed lock stands behind it
   `apps/api/src/queue/queues.ts:28`
   → one sweep at a time
3. `process()` runs four arms in order; each of the last three is `.catch` -logged so one failure does not skip the next
   `apps/api/src/attempts/attempt-sweeper.processor.ts:41`
   → -
4. Arm 1, `endStranded` : pages `IN_PROGRESS` sittings whose `endsAt` is past `SAVE_GRACE_SEC` (200 per page, keyset on `endsAt, id`), keeps those with no live Redis key or whose key is abandoned and flushed, and ends them through `submit.expire` in lanes of 8
   `apps/api/src/attempts/attempt-sweeper.processor.ts:55`
   → sittings `SUBMITTED` and queued for scoring by the normal submit path
5. Arm 2, `rollup.sweep()` : asks for the cohort counting pass (a keyed, 5 s delayed job)
   `apps/api/src/attempts/rollup-queue.ts:27`
   → rollup job
6. Arm 3, `askAgainForUnscored` : first sets the `scoring_backlog` gauge to the count of sittings unscored for over 5 minutes
   `apps/api/src/attempts/attempt-sweeper.processor.ts:97`
   → gauge

7. Then `neverScored`: `SUBMITTED` with `score IS NULL` and `submittedAt` older than 30 s (`SUBMIT_QUEUE_GRACE_SEC`), oldest first, `LIMIT 1000`
   `apps/api/src/attempts/attempt-sweeper.processor.ts:149`
   → up to 1000 ids
8. `scoring.queue(page)` uses the SAME `scoring-<attemptId>` id, so a job BullMQ still holds swallows the re-ask; only a job that is really gone is recreated
   `apps/api/src/attempts/attempt-sweeper.processor.ts:99`
   → lost jobs recreated
9. Arm 4, `askForRescores`: pages `EVALUATED` sittings with `scoredRevision < Test.paperRevision` where `paperRevision > 0`, in 1000-row keyset pages on `(testId, scoredRevision, id)`, until a short page
   `apps/api/src/attempts/attempt-sweeper.processor.ts:123`
   → `Behind[]`
10. `scoring.rescore(page)` enqueues under `scoring-rescore-<attemptId>-<revision>`: one drop queues once, a second drop is a new ask
    `apps/api/src/queue/queues.ts:79`
    → re-score jobs

The status in each query is a raw literal (`UNSCORED`, `EVALUATED`, `IN_PROGRESS`, :174-180), not a bound parameter. A bound enum cannot prove a partial-index predicate, so the planner would skip `Attempt_unscored_idx`, `Attempt_rescore_idx` and `Attempt_expiring_idx` (the 2026-09-30 incident on live-ops is recorded in `docs/03-conventions.md`).

Why it is shaped this way

- **A stamp, not an outbox.** `scoredRevision` is written in the same statement as the marks. Earlier a drop wrote one outbox row per sitting, a relay forwarded them, a repair joined back, and a nightly prune cleaned up; migration `20260930140000` replaced that and `20260930170000` removed the outbox. The cost is that finding work is an index scan, not a queue of rows.
- **The rescore index carries the id.** `Attempt_rescore_idx ("testId","scoredRevision","id") WHERE status='EVALUATED'` (migration `20260930160000`): 6,000 sittings of one test share a revision, so a keyset without the id skips rows, and a sort per page sorted the whole matched set.
- **Ids in the job, never data.** The payload is `{attemptId, testId}` (`queues.ts:68`), so a job held across a deploy or a retry cannot carry a stale sheet. The price is one read per attempt.
- **Keyed jobs are the dedupe.** BullMQ holds one job per id, so asking twice is asking once. `keyedJob` sets `removeOnFail: true` (`queues.ts:63`): a retained failed job under an id would silently swallow every later add. The consequence is that a scoring job which exhausts its attempts is gone; `QueueFailures` logs it and counts it (`queue-failures.ts:17`), and the sweeper recreates it.
- **Positional sheets.** A sheet slot is the paper row's position, not a question id, which keeps the row small. It means a sheet can only be decoded with the paper in order, and that the paper cannot move once sat: two database triggers (`paper_question_sat_guard`, `question_version_sat_guard`) freeze it, which is also why the terms cache needs no invalidation beyond the revision key. `status` is the one column that may change, and `paperRevision` keys it.
- **Packed `sectionScores`.** Stored as 6-tuples `[baseConfigSectionId, score, correct, wrong, unattempted, timeSpentSec]` (`packedSections`, score-paper.ts:52) because repeated field names were most of the value. Read only through `sectionScoresIn` (:64), which drops anything that is not a 6-array. Raw SQL readers index positionally: `rollup.service.ts:209` sums element 5.
- **Time is derived, not stored per request.** `timeTakenSec` is start to submit, capped at 86,400, and 86,400 if never submitted (`leaderboard-score.ts`), so less time wins a tie.

What must already be true

- An `AttemptSheet` row exists (created in the start transaction).
- The sitting is `SUBMITTED` or `EVALUATED` with `submittedAt` set.
- `PaperQuestion` rows are frozen by the sat guards, and `optionIds` are filled by trigger.
- `REDIS_URL` is set, and `API_ROLE` is `worker` or `all`, or neither the processor nor the sweep scheduler exists.

The scorer reads and writes no application Redis key; BullMQ's own keyspace is its only Redis traffic. No prefix is configured, so staging and production must not share an instance (`docs/04-infrastructure.md`). Exact key names were not verified.

Data it touches

Column	Written at	Note
`Attempt.status` to `EVALUATED`	scoring.processor.ts:158	
`Attempt.evaluatedAt`	:159	first evaluation only (`COALESCE`)
`Attempt.score`	:160	`Decimal(8,2)`
`correctCount`, `wrongCount`, `unattemptedCount`	:161-163	
`Attempt.sectionScores`	:164	packed tuples
`Attempt.timeTakenSec`	:165	
`Attempt.scoredRevision`	:166	the `Test.paperRevision` marked against
`AttemptSheet.verdicts`	:175	`[isCorrect, marksAwarded]` per slot, at the paper row's position
`Notification`	:190, :203	`RESULT_READY` / `RESULT_UPDATED`
`StudentStat`, `StudentSubjectStat`	rollup.service.ts:323	first evaluation only, incremental add

`AttemptSheet.answers` slot shape is `[stateIndex, option, timeSpentSec, firstActionSec, answeredSec]`, plus a 6th element `typedAnswer` only when one was typed (`answer-sheet.ts:21`, :105). `option` is a position into `PaperQuestion.optionIds`, or the raw id when that array does not hold it (`optionOf`, :87), which then scores wrong. `SLOT_STATES` (:15) is append-only.

The SQL view `AttemptSheetAnswer` (`prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:418`) unpacks the sheet and verdicts into rows for ad-hoc SQL. Nothing in TypeScript queries it. *Read, not measured:* it recomputes `row_number()` over the paper for every slot, which is quadratic in paper length per sitting.

Who sees the result

There is no scoring endpoint. Students read results at `GET /me/attempts/:id/scorecard` and `/solutions` (`attempt-report.controller.ts`); both refuse with CONFLICT until the sitting is `EVALUATED` (`attempt-report.service.ts:139`, `attempt-report.ts:18`). Admins reach re-marking only through the status PATCH above.

Cost and performance

Per job: one `findUnique`, zero or one paper read (an LRU hit is free), then one transaction holding one raw statement for both table updates, two rollup upserts and one `createMany`. There is no N+1: the sheet-to-terms join is an array index. `docs/04-infrastructure.md` records the 25 September 2026 measurement (5,000 sittings of 100 questions evaluated in 4.75 s at concurrency 8); that figure was not re-run for this chapter. Scoring is idempotent, so the concurrency of 8 is a throughput choice, not a safety one.

What is wrong with it

- Stale: `apps/api/src/attempts/attempts.module.ts:89` says "Neither processor is here on purpose", but four processors are registered at :86 and the exports list holds services only; the comment predates the other processors.
- Stale: `apps/api/src/queue/queues.ts:20` says "6B retunes this table under load"; "6B" is a plan phase named only again at `metrics.service.ts:2` and in no tracked doc, so a reader cannot find what it refers to.
- Correctness (minor): `apps/api/src/attempts/scoring.processor.ts:103` calls `recount` for every re-score of an `EVALUATED` sitting even when the total did not move (`attempt.status === EVALUATED` alone is enough); the keyed 1-minute debounce makes it cheap but it does queue two rollup jobs for a no-op.
- Correctness (ceiling): `apps/api/src/attempts/attempt-sweeper.processor.ts:154` re-asks only the oldest 1000 unscored sittings per sweep; while more than 1000 are legitimately still queued, a genuinely lost newer job waits until older ones drain. It is bounded and recovers, but no gauge or log marks the batch as full (the delivery sweep has one).

The score card, the key and the per-question report

Where it lives

Layer	Path
Routes (student reads)	`apps/api/src/attempts/attempt-report.controller.ts`, `apps/api/src/attempts/question-report.controller.ts`, `apps/api/src/attempts/performance.controller.ts`
Score card, field effort, admin report	`apps/api/src/attempts/performance.service.ts`
Solutions, typing, trend	`apps/api/src/attempts/attempt-report.service.ts`
Per-question table	`apps/api/src/attempts/question-report.service.ts`, `apps/api/src/attempts/question-report.ts`
Shared refusal and mapping	`apps/api/src/attempts/attempt-report.ts`
The paper cache	`apps/api/src/attempts/paper-sheet.service.ts`
Sheet codec	`apps/api/src/attempts/answer-sheet.ts`
Web	`apps/exams/src/features/performance/`, `apps/exams/src/features/review/`
Mobile	`apps/mobile/src/components/report/`
Shapes and cache keys	`packages/contracts/src/attempts.ts`, `packages/contracts/src/stats.ts`, `packages/app-kit/src/student-queries.ts`

All three controllers are registered on the `CORE` API role only (`apps/api/src/attempts/attempts.module.ts:49`), so a results rush never lands on the containers serving the live hall. Every one carries `@Actors(ActorTypes.STUDENT)`; row ownership is part of every `WHERE`, not a guard.

What this is for

Once the scoring job has marked a sitting, the student reads what it was worth, where it put them in the cohort, and why each answer was right or wrong. Four student reads sit behind the report shell at `/attempts/:attemptId/report`, plus two that need no marks:

Route	Returns	Gate
`GET /me/attempts/:id/scorecard`	marks, standing, sections, composition, time, curve	own sitting, `EVALUATED`
`GET /me/attempts/:id/solutions?sectionId=&questionId=`	one section (or one question) with the key	own sitting, `EVALUATED`
`GET /me/attempts/:id/question-report`	one row per question, cohort beside it, the key's verdict	own sitting, `EVALUATED`
`GET /me/attempts/:id/typing`	each typing section's passage and what was typed	own sitting, `EVALUATED`
`GET /me/attempts/:id/field`	cohort effort beside a handed-in paper	own sitting, `SUBMITTED` or `EVALUATED`
`GET /me/performance`	the trend: every marked sitting, oldest first	the student

The web shell offers five tabs (`packages/app-kit/src/report-tabs.ts`): Score card, Subject report, Solution report, Question report, Compare. Subject report and Compare read the score card; they have no endpoint of their own. A paper with `maxMarks === 0` (a typing test alone) shows only the Score card tab (`apps/exams/src/features/performance/report.tsx:161`).

Flow: the score card

GET /ME/ATTEMPTS/:ID/SCORECARD — OPENING A RESULT

1. The report index route mounts `ScoreCardPanel`; the shell above it reads only the trend
   `apps/exams/src/App.tsx:216`
   → `useQuery(scoreCardQuery)`, key `['me','attempts',id,'score-card']`
2. The query waits on a queued marking job instead of showing Retry
   `packages/app-kit/src/student-queries.ts:163`
   → `retryWhileMarking`: up to 3 retries on a 409 `CONFLICT`, error toast suppressed
3. Guards run in registration order, then the controller
   `apps/api/src/app.module.ts:89`
   → JWT, actor type, feature, super-admin, throttler
4. The controller hands straight to the analytics service
   `apps/api/src/attempts/attempt-report.controller.ts:27`
   → `analytics.scoreCard(user.id, id)`
5. One `findFirst` with owner AND `status = EVALUATED` in the WHERE
   `apps/api/src/attempts/performance.service.ts:207`
   → the Attempt row plus test, blueprint and sections; no `questionVersion`
6. No row: a second tiny read decides the refusal
   `apps/api/src/attempts/attempt-report.ts:114`
   → `NOT_FOUND` "No such sitting" if not theirs; `SITTING_VOIDED` (409) if voided; else `CONFLICT` (409) "not been marked yet"
7. Test rollup row and the live standing, in parallel
   `apps/api/src/attempts/performance.service.ts:213`
   → `TestStat` row; one `standingsSql` raw query (rank, percentile, cohort size)

8. `figuresOf` runs five reads in parallel
   `apps/api/src/attempts/performance.service.ts:310`
   → paper rows, `AttemptSheet`, `TestSectionStat`, `topperOf`, `curveOf`
9. The sheet is joined to the paper by position and put in the student's display order
   `apps/api/src/attempts/answer-sheet.ts:198`
   → `ServedAnswer[]` carrying state, time, verdict, marks
10. The cohort curve is counted once per rollup, not per reader
    `apps/api/src/attempts/performance.service.ts:355`
    → in-process promise keyed `testId:computedAt`
11. Composition, sectional standing, time use and pace are derived; every rate carries its denominator
    `apps/api/src/attempts/performance.service.ts:335`
    → a figure over nobody is `null`, never `0`
12. The card is assembled: headline counts, rank/percentile/cohortSize, `hasWritten`
    `apps/api/src/attempts/performance.service.ts:224`
    → `ScoreCard` JSON, wrapped by `ResponseInterceptor` as `{success, data, meta}`
13. The client parses it; a 409 becomes the "No marks yet" EmptyState, `SITTING_VOIDED` the set-aside one
    `apps/exams/src/features/performance/score-card-gate.tsx:20`
    → panel drawn

The score card is one request for the card, Subject report and Compare tabs, pinned by `QUERY_BUDGET = { SCORE_CARD: 14, REFUSAL: 2 }` (`apps/api/test-db/attempt-report.db.test.ts:44`). The same `figuresOf` serves the admin's `GET /admin/students/:studentId/performance` (`apps/api/src/attempts/performance.controller.ts:36`), so the two cannot drift. `maxMarks` comes from the paper's own rows (`composition.maxMarks`), not the blueprint, so a per-question mark cannot make the card and the trend read different percentages.

The card carries no question and no key. `performance.service.ts` has no select that loads `questionVersion`; that absence is the privacy boundary for the card.

Headline and sections on screen

The hero leads with the percentile ("better than N of M") when the sitting is ranked, and with marks when it is not (`apps/exams/src/features/performance/score-card.tsx:164`). A retake is marked but outside the ranking (`isGraded: false`): rank, percentile and the cohort curve are `null`, and the shell shows an Info tooltip (`RETAKE_NOTICE`). A paper with a typing section appends a `TypingSection` per section, scored client-side from `/typing` with `scoreTyping`; a typing-only paper shows the typing result as the whole card. A paper with a written section shows an info Alert (`hasWritten`).

Flow: the answer key

GET /ME/ATTEMPTS/:ID/SOLUTIONS — READING THE KEY

1. Solution tab mounts with section `''`; changing tab changes the query key
   `apps/exams/src/features/review/review.tsx:18`
   → key `['me','attempts',id,'solutions',sectionId]`, previous section kept as placeholder
2. Query string validated
   `apps/api/src/attempts/attempt-report.controller.ts:54`
   → `{sectionId?, questionId?}` through `solutionsQuerySchema`
3. The sitting alone is read, with no key in the select
   `apps/api/src/attempts/attempt-report.service.ts:134`
   → Attempt, `AttemptSheet`, Test, blueprint sections
4. The whole gate
   `apps/api/src/attempts/attempt-report.service.ts:138`
   → not theirs: `NOT_FOUND`; voided: `SITTING_VOIDED`; not `EVALUATED`: `CONFLICT`
5. The paper with its key comes from a per-process promise cache
   `apps/api/src/attempts/paper-sheet.service.ts:149`
   → `PaperQuestion` + `Question` + `QuestionVersion{content,options,answerKey}`, keyed `testId:paperRevision`
6. Sheet and verdicts are laid over the rows by position
   `apps/api/src/attempts/answer-sheet.ts:172`
   → each row gets this sitting's answer, verdict and marks
7. Content and options are narrowed to the section's language set
   `apps/api/src/attempts/attempt-report.service.ts:186`
   → a section's language rule ("English Language reviews in English") wins over the sitting's
8. The same sequencer the exam used reorders questions and options
   `apps/api/src/attempts/attempt-report.service.ts:193`
   → `servedQuestions(..., shuffleSeed, shuffleQuestions, shuffleOptions)`; option "C" is still "C"
9. Typing passages are dropped (no key to review); sections listed without them
   `apps/api/src/attempts/attempt-report.service.ts:204`
   → `sections[]` for the tab bar
10. Narrowing happens AFTER the sequencer: `questionId` wins, else `sectionId`, else the first section
    `apps/api/src/attempts/attempt-report.service.ts:233`
    → one section's questions only
11. Image keys of the narrowed questions only are turned into URLs
    `apps/api/src/attempts/attempt-report.service.ts:214`
    → `servedQuestion` replaces keys in stem, passage, solution, options

12 Response
`apps/api/src/attempts/attempt-report.service.ts:216`
→ `{attemptId, testId, testTitle, languages, sections, sectionId, questions[]}`; each question carries `options[].isCorrect`, `answerKey`, the worked solution

The paper cache

`PaperSheetService` holds four per-process maps (`rows`, `terms`, `served`, `solutions`; `apps/api/src/attempts/paper-sheet.service.ts:116`). `hold` stores the *promise*, so a results rush reads a cold paper once; a rejected read deletes itself so the next reader retries (`:108`). `recall` re-inserts on a hit and `remember` evicts the longest-unread key past `HELD_PAPERS = 64`, so it is LRU, with no TTL and no byte bound. The solutions key includes `Test.paperRevision`, so a dropped question (which bumps the revision) makes the old copy unreachable.

Is there a solutions gate?

No. The only gate is the sitting's own: yours, and `EVALUATED`. The moment the scoring job marks one student's sitting, that student can read the key while the rest of the batch is still sitting. This is accepted: a student who has sat can hand the key to one who has not whatever the system does. The old stale comments that described a `solutionsAreOpen` gate in `packages/contracts/src/stats.ts` are gone.

Flow: the per-question report

GET /ME/ATTEMPTS/:ID/QUESTION-REPORT — THE TABLE

1 Question tab mounts; its query is the same marking-aware kind
`apps/exams/src/features/performance/question-report.tsx:48`
→ key `['me','attempts',id,'question-report']`

2 Controller
`apps/api/src/attempts/question-report.controller.ts:12`
→ `questions.forAttempt(user.id, id)`

3 Owner AND `EVALUATED` in the WHERE; the attempt select has no `questionVersion`
`apps/api/src/attempts/question-report.service.ts:99`
→ Attempt + sheet + blueprint

4 No row: same refusal as the card
`apps/api/src/attempts/attempt-report.ts:114`
→ `NOT_FOUND` / `SITTING_VOIDED` / `CONFLICT`

5 Five reads in parallel
`apps/api/src/attempts/question-report.service.ts:106`
→ `TestQuestionStat` rows, `TestStat` totals, `topperOf`, the whole paper WITH the key, live cohort size

6 Sheet and verdicts decoded onto the rows; option order and question order as served; typing passages filtered
`apps/api/src/attempts/question-report.service.ts:119`
→ `ReportPaperRow[]`

7 Each option is numbered by the seat this sitting was served it in; `correctAnswer` is the first accepted string, only for `TEXT_FIELD`
`apps/api/src/attempts/question-report.service.ts:220`
→ `Map<questionId, KeyedQuestion>`

8 Cohort columns are blanked under the floor
`apps/api/src/attempts/question-report.service.ts:151`
→ `evaluatedCount >= 10` (`COHORT_COMPARISON_FLOOR`, `packages/contracts/src/stats.ts:778`), else `null`

9 One row per served question: the student's half by slot, the cohort's by `paperQuestionId`, the key's by `questionId`
`apps/api/src/attempts/question-report.ts:48`
→ attempt rate, accuracy (p-value), average and topper time, option shares, `correctOptionId`, `correctAnswer`

10 Pace index: whole paper against the cohort's average, null under the floor or with no clock
`apps/api/src/attempts/question-report.ts:70`
→ `paceIndex`

11 Response
`apps/api/src/attempts/question-report.service.ts:163`
→ `{attemptId, testId, testTitle, cohortSize, paceIndex, sections, questions}`

The table reads the paper straight from Postgres every time; it does not use the solutions cache. Cohort columns come only from the rollup tables, so a question no rollup pass has counted shows a dash, never a live scan. The screen's note says which: fewer than 10 sat, or the pass that counts them runs on a schedule (`cohortNoteSays`, `packages/app-kit/src/student-figures.ts:137`). `cohortSize` is a live count while the floor is checked against the rolled `evaluatedCount`, so the two can briefly disagree.

The web table (`apps/exams/src/features/performance/question-report.tsx`) offers a result filter (all, correct, incorrect, unattempted), a stat band (pace, sittings, blank-but-answerable, time that bought nothing) and, where any option has a count, an expandable per-question option distribution with a warning naming a wrong option most of the field chose.

Compare

Compare is the score card plus the leaderboard. When the card's cohort has sittings, the tab draws You / Average / Topper bars and the paper's board (`GET /me/leaderboard`, page by page, own seat pinned). When nobody is ranked, it stands the sitting against the student's own attempts of the same paper and plots retakes (`apps/exams/src/features/performance/attempt-compare.tsx:34`), with an info Alert saying so (`compare.tsx:69`).

The data it touches

Column	Role here	Written by
`Attempt.status`	`EVALUATED` is the whole gate for the card, key and table	`apps/api/src/attempts/scoring.processor.ts:158`
`Attempt.score`, counts, `sectionScores`	headline figures	scoring processor

Column	Role here	Written by
`Attempt.shuffleSeed`	re-derives the student's display order	attempt start
`Attempt.isGraded`	false for a retake: no rank, no curve	attempt start
`AttemptSheet.answers` / `verdicts`	positional arrays, slot N is the Nth `PaperQuestion` by `order`	autosave/flush; scorer
`QuestionVersion.options` / `answerKey` / `content`	the key (frozen once sat)	authoring
`Test.paperRevision`	keys the solutions cache	paper edit
`TestStat`, `TestSectionStat`, `TestQuestionStat`	the cohort halves	rollup

There is no Redis and no queue on any read here. The `AttemptSheetAnswer` SQL view exists for ad-hoc support SQL only; no TypeScript reads it.

Who calls what

Read	Web	Mobile
`scorecard`	`features/performance/score-card.tsx`, `subject-report.tsx`, `compare.tsx`	`components/report/score-card-panel.tsx`, `subjects-panel.tsx`, `compare-panel.tsx`
`solutions`	`features/review/review.tsx`; `saved-question-dialog.tsx` by `?questionId=`	`components/report/solutions-panel.tsx`
`question-report`	`features/performance/question-report.tsx`	`components/report/questions-panel.tsx`
`typing`	`score-card.tsx` (enabled once the card succeeds)	`score-card-panel.tsx`
`field`	`features/exam/submitted.tsx`, before marks exist	read, not traced
`performance`	shell picker and `overview.tsx`	`compare-panel.tsx`

`submitted.tsx` also prefetches the card with `scoreCardAheadQuery` (one retry on a 409, never a banner), so the result is often warm when the student taps through. Old bookmarks (`/attempts/:id/scorecard`, `/review`, `/questions`) redirect into the tabbed shell (`apps/exams/src/App.tsx:237`).

> **Note** The 409 `CONFLICT` and `SITTING_VOIDED` share a status; the client reads `error.code` (`packages/app-kit/src/marking.ts:6`). Marking-pending retries; voided never does. A voided sitting carries no marks or rank.

> **Watch** Adding a field to any `*_SELECT` in `performance.service.ts` that reaches `questionVersion` would put the key on the score card path, which every report tab shares. The header says to keep it out; nothing mechanical does.

What changed

- The `EVALUATED` check moved into the card's WHERE, and a refusal costs two queries; the question report now does the same (`question-report.service.ts:99`). Both share `refuseUnreadable`.
- A voided sitting answers `SITTING_VOIDED` instead of "not marked yet" (c2aaa4d), and every tab draws a "set aside" state.
- Typing: `GET /attempts/:id/typing`, a typing-only paper's card, and typing passages excluded from the key and the table (7824421, 7fcd496).
- `GET /attempts/:id/field` gives the submitted page a cohort-effort comparison before marking runs (6ea1727).
- Per-section languages: a section can be reviewed in its own language rule (fd39f07).
- Dropped and bonus questions carry a `disposition` on the card, key and table rows.
- Passages: a question's passage rides in content and is resolved to image URLs (09862f0).
- The solutions-gate comments in contracts were corrected (84a4b40).

What is wrong with it

- `apps/api/src/attempts/attempt-report.controller.ts:50` — Correctness (comment): says solutions is "the ONLY endpoint carrying" the key; `question-report` also returns `correctOptionId`, `options[].isCorrect` shares and `correctAnswer` (`question-report.service.ts:74`). Stale.
- `apps/api/src/attempts/attempt-report.service.ts:3` — Stale: "the one place a student's `questionVersion` loads"; `KEY_ROW_SELECT` loads it for the question report too.
- `apps/api/src/attempts/question-report.service.ts:36` — Stale: "No `questionVersion` anywhere in here" is true of the attempt select only; `KEY_ROW_SELECT` at `:71` loads it.
- `apps/api/src/attempts/question-report.service.ts:110` — Correctness (cost): the report re-reads the whole paper with its key from Postgres on every open, bypassing the `solutionsOf` cache the review uses; a 6K results rush pays the paper read per student.
- `apps/api/src/attempts/attempt-report.service.ts:212` — Correctness: a `sectionId` that is not on the paper returns `questions: []` with that id echoed back, not a refusal.
- `apps/api/src/attempts/attempt-report.service.ts:139` — Privacy: the first student marked can read the key and worked solutions while the rest of the batch is still sitting. Accepted by design; no cohort-wide release instant exists.

Rank, percentile, the board and the rollups

Where it lives

Path	What it is
`apps/api/src/attempts/ranking-sql.ts`	Every ranking query as `Prisma.Sql`; the only place rank SQL is written.
`apps/api/src/attempts/leaderboard.service.ts`	`standing`, `cohortSize`, `standingsOfStudent`, `standingsOf`, `standingsOfTests`, `cohortsOf`.
`apps/api/src/attempts/leaderboard-view.service.ts`	The student board: the reader's own seat live, the page of seats held.
`apps/api/src/attempts/leaderboard-board.ts`	Pure shaping: `boardName`, `pageRanks`, `splitBoard`.
`apps/api/src/attempts/rollup.service.ts`	The five aggregates: sweep, recount, rebuild, delta fold.
`apps/api/src/attempts/rollup-fold.ts`	What one sitting adds, with no database.
`apps/api/src/attempts/rollup.processor.ts`, `apps/api/src/attempts/rollup-queue.ts`	The BullMQ worker and the producer side.
`apps/api/src/attempts/overview.service.ts`	`/me/overview`: stored tallies plus live standings.
`apps/api/src/attempts/performance.service.ts`	The score card and the admin report: trajectory, curve, topper, section cohort.
`apps/api/src/dashboard/`	The admin dashboard counts, gated per band by `dashboard-bands.ts`.
`packages/contracts/src/leaderboard.ts`	The board's schema, `LEADERBOARD_PODIUM`, `LEADERBOARD_PAGE_SIZE`, `leaderboardPageOf`.

What it is for

A standing must be true at the instant it is read, and the live hall must not pay for it. So rank, percentile, cohort size, the score curve and the topper are **counted from `Attempt` on every read and stored nowhere** (`prisma/schema.prisma:1246` says the spread and the topper are counted live; no rank, percentile or topper column exists). What is stored is the additive part: a student's career tallies, a test's cohort size and clock, per-section totals and per-question item analysis. The student's two tables are folded as a delta inside the scorer's transaction; the test's three are recounted by a periodic pass.

The cohort and the order

A test's cohort is the sittings that satisfy `IN_COHORT` (`apps/api/src/attempts/ranking-sql.ts:11`): `"isGraded" AND "status" = 'EVALUATED' AND "score" IS NOT NULL`. Order is `RANK_ORDER` (`:21`): marks descending, then `timeTakenSec` ascending with no time last, then `id`. The tiebreak inside the standing query writes the same rule a second way, `COALESCE(timeTakenSec, 2147483647)`, held in `SLOWEST_SEC` (`:24`). Percentile is the SQL function `sitting_percentile(outscored, tied, cohort)` (migrations `20260911190000_rank_is_counted_in_postgres`, `20260912010000_percentile_is_parallel_safe`): a tie on marks counts half, time does not enter, a field of one is 100.

The filter stays literal SQL, not bound parameters, so the planner can prove the partial index predicate. The index is `Attempt_ranking_idx`, `("testId", "score" DESC, "timeTakenSec", "id") INCLUDE ("studentId", "submittedAt") WHERE "isGraded" AND "status" = 'EVALUATED' AND "score" IS NOT NULL`, rebuilt in `20260929090000_the_bank_boards_and_ledger_read_through_indexes` so every board read is index-only. Beside it, hand-written in migrations: `Attempt_graded_per_test_key` (unique `(testId, studentId)` where `isGraded`, so a cohort is one sitting per student) and `Attempt_student_drift_idx` (only sittings re-scored or voided after their first evaluation, `20260928203000`). The schema declares `@@index([testId, updatedAt])` for the cohort sweep and `@@index([submittedAt])` for the reports (`prisma/schema.prisma:777`, `:779`).

The queries in `ranking-sql.ts`, by consumer:

Function	Shape	Read by
`standingsSql` (`:56`)	One LATERAL cohort count per chosen sitting; `{ attemptId }`, `{ attemptIds }` or `{ studentId }` with an optional `newest` bound applied before the laterals.	Score card, trajectory, `/me/overview`, the board's gate.
`cohortSizeSql` (`:95`)	`COUNT(*)` of the cohort.	`LeaderboardService.cohortSize`.
`rankedCohortSql` (`:130`)	One window pass over the whole cohort, named (student, branch).	The held ranking behind the board.
`testBoardSql` (`:139`)	The same pass, filtered to the podium, one page of ranks and the reader's seat.	The board when the held ranking cannot be used.
`testResultsSql` (`:162`)	One window pass in `standingsSql`'s order, then the unranked sittings.	The admin result export (`apps/api/src/attempts/test-report.ts:153`).
`cohortStandingsSql` (`:195`)	Window pass per test; a date window narrows AFTER ranking.	Period and cohort reports (`apps/api/src/reports/`).
`cohortFiguresSql` (`:231`)	Size, mean, highest and the topper per test.	Report tests table.
`rankOneSql` (`:27`)	`LIMIT 1` in index order.	`topperIdOf` (`apps/api/src/attempts/topper.ts:41`).

Flow: a score card's rank and percentile

GET /ME/ATTEMPTS/:ID/SCORECARD — ONE SITTING'S STANDING

1. Route on `@Controller('me')` with `@Actors(STUDENT)`; the id is the sitting
   `apps/api/src/attempts/attempt-report.controller.ts:27`
   → `user.id` from the token
2. Global guards run in order: JWT, actor, feature, super-admin, throttler; then the interceptors
   `apps/api/src/app.module.ts:89`
   → request admitted
3. The sitting is read with owner and `status: EVALUATED` in the WHERE; a miss goes to `refuseUnreadable`
   `apps/api/src/attempts/performance.service.ts:205`
   → row, or a refusal that says whether it is voided or unmarked
4. In parallel: the test's `TestStat` row and the live standing
   `apps/api/src/attempts/performance.service.ts:213`
   → `testStats` map, `Standing`
5. `standing()` runs `standingsSql({ attemptId })`; the chosen CTE keeps the sitting only if it is in the cohort
   `apps/api/src/attempts/leaderboard.service.ts:46`
   → one row, or none for a retake, void or unscored sitting
6. The LATERAL counts cohort, outscored, tied and rank over `Attempt_ranking_idx`; `sitting_percentile` is applied in the same statement
   `apps/api/src/attempts/ranking-sql.ts:76`
   → `rank`, `percentile`, `cohort_size`
7. `test_id` must equal the asked test, else the standing is null
   `apps/api/src/attempts/leaderboard.service.ts:48`
   → `Standing` or null
8. `figuresOf` adds composition, sections against the cohort, the topper's clock and the curve
   `apps/api/src/attempts/performance.service.ts:303`
   → `ScoreCard` fields
9. `rank`, `percentile`, `cohortSize` are `standing?.x ?? null`; the interceptor wraps
   `apps/api/src/attempts/performance.service.ts:248`
   → `{ success: true, data, meta }`

Nothing is cached for the standing. The cohort curve and the field-effort figures are held in process per `TestStat.computedAt` (`performance.service.ts:355`, `:297`), so a rollup moving the watermark is what retires them.

Flow: the leaderboard

The board is one paper. Marks do not compare across papers, so no board spans two. It is a podium of three (`LEADERBOARD_PODIUM`, `packages/contracts/src/leaderboard.ts:15`), then the whole ranking from fourth place, 20 to a page (`LEADERBOARD_PAGE_SIZE`, `:18`), with the reader's own seat carried with every page and pinned under the table when it is on another.

GET /ME/LEADERBOARD?TESTID=&PAGE=&PAGESIZE= — A PAGE OF THE BOARD

1. `@Query(new ZodQuery(leaderboardQuerySchema))`; `page` defaults 1, `pageSize` defaults 20 and is capped by `PAGE_SIZE_MAX`; the reader is `@CurrentUser()`, never a query value
   `apps/api/src/attempts/leaderboard.controller.ts:23`
   → `LeaderboardQuery`
2. Mounted on the EXAM role only
   `apps/api/src/attempts/attempts.module.ts:49`
   → route exists on exam containers
3. Gate: the reader's own graded EVALUATED sitting on this test, with the test title and `TestStat.computedAt` in one select; no row throws `NOT_FOUND`
   `apps/api/src/attempts/leaderboard-view.service.ts:42`
   → `mine.id`, watermark
4. Own standing counted live by the call the score card makes, so the two can never quote different ranks
   `apps/api/src/attempts/leaderboard-view.service.ts:55`
   → `Standing`
5. `standing === null` (retake, void, unscored) returns an empty board with `cohortSize: 0`
   `apps/api/src/attempts/leaderboard-view.service.ts:57`
   → `emptyBoard`
6. `pageRanks` turns page into a rank window: `from = 3 + (page-1)*pageSize + 1`
   `apps/api/src/attempts/leaderboard-board.ts:16`
   → `{ from, to }`
7. `heldSeats`: with no `TestStat` yet, return null
   `apps/api/src/attempts/leaderboard-view.service.ts:90`
   → live path
8. `hold()` keeps one promise per `testId:computedAt-ms` in an 8-entry in-process map; a miss runs `rankedCohortSql` over the whole cohort
   `apps/api/src/attempts/leaderboard-view.service.ts:92`
   → `CohortSeatRow[]` in memory
9. Agreement test: the held seat's rank and cohort size must equal the live standing; if not, null
   `apps/api/src/attempts/leaderboard-view.service.ts:99`
   → held or live
10. Agree: `seatsOn` cuts podium, the page, and the reader's own seat
    `apps/api/src/attempts/leaderboard-view.service.ts:105`
    → seats

11. Disagree: `testBoardSql(testId, mine.id, ranks)` selects the same three groups live
`apps/api/src/attempts/leaderboard-view.service.ts:61`
→ seats

12. Rows are shaped in Node: `isYou` from the gate's attempt id, `percentile` only on the reader's own row, a blank name reads `Student`
`apps/api/src/attempts/leaderboard-view.service.ts:63`
→ `LeaderboardRow[]`

13. `splitBoard` separates podium (rank at most 3) from the page's rows; `total = cohortSize - 3`; `yourPage = leaderboardPageOf(rank, pageSize)`, null on the podium
`apps/api/src/attempts/leaderboard-view.service.ts:72`
→ `{ testId, label, cohortSize, podium, rows, page, pageSize, total, you, yourPage, generatedAt }`

The reader's rank is always the live count; only the *other* seats can be up to one rollup old, and only until the watermark moves. The held ranking is keyed by `TestStat.computedAt`, which moves whenever the cohort is recounted, and a sitting that joined or left the cohort since makes the reader's rank disagree with the held one, which sends that request to the live path. A rejected read is deleted from the map so the next reader retries (`hold`, `apps/api/src/attempts/paper-sheet.service.ts:96`). No row carries a mobile, an email, an attempt id or an answer; the privacy argument for the board is that it is behind the JWT guard and `@Actors(STUDENT)` and has no public twin (`leaderboard.controller.ts:1`).

Clients: the exams SPA's page `apps/exams/src/features/leaderboard/leaderboard.tsx:56` (query key `[...leaderboardQueryKey(testId), page]`, `keepPreviousData` so a page turn does not blink), the table and pager in `apps/exams/src/features/leaderboard/board.tsx:78`, the compare tab `apps/exams/src/features/performance/compare.tsx:25`, and mobile `apps/mobile/src/components/performance/leaderboard-panel.tsx:31`. The reader's pinned seat is shown when `yourPage` differs from the page on screen, and pressing it turns to that page.

Flow: the rollups

There are five tables. Two are per student and written as a delta with the marks; three are per test and recounted.

Table	Key	Holds	Written by
`StudentStat`	`studentId`	`testsEvaluated`, `sumScore`, `totalCorrect/Wrong/Unattempted`, `sumTimeSec`, `retakeCount`, `lastAttemptAt`, `computedThrough`, `computedAt` (`prisma/schema.prisma:1213`)	`addToStudent` delta; `writeStudent` outright
`StudentSubjectStat`	`(studentId, subjectId, scope)`	`attempted`, `correct`, `sumTimeSec` (`:1231`)	`addToSubjects` delta; `writeStudent` delete and create
`TestStat`	`testId`	`evaluatedCount`, `sumTimeSec`, `computedAt` only (`:1247`)	`writeTestTotals`
`TestSectionStat`	`(testId, baseConfigSectionId)`	`attempted`, `sumScore`, `sumTimeSec` (`:1257`)	`writeSectionTotals`
`TestQuestionStat`	`(testId, paperQuestionId)`	correct, wrong, skipped, `sumTimeSec`, `optionCounts` (`:1273`)	`writeItemTotals`

Every table keeps sums and counts; averages, accuracy and p-values are derived on read. `TestStat` is down to three columns because the mean, highest, lowest and topper are counted live. Its `computedAt` doubles as the cache key for the held ranking, the curve and the field effort. `sectionScores` is a packed array per section, `[sectionId, score, correct, wrong, unattempted, timeSpentSec]` (`rollup.service.ts:225`); index 5 feeds both `sumTimeSec` columns, index 1 feeds `TestSectionStat.sumScore`. Redis is not touched by any of this.

THE FIRST EVALUATION OF A SITTING — THE STUDENT'S DELTA

1. The scoring job runs `persist` in one `SHORT` transaction (5 s wait, 10 s limit)
`apps/api/src/attempts/scoring.processor.ts:119`
→ tx open

2. `mark` claims and marks the sitting in ONE statement: it locks the row, sets `EVALUATED`, `evaluatedAt = COALESCE(...)`, score, counts, packed sections, `timeTakenSec`, `scoredRevision`; `first` is `evaluatedAt IS NULL`
`apps/api/src/attempts/scoring.processor.ts:144`
→ `Attempt` row, `first`

3. If the status moved or a newer paper's marks landed, nothing applies
`apps/api/src/attempts/scoring.processor.ts:128`
→ `applied: false`

4. On `first`, `foldStudentSitting` runs in the same transaction
`apps/api/src/attempts/scoring.processor.ts:131`
→ totals of one sitting

5. `addToStudentTotals` works the sitting out with no database: every sitting adds answers and time; a graded one adds `testsEvaluated` and score, an ungraded one adds `retakeCount`; typing passages fold nothing
`apps/api/src/attempts/rollup-fold.ts:84`
→ `StudentTotals`

6. `StudentStat` is written first: `INSERT ... ON CONFLICT DO UPDATE` adding each column, `lastAttemptAt = GREATEST(...)`
`apps/api/src/attempts/rollup.service.ts:335`
→ one row incremented

7. `StudentSubjectStat`, one `unnest` upsert in subject order
`apps/api/src/attempts/rollup.service.ts:363`
→ one row per subject and scope incremented

8. Commit: marks and totals land together; a retry cannot count twice because the claim is already taken
`apps/api/src/attempts/scoring.processor.ts:119`
→ durable

`StudentStat` is locked first on purpose, as `rebuildStudent` locks it first, so a rebuild running mid-evaluation queues behind the scorer rather than racing it. A first evaluation queues no cohort job at all: the sitting's `updatedAt` is what the sweep finds.

ATTEMPT-SWEEP TICK (EVERY 2 MINUTES) — THE COHORT RECOUNT

1. A repeat scheduler `attempt-sweep`, `every: ATTEMPT_SWEEP_EVERY_MS` (2 min), registered by the WORKER role only under a fixed scheduler id
   `apps/api/src/attempts/attempts.module.ts:114`
   → one tick per interval
2. The sweeper processor ends stranded sittings, then calls `rollup.sweep()`; a failure here is logged, not thrown
   `apps/api/src/attempts/attempt-sweeper.processor.ts:43`
   → job `sweep-cohorts` added
3. `RollupQueue.sweep` adds `sweep-cohorts` under the one id `rollup-sweep-cohorts` with a 5 s delay, so a burst is one job
   `apps/api/src/attempts/rollup-queue.ts:27`
   → delayed job, `removeOnComplete`
4. The rollup worker (concurrency 2, 5 attempts, 5 s backoff) dispatches on `job.name`; legacy `fold-attempt` and `fold-pending` names map to the same sweep
   `apps/api/src/attempts/rollup.processor.ts:27`
   → `sweepCohorts()`
5. `changedTests`: tests with a sitting that MOVES the cohort (voided, or graded and evaluated) whose `updatedAt` beats `TestStat.computedAt` minus 2 minutes; at most 50
   `apps/api/src/attempts/rollup.service.ts:147`
   → test ids
6. `recountTest` per test, each in `tried()` so one failure does not stop the pass; a MEDIUM transaction (10 s wait, 15 s)
   `apps/api/src/attempts/rollup.service.ts:194`
   → tx open
7. `writeTestTotals`: one `INSERT ... ON CONFLICT` over `IN_COHORT` sittings summing the packed time
   `apps/api/src/attempts/rollup.service.ts:207`
   → `TestStat` row with a new `computedAt`
8. `writeSectionTotals`: delete the test's rows, then one `INSERT ... SELECT` over `jsonb_array_elements(sectionScores)` with `IN_COHORT`
   `apps/api/src/attempts/rollup.service.ts:231`
   → `TestSectionStat` rows
9. `staleItems`: tests with `evaluatedCount > 0`, no item row newer than `TestStat.computedAt`, and none newer than now minus 15 minutes (`COHORT_COUNT_EVERY_MIN`); at most 50
   `apps/api/src/attempts/rollup.service.ts:162`
   → test ids
10. `recountTestItems` in a BULK transaction (120 s): `firstSittings` picks one sitting per student, oldest first, then `replay` reads them 200 ids at a time with each paper once
    `apps/api/src/attempts/rollup.service.ts:243`
    → in-memory `Map<paperQuestionId, QuestionTotals>`
11. `writeItemTotals`: `deleteMany` then `createMany`
    `apps/api/src/attempts/rollup.service.ts:255`
    → `TestQuestionStat` rows
12. `driftedStudents`: `StudentStat` rows with a sitting EVALUATED or VOIDED whose `updatedAt` is past `evaluatedAt` and past `computedThrough` minus 2 minutes, through `Attempt_student_drift_idx`; at most 50
    `apps/api/src/attempts/rollup.service.ts:179`
    → student ids
13. `rebuildStudent` in a BULK transaction: upsert `StudentStat` first, replay every EVALUATED sitting (retakes included), then `writeStudent`
    `apps/api/src/attempts/rollup.service.ts:277`
    → `StudentStat` set outright, `computedThrough = now`, `StudentSubjectStat` deleted and recreated

The other ways a rebuild is asked for

Trigger	Where	Job id
A re-score (a dropped question, a changed paper)	`scoring.processor.ts:108`, `recount` queues `rebuild(testId)` and `rebuildStudent(studentId)`	`rollup-rebuild-<testId>` and `rollup-rebuild-student-<studentId>`, 1 minute delay, so a drop's re-scores collapse to one
A void	`apps/api/src/attempts/attempt-resolution.service.ts:168` (`reverse`) queues the same pair	same ids
Admin re-sync, `POST /admin/tests/:id/analytics/resync`	`apps/api/src/attempts/test-analytics.service.ts:100` calls `rebuildNow`	`rollup-rebuild-<testId>-now`, zero delay, a separate id so a waiting debounce cannot swallow it
Backfill	`apps/api/src/enqueue-rollup-rebuild-all.ts` (`pnpm rollup:rebuild-all` in `apps/api`)	the fixed `rollup-rebuild-all`, kept on neither outcome

`rebuildAll` (`rollup.service.ts:294`) tries every test and every student on its own, logs failures and says how many landed, so one bad row no longer fails the job and restarts all five retries from the first test. Reversal is always a bounded rebuild, never an undone delta: running a delta twice corrupts the total, running a rebuild twice writes the same answer.

Performance trends and the overview

- `GET /me/performance` (`attempt-report.controller.ts:61`, `attempt-report.service.ts:98`) reads the newest 500 EVALUATED sittings of the student (`HISTORY_LENGTH`), turns them oldest first, and puts live standings on the newest 20 only (`TREND_LENGTH`) through one `standingsOf(ids)` call. Marks are percentages of the paper's own marks (`paperMarks`).

- **The report trajectory** (`performance.service.ts:169`) takes up to 20 sittings in the scope (`SCOPE_ATTEMPT_CAP`), the anchor is the newest graded one, and `toPoint` (`:445`) gives each point its live percentile, rank and cohort size, falling back to `TestStat.evaluatedCount` for the size. Only a single-sitting scope has a curve (`curveOf`, `:345`).
- `GET /me/overview` (`overview.service.ts:55`): `StudentStat` and `StudentSubjectStat` for the tallies, three queries in parallel, plus `standingsOfStudent(studentId, RECENT_STANDINGS)`, which is five laterals. `avgPercentile` and `bestPercentile` are over those five newest graded sittings; a student with no row is a clean slate, not an error. Mounted on the CORE role.
- `GET /me/performance/days` counts sittings by institute day since the account opened; no paper is read.
- **Admin dashboard** (`apps/api/src/dashboard/dashboard.service.ts`): counts and `groupBy`s, each band behind `bandsFor` so a count the caller may not see is never run. `sittings` (`:173`) is the one band that reads a rollup, `TestStat.evaluatedCount`, for the recent active tests.

What must already be true

Both `sitting_percentile` migrations are applied, or every standing and board query errors. A sitting is in a cohort only when `isGraded AND status = 'EVALUATED' AND score IS NOT NULL`, and a board needs the reader's own such sitting. Held neighbours need a `TestStat` row, so one rollup pass. The sweep scheduler registers on the WORKER role only; the board is mounted on EXAM, the overview on CORE (`attempts.module.ts:49`, `:52`).

Why it is shaped this way

- **Counted, not stored.** A stored rank is wrong when the next student submits. The first implementation used a Redis sorted set (`20260911190000`), a second source of truth with its own rebuild.
- **Student tallies are a delta, cohort tallies a recount.** Folding the cohort per submit would serialise thousands of writers on one `TestStat` row; recounting students would replay their whole history each pass.
- **The board holds a ranking per rollup, not per reader.** A cold read per reader at a 6K cohort is millions of rows for a dozen answered; one read per watermark, sliced in memory, makes it one. The reader's own rank is never from the held copy, so the board and the score card cannot disagree. A neighbour's name can be one sweep stale.
- **The page is cut from the same held ranking.** Page turns cost no query when the held ranking agrees; they cost one window pass over the cohort when it does not.

Is it optimal

Score-card standing is one query, one LATERAL pass, index-only. `/me/leaderboard` is two queries plus a ranking read amortised over the sweep window on a hit; on a miss or disagreement it is two queries plus a window pass over the whole cohort, per request. `/me/overview` is three queries and five laterals. Rollup sweep is three selections, then per test either two statements or a 200-id-paged replay of every sheet. Figures quoted in migration headers and commit bodies (`20260918100000`, `20260929090000`, `b610946`) are read, not measured here.

What is wrong with it

- `packages/contracts/src/leaderboard.ts:13` — two consecutive doc comments sit on `LEADERBOARD_PODIUM`; the first ("ONE paper…") belongs to the file, not to the constant. Stale.
- `packages/contracts/src/stats.ts:39` — two consecutive doc comments before `RECENT_STANDINGS`; the first describes the overview schema below it. Stale.
- `apps/api/src/attempts/overview.service.ts:192` — "Every graded sitting's current percentile" is only the newest five (`RECENT_STANDINGS`). Stale.
- `apps/api/src/queue/queues.ts:92` — the `ROLLUP_JOBS` comment says "one sitting to fold in"; no such job exists. Stale.
- `apps/api/src/queue/queues.ts:111` — `RollupJobData.attemptId` is read by nothing (only `testId` and `studentId` are). Dead.
- `apps/api/src/queue/queues.ts:101`, `apps/api/src/attempts/rollup.processor.ts:29` — `DRAINED_ROLLUP_JOBS` remaps `fold-attempt` and `fold-pending`, jobs nothing enqueues now. Dead.
- `docs/01-architecture.md:225` — the live-rank paragraph still describes points boards, a series board of 128 ms and a 60-second per-scope cache; points boards are deleted and the board is held per rollup watermark. Stale.
- `apps/api/src/attempts/rollup.service.ts:162` — `staleItems` takes up to 50 tests, each a BULK replay, on a concurrency-2 queue; a pass can outlast the 2-minute interval and the next `sweep-cohorts` add is dropped by its shared id. Self-correcting, but item figures can lag the 15-minute clock the copy promises. Correctness (read, not measured).
- `apps/api/src/attempts/leaderboard-view.service.ts:61` — the live fallback runs a full-cohort window pass per request whenever the held ranking disagrees (any cohort change between sweeps, or no `TestStat` row yet), so a busy test just after a mass submit pays that per reader. Correctness (read, not measured).

Part II — Making a paper

Building a test, and freezing it

A paper does not appear; it is assembled, reviewed and then frozen. This chapter follows a test from the blueprint it is cut from, through the draw or the typist that fills it, to the one save that offers it to students, and then the walls that stop it moving.

The hinge is the freeze, and there are two of them. The **offer** (`Test.finalizedAt`) is a service rule: from it, every paper edit is refused by code. The **first sitting** is a database rule: from it, triggers refuse any change to the paper rows and to the question versions they pin, whatever code or script tries. The first is a state an admin walks into on purpose; the second is the one no new code path can forge.

Where it lives

`apps/api/src/tests/` is the module (11 files, about 2,850 lines), plus `apps/api/src/common/paper-edit.ts` and the blueprint half in `apps/api/src/configs/`.

File	Owns
`apps/api/src/tests/tests.controller.ts`	`TestsController` (`admin/tests`), `SeriesTestsController`, `TypistDoneController`
`apps/api/src/tests/tests.service.ts`	`Test` itself: list, detail, create, update, delete
`apps/api/src/tests/paper.service.ts`	the paper: read, add, fill, replace, remove, typist Done, hand-over, drop or bonus
`apps/api/src/tests/draw-engine.ts`	the one pure draw (`drawSection`)
`apps/api/src/tests/finalize.service.ts`	the freeze and every gate in front of it
`apps/api/src/tests/offering.service.ts`	the Offer step's save, series moves, program openings
`apps/api/src/tests/test-rules.ts`	pure rules: scope, title, series fit, completeness, what moves a paper
`apps/api/src/tests/paper-languages.ts`	the SQL that finds questions missing a language their section is sat in
`apps/api/src/tests/edit-lock.ts`	the test's key bound to the shared advisory lock
`apps/api/src/common/paper-edit.ts`	`beginPaperEdit` / `beginDraftPaperEdit`: the `SELECT … FOR UPDATE` on `Test`
`apps/api/src/configs/base-configs.service.ts`	the blueprint: create, update, clone, delete, `assertUsable`
`apps/api/src/configs/base-config-rules.ts`	blueprint rules: shape per timer template, what a locked config still allows

What it is for

A `BaseConfig` describes a paper's shape; a `Test` is one instance of it with its own scope, skin, series and draw settings. There is no `Paper` model and no per-test section row. A test has exactly one paper, its `PaperQuestion` rows (`prisma/schema.prisma:601`), and its sections are the `BaseConfigSection` rows its scope covers (`packages/contracts/src/tests.ts:133`, `scopedSections`). Every sitting is served that one paper, which is what makes a rank mean anything and keeps any pool query off the start path.

The blueprint a test is cut from

A config is a stage's reusable shape: timer template, navigation, languages, skin (`examTemplate`), calculator, shuffles, and its modules and sections (`prisma/schema.prisma:343`, `:388`, `:402`). A stage holds many; exactly one is the default.

- **A test is born inside a config.** `TestsService.create` calls `BaseConfigsService.assertUsable` (`apps/api/src/configs/base-configs.service.ts:361`), which refuses only a retired config. A *locked* config still takes new tests.
- **A config locks on the first sitting, not at the offer.** `lockTheBlueprint` (`apps/api/src/attempts/attempts.service.ts:182`) sets `locked` inside the transaction that creates the first `Attempt` (call at `:175`). Before then, a config with tests built on it still keeps its sections and modules: a save that rewrites them mints new section ids, and a test's paper rows, assignments, scope and draw spec all name sections by id, so `update` refuses with `BUILT_ON_CONFIG_MESSAGE` (`base-configs.service.ts:197`). A change to only a section's languages is the exception; it is applied in place (`:193-195`, `:240`).
- **Once locked, three fields move.** `UNFROZEN_FIELDS` is `name`, `isDefault`, `isActive` (`apps/api/src/configs/base-config-rules.ts:20`); `locksOutEdit` (`:35`) refuses anything else with a sentence, and the database's `base_config_guard` trigger (current body in `prisma/migrations/20261001100000_a_locked_config_freezes_its_skin/migration.sql`) refuses the same columns to a script. The route to change a locked shape is `clone` (`base-configs.service.ts:260`): the copy starts unlocked, not default, version + 1, and points back through `clonedFromId`.
- **Deleting** a config is blocked while it is locked, has tests, or has clones (`configDeletionBlocker`, `base-config-rules.ts:224`); retire it instead.
- **The skin is copied at creation.** `Test.examTemplate` starts as the config's and then belongs to the test, so a config re-skin cannot repaint a paper already sat (`tests.service.ts:224`).

Creating the test

POST /ADMIN/TESTS — A NEW DRAFT

1. `@Audit(TEST, CREATE)` and `@RequiresFeature(TEST_MANAGEMENT, WRITE)`; body parsed by `createTestSchema` (title required, `baseConfigId`, `testSeriesId`)
   `apps/api/src/tests/tests.controller.ts:79`
   → validated body

2. Config read and proved not retired
   `apps/api/src/tests/tests.service.ts:207`
   → `BaseConfigDetail` with sections
3. Series read; it must exist and be for the config's stage (or carry no stage)
   `apps/api/src/tests/tests.service.ts:236`
   → `VALIDATION_ERROR` on `testSeriesId` otherwise
4. Title free inside that series, case-insensitively
   `apps/api/src/tests/tests.service.ts:409`
   → `CONFLICT` on `title`; the unique index on `(testSeriesId, lower(title))` is the real guarantee
5. Scope names a real module or section of this config; a `FULL` scope names none
   `apps/api/src/tests/tests.service.ts:395`
   → `VALIDATION_ERROR` on `scopeRef`
6. Any difficulty mix must add up to its section's count
   `apps/api/src/tests/tests.service.ts:383`
   → `VALIDATION_ERROR` on `questionPoolFilter`
7. One insert, no transaction. Stage comes off the config, never the body
   `apps/api/src/tests/tests.service.ts:217`
   → `Test` row: `status = DRAFT`, `version = 0`, `paperSource = null`, `finalizedAt = null`
8. Read back whole
   `apps/api/src/tests/tests.service.ts:189`
   → `TestDetail`: the row, its schedule, the config, who holds the edit claim, and the paper's language gaps

No `PaperQuestion` row exists yet and `paperSource` is null, which blocks every paper action (`assertSourceChosen`, `apps/api/src/common/paper-edit.ts:34`). The admin chooses the source once, from the Paper step (`apps/admin/src/features/tests/test-builder-assign-step.tsx:386`), by `PATCH /admin/tests/:id { paperSource }`. `assertPaperSourceOpen` (`tests.service.ts:374`) then refuses any later change, super admin included, because every assignment on the test rests on the choice.

Scope is `FULL`, `MODULE` or `SECTIONAL` (`packages/contracts/src/tests.ts:32`). Question count, duration and marks are all derived over the scoped sections (`testShapeOf`, `apps/api/src/tests/test-rules.ts:36`); a scoped test is never the config's whole total. Narrowing the scope after rows exist deletes the rows it no longer covers, stands down the assignments on those sections (`replacedAt`), and records how many it dropped in the audit diff (`tests.service.ts:343`).

> **Note** `TestsService.update` is the one writer of title, scope, skin, source and draw spec. It stacks four gates in this order: the screen's `expectedUpdatedAt` (`:255`), the Redis edit claim (`:258`), the sat gate (`:265`), the offered gate (`:269`). The row is finally written with `updateMany … where updatedAt = <what was judged>` (`:294`); a save that lost the race is refused, not merged.

Where the questions come from

`PaperSource` is `FRAMED` (typed for this test by a typist) or `PICKED` (drawn from the bank), declared once (`prisma/schema.prisma:139`). Both sources take a typist and a proof-reader per section.

	PICKED	FRAMED
Who puts rows on the paper	the test owner: hand-pick (`addQuestions`) and/or the draw (`fillSection`)	the section's typist, at Done (`typistDone`)
Candidate pool	the bank, narrowed by the section's subject and the spec's topics	only questions written under an assignment on this test and section
Who else may edit the paper	anyone with TEST_MANAGEMENT write, until the section is with its reader	a super admin only; `assertNotTyped` (`paper.service.ts:783`) refuses the rest
How a section reaches its reader	the owner's hand-over, once the section is full	the typist's Done, which stamps the reader's `handedAt`
What the typist owes	nothing; they only fix what comes back	Done, exactly the section's count

A whole typed test can also be imported from one sheet on the Paper step, each row filed under its section's typist as a draft (`225ab39`; the import itself belongs to the question-import chapter). Done, hand-over and reading are unchanged by it.

The draw

There is one draw and it is pure: `drawSection` in `apps/api/src/tests/draw-engine.ts:68`. It is handed a section, a candidate pool, an optional spec, a seed and a set of **pins** (rows already on the paper), and returns the section's rows or a shortfall. It reads no database.

1. The pool is sorted by id, shuffled with the seed (`seededRandom`), and filtered: pins are excluded and each candidate must match the section's subject, the spec's topics, and any one of the spec's tags (`matches`, `:121`).
2. Without a `mix`, the first `questionCount − pins` survivors are taken. With a `mix`, each difficulty is filled to its own count, and a pin counts against the bucket it belongs to (`fill`, `:102`).
3. Pins come first in the result. If fewer than `questionCount` rows come out, the result is `{ ok: false, shortfall }`; a short section never reads as whole.
4. Marks and negative marks on every row come from the **section**; the version comes from the **question** (`:94–96`).

The service wraps that purity with the I/O (`PaperService.drawRemainder`, `paper.service.ts:285`):

- **Pool** (`poolFor`, `:735`): not archived, a current version, of the section's answer kind (choice, typed or written), and not locked behind another test's open assignment (`drawableFor`, `apps/api/src/questions/question-core.ts:222`). `PICKED` narrows by `subjectId` and `topicIds` in SQL;

tags and the split are the engine's. `FRAMED` narrows to the section's own typist's work instead.

- **Pins** (`pinsOf` , `:331`): the section's existing rows, resolved at the version the *row* pins, not the question's current one, so a question the bank has moved on from still counts as one.
- **Split guard** (`assertWithinSplit` , `:914`): a section already holding more of one difficulty than its mix allows is refused before the draw, naming "take one off first".
- **Seed** (`freshSeed` , `:928`): `randomInt(2**31)` on every call. A re-fill gives a different paper; nothing lets an admin supply a seed.
- **Shortfall**: `DRAW_SHORTFALL` (HTTP 422) with a field error keyed by section id: "needs N, and the bank holds M" for a picked paper, or "its typist has written M" for a framed one (`:313–324`).

The draw only ever **adds**: it never withdraws a hand-picked row. Rows it returns are renumbered from the paper's current highest `order` + 1, because the engine numbers from 1 and knows nothing of the rows already there (`:263–271`).

POST /ADMIN/TESTS/:ID/PAPER/SECTIONS/:SECTIONID/FILL — DRAW WHAT A SECTION LACKS

1. `@Audit(TEST, UPDATE)` , `@RequiresFeature(TEST_MANAGEMENT, WRITE)` ; `user` becomes the `Editor`
   `apps/api/src/tests/tests.controller.ts:156`
   → editor `{ id, isSuperAdmin }`
2. Test read with its attempt count
   `apps/api/src/tests/paper.service.ts:861`
   → `{ finalizedAt, paperSource, scope, questionPoolFilter, _count.attempts }`
3. Advisory claim in Redis: `test:edit:<testId>` = admin id, 900 s, no heartbeat; refused if another admin holds it (a super admin takes it over)
   `apps/api/src/tests/edit-lock.ts:10`
   → Redis key set or refreshed
4. Refused if offered or sat
   `apps/api/src/tests/paper.service.ts:789`
   → `CONFLICT` with `OFFERED_TEST_MESSAGE` or `SAT_TEST_MESSAGE`
5. Refused while `paperSource` is null
   `apps/api/src/common/paper-edit.ts:34`
   → `CONFLICT`
6. Section must be in the scope; a FRAMED paper refuses a non-super-admin; a section with its reader (handed, not released) refuses a non-super-admin
   `apps/api/src/tests/paper.service.ts:140`
   → `NOT_FOUND` / `CONFLICT`
7. Transaction `TX_LIMITS.SHORT` (maxWait 5 s, timeout 10 s) opens and takes `SELECT … FOR UPDATE` on the `Test` row; re-reads `finalizedAt` under it
   `apps/api/src/common/paper-edit.ts:22`
   → an offer that landed in between is refused
8. Every paper row read once, under the lock
   `apps/api/src/tests/paper.service.ts:259`
   → `HeldRow[]`
9. Pool, pins, split guard, `drawSection` ; a shortfall throws
   `apps/api/src/tests/paper.service.ts:311`
   → `DrawnQuestion[]` or `DRAW_SHORTFALL`
10. `paperQuestion.createMany` , numbered from highest `order` + 1
    `apps/api/src/tests/paper.service.ts:264`
    → new `PaperQuestion` rows; `optionIds` filled by trigger
11. `Test.version + 1` , so any Offer step opened earlier is refused
    `apps/api/src/tests/paper.service.ts:100`
    → `Test.version`
12. A now-full section that holds an unchecked question goes back to its released reader
    `apps/api/src/assignments/unread-questions.ts:10`
    → reader's `finalizedAt` cleared
13. Audit row records section name and count added; paper re-read and returned
    `apps/api/src/tests/paper.service.ts:646`
    → `TestPaper`

`POST :id/paper/questions` (`addQuestions` , `paper.service.ts:155`) is the hand-pick variant: same gates, plus `assertNoRepeats` , an already-on-paper check, a section-full check and `quotaWithPicks` against the split (`picksFor` , `:197`). It never draws. `PATCH :id/paper/:rowId` (`replaceQuestion` , `:347`) swaps one row's question in place, keeping its position, judged as one more pick against the same split. `DELETE :id/paper/questions` (`removeQuestions` , `:571`) resolves every row before deleting any. All three bump `Test.version` in the same transaction.

> **Watch** `@@unique([testId, questionId])` means a question sits on a paper once. The draw excludes everything already on the paper from its pool (`:295–298`), so a section cannot be refilled with a question another section holds.

A typed section: the typist's Done

POST /ADMIN/SECTIONS/:TESTID/:SECTIONID/DONE — A TYPIST HANDS IN

1. `@RequiresFeature(QUESTION_AUTHORING, WRITE)` : the key is the authoring one, not TEST_MANAGEMENT
   `apps/api/src/tests/tests.controller.ts:238`
   → `{ selected, discard }`
2. The caller proved to hold a live, unfinished TYPIST seat on this pair
   `apps/api/src/tests/tests.controller.ts:249 (` SectionWorkService.actingTypist `)`
   → the typist's assignment id

3. Assignment must be a live typist's and not already done
   `apps/api/src/tests/paper.service.ts:406`
   → `NOT_FOUND` / `CONFLICT` (`ALREADY_DONE_MESSAGE`)
4. Test not offered or sat; Done must be open for this source (a picked paper has no Done)
   `apps/api/src/tests/paper.service.ts:414`
   → `CONFLICT`
5. Selected questions resolved and checked drawable
   `apps/api/src/tests/paper.service.ts:425`
   → `DrawableContext`
6. Transaction `TX_LIMITS.SHORT` , `Test` row locked, offered re-checked
   `apps/api/src/common/paper-edit.ts:22`
   → lock held
7. Typist's `finalizedAt` stamped, conditional on still being null
   `apps/api/src/tests/paper.service.ts:430`
   → `QuestionAssignment.finalizedAt`
8. Whatever any typist of the section wrote is read under the lock; the selection is validated whole: exactly the count, all the typist's own, no repeats, difficulty split met exactly
   `apps/api/src/tests/paper.service.ts:489`
   → `VALIDATION_ERROR` on `selected` before anything is written
9. The section's rows become exactly the choice: unticked rows deleted, new ones appended after the paper's highest `order`
   `apps/api/src/tests/paper.service.ts:455`
   → `PaperQuestion` rows
10. `Test.version + 1` ; the section's reader's `handedAt` stamped in the same transaction
    `apps/api/src/tests/paper.service.ts:439`
    → `QuestionAssignment.handedAt`
11. Leftovers settled: discarded questions deleted, the rest go to the bank as ordinary questions
    `apps/api/src/tests/paper.service.ts:446 (` QuestionsService.settleTyped `)`
    → no question left owned by a finished job

A PICKED section reaches its reader by `POST :id/paper/sections/:sectionId/hand-over` (`handOver` , `paper.service.ts:537`), gated by `handOverGap` (`:886`): not offered, source is PICKED, the section holds its full count, a live reader exists, and it has not already been handed. It stamps the reader's `handedAt` , releases the section's Redis edit claim, and does **not** take the Test row lock.

The offer: the freeze

One road only. There is no separate "finalize" endpoint: the Offer step's single save is the way a test becomes `ACTIVE` .

PUT /ADMIN/TESTS/:ID/OFFERING — OFFER, RETIRE, OR RESCHEDULE

1. `@Audit(TEST, UPDATE)` , `@RequiresFeature(TEST_MANAGEMENT, WRITE)` ; body `saveOfferingSchema` : `opensAt` , `programOpenings[]` , `offered` , `waiveLanguageGaps?` , `expectedVersion`
   `apps/api/src/tests/tests.controller.ts:182`
   → offerer `{ adminId, isSuperAdmin }`
2. Transaction `TX_LIMITS.MEDIUM` (maxWait 10 s, timeout 15 s); the `Test` row taken `FOR UPDATE` first
   `apps/api/src/tests/offering.service.ts:272`
   → lock held for the whole save
3. Optimistic lock: `version !== expectedVersion`
   `apps/api/src/tests/offering.service.ts:274`
   → `CONFLICT` "changed after you opened its Offer step"
4. Opening written if the minute changed: refused on a sat test, refused if not more than five minutes ahead, judged against sibling openings in a sequential series
   `apps/api/src/tests/offering.service.ts:320`
   → `Test.opensAt`
5. Program openings: duplicates refused; a set row must be earlier than the test's opening; the kept set replaces the stored one
   `apps/api/src/tests/offering.service.ts:341`
   → `TestProgramUnlock` rows
6. `offered` and status not yet `ACTIVE` : hand to `offerWithin`
   `apps/api/src/tests/offering.service.ts:289`
   → the gates below
7. Row re-read inside the lock; if already finalized, only status is lifted to `ACTIVE` and the call returns; none of the gates below run again
   `apps/api/src/tests/finalize.service.ts:59`
   → `Test.status`
8. Gate (not a super admin): every scoped section has a current proof-reader
   `apps/api/src/tests/finalize.service.ts:147`
   → `VALIDATION_ERROR` naming the sections
9. Gate (not a super admin): no live proof-reader seat, and no live FRAMED typist seat, is unfinished
   `apps/api/src/tests/finalize.service.ts:164`
   → `VALIDATION_ERROR` "still being proof-read"
10. Gate (not a super admin): every question on a read section carries its reader's tick
    `apps/api/src/tests/finalize.service.ts:188`
    → `VALIDATION_ERROR` per section
11. Gate (everyone): the paper is whole, each scoped section at its exact count, nothing in a section the config no longer has
    `apps/api/src/tests/finalize.service.ts:110`
    → `VALIDATION_ERROR` listing every shortfall

12. Gate (everyone): every non-dropped question carries every language its section is sat in, unless the offering admin ticked the waiver
`apps/api/src/tests/finalize.service.ts:127`
→ `VALIDATION_ERROR`, or the list of languages waived

13. The freeze: one `UPDATE` setting `finalizedAt = now`, `status = ACTIVE`, `version + 1`, and, only if languages were waived, `languageWaivedById`, `languageWaivedAt`, `languagesSkipped`
`apps/api/src/tests/finalize.service.ts:75`
→ the durable result

14. `offered` false and status `ACTIVE`: `status = INACTIVE` only
`apps/api/src/tests/offering.service.ts:295`
→ `Test.status`

15. `version + 1` once more, closing the transaction
`apps/api/src/tests/offering.service.ts:298`
→ `Test.version`

16. Re-read; audit diff of status, opening and program openings; `ACCESS_CATALOG_CHANGED` for the series
`apps/api/src/tests/offering.service.ts:302`
→ shared catalog rebuilt; `TestOffering` returned

Points that change the outcome:

- **A super admin skips the reading, never the paper.** Hops 8-10 are skipped for a super admin (`finalize.service.ts:67`); hops 11-12 are not. The reasoning in the commit body is that an absent reader must not stop a paper going out on test day.
- **The language check is an offer-time gate, with a named escape.** A question "carries" a language if its pinned version's `content` has that key; a section with its own languages owes no others (`paper-languages.ts:23`). Any admin may offer past a gap by ticking `waiveLanguageGaps`; the waiver is recorded against the signed-in admin and shown on the test for good. A `CHECK` keeps the two waiver columns both set or both null (`prisma/migrations/20261014130000_an_offer_past_a_missing_language_names_who_waived_it/migration.sql`). The languages waived are copied to `Test.languagesSkipped`, and a candidate is not offered them (`offeredLanguages`, `packages/contracts/src/exam-order.ts:97`, used at `apps/api/src/attempts/attempts.service.ts:167`).
- **A re-offer proves nothing again.** A frozen paper was judged at its first offer; offering a retired test only flips its status back. It never re-freezes or redraws (`437da2a` made this explicit after a super-admin-offered test could be revived only by a super admin; `f7a66b0` declined to re-check completeness here because a finalized paper that is not whole is a state nothing can reach).
- **The Test row lock makes the offer and a paper edit take turns.** Every paper write calls `beginPaperEdit` or `beginDraftPaperEdit` first. The same lock order (Test before Question) is what the question-version trigger relies on, so two admins on one unfrozen test cannot deadlock on foreign-key parents (comment at `apps/api/src/common/paper-edit.ts:1-6`).

Test status

`TestStatus` is `DRAFT`, `ACTIVE`, `INACTIVE` (`prisma/schema.prisma:126`). There is no switch other than the Offer step's save.

Transition	Written by
(new) to `DRAFT`	column default at create
`DRAFT` to `ACTIVE`	the freeze, in the same `UPDATE` as `finalizedAt` (`finalize.service.ts:75`)
`ACTIVE` to `INACTIVE`	the Offer save with `offered` false (`offering.service.ts:295`)
`INACTIVE` to `ACTIVE`	the Offer save with `offered` true, on an already-frozen paper (`finalize.service.ts:60`)

Nothing returns a test to `DRAFT`. `ACTIVE` is the only status the shared catalog carries (`apps/api/src/access/access-resolver.service.ts:46`), the only one the start gate allows (`testStartBlocker`, `apps/api/src/attempts/attempt-rules.ts:13`), and the only one the "a test has opened" sweep announces (`apps/api/src/notifications/test-opening.service.ts:42`). Because status and `finalizedAt` ride one write, `ACTIVE` always implies a frozen paper; the start gate checks `finalizedAt` as well anyway (`attempt-rules.ts:19`).

Openness is derived on every read, never stored: a test opens at `opensAt` and never shuts, and a sitting may begin five minutes early (`testIsOpen`, `packages/contracts/src/access.ts:399`). A student's effective opening is the earliest of their own programs' unlocks and the test's. Students are told once: a five-minute sweep (`TESTS_OPENED_SWEEP_EVERY_MS`, `apps/api/src/queue/queues.ts:158`) finds `ACTIVE` tests with `announcedAt` null and an opening in the past, stamps `announcedAt` first, then pushes. The stamp is the bell entry, and a crash after the stamp costs the push, never a second one.

What the freeze closes

After...	Refused	Where
the offer	adding, replacing, removing, filling, hand-over and typist Done	`assertAssemblable` (`paper.service.ts:789`), `beginDraftPaperEdit` re-check under the lock
the offer	any test edit that could change what the paper holds: scope, scope ref, draw spec	`movesThePaper` (`test-rules.ts:128`) at `tests.service.ts:269`
the offer	a seat's work: typing, reading and release on a test that is offered or a job that is over	`assertJobOpen` (`apps/api/src/assignments/assignment-gates.ts:46`)
the first sitting	every Setup edit but `title` (an offered, unsat test may still change `title`, `examTemplate`: `PAPER_NEUTRAL_FIELDS`, `test-rules.ts:99`)	`TEST_UNFROZEN_FIELDS` (`test-rules.ts:96`); `locksOutTestEdit` (`:122`)
the first sitting	a series move, a change of opening time, and deleting the test	`assertUnsat` (`offering.service.ts:500`), `testDeletionBlocker` (`test-rules.ts:169`)

After…	Refused	Where
the first sitting	any change to a `PaperQuestion` column except `status`, and any change to the options, key or content of a pinned version	the triggers below

The one write that survives both is a question's **disposition**. `PATCH :id/paper/:rowId/status` (`setQuestionStatus`, `paper.service.ts:606`) refuses while `finalizedAt` is null (`:612`, "Only an offered test can have a question dropped or made a bonus"), sets the status `ACTIVE`, `DROPPED` or `BONUS` on the row by question id, and increments `Test.paperRevision` in the same transaction (`:627`). The write itself is the gate: `updateMany … where status <> new` means two admins clicking cannot both win. The sweeper re-scores every sitting whose `scoredRevision` is behind the test's `paperRevision`.

The triggers

All four were written in `prisma/migrations/20260917100000_a_sitting_keeps_its_answers_on_one_sheet/migration.sql`, recreated verbatim in `20260918120000_every_id_is_a_uuid` (`:415–417`), and are unchanged since except for the version guard's lock (below). They enforce the stronger rule, *a paper somebody has sat*, not the offer. An offered, unsat paper is protected by services alone; there is no trigger on `finalizedAt`.

- `paper_question_sat_guard`, `BEFORE INSERT OR UPDATE OR DELETE ON "PaperQuestion"` (`:123–143`). If the update changes nothing but `status`, it returns at once: no lock, no lookup, so a drop never blocks a student starting. Otherwise it does `PERFORM 1 FROM "Test" WHERE id IN (NEW.testId, OLD.testId) FOR UPDATE`, then raises `integrity_constraint_violation` if an `Attempt` exists for the test. The `FOR UPDATE` is the point. An `Attempt` insert takes only `KEY SHARE` on its `Test` through the foreign key, which conflicts with nothing an ordinary `UPDATE` holds. Without the explicit lock, a paper edit and the first sitting could each pass their own check and both commit. With it they take turns (`prisma/migrations/20260918150000_the_version_guard_takes_the_lock_its_sibling_takes/migration.sql`).
- `question_version_sat_guard`, `BEFORE UPDATE ON "QuestionVersion"` when `options`, `answerKey` or `content` differ. It locks every test pinning the version, in `ORDER BY "id"` so two backends cannot take them in opposite orders and deadlock with `40P01`, then raises if any of them has a sitting. Current body: `prisma/migrations/20260918160000_the_version_guard_locks_its_tests_in_one_order/migration.sql:20–36`.
- `paper_question_option_ids`, `BEFORE INSERT OR UPDATE OF "questionVersionId", "optionIds"` (`:58–60`). Copies the pinned version's option ids, in stored order, onto the row. An `AttemptSheet` stores a *position* into this array, so it is load-bearing for scoring.
- `question_version_option_ids_fanout`, `AFTER UPDATE OF "options" ON "QuestionVersion"` (`prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql:25–37`). Re-assigns `optionIds = optionIds` on every pinning row so the previous trigger rebuilds it.

> **Watch** The fan-out leans on Postgres firing same-event `BEFORE` triggers alphabetically: `paper_question_option_ids` (o) runs before `paper_question_sat_guard` (s), so the self-assignment is a real change by the time the sat guard compares. Renaming either trigger reverses the order and silently drops the sat guard's own check on that path (explained in the header of the fan-out migration).

Beside them: `Test_testSeriesId_title_key`, a unique index on `("testSeriesId", lower("title"))` (`prisma/migrations/20260912170000_a_name_is_taken_once/migration.sql:67`), is the real guarantee behind `assertTitleFree`; the service check exists so the refusal lands on the field. The only `CHECK` left on `Test` is `Test_language_waiver_whole`; the ranked and practice-window checks were dropped (`20260911150000`, `20260909180000`).

Data touched

Model.column	Holds	Written by
`Test.title`	text, unique per series (case-insensitive)	create, update
`Test.baseConfigId`, `examStageId`	the blueprint; the stage is denormalised so the composite FK holds	create only
`Test.scope`, `scopeRef`	`FULL` / `MODULE` / `SECTIONAL`, and `{ moduleId?, sectionId? }`	create, update
`Test.questionPoolFilter`	per-section `{ topicIds?, tags?, mix? }` (`packages/contracts/src/tests.ts:88–119`)	create, update
`Test.paperSource`	`FRAMED` / `PICKED`, null until declared	update, once
`Test.examTemplate`	the skin, copied from the config at create	create, update
`Test.status`	`DRAFT` / `ACTIVE` / `INACTIVE`	the freeze, the Offer save
`Test.version`	the optimistic lock behind the Offer step	every paper write, the freeze, each Offer save, a series move
`Test.paperRevision`	monotonic key for re-scoring and cached paper terms	a question's disposition only (`paper.service.ts:627`)
`Test.finalizedAt`	the freeze; set once, never cleared	`finalize.service.ts:75`
`Test.opensAt`	the whole of the test's timing	`offering.service.ts:337`
`Test.announcedAt`	the opening sweep's watermark	`test-opening.service.ts`
`Test.languageWaivedById`, `languageWaivedAt`, `languagesSkipped`	who offered past a language gap, when, and which languages	the freeze, only when something was waived
`Test.seriesOrder`	position in the series; cleared by a move	`offering.service.ts:248`
`PaperQuestion.{testId, baseConfigId, baseConfigSectionId}`	the row's place; composite FKs	paper writes

Model.column	Holds	Written by
`PaperQuestion.{questionId, questionVersionId}`	the pinned version	paper writes
`PaperQuestion.order`	unique per test; the sheet's slot order	paper writes
`PaperQuestion.{marks, negativeMarks}`	copied from the section	paper writes
`PaperQuestion.status`	`ACTIVE` / `DROPPED` / `BONUS`, the only post-sitting change	`paper.service.ts:621`
`PaperQuestion.optionIds`	the version's option ids in stored order	trigger only
`TestProgramUnlock.{testId, programCode, opensAt}`	which programs open it earlier	`offering.service.ts:373–382`
`QuestionAssignment.{finalizedAt, handedAt, replacedAt}`	the state of a seat's work	`paper.service.ts:430`, `:441`, `:560`; `tests.service.ts:352`

Redis: `test:edit:<testId>` (`apps/api/src/redis/redis.keys.ts:61`) is the advisory edit claim; value is the admin id, TTL `EDIT_LOCK_TTL_SEC` = 900 s (`:72`). It is taken before the Postgres transaction deliberately, because Redis is not transactional with Postgres and a claim taken inside a rolled-back transaction would be a claim nobody released. There is no heartbeat and no release on save.

> **Note** The question's "in use" state is not a counter. `docs/03-conventions.md` now says so correctly: what stops a used question changing is `question_version_sat_guard`, and in-use is derived per read from whether any `PaperQuestion` or `TestQuestionStat` points at it. The offer writes nothing on `Question`.

Prerequisites

- An active `BaseConfig` with sections (`assertUsable`); a `TestSeries` whose `examStageId` is null or equal to the config's.
- An admin with TEST_MANAGEMENT write. The typist's Done needs QUESTION_AUTHORING write.
- `Test.paperSource` set before any paper action.
- For FRAMED, a live TYPIST `QuestionAssignment` per section with questions written under it.
- To offer without being a super admin: a live proof-reader on every scoped section, every reader and FRAMED typist seat finished, and every paper question on a read section ticked. A super admin needs the paper whole and nothing else (plus the language waiver, if a gap exists).
- Migrations `20260917100000` and later applied (the triggers and `optionIds`).

Why this shape

- **No `Paper` table and no per-test section rows.** Sections are read back off the config, so a scope change needs no copy. The cost is a config whose sections are protected by id: the sections of a config with tests cannot be rewritten, and a locked config cannot change at all.
- **`finalizedAt` is the freeze, not the existence of rows.** The paper is built incrementally; nothing about row count says it is ready.
- **Marks and negatives are copied onto each row** so a paper's scoring never moves with the config. It costs duplication and a stale copy if the config were edited, which the lock forbids.
- **Immutability of a sat paper is in Postgres, not the service.** A script or a bug bypasses an application guard; it cannot bypass a trigger. The service rules above that exist to give the admin a sentence instead of a constraint violation.

Front end (apps/admin only)

The student SPA never calls these routes. Route strings are `ADMIN_TEST_ROUTES` in `packages/contracts/src/tests.ts:510`; the client wrappers are at `packages/contracts/src/client/admin.ts:567–662`.

The builder (`apps/admin/src/features/tests/test-builder.tsx`) has three steps (`TEST_BUILDER_STEP`, `packages/contracts/src/tests.ts:405`): Setup, Paper, Offer. Reopening a test lands on the first step still owing work (`testBuilderStepOf`, `:422`; `owesAPaper`, `:417`). The Paper step's Next stays shut until the paper could be offered as the viewer would be allowed to: `usePaperBlocker` (`test-builder.tsx:364`) reads the test's assignments and calls `paperBlockerOf` (`packages/contracts/src/assignments.ts:368`), which mirrors the server's reading gates and takes `skipsReading` from the viewer. It does not check languages; the Paper step shows the gaps as a warning, and the waiver tick lives on the Offer step.

Action	Wrapper	Screen
Create / Setup	`api.admin.tests.create` / `.update`	`test-builder.tsx:187`
Choose the source	`api.admin.tests.update(id, { paperSource })`	`test-builder-assign-step.tsx:386`
Hand-pick	`addPaperQuestions`	`test-paper.tsx:509`
Draw	`fillPaperSection`	`test-paper.tsx:521`
Replace a row	`replacePaperQuestion`	`paper-questions.tsx:259`
Remove rows	`removePaperQuestions`	`paper-questions.tsx:163`
Hand over	`handOverSection`	`test-paper.tsx:673`, `test-builder-assign-step.tsx:475`
Offer / retire / schedule	`saveOffering`	`test-builder-offering.tsx:392`; also `apps/admin/src/features/test-series/test-series-tests.tsx:278`
Drop or bonus	`setPaperQuestionStatus`	`paper-disposition.tsx:115`

Action	Wrapper	Screen
Move series	`moveToSeries`	`test-series-tests.tsx:323`

Cost

Read from the code, not measured with `EXPLAIN`. `GET /admin/tests` is two queries in one `$transaction` plus two `groupBy` scoped to the page's ids (`tests.service.ts:151`). `PUT :id/offering` is a long transaction with an N+1 over changed program openings (`requireProgram` per changed row, `offering.service.ts:365`). Fill's pool read is an unbounded `findMany` over the drawable bank, narrowed by subject and topic (`paper.service.ts:742`); at the sizes seen so far this is a one-admin action, not a hot path. The offer path, like all of this chapter, is off the live-test hot path entirely.

What is wrong with it

- `apps/api/src/tests/tests.service.ts:381` — Stale: a doc comment about `assertCovers` ("The scope has to name a part of THIS config...") sits stacked above `assertMixesAddUp`'s own, two lines before the method it belongs to.
- `apps/api/src/tests/paper.service.ts:927` — Stale: "A re-draw the admin did not seed" implies a seed an admin can give; `freshSeed` is the only caller of the seed and no request field carries one.
- `apps/api/src/tests/draw-engine.ts:61` — Stale: "Same seed, same pool, same rows — which is what makes a re-draw a decision" describes a property the service never uses; every fill draws a fresh seed.
- `docs/02-domain-rules.md:84` — Stale: "nothing else gates the draw" is false; `drawableFor` (`apps/api/src/questions/question-core.ts:222`) also restricts the pool to the section's answer kind and hides work typed under another test's open assignment.
- `apps/api/src/tests/paper.service.ts:606` — Correctness (minor): a drop or bonus changes what every sitting is marked against but does not take the `Test.version` bump or the Redis edit claim; an Offer step opened before it saves without a conflict. Harmless today because nothing on the Offer step depends on dispositions.

The question bank

Where it lives

`apps/api/src/questions/` holds the bank, the importer, the typist's authoring surface and the taxonomy. This chapter covers the bank itself; the importer and section work have their own chapters and only appear where they share a rule.

File	Owns
`apps/api/src/questions/questions.service.ts`	`Question` and `QuestionVersion`, the only module that writes them
`apps/api/src/questions/questions.controller.ts`	`admin/questions`: list, export, images, availability, detail, versions, create, update, archive, unarchive, delete
`apps/api/src/questions/question-core.ts`	`buildContent`, stem hash, `drawableFor`, the server's `validateQuestion`
`apps/api/src/questions/question-versions.ts`	the edit-in-place versus new-version rule
`apps/api/src/questions/question-query.ts`	`reachableTest`, `questionWhere`, `questionOrderBy`
`apps/api/src/questions/question-rows.ts`	the row include, `toSummary`, `toDetail`
`apps/api/src/questions/question-audit.ts`	the content fingerprint and the audit diff
`apps/api/src/questions/question-images.ts`	key minting, `<img>` rewriting, size and pixel limits
`apps/api/src/questions/question-sanitize.ts`	the DOMPurify gate
`apps/api/src/questions/taxonomy.service.ts`	`Subject` and `Topic`
`apps/api/src/questions/stem-rehash.service.ts`	the boot-time rehash worker
`packages/contracts/src/question-rules.ts`	the one definition of "valid", shared by the editor, the API and the importer
`apps/api/src/saved/saved-questions.service.ts`	`SavedQuestion`, the student's starred questions

The schema is `prisma/schema.prisma:468` (`Question`) and `prisma/schema.prisma:518` (`QuestionVersion`). The controllers mount only on the `CORE` API role; `StemRehashService` runs only on `WORKER` (`apps/api/src/questions/questions.module.ts:31`).

What it is for

The bank holds every question the platform can put on a paper, in up to three languages (English, Hindi, Telugu), filed under a Subject, then an optional Topic, plus free-text tags. A paper never copies a question: each `PaperQuestion` pins one `QuestionVersion`, so fixing a typo cannot change what a student already sat.

Four question types share the one table (`prisma/schema.prisma:41`):

Type	Answered by	Version payload
`SINGLE_MCQ`	one of 2 to 6 options	`options`
`TEXT_FIELD`	a typed answer, exact or numeric with a tolerance	`answerKey`
`TYPING`	the stem is a passage to be typed; no options, no key	none
`DESCRIPTIVE`	written free and marked by rule against key points	`answerKey` with a rubric

The module enforces the two rules the schema cannot state: when an edit may rewrite the existing version row and when it must append a new one, and that one question is never typed into the bank twice.

Data

Column	Meaning
`Question.questionCode`	the institute's own reference, upper-cased, unique, optional
`Question.subjectId`, `topicId`	the taxonomy; frozen once anything uses the question
`Question.difficulty`	a bank tag used by a section's difficulty mix; never shown to a student
`Question.status`	`ACTIVE` or `ARCHIVED` only
`Question.currentVersionId`	the version the bank shows; a composite FK `[id, currentVersionId]` to `QuestionVersion[questionId, id]`, `Restrict`
`Question.tags`	lowercased, at most 10, 2 to 32 characters, letters, digits and single spaces; GIN indexed
`Question.stemHash`, `stemHashVersion`	the duplicate key and the fold it was made with; `stemHash` is `@@unique`
`Question.searchText`	derived by trigger; the only column the search reads; omitted from every Prisma read (`apps/api/src/prisma/prisma.service.ts:36`)
`Question.assignmentId`	the typing assignment a question was written under, `SetNull`
`QuestionVersion.content`	`{ en?: { stem, passage?, solution? }, hi?, te? }`, each field a list of `{ type: 'TEXT', text: html }` nodes
`QuestionVersion.options`	`[{ id, position, isCorrect, text: { en?, hi?, te? } }]`; stored order is the order of `PaperQuestion.optionIds`

Column	Meaning
`QuestionVersion.answerKey`	`{ mode, answers, tolerance?, rubric? }`
`QuestionVersion.createdById`, `createdAt`	the hand and the moment; reassigned by an in-place revision
`Subject.language`	set when the subject is a language (Hindi Comprehension); chosen at creation, never changed

Language is a key inside the content JSON, not a column. `QuestionSummary.languages` is recomputed from the stored content on every read (`apps/api/src/questions/question-rows.ts:137`). Neither `Question` nor `QuestionVersion` carries a render mode; that lives on the test's `BaseConfig`.

The split into two tables is deliberate. `PaperQuestion` holds a composite `Restrict` FK to the version, so "which wording did this student read" is a database fact rather than a convention. Options live in a jsonb column rather than an `Option` table so one read is the whole question on the exam path; the price is that no SQL constraint can say "exactly one option is correct" (`validateQuestion` carries that) and ids must be merged by hand.

Indexes on `Question`: `(subjectId, difficulty, status)`, `(topicId)`, `(createdById)`, `(assignmentId)`, GIN `(tags)`, GIN trigram `(searchText)`, `(status)`, `(createdAt, id)`, and the unique `(stemHash)` (`prisma/schema.prisma:503`).

Flow: authoring a question in the bank

The bank page at `/questions/new` is the same authoring workspace every section uses. It builds a draft from the workspace state and posts it.

POST /ADMIN/QUESTIONS — WRITING A NEW QUESTION

1. `BankQuestionPage` hands the workspace a `create` source; Save calls `api.admin.questions.create` and maps a unique-code refusal onto the code field
   `apps/admin/src/features/questions/bank-question.tsx:131`
   → draft posted
2. Global JWT guard, `@Actors(ADMIN)`, `@RequiresFeature(QUESTION_MANAGEMENT, WRITE)`, `@Audit(QUESTION, CREATE)`
   `apps/api/src/questions/questions.controller.ts:131`
   → `AuthenticatedUser` on the request
3. `ZodBody(questionDraftSchema)` parses: tags are trimmed, lowercased, de-duplicated and capped at 10; the code is upper-cased and pattern checked
   `packages/contracts/src/questions.ts:403`
   → a `QuestionDraft`
4. `create(body, user.id)`: the author comes from the token, never the body
   `apps/api/src/questions/questions.controller.ts:139`
   → `createdById`
5. `validated` reads exactly the one subject and one topic the draft names, then runs the shared rules with KaTeX bound in; the first issue throws `VALIDATION_ERROR` with `fieldErrors`
   `apps/api/src/questions/questions.service.ts:456`
   → no row yet
6. `buildContent`: per language that has a stem, each field passes DOMPurify, loses any `src` on a keyed image, and is wrapped in exactly one `<div>`
   `apps/api/src/questions/question-core.ts:62`
   → `content`, `options`, `answerKey`, `stemHash`
7. `assertNotDuplicate(stemHash, null)` is a single query; a hit throws `CONFLICT` with `fieldErrors['stem.en']` and `details.duplicateOf`, naming an archived twin as archived
   `apps/api/src/questions/questions.service.ts:487`
   → refusal or pass
8. `$transaction(…, TX_LIMITS.SHORT)` (5 s wait, 10 s timeout); a typist's create first calls `assertJobOpen`
   `apps/api/src/questions/questions.service.ts:236`
   → transaction open
9. `question.create` with the taxonomy columns and `createdById`; `currentVersionId` is still null
   `apps/api/src/questions/questions.service.ts:238`
   → one `Question` row
10. `questionVersion.create` as version 1; every option gets a fresh `randomUUID()`
    `apps/api/src/questions/question-versions.ts:20`
    → one `QuestionVersion` row
11. `question.update { currentVersionId }`; the `question_search_text` BEFORE trigger fires on that column and fills `searchText`
    `apps/api/src/questions/questions.service.ts:254`
    → the composite FK is satisfied, search text set
12. `served` rewrites every image key to `${MEDIA_BASE_URL}/<key>`; the interceptor wraps the answer
    `apps/api/src/questions/questions.service.ts:75`
    → `{ success: true, data: QuestionDetail, meta }`

If two admins type the same stem at once, both can pass step 7. The unique index on `stemHash` then fails the second commit, and the exception filter answers `CONFLICT` "That already exists" without `fieldErrors` (`apps/api/src/common/all-exceptions.filter.ts:130`). The precheck exists to give the friendly message; the index is the guarantee.

The rule: edit in place or add a version

This is the heart of the module. A save either rewrites the current version row, appends a new one, or writes nothing.

PATCH /ADMIN/QUESTIONS/:ID — EDITING A QUESTION

1. The workspace saves with `expectedUpdatedAt` set to the stamp it loaded
   `apps/admin/src/features/questions/bank-question.tsx:105`
   → stale-screen guard armed
2. `@RequiresFeature(QUESTION_MANAGEMENT, WRITE)`, `@Audit(QUESTION, UPDATE)`; `assertNotSectionWork` refuses a question a section's typist still holds, because that is changed on the section's page
   `apps/api/src/questions/questions.controller.ts:142`
   → refusal `CONFLICT`, or continue
3. `validated` and `assertNotDuplicate(stemHash, id)`: same checks as create, excluding itself
   `apps/api/src/questions/questions.service.ts:268`
   → `BuiltQuestion`
4. `$transaction(TX_LIMITS.SHORT)` into `writeEdit`; the row is read inside, so what is decided on is what is written
   `apps/api/src/questions/questions.service.ts:282`
   → transaction open
5. `assertScreenIsCurrent` compares `expectedUpdatedAt` with the row; a mismatch is `CONFLICT` on `status`
   `apps/api/src/questions/questions.service.ts:525`
   → stale form refused
6. `assertIdentitySettled`: changing type, subject, or an already-set topic on a question any `PaperQuestion` or `TestQuestionStat` points at is refused; filling in a topic it never had is allowed
   `apps/api/src/questions/questions.service.ts:404`
   → refusal or pass
7. Raw `SELECT … FROM "Test" … ORDER BY "id" FOR UPDATE` over every test pinning the question and every unfinalized test holding only its review tick
   `apps/api/src/questions/questions.service.ts:295`
   → test rows locked in `id` order, before the question
8. `question.updateMany({ where: { id, updatedAt } })` writes the taxonomy, tags, code and hash; `count !== 1` throws the edited-elsewhere refusal
   `apps/api/src/questions/questions.service.ts:304`
   → `Question` columns written, `updatedAt` moved
9. `optionsWithIds` carries each option id over by position and mints one only for a position that had none
   `apps/api/src/questions/question-versions.ts:10`
   → merged option list
10. `versionFor` fingerprints content, options and key; if it equals the stored version's fingerprint nothing is written and the version is not "reworded"
    `apps/api/src/questions/question-versions.ts:38`
    → `{ id: currentVersionId, reworded: false }`
11. Otherwise `revisableVersionId` counts `PaperQuestion` rows pinning this version on a `reachableTest`; none means `revise`
    `apps/api/src/questions/question-versions.ts:58`
    → decision

12a. Not reached: `revise` updates the existing version's `content`, `options`, `answerKey`, and reassigns `createdById` and `createdAt`
    `apps/api/src/questions/question-versions.ts:72`
    → same version id, new words

12b. Reached: `insertVersion` appends `version = current + 1`
    `apps/api/src/questions/question-versions.ts:94`
    → a new immutable `QuestionVersion`

13. If reworded, `uncheckReworded` clears every proof-reader tick on unfinalized tests and reopens the reader's section
    `apps/api/src/assignments/unread-questions.ts:40`
    → `QuestionReview.checkedAt = null`
14. `question.update { currentVersionId }` (a no-op for 12a) with the full include; triggers fan out
    `apps/api/src/questions/questions.service.ts:316`
    → `searchText` and every unreached paper's `optionIds` rebuilt
15. `auditContext.setChanged(questionDiff(before, after))`: one leaf per `stem.EN`, `solution.HI`, `option.3.TE`, plus the correct positions
    `apps/api/src/questions/question-audit.ts:26`
    → audit row on commit

Reachable. `reachableTest(now)` (`apps/api/src/questions/question-query.ts:15`) selects tests that have been offered (`finalizedAt` not null, not merely out of `DRAFT`) and whose earliest opening, including any program unlock, is at or before `now + START_GRACE_MS`. A null `opensAt` means open now. The grace is five minutes (`packages/contracts/src/access.ts:396`), so a paper that is about to open is already off limits to in-place edits.

What breaks if the rule is wrong. An answer sheet stores a position into `PaperQuestion.optionIds`, not an option id. Rewrite a version a reachable test pins and the stem and options change under people mid-sitting, a stored position may decode to a different option and score against the new key, and a dropped option makes the answer vanish.

The database is a backstop, not the policy. `question_version_sat_guard` (`prisma/migrations/20260918160000_the_version_guard_locks_its_tests_in_one_order/migration.sql:20`) raises `integrity_constraint_violation` on a content, options or key change to any version pinned by a test that has an `Attempt`. It trips only at the first attempt, so the service is deliberately stricter. The surfaced error is a raw Postgres one, which is why the service decides first.

Option ids and the two triggers. `paper_question_option_ids` is BEFORE INSERT OR UPDATE OF `questionVersionId`, `optionIds` on `PaperQuestion` and rebuilds `optionIds` from the pinned version. `question_version_option_ids_fanout` is AFTER UPDATE OF `options` on `QuestionVersion` and re-touches `optionIds` on every pinning paper, which re-fires the BEFORE trigger. An in-place revision therefore repairs every other unopened paper holding the question. It relies on Postgres running same-event BEFORE triggers alphabetically (`paper_question_option_ids` before `paper_question_sat_guard`), so renaming either reverses the order.

Lock order. Every path touching both takes the `Test` before the `Question`, tests in `id` order. Nothing enforces it but the rule; two admins crossing on one order deadlock and Postgres kills one save.

> **Note** `versionFor` runs the fingerprint comparison before it decides anything, so saving an unchanged form writes no version and puts no new author on the row. Only a real change moves `createdById` and `createdAt` on an in-place revision.

Archive, unarchive, delete

Action	Rule	Where
Archive	the soft remove: out of the bank list and out of every draw, reversible, loses nothing	apps/api/src/questions/questions.service.ts:343
Unarchive	the only way back; status becomes `ACTIVE`	apps/api/src/questions/questions.service.ts:348
Delete	one hard delete, at any status, only if no `PaperQuestion` and no `TestQuestionStat` points at the question; clears `currentVersionId`, deletes the versions, then the question	apps/api/src/questions/questions.service.ts:353

Each is pinned to the `updatedAt` it read, so a decision made against a status somebody has since changed is refused. A typist's Done calls `settleTyped` inside its own transaction: discarded drafts are deleted by the same rule, the rest are detached from the assignment and stay in the bank (`apps/api/src/questions/questions.service.ts:371`).

Content: validation, languages, figures

Validation is one function in contracts (`packages/contracts/src/question-rules.ts:89`) used by the editor, the API and the importer. It reports every problem rather than throwing on the first. The server adds KaTeX (`apps/api/src/questions/question-math.ts:4`) because a draft posted straight at the API can carry any LaTeX, and `emptiedTranslations` refuses a translated stem that sanitizes to nothing instead of silently dropping its language (`apps/api/src/questions/question-core.ts:156`). Per type:

- MCQ: 2 to 6 options, numbered from 1 with no gaps, exactly one correct, option text in every language the stem has, no two options saying the same thing (`question-rules.ts:248`).
- Typed answer: no options; the lead-language answer is required; in `NUMERIC` mode every answer must parse as a number and the tolerance must be in range (`question-rules.ts:325`).
- Typing: no options, no key, no picture or formula, bounded in key depressions both ways (`question-rules.ts:385`).
- Descriptive: no options; minimum words must not exceed maximum (`question-rules.ts:434`).

Languages. `SUPPORTED_LANGUAGES` is `en`, `hi`, `te`. A language reaches the stored row only if it has a stem (`languagesIn`), so a translated option with no translated stem is a validation error, not a quiet drop. English is required for an ordinary subject. A subject with `Subject.language` set requires that language in its place and refuses any other beside it (`SUBJECT_LANGUAGE_REQUIRED`, `LANGUAGE_OUTSIDE_SUBJECT`); the rest of the question is judged in that language alone (`question-rules.ts:151`). Every rule that used to read English as the first language now reads the question's lead language (`leadLanguageIn`, `question-rules.ts:500`): the duplicate key, the rehash, the preview, a typed answer and a typing passage. A question that has English still leads with English, so no existing hash moved.

Passage. A question may carry the passage it is asked about (`passage` per language). Each question holds its own copy; there is no grouping. It is folded into the duplicate key only when present, so a question without one keeps its hash.

Figures and formulas. There is no attachment table and no row linking an image to a question.

POST /ADMIN/QUESTIONS/IMAGES — ADDING A FIGURE

1. The browser shrinks the image, then posts multipart field `file`
   `apps/admin/src/lib/upload-question-image.ts:7`
   → `File`
2. `@RequiresAnyFeature([QUESTION_MANAGEMENT, QUESTION_AUTHORING, QUESTION_PROOFREAD], WRITE)`; `FileInterceptor` capped at `QUESTION_IMAGE_MAX_BYTES` (2 MB)
   `apps/api/src/questions/questions.controller.ts:93`
   → buffer in memory
3. `checkQuestionImage` refuses an empty file; the leading bytes decide the type (PNG, JPEG, GIF, WebP), the browser's content type is only a hint; SVG is excluded because it can carry script; over 12 megapixels is refused by dimensions
   `apps/api/src/questions/question-images.ts:55`
   → `CheckedImage` or a `VALIDATION_ERROR` on `file`
4. `questionImageKey` mints `questions/images/<uuid>.<ext>`, unscoped because the image exists before its question does; the importer instead keys by content digest
   `apps/api/src/questions/question-images.ts:20`
   → an S3 key
5. `storage.upload(key, buffer, type, IMMUTABLE_CACHE_CONTROL)` is one `PutObject` with `public, max-age=31536000, immutable`
   `apps/api/src/questions/questions.service.ts:87`
   → object in S3 or MinIO
6. The response is `{ key, url }`; the editor writes `<img data-key="…">` into the content
   `apps/api/src/questions/questions.service.ts:89`
   → key held only inside question HTML

On save, `stripImageSrc` removes any `src` from a keyed image and `withoutForeignImages` drops an `<img>` that has no key (`apps/api/src/questions/question-images.ts:107`). On read, `applyImageUrls` builds a fresh unsigned `${MEDIA_BASE_URL}/<key>` (`question-images.ts:119`), so an old row pointing anywhere else is neutralised. A formula is not media: it is `<span data-type="inline-math" data-latex="…">` in the same HTML (`apps/api/src/questions/question-content.ts:111`). DOMPurify allows a fixed tag list and the attributes `colspan`, `rowspan`, `src`, `alt`, `width` plus data attributes (`apps/api/src/questions/question-sanitize.ts:37`).

Duplicates and the stem hash

`computeStemHash` is a sha256 of `canonicalStemKey` (`apps/api/src/questions/question-core.ts:171`): the stem, the options as a set, and the correct option's text, in the lead language, folded for case, spacing, markup and stray punctuation. Signs, dashes, `%`, `/` and a decimal point between digits survive the fold, so "x = 2.5" and "x = 25" are different questions. `STEM_HASH_VERSION` is 2 (`question-core.ts:169`); `StemRehashService` rehashes any row below it at worker boot, 500 rows a batch, with a raw `UPDATE … WHERE "updatedAt" = seen` so it never overwrites a concurrent edit and never moves `updatedAt` (`apps/api/src/questions/stem-rehash.service.ts:31`). The stored-content hash goes through the same plain-text path the editor would post back, so a rehash equals the next edit's hash (`question-core.ts:184`).

Search

GET /ADMIN/QUESTIONS?Q= — SEARCHING THE BANK

1. The list screen calls `api.admin.questions.list` through `useListScreen`
   `apps/admin/src/features/questions/questions.tsx:187`
   → query params
2. `@RequiresAnyFeature([QUESTION_MANAGEMENT, TEST_MANAGEMENT], READ)`; the test builder's picker reads the same endpoint
   `apps/api/src/questions/questions.controller.ts:73`
   → guard passed
3. `ZodQuery(questionListQuerySchema)`: `q` is trimmed, capped at 64 characters (`SEARCH_QUERY_MAX`) and has its `\`, `%`, `_` escaped; there is no minimum length
   `packages/contracts/src/common.ts:223`
   → escaped term
4. `questionWhere` splits filters into those that always narrow (archived hidden unless ARCHIVED is named, `drawable`, `writtenFor`, search, tag, author, date range) and those the `match` toggle governs (subject, topic, type, difficulty, status, language)
   `apps/api/src/questions/question-query.ts:29`
   → one `AND` list
5. `{ searchText: { contains: q, mode: 'insensitive' } }`, which Prisma 6.19 renders as `"searchText" ILIKE $1`
   `apps/api/src/questions/question-query.ts:51`
   → predicate, no ids on the wire
6. The trigram GIN index `Question_searchText_trgm_idx` serves the ILIKE on the bare column
   `prisma/schema.prisma:510`
   → bitmap index scan
7. `$transaction([findMany(QUESTION_INCLUDE, orderBy, page), count])`, ordered `createdAt` then `id` descending (or ascending)
   `apps/api/src/questions/questions.service.ts:100`
   → one page and a total
8. `toSummary` builds a 140-character stem preview; `inUse` comes from `take: 1` on `paperQuestions` and `questionStats`
   `apps/api/src/questions/question-rows.ts:65`
   → `Paginated<QuestionSummary>`, with `meta.page/pageSize/total`

`Question.searchText` is the current version's field text (stem, passage, solution, every language), the question code and the tags as one string with markup stripped and `< > &` decoded. It is rebuilt by `questionSearchText` (`prisma/migrations/20261008120000_the_bank_search_reads_the_words_not_the_json_around_them/migration.sql`), which reads the JSON path `$.*.*[*].text` so the JSON's own key names and escaping are not searchable. Option text is not searched. Two triggers maintain it: `question_search_text` (BEFORE INSERT OR UPDATE OF `currentVersionId`, `questionCode`, `tags`, `searchText` on `Question`) and `question_version_search_text_fanout` (AFTER UPDATE OF `content` on `QuestionVersion`, only when the content changed). Writing `searchText = searchText` is how a backfill fires the trigger; any hand-set value is recomputed. The index sits on the bare column because that is what the ILIKE emits; an index on `lower("searchText")` would never be used.

Other filters: tag is `tags has`, author is a contains match on the author's name or sign-in email, dates use `withinInstituteDays` (IST days), and language tests that the current version's `content` has the language key (`apps/api/src/questions/question-query.ts:82`). The language filter is a JSON path predicate with no index; it degrades with bank size. The list serves only admins; students and mobile never call these routes.

> **Watch** The picker (`drawable=true`) asks `drawableFor` (`apps/api/src/questions/question-core.ts:222`): not archived, the section's own kind of type, a current version, and either unassigned, written for this test, or written under a test with no open assignment. It is the same predicate the draw uses, so the picker cannot offer a row the fill refuses.

Availability, versions, export

- `GET /admin/questions/availability` counts what a section could draw, grouped by difficulty, in the database. It excludes what the test already holds, because a question sits on a paper once (`apps/api/src/questions/questions.service.ts:153`). The draw-spec screen calls it (`apps/admin/src/features/tests/draw-spec.tsx:46`).
- `GET /admin/questions/:id/versions` lists the chain newest first, with the author and the titles of the tests pinning each link (`apps/api/src/questions/questions.service.ts:188`). The chain is short by construction: a link is added only when a reachable paper pins the old one.
- `POST /admin/questions/export` is not a download. It returns 202 and starts an export job (`apps/api/src/questions/questions.controller.ts:82`); the worker builds the workbook through `exportWorkbook` (`apps/api/src/exports/export-builders.ts:52`). `exportRows` runs the list's own predicate with no page, refuses more than `EXPORT_MAX_ROWS` through `assertExportable`, and adds a per-author count by status (`apps/api/src/questions/questions.service.ts:120`). Image cells carry the key, not a signed URL.

Taxonomy

`Subject` and `Topic` are two levels only; anything finer is a tag. `TaxonomyController` (`apps/api/src/questions/taxonomy.controller.ts:49`) serves `/admin/subjects` and `/admin/topics` with `QUESTION_MANAGEMENT` for writes; reads are open to the roles that file or draw (`TAXONOMY_READERS`). Names arrive canonical from the schemas, so a case- or space-different duplicate cannot be created; a topic name is unique within its subject. Both can be renamed. A subject's `language` is accepted on create and not on update (`apps/api/src/questions/taxonomy.service.ts:127`, `:144`).

Deletion is allowed only while nothing carries the row: a subject is blocked by topics, questions, base-configuration sections and student subject statistics; a topic by questions and by any test whose `questionPoolFilter` names it, found with a `jsonb_path_exists` query because the foreign key cannot see ids inside JSON (`apps/api/src/questions/taxonomy.service.ts:177`, `:280`, `:298`). The refusal names each holder with its count.

Saved questions

A student stars a question from the review of a sitting; the star is provenance and permission in one. `SavedQuestion` has `@@unique([studentId, questionId])`, `attemptId` and `paperQuestionId` both `SetNull` (`prisma/schema.prisma:1337`). Routes are `me/saved`, student-only (`apps/api/src/saved/saved.controller.ts:28`).

POST /ME/SAVED/BOOKMARKS — STARRING A QUESTION

1. The review surface posts `{ attemptId, questionId }`
   `packages/contracts/src/client/me.ts:241`
   → body
2. `@Actors(STUDENT)`; `ZodBody(bookmarkQuestionSchema)`
   `apps/api/src/saved/saved.controller.ts:48`
   → `BookmarkQuestionBody`
3. `servedPastTheGate` finds the caller's own attempt (a stranger's reads as missing) and the `PaperQuestion` for `(testId, questionId)`; absent is `NOT_FOUND` "You did not sit this question"
   `apps/api/src/saved/saved-questions.service.ts:225`
   → `paperQuestionId`
4. The attempt must be `EVALUATED`, otherwise `CONFLICT` "not marked yet"
   `apps/api/src/saved/saved-questions.service.ts:241`
   → refusal or pass
5. `savedQuestion.create`; a unique violation is swallowed, so starring twice is one row
   `apps/api/src/saved/saved-questions.service.ts:174`
   → one `SavedQuestion` row
6. The row is read back with its subject, topic and current content preview
   `apps/api/src/saved/saved-questions.service.ts:186`
   → `SavedQuestion` response

The list reads newest first through the `(studentId, createdAt)` index, optionally narrowed by subject and by test (a test narrows by the caller's own attempt ids), then reads each sitting's sheet once to show time spent (`apps/api/src/saved/saved-questions.service.ts:53`, `:91`). `facets` reads the whole saved set so the filter options do not shift with the page (`:143`). `bookmarkedIn` matches on the questions the sitting served, so a star made on an earlier paper still shows (`:200`). Delete puts the owner in the `WHERE`, so another student's row reads as missing (`:194`). Opening a saved question reuses the solutions query of its sitting, so it shows the wording that was sat; a voided sitting falls to "no longer available" with no Retry (`apps/exams/src/features/review/saved-question-dialog.tsx:34`).

Cost, read not measured

The list is two queries in a transaction, with no N+1 (`take: 1` on both child relations replaced a `_count` that grouped whole tables per page). The search measured on a 50K-row bank in the migration notes: about 0.1 to 0.2 ms for the page, 71.6 ms worst case at 98% selectivity where the planner abandons the index. Short terms under three characters have no trigrams and fall back to a scan rather than failing. An edit is three queries before the transaction, five or six inside, plus triggers; the option-id fan-out is one BEFORE trigger per pinning paper.

Tests: `apps/api/test-db/questions-service.db.test.ts` pins the version rule across draft, unreached, inside-grace and open papers; `apps/api/test-db/question-version-option-ids.db.test.ts` pins the fan-out; `apps/api/test-db/saved-questions.db.test.ts`, `apps/api/test-db/taxonomy-service.db.test.ts`.

What is wrong with it

- Correctness: `packages/contracts/src/question-rules.ts:344` reports a missing typed answer on field `answerKey.answers.en` while the message names the lead language; for a Hindi subject the error lands on a field the form does not show. The typing-passage refusal at `:402` hard-codes the same field.
- Correctness: `apps/api/src/saved/saved-questions.service.ts:272` previews a saved question from the question's current version, while opening it shows the version that was sat, so a reworded question can list under new words and open under old ones.
- Stale: `apps/api/src/questions/questions.service.ts:474` stacks two doc comments; the first, about `assertNotDuplicate`, sits on `duplicateOf`.
- Stale: `apps/api/src/questions/question-core.ts:37` is a doc comment attached to a re-export, describing nothing it sits on.
- Stale: `docs/02-domain-rules.md:577` says a typing passage's "English stem" is the passage, and `docs/02-domain-rules.md:640` says "English is required"; both hold only for a subject that is not a language.

The authoring workshop

Where it lives

A paper is typed and proof-read before it is offered. Two people hold one section of one test: a **typist** writes its questions, a **proof-reader** reads them. The module records who holds which half, moves the section between them, and carries the reader's verdict back to the typist with a reason, one question at a time. The test's owner works the same screen from a third seat. `Test.finalizedAt` (the offer) ends all of it.

Piece	Path	Owns
Assignments	`apps/api/src/assignments/assignments.service.ts`	assign, pass a role on, change the due day, remove, the per-admin queue, a reader's release and its gap rule
Gates	`apps/api/src/assignments/assignment-gates.ts`	`doneOpen`, `readOpen`, `owed`, `standingOf` and `assertJobOpen`: the predicates API and UI share
Unread rules	`apps/api/src/assignments/unread-questions.ts`	"unchecked on this test", and the two reopeners a paper edit calls
Progress	`apps/api/src/assignments/section-progress.service.ts`	the super-admin grid and the queue's test and section pickers
Thread	`apps/api/src/assignments/section-thread.service.ts`	the (test, section) discussion and its revision trail
Section work	`apps/api/src/questions/section-work.service.ts`	the screen's whole read, and every per-question verdict
Seat rules	`apps/api/src/questions/section-rules.ts`	`editable`, `deletable`, `actingAs`, `typingRow`, `heldNow` and the refusal texts
Scoped read	`apps/api/src/questions/section-scoped.ts`	which questions a seat sees
Typist's Done	`apps/api/src/tests/paper.service.ts:395`	`typistDone`, here because its effect is the paper
Edit claim	`apps/api/src/common/edit-lock.ts`	the advisory per-section claim in Redis
Screen	`apps/admin/src/features/authoring/section-authoring.tsx`	the one screen all three seats work from
Schema	`prisma/schema.prisma`	`QuestionAssignment`, `QuestionReview`, `QuestionWorkTime`, `SectionComment`

The authoring directory also holds the bank's own editor (`authoring-editor.tsx`, `authoring-history.tsx`) and the shared card workspace (`authoring-workspace.tsx`) that both screens render. This chapter covers the section screen; the bank editor is the questions chapter's.

What a section is, in state

State is stamps, not an enum: each of `finalizedAt`, `handedAt` and `replacedAt` on `QuestionAssignment` says *when*, so no state machine can drift from the dates. The cost is that "what state is this row" is a TypeScript predicate SQL cannot enforce.

Stamp	On	Means	Written by
`finalizedAt`	either row	the holder's job is done; null is outstanding	typist's Done `apps/api/src/tests/paper.service.ts:432`, reader's release `apps/api/src/assignments/assignments.service.ts:477`
`handedAt`	reader's row	the section has reached the reader	Done `apps/api/src/tests/paper.service.ts:443`, owner's hand-over `apps/api/src/tests/paper.service.ts:537`
`replacedAt`	either row	the role passed to someone else, or the section left the test's scope; the row stays as the record	`apps/api/src/assignments/assignments.service.ts:246`
`dueAt`	either row	the day it is wanted by, kept as that day's last instant at the institute; gates nothing	`apps/api/src/assignments/assignments.service.ts:257`

One active holder per role per section is a hand-written partial unique index, `QuestionAssignment_active_role_key ... WHERE "replacedAt" IS NULL` (migration `20260928160000`), invisible to Prisma. Passing a role inserts a new row and stamps the old one rather than updating `assigneeId`, so the record survives.

The reader's verdict is a separate table, `QuestionReview`, one row per `(testId, questionId)`. The state a card shows is derived by `reviewOf` (`apps/api/src/questions/section-rules.ts:114`): an open send-back (`sentBackAt` set, `fixedAt` null) is SENT_BACK, else a `checkedAt` is CHECKED, else a `fixedAt` is FIXED, else UNCHECKED. Actor ids on it have no foreign key, so a verdict outlives whoever gave it.

`QuestionWorkTime` keeps seconds per `(test, question, admin, seat)`, added to in batches; nothing reads a history of it.

The three seats

`SectionWorkService.load` (`apps/api/src/questions/section-work.service.ts:402`) decides who the viewer is on the section, once per request. It reads the test, the section, and every assignment row on the pair (replaced ones included). The rule is short:

- The viewer's own row is the one they hold now, else the last they held (`:436`).
- With no row and no TEST_MANAGEMENT grant the answer is NOT_FOUND "No such section", never FORBIDDEN: not yours reads as not there (`:439`).
- The seat is TYPIST or READER by the row's role; a test owner whose own seat has passed on reads the section as its OWNER again (`:442–445`).
- A test with no `paperSource` is refused (`assertSourceChosen`, `:440`).

A feature key is therefore not a seat. The controller's guard only says the caller works on sections (`SECTION_KEYS`: authoring, proof-read or test management); the service says whether *this* section is theirs. Role is read afterwards, only to label the seat.

Flow: assigning a typist or a proof-reader

POST /ADMIN/ASSIGNMENTS/TESTS/:TESTID — GIVING A SECTION A HOLDER

1. Owner submits the Assign dialog on test builder step 2
   `apps/admin/src/features/tests/test-builder-assign-step.tsx:677`
   → body `{baseConfigSectionId, assigneeId, role, dueAt}`; the due day is required
2. Route guard: `@RequiresFeature(TEST_MANAGEMENT, WRITE)`, `ZodBody(createAssignmentSchema)`, 201
   `apps/api/src/assignments/assignments.controller.ts:88`
   → validated body, `user.id` as actor
3. Test must exist and must not be offered
   `apps/api/src/assignments/assignments.service.ts:207`
   → CONFLICT "has been offered, so nobody new can be given its sections"
4. Source must be chosen (FRAMED or PICKED)
   `apps/api/src/assignments/assignments.service.ts:209`
   → `assertSourceChosen`
5. Due day must be today or later
   `apps/api/src/assignments/assignments.service.ts:210`
   → VALIDATION_ERROR on `dueAt`
6. Section must belong to the test's config and sit inside its scope
   `apps/api/src/assignments/assignments.service.ts:518`
   → NOT_FOUND "No such section" otherwise
7. Assignee must hold the role's key at WRITE: QUESTION_AUTHORING for a typist, QUESTION_PROOFREAD for a reader
   `apps/api/src/assignments/assignments.service.ts:541`
   → `fieldErrors.assigneeId` refusal
8. Nobody reads their own typing: not the section's active typist, and not anyone who ever typed a question for it
   `apps/api/src/assignments/assignments.service.ts:556`
   → `fieldErrors.assigneeId` refusal
9. Transaction opens on the app's one lock order: `SELECT ... FOR UPDATE` on `Test`, refused if the test was offered meanwhile
   `apps/api/src/assignments/assignments.service.ts:232`
   → the test row is held
10. Current holder re-read under the lock; the same admin again is refused
    `apps/api/src/assignments/assignments.service.ts:235`
    → VALIDATION_ERROR "already holds that role"
11. Progress belongs to the role, not the person: a holder's `handedAt` carries over; with no holder, a reader arriving on a FRAMED section whose typist is done starts handed
    `apps/api/src/assignments/assignments.service.ts:244`
    → `handedAt` for the new row
12. Old row stands down, never deleted
    `apps/api/src/assignments/assignments.service.ts:246`
    → `replacedAt = now()`
13. New row inserted with role, `dueAt` (end of the institute day), inherited `finalizedAt` and `handedAt`, `createdById`
    `apps/api/src/assignments/assignments.service.ts:251`
    → `QuestionAssignment` row
14. A race loses on the partial unique and is told why
    `apps/api/src/assignments/assignments.service.ts:270`
    → CONFLICT "That section already has a typist/proof-reader"
15. The replaced holder's section claim is released so it cannot keep the new holder out
    `apps/api/src/assignments/assignments.service.ts:267`
    → Redis key `section:edit:{test}:{section}` deleted if theirs
16. Row returned with section facts; the answer carries nobody's time
    `apps/api/src/assignments/assignments.service.ts:269`
    → `Assignment` in the envelope, `secondsSpent: null`

Related writes on the same controller: `PATCH tests/:testId/:id` moves the due day without passing the seat on (refused once the test is offered or the seat is finished, because a finished seat's standing is read off the day; `apps/api/src/assignments/assignments.service.ts:281`). `DELETE tests/:testId/:id` takes back a row only while nothing has been done under it: no question typed, no review given, no comment, no audited edit since the row began (`apps/api/src/assignments/assignment-removal.ts:16`). `GET assignable` lists active admins who hold the role's key at WRITE, with those *called* typist or proof-reader sorted first; the name decides order, never eligibility (`apps/api/src/assignments/assignments.service.ts:198`).

An offered test takes nobody new, and the offer itself now refuses any in-scope section with no current proof-reader (`apps/api/src/tests/finalize.service.ts:147`), then any reader who has not released, then any question on a read section without a tick (`:164`, `:188`). A super admin skips the reader checks (`:67`) but never the whole-paper check.

Flow: the typist hands the section over (Done)

There is no per-question typist tick. The typist's one hand-over is Done, and it chooses the paper.

POST /ADMIN/SECTIONS/:TESTID/:SECTIONID/DONE — A TYPIST MARKS A SECTION DONE

1. "Mark done" opens the dialog, shown only when the section says `canMarkDone` and no card holds an unsaved edit
   `apps/admin/src/features/authoring/section-authoring.tsx:257`
   → `TypistDoneDialog` open
2. Dialog re-reads the section; typed drafts are the rows to tick, paper rows not typed here are listed as leaving the paper
   `apps/admin/src/features/authoring/typist-done-dialog.tsx:48`
   → `chosen` and `discarded` sets
3. Submit
   `apps/admin/src/features/authoring/typist-done-dialog.tsx:60`
   → body `{selected, discard}`
4. Route: `@Audit(TEST, UPDATE)`, `@RequiresFeature(QUESTION_AUTHORING, WRITE)`, 200, `ZodBody(typistDoneSchema)` (at least one selected)
   `apps/api/src/tests/tests.controller.ts:232`
   → validated body
5. Seat resolved to a typing row; a super admin acts through the current holder
   `apps/api/src/questions/section-rules.ts:179`
   → the `Assignment` Done is taken under
6. Row re-checked: must be a TYPIST row, not replaced, not already finalized
   `apps/api/src/tests/paper.service.ts:395`
   → NOT_FOUND or CONFLICT otherwise
7. Gate `doneOpen`: TYPIST, not replaced, not finalized, paper FRAMED, test not offered
   `apps/api/src/assignments/assignment-gates.ts:27`
   → CONFLICT otherwise
8. Transaction opens with `FOR UPDATE` on the test
   `apps/api/src/tests/paper.service.ts:429`
   → lock held
9. `finalizedAt = now()`, guarded on `finalizedAt: null`; a second Done matches zero rows
   `apps/api/src/tests/paper.service.ts:432`
   → CONFLICT "already done" for the loser
10. Typed drafts re-read under the lock, so a question typed while Done waited is settled with the rest; the choice must be exactly the section's count, inside its difficulty mix, and all the typist's own
    `apps/api/src/tests/paper.service.ts:437`
    → refusal before the paper moves
11. The choice becomes the paper: unticked paper rows deleted, new ones appended after the highest `order`
    `apps/api/src/tests/paper.service.ts:438`
    → `PaperQuestion` rows
12. The unlock: `handedAt = now()` on the active reader
    `apps/api/src/tests/paper.service.ts:443`
    → the section has reached the reader
13. Unchosen drafts settled: discarded ones deleted, the rest detached (`assignmentId` null) so they go to the bank as ordinary questions
    `apps/api/src/questions/questions.service.ts:371`
    → `Question.assignmentId` nulled
14. Typing claim released (nobody types here now) and the section re-read
    `apps/api/src/questions/section-work.service.ts:177`
    → Redis claim key deleted; `SectionWork` returned

`handedAt` is the whole unlock: `readOpen` requires it (`apps/api/src/assignments/assignment-gates.ts:61`), `scopedQuestions` returns nothing to a reader without it (`apps/api/src/questions/section-scoped.ts:17`), and `requireReading` refuses with "has not reached you yet" (`apps/api/src/questions/section-work.service.ts:475`). On a PICKED paper there is no Done; the owner hands over instead (`POST /admin/tests/:id/paper/sections/:sectionId/hand-over`, `apps/api/src/tests/paper.service.ts:537`), refused until the paper holds the section's count.

Flow: a proof-reader sends one question back

POST /ADMIN/SECTIONS/:TESTID/:SECTIONID/QUESTIONS/:QUESTIONID/SEND-BACK — ONE QUESTION TO THE TYPIST

1. "Send back" on the card, offered only when the section has a typist
   `apps/admin/src/features/authoring/section-question-card.tsx:153`
   → `SendBackDialog`
2. Reason (radio: spelling, data correction, answer option) and an optional note of at most 500 characters
   `apps/admin/src/features/authoring/section-question-card.tsx:234`
   → body `{reason, note?}`
3. Route: `@RequiresAnyFeature(SECTION_KEYS, WRITE)`, 200, `ZodBody(sendBackSchema)`
   `apps/api/src/questions/section-work.controller.ts:213`
   → validated body
4. Context read once
   `apps/api/src/questions/section-work.service.ts:402`
   → seat, rows, typist, reader
5. Viewer must hold the reading seat now, test not offered, section handed, not yet released
   `apps/api/src/questions/section-work.service.ts:469`
   → FORBIDDEN or CONFLICT otherwise
6. A section with no active typist refuses
   `apps/api/src/questions/section-work.service.ts:344`
   → CONFLICT "no typist to send a question back to"

7. Question must be on this section's paper
   `apps/api/src/questions/section-work.service.ts:480`
   → NOT_FOUND otherwise
8. Transaction: `FOR UPDATE` on the test, then the reader's job re-read as open, so a release that landed first wins
   `apps/api/src/assignments/assignment-gates.ts:46`
   → CONFLICT "finished or passed to somebody else"
9. Upsert on `@@unique([testId, questionId])`: sets `sentBackAt`, `reason`, `note`, `sentBackById`; clears `fixedAt`, `fixedById`, `checkedAt`, `checkedById`
   `apps/api/src/questions/section-work.service.ts:357`
   → `QuestionReview` row
10. The reader's own claim is released and the section re-derived; the question's state is now SENT_BACK
    `apps/api/src/questions/section-work.service.ts:183`
    → `SectionWork` returned
11. The card turns the reader's actions into a "With the typist" badge; the typist's card shows the reason and note as a warning chip
    `apps/admin/src/features/authoring/section-question-card.tsx:137`
    → no further write
12. The question is editable by the typist again even after Done
    `apps/api/src/questions/section-rules.ts:148`
    → `editable` true on SENT_BACK
13. Typist saves, then "Mark fixed"
    `apps/api/src/questions/section-work.service.ts:368`
    → `fixedAt`, `fixedById`; state FIXED, back to the reader

Check is the same shape (`apps/api/src/questions/section-work.service.ts:304`): `reviewing()` runs it under the test lock, refuses a question still with the typist (`WITH_TYPIST_MESSAGE`, `:312`), and upserts `checkedAt` and `checkedById`. *Uncheck* nulls both (`:322`). A reword drops every tick on every draft that holds the question (`apps/api/src/assignments/unread-questions.ts:40`), and a released reader is pulled back if the section then holds an unchecked question (`:10`). A reader's own saved edit of any kind also drops their tick on that test (`apps/api/src/questions/section-work.service.ts:265`), and the Mark fixed, Mark done and Release buttons are hidden while any card holds an unsaved edit, because the hand-on closes the card.

No notification fires on a send-back: nothing in `apps/api/src/assignments/` or `section-work.*` touches `NotificationsService`. The typist learns of it on the screen.

Flow: the reader releases the section

POST /ADMIN/SECTIONS/:TESTID/:SECTIONID/RELEASE — "MARK READ"

1. "Release" opens a confirm dialog naming the question count; offered only when the server's `canRelease` is true
   `apps/admin/src/features/authoring/section-authoring.tsx:263`
   → `FinalizeAssignmentDialog`
2. Route: `@RequiresFeature(QUESTION_PROOFREAD, WRITE)`, 200
   `apps/api/src/questions/section-work.controller.ts:115`
   → caller
3. Reading row resolved (a super admin acts through the holder)
   `apps/api/src/questions/section-work.service.ts:221`
   → reader `Assignment`
4. Row checks: not a typist row, not replaced, handed, test not offered
   `apps/api/src/assignments/assignments.service.ts:441`
   → CONFLICT otherwise
5. Transaction takes the test row `FOR UPDATE`; a stamp already standing is returned as is (idempotent)
   `apps/api/src/assignments/assignments.service.ts:462`
   → lock held
6. Release gap: no typing job open, the paper holds the section's count, every question checked
   `apps/api/src/assignments/assignment-counts.ts:50`
   → refusal text names the gap
7. `finalizedAt = now()` on the reader's row
   `apps/api/src/assignments/assignments.service.ts:477`
   → the reader's authority over the section ends
8. Own claim released, section re-read
   `apps/api/src/questions/section-work.service.ts:223`
   → `SectionWork` returned

After release, a test owner may edit the section again (`withOwner`, `apps/api/src/questions/section-rules.ts:212`); a question added or swapped on later that the reader has not checked reopens the reader's `finalizedAt` once the section is whole again.

The comment thread

The thread is scoped to `(testId, baseConfigSectionId)`: the section, never a question. Both seats share it. It has its own controller (`apps/api/src/assignments/assignments.controller.ts:167`) and its own pair of rules:

- **Read** (`apps/api/src/assignments/section-thread.service.ts:119`): TEST_MANAGEMENT (a super admin always holds it) reads every section; anyone else needs any assignment row on the pair, replaced rows included, so a stood-down holder still reads. Otherwise NOT_FOUND "No such section".
- **Write** (`:132`): refused once any attempt exists on the test ("This test has been sat", `:139`); otherwise only a *current* holder writes. A super admin may write to an unstaffed section, after the pair is proved to exist (`:150`).

- **Reword** (`:71`): the author only, never a super admin. The previous `{body, at}` is pushed onto `revisions`, oldest first, and `editedAt` is set. A reword is a write, so a stood-down author cannot reword.
- The whole thread comes back unpaged, oldest first, with image keys signed once per distinct key (`:112`). The client component is `apps/admin/src/components/section-thread.tsx:111`.

The edit claim

Two people editing one section minutes apart would overwrite each other; the optimistic stamp only refuses the second save after the work. So a section carries an advisory claim in Redis: one key per section, `section:edit:{testId}:{baseConfigSectionId}` (`apps/api/src/redis/redis.keys.ts:67`), TTL 15 minutes (`:72`). The key holds the admin id so a refusal can name them.

- **Taken** before the transaction, on create, edit and delete (`apps/api/src/questions/section-work.service.ts:193` , `:259` , `:289`), by an atomic Lua take-over (`apps/api/src/redis/redis.service.ts:43`): free or lapsed or the caller's own is re-armed for 15 fresh minutes; anyone else's refuses with "X is editing this section". A super admin steals it. It is taken outside the transaction because Redis is not transactional with Postgres.
- **No heartbeat.** It lapses 15 minutes after the last real edit (`apps/api/src/common/edit-lock.ts:1-5`).
- **Given up** at each hand-over: Done (`apps/api/src/questions/section-work.service.ts:177`), a release (`:223`), a send-back, a check and a fixed (`passedOn` , `:183`), a role passing to someone else (`apps/api/src/assignments/assignments.service.ts:267`), an owner's hand-over (`apps/api/src/tests/paper.service.ts:537`), and a reword that sends a released section back to its reader (`apps/api/src/questions/section-work.service.ts:276`).
- **Shown** as `SectionWork.editingBy` . The screen blocks every card for a non-super-admin while someone else holds it, polls every 10 seconds for the claim to lapse (`apps/admin/src/features/authoring/section-authoring.tsx:64`), and re-reads the open cards the moment it frees (`:81`). A super admin is told, never stopped; their save takes the claim.

Reads, checks and ticks do not take the claim; they take the test row lock for the length of their transaction instead.

Who may do what to a question

`editable` (`apps/api/src/questions/section-rules.ts:141`) is the per-question rule, sent to the client as `SectionQuestion.editable` and judged again on save:

Viewer	May edit
Anyone, test offered	only a super admin
Super admin	always, otherwise
Typist (held now, or the section's last typist when nobody took the seat after them)	a question sent back to them; or, on a FRAMED paper before Done, a question they typed
Proof-reader	a question on the paper, once handed and before release, unless it is with the typist
Test owner (TEST_MANAGEMENT at WRITE)	only when the section is back with them: a picked section before hand-over, or any section once its reader has released

`deletable` adds: typed for this section, off the paper, and either the viewer wrote it or they are a super admin. A paper row is never deleted here. A stood-down typist only reads, except that the last typist with nobody after them still fixes the section's drafts.

Time on a question

The client counts seconds only while a card is at rest in view, the tab is visible, and the question awaits the viewer (`awaitsViewer` , `apps/admin/src/features/authoring/work-clock.ts:5`): a blank card or one being edited for a typist, a question sent back for a typist until fixed, and for a reader any question not yet checked. It reports in batches every 30 seconds and on tab hide with `keepalive` (`apps/admin/src/features/authoring/use-work-clock.ts:10`), through `POST .../questions/:questionId/time` (at most 3600 seconds per report). A failed report that got no answer is given back to be retried. The server upserts an increment per `(test, question, admin, seat)` and counts nothing once the test is offered (`apps/api/src/questions/section-work.service.ts:150`). A holder sees their own time; the owner and a super admin see each seat's.

The data it touches

Column	Holds	Written by
`QuestionAssignment.testId / baseConfigSectionId`	composite FKs to `Test(id, baseConfigId)` and `BaseConfigSection(baseConfigId, id)`	assign
`.assigneeId` , `.role`	the admin (FK RESTRICT); TYPIST or PROOFREADER	assign
`.dueAt` , `.finalizedAt` , `.handedAt` , `.replacedAt`	the stamps above	see table at the top
`QuestionReview.checkedAt/checkedById`	the reader's tick	`apps/api/src/questions/section-work.service.ts:304`
`.sentBackAt/reason/note/sentBackById`	the rejection; reason is SPELLING, DATA_CORRECTION or ANSWER_OPTION	`:357`
`.fixedAt/fixedById`	the typist's answer	`:383`
`QuestionWorkTime.seconds`	running total per seat	`:163`

Column	Holds	Written by
`SectionComment.body/images/revisions/editedAt`	the message, S3 keys signed on read, earlier wordings	`apps/api/src/assignments/section-thread.service.ts:63`, `:94`
`Question.assignmentId`	the typing job a draft was written under; SetNull	`apps/api/src/questions/questions.service.ts:386`

Redis holds one key (the claim). There is no queue and no BullMQ job anywhere in this module.

The screen

`SectionAuthoringPage` (`apps/admin/src/features/authoring/section-authoring.tsx:67`) is mounted at three route patterns that render the same component (`apps/admin/src/App.tsx:226`): `/authoring/assignments/tests/:testId/sections/:sectionId`, `/proofreading/assignments/tests/:testId/sections/:sectionId` and `/tests/:testId/sections/:sectionId` (`apps/admin/src/lib/constants.ts:100-106`). It reads `GET /admin/sections/:t/:s` and renders the cards through the shared `AuthoringWorkspace`.

- One card per question, one screen each. The card's foot carries the review badge, the viewer's clock, each seat's time for those who may see both, and notices as chips (a send-back reason, or a warning that an edit also lands on other tests).
- `seatOf` (`apps/admin/src/features/authoring/section-seat.ts:12`) turns the server's facts and the viewer's grants into the five booleans every control reads: `writes`, `reading`, `typing`, `fixing`, `releasing`. Mark done appears only when the server's `canMarkDone` is true; Release only when `canRelease` is.
- Saving sends `expectedUpdatedAt` (the stamp the card loaded) so an edit made elsewhere is refused rather than overwritten (`apps/admin/src/features/authoring/section-authoring.tsx:216`).
- Where the section stands is said once as a toast when it changes (`:388`), not as a banner: offered, passed to somebody else, not yet handed, released, marked done.
- A section sat in one language asks only for that language in its checklist (`b26a88a`); the bank editor still asks for all.
- The progress panel (`apps/admin/src/features/authoring/section-progress-panel.tsx:43`) shows both holders, due day, time, and a tile per question that jumps to its card.

Endpoint	Caller (apps/admin/src)	Notes
`GET /admin/assignments/mine`	features/authoring/assignment-queue.tsx	one admin's rows; hard-scoped to `user.id` (`assignments.controller.ts:112`)
`GET /admin/assignments/progress`	features/authoring/section-progress.tsx	super admin only; every unfrozen test's sections
`GET /admin/assignments/tests`, `.../tests/:id/sections`	features/authoring/assignment-scope-picker.tsx	the queue's pickers
`GET/POST/PATCH/DELETE /admin/assignments/tests/:testId...`	features/tests/test-builder-assign-step.tsx	owner only
`GET /admin/sections/:t/:s`, question reads and writes, `check`, `send-back`, `fixed`, `time`, `other-tests`	features/authoring/section-authoring.tsx	any section key; the service decides
`POST .../done`	features/authoring/typist-done-dialog.tsx	typist
`POST .../release`	features/authoring/finalize-assignment-dialog.tsx	reader
`POST .../import/preview`, `.../import/commit`	features/imports/import-questions.tsx	typist's sheet lands in the section
`GET/POST/PATCH .../comments`	components/section-thread.tsx	the thread

A whole typed test can now be imported from one sheet on the Paper step, a Section column routing each row into that section's typist's drafts (`225ab39`); Done and reading do not change.

What it leans on

The response envelope and exception filter (controllers return data or throw `AppException`); `ZodBody` over the contracts schemas (`createAssignmentSchema`, `typistDoneSchema`, `sendBackSchema`, `workTimeSchema`); the four security decorators for the coarse gate, leaving the per-section rule to the services; `common/paper-edit.ts` for the one lock order (Test before Question); `common/edit-lock.ts` for the claim; `common/time/institute-day.ts` for due days; `@Audit` on question writes and on Done and the due-day change.

Cost, read not measured

No database was queried. By reading: `GET mine` batches its counts across the page (`apps/api/src/assignments/assignments.service.ts:374`); the progress grid materialises every unfrozen test's sections in memory and pages after the cross (`apps/api/src/assignments/section-progress.service.ts:144`), fine at thousands of rows; `GET /admin/sections/:t/:s` runs several statements (test, section, assignments, the scoped read, times, a Redis GET, a release-gap count) in parallel; a check or send-back is a load, a transaction with the test lock, and a full `workOf` re-read, because the re-read is the payload.

What is wrong with it

- Stale — `apps/admin/src/features/authoring/section-authoring.tsx:39` — three empty imports (`import {} from './use-delete-question'`, `./cross-test-warning`, `./notice-chip`, lines 39, 41, 42) import nothing.

- Stale — `apps/api/src/assignments/assignments.service.ts:555` — "Spec §11's convention" and `apps/api/src/assignments/section-thread.service.ts:31` "spec §9" cite specs that are not tracked in the repo.
- Correctness — `apps/api/src/assignments/assignments.controller.ts:89` and `:151` — assigning and removing an assignment carry no `@Audit`, while `changeDueDay` (`:100`) does; read, not verified whether another path records them.
- Correctness — `apps/api/src/questions/section-work.service.ts:150` — `spend` accepts time for any question in scope of a held seat regardless of whether it awaits the holder; the "only when it waits on you" rule lives only in the client (`work-clock.ts:5`), so a modified client can add time to any card (capped at 3600 per report).
- Dead — `apps/api/src/questions/section-work.service.ts:114` — `seatReplaced` is written `context.mine?.replacedAt !== null && context.mine !== null`; the second clause only exists to cancel the first when `mine` is null.

Importing in bulk

Where it lives

Path	What it holds
`apps/api/src/common/importing`	the half every importer shares: upload limit, sheet reading, CSV, date cells, the errors workbook, the stored-file key
`apps/api/src/imports`	the three roster importers (students, event candidates, program enrolment) and their run lifecycle
`apps/api/src/questions/question-import.service.ts`	the question importer: preview, review-window edits, remaps, commit
`apps/api/src/questions/question-import.ts`	the pure judgement of a question sheet, row by row
`apps/api/src/questions/question-import-cells.ts`	reading one row's cells into a `QuestionDraft`
`apps/api/src/questions/question-import-causes.ts`	folding per-row reasons into counted causes and fixable missing values
`apps/api/src/audit/audit.service.ts:225`	the only read-back of a run: `listImports`, `importFile`
`apps/admin/src/features/imports`	the four screens; `packages/app-kit/browser/use-import-screen.ts` is the shared preview-then-commit hook

Every bulk entry point is upload, review, then commit. Four things import: students, event candidates, program enrolments and questions. The roster importers and the question importer share the reading and the vocabulary but not the shape of a run. A roster has no review step, so its preview writes nothing and its commit takes the file again. A question sheet has a review window with per-row corrections, so its preview stores the file and opens a run, and its commit names that run.

Both re-read and re-plan at commit rather than accept a plan back from the client. A client that can send a plan can send any plan, and the database may have changed since the preview (a number registered, a question added).

The shared half

POST /IMPORTS/STUDENTS/PREVIEW — A FILE BECOMES A TABLE

1. The controller takes a multipart field named `file`; the class is `@Actors(ADMIN)` and the route needs `STUDENT_MANAGEMENT` READ
   `apps/api/src/imports/imports.controller.ts:54`
   → the file as an in-memory buffer
2. Multer enforces the byte ceiling while the body arrives (`importLimitBytes`, one file), so an oversize sheet never reaches memory
   `apps/api/src/common/importing/upload.ts:7`
   → a refusal before the handler runs
3. A missing file becomes a field error on `file`
   `apps/api/src/common/importing/upload.ts:24`
   → `VALIDATION_ERROR` with `fieldErrors.file`
4. Empty bytes are refused; the first four bytes are matched against `PK\x03\x04`. A non-zip with a NUL in the first 512 bytes, or a PDF header, is refused as not a spreadsheet; anything else is parsed as CSV
   `apps/api/src/common/importing/sheet-reader.ts:80`
   → a `CsvTable`, or a validation refusal
5. For a workbook the zip's central directory is read without inflating anything; a declared total over 200 MB is refused before ExcelJS loads it
   `apps/api/src/common/importing/sheet-reader.ts:103`
   → zip-bomb refusal
6. The sheet is chosen: the named one (questions ask for `Questions`), else the first visible one, else the first. Row 1 becomes headers (normalised) plus the labels as written
   `apps/api/src/common/importing/sheet-reader.ts:124`
   → `headers`, `labels`
7. Each later row is read cell by cell (rich text joined, a formula's result, a Date through `toIsoDate`, numbers with no exponent). Floating pictures are attached to the cell holding their top-left corner. An all-empty row is dropped; `line` is the sheet's own row number
   `apps/api/src/common/importing/sheet-reader.ts:146`
   → `CsvTable.rows[].{line, values, pictures?}`

CSV goes through the same table type (`parseCsvRows` at `apps/api/src/common/importing/csv.ts:11`, which strips a BOM and keeps the true starting line of a quoted multi-line field), so validation, line numbers and the preview are one code path.

`toIsoDate` (`apps/api/src/common/importing/date-cell.ts:22`) accepts ISO-like, named-month (either order), numeric day/month/year and an Excel serial. For an ambiguous numeric date it reads day first, because this institute writes it that way. A value nothing recognised comes back unchanged; the caller decides what that means.

`rowsWithErrors` (`apps/api/src/common/importing/rows-with-errors.ts:11`) builds the "rejected lines" workbook: the original columns, then an `Errors` column. A previously downloaded `Errors` column is replaced, not duplicated, so the fixed file can be uploaded again as it is. `refuseFileErrors` turns file-level errors into a refusal when there are no rows to hand back.

The stored file's extension follows its bytes (`importFileKey` at `apps/api/src/common/importing/import-log.ts:7`): `.csv` for a non-workbook, else `.xlsx`. `importFileContentType` reads the type back off the key, so a download's name and type agree.

The roster importers

The same two-request shape, three times: `preview` (READ, writes nothing), `errors` (READ, returns the refused lines as a workbook) and `commit` (WRITE). Each has a `template` GET generated from the column list in `packages/contracts/src/imports.ts`, so a sample can never document a

format the parser rejects.

Importer	Routes	Row action	Creates students?
Students	`/imports/students/*`	`create` / `update` / `skip`	yes
Event candidates	`/imports/events/:eventId/candidates/*`	`create` / `add` / `skip`	yes, as `NON_IACE`, reaching that event only
Program enrolment	`/imports/programs/:code/students/*`	`enrol` / `already` / `skip`	never

The event and the program must exist before anything is read (`eventCandidateTable` at `apps/api/src/imports/imports.service.ts:346` and `programStudentTable` at :356 run the lookup beside the file read), so a stale page cannot fill a deleted roster.

PLAN A STUDENT ROSTER (PREVIEW, ERRORS AND COMMIT ALL RUN THIS)

1. `contextFor` loads four things in parallel: active branches, active exam codes, active program codes, and only the students whose mobiles the file mentions (live, by `mobile IN (...)`)
   `apps/api/src/imports/imports.service.ts:392`
   → an `ImportContext`
2. `planRoster` refuses a whole file for being empty, missing a required column (named as the sample names it), or over `IMPORT_MAX_ROWS` (1000). On any of these zero rows are planned
   `apps/api/src/imports/student-import.ts:71`
   → `fileErrors[]`, `rows: []`
3. `readContact` : name through `personNameSchema` , mobile through `mobileSchema` , and a repeat of a number inside the file is "already on line N"
   `apps/api/src/imports/student-import.ts:112`
   → `mobile` , `fullName` , errors
4. Student type, branch (active branch by canonical name; blank is legal only for `NON_IACE` ; the type/branch pairing the admin screens refuse is refused here too), courses, exam and program codes against the loaded catalogs
   `apps/api/src/imports/student-import.ts:214`
   → per-row errors
5. The optional profile: a bad value is an error, a blank one is absent. A date of birth passes `toIsoDate` then `dobSchema` ; "not a date we can read" is said only when `toIsoDate` changed nothing
   `apps/api/src/imports/student-import.ts:281`
   → `StudentImportProfile`
6. The row is planned as what it leaves the student holding: the union of what they hold and what the sheet adds. A row granting no course, exam or program is refused
   `apps/api/src/imports/student-import.ts:355`
   → `action` , `errors`
7. `skip` on any error, else `update` if the mobile is known, else `create`
   `apps/api/src/imports/student-import.ts:177`
   → `StudentImportPlan { rows, summary, fileErrors }`

An import only adds access. A re-upload merges courses, exams and programs into what the student holds and never removes any. A blank name column means no opinion, not "clear the name". A row for an existing student with no branch disconnects the branch, and only a `NON_IACE` row can reach that state.

There is no PIN or password anywhere in the roster importers. A student signs in by an SMS code, so an imported student is just a row that exists.

POST /IMPORTS/STUDENTS/COMMIT — APPLYING A ROSTER

1. `STUDENT_MANAGEMENT` WRITE, `FileInterceptor` ; the caller id comes from `@CurrentUser()`
   `apps/api/src/imports/imports.controller.ts:74`
   → file + actor id
2. The file is re-read and re-planned against the live database; the client's earlier plan is not consulted
   `apps/api/src/imports/imports.service.ts:86`
   → a fresh plan
3. `withRun` opens the run first: an `ImportLog` row with `feature STUDENT` , `source SHEET` , `actorId` , `total` , `status PREVIEWED` , plus `errors.fileErrors` if any
   `apps/api/src/imports/imports.service.ts:479`
   → one `ImportLog` row
4. Inside the run's try, the bytes go to S3 under `imports/student/<logId>.<ext>` and `fileS3Key` is written back. A storage outage therefore closes the run FAILED instead of leaving it open
   `apps/api/src/imports/imports.service.ts:499`
   → S3 object, `fileS3Key`
5. Row loop, not in one transaction. A `skip` row is passed over; the rest go to `writeRow`
   `apps/api/src/imports/imports.service.ts:97`
   → per-row outcome
6. A new student is created. If the number was registered since the plan, the unique-violation is caught, the live holder is re-read, and the row becomes an update for that student
   `apps/api/src/imports/imports.service.ts:223`
   → `Student` (+ nested `StudentProfile`) row
7. An update is live-only (`deletedAt: null`) and runs with a guarded access-append statement in one `$transaction` : name, branch and profile, plus a single `UPDATE` that appends only the courses, exams and programs the student does not hold at that moment
   `apps/api/src/imports/imports.service.ts:255`
   → `Student` columns; arrays appended
8. If the student was erased between plan and write, the row is counted skipped and its reason is kept by sheet line
   `apps/api/src/imports/imports.service.ts:102`
   → `outcome.rowErrors[]`

9. A throw anywhere closes the run FAILED with `failed = total − created − updated − skipped`, the error message and any row errors, then rethrows
   `apps/api/src/imports/imports.service.ts:464`
   → `ImportLog.status = FAILED`
10. On success `closeRun` writes one `RowActionLog` per touched student (a failure to record these is logged and swallowed, since the writes already happened), then the counts, `status COMMITTED` and `finishedAt`
    `apps/api/src/imports/imports.service.ts:506`
    → `RowActionLog` rows, closed `ImportLog`
11. The response is the plan summary plus `created`, `updated`, `skipped`, where `skipped` is invalid rows plus rows erased mid-run
    `apps/api/src/imports/imports.service.ts:113`
    → `StudentImportResult`

The other two commits reuse `withRun`:

- **Event candidates** (`apps/api/src/imports/imports.service.ts:128`). A number already on file only joins the roster; nothing on that student changes. An unknown number becomes a `NON_IACE` account (`createCandidate`, :210), with the same lost-race fallback. The ids are handed to `events.addCandidates` in a `finally`, so an account a failing run created is not left on no event. A number belonging to a deleted student is refused with its own message.
- **Program enrolment** (`apps/api/src/imports/imports.service.ts:180`). One statement for the whole file: `UPDATE "Student" SET programs = array_append(...) WHERE id = ANY(...) AND NOT (programs @> ARRAY[code]) RETURNING id`. The guard means a student enrolled since the preview cannot hold the code twice. A student already carrying the code is `already`, not an error, because re-uploading last term's list is normal.

`IMPORT_MAX_ROWS` is 1000 for rosters. The student commit is one round trip or more per non-skipped row, with no enclosing transaction, so a failure part-way leaves the earlier rows written, and the run's counts say how far it got.

The question importer

A question sheet is judged by the same function at preview, during review and at commit (`planQuestionImport`, `apps/api/src/questions/question-import.ts:111`). Every row becomes the same `QuestionDraft` the single-question form posts and goes through the same `validateQuestion`, so what makes a question valid lives in the core and the sheet only reports what it alone can get wrong (an unreadable number, a subject not in the bank).

Where a sheet may land, which fixes its row ceiling and its judging scope:

Target	Entry	Stored as `ImportLog.target`	Row ceiling	Permission
The bank	`POST /imports/questions/preview`	`bank`	100 (`QUESTION_IMPORT_MAX_ROWS`)	`QUESTION_MANAGEMENT`
One section of a typed test	`POST /admin/sections/:testId/:sectionId/import/preview`	`<testId>/<baseConfigSectionId>`	50 (`SECTION_IMPORT_MAX_ROWS`)	`QUESTION_AUTHORING`, and the caller must hold that section's open typing row
A whole typed test	`POST /admin/tests/:testId/import/preview`	`test:<testId>`	50 per section	`TEST_MANAGEMENT` WRITE

The first two controllers are `apps/api/src/questions/question-import.controller.ts:72` and `apps/api/src/questions/section-work.controller.ts:88`; the third is `TestImportController` at `apps/api/src/questions/question-import.controller.ts:161`. A section run is held to the section's subject and to the kind of question its paper can draw (choice, typing or descriptive). A whole-test sheet adds a Section column; each row is judged by the section it names and written as that section's typist's draft. The sheet carries up to six options per language (`MCQ_OPTION_MAX`).

POST /IMPORTS/QUESTIONS/PREVIEW — OPENING A RUN

1. The bank route needs `QUESTION_MANAGEMENT` READ and the previewer's id; a whole-test target first checks that the test is typed and not offered
   `apps/api/src/questions/question-import.service.ts:109`
   → scope chosen
2. Read, resolve, judge: the sheet is read (preferring a sheet named `Questions`), the taxonomy loaded, the scope resolved, and the table planned twice. The first pass only harvests stem hashes and codes, then one `findMany` asks the bank which of them exist
   `apps/api/src/questions/question-import.service.ts:420`
   → `QuestionImportPlanning`
3. If the file itself cannot be planned (no rows, a missing required column, too many rows) the plan is returned with an empty `importLogId` and nothing is stored
   `apps/api/src/questions/question-import.service.ts:113`
   → no `ImportLog`, no S3 object
4. Otherwise the run is created `PREVIEWED` with the target and total
   `apps/api/src/questions/question-import.service.ts:115`
   → `ImportLog` row
5. The sheet is uploaded under `imports/question/<logId>.<ext>` and the key written back
   `apps/api/src/questions/question-import.service.ts:128`
   → `fileS3Key`
6. Pictures are stored now, keyed by content hash, because the review window must draw them before anything is written
   `apps/api/src/questions/question-import.service.ts:132`
   → S3 objects
7. The plan goes to the client without the drafts, hashes and pictures
   `apps/api/src/questions/question-import.ts:459`
   → `QuestionImportPlan { importLogId, rows, summary, fileErrors, causes? }`

Row outcomes (`QuestionImportAction`): `create`, `duplicate` (a stem already in the bank, or earlier in the same file, skipped without error), `skip` (has issues) and `left_out` (set aside in the review window). A duplicate names its source in `duplicateOf`: a question id for the bank, or `line N` for the same file. A row only claims its stem and its question code if it would really be written, so a skipped or left-out row does not turn the next good copy into a duplicate. In a whole-test sheet a section's room is spent only by a row that would be created, and a row past the room is skipped with that reason and never saved.

The review window

The preview's rows open on the authoring page, one card per row. Everything the admin does there is held against the run, never in the bank, and every action answers with the whole sheet judged again.

Action	Route	Held in
Correct a row	`PUT /imports/questions/:importLogId/rows/:line`	`ImportRowEdit.draft` (image `src` stripped to keys)
Leave a row out, or bring it back	`PUT .../rows/:line/left-out`	`ImportRowEdit.leftOut`
Read a missing subject, topic or difficulty as one the bank holds	`POST .../:importLogId/remap`	`ImportLog.remaps`
Judge again, after a subject was created	`POST .../:importLogId/rejudge`	nothing
Fetch the rows as drafts	`GET .../:importLogId/drafts`	nothing

All of them go through `openRun` (`apps/api/src/questions/question-import.service.ts:372`), which refuses a run that is not a question run, has no stored file, belongs to another admin (a super admin included), or is already `COMMITTED`. `changeRow` (:292) rejects a line the file does not have.

`causesOf` (`apps/api/src/questions/question-import-causes.ts:31`) folds the skipped rows' issues into one line per reason with its count; a reason naming a subject, topic or difficulty the bank lacks carries the three closest bank values by shared words. The screen then offers "Use X" (a remap), "Choose another", or, for someone with `QUESTION_MANAGEMENT` WRITE, "Create X" (creates the subject or topic, then rejudges). A remap target is checked to exist, and a topic must sit under its subject (`assertRemapTarget`, :356).

POST /IMPORTS/QUESTIONS/COMMIT — WRITING THE QUESTIONS

1. Body `{ importLogId }` through `ZodBody`; `QUESTION_MANAGEMENT` WRITE. The section route also needs `QUESTION_AUTHORING` WRITE and re-checks the typing row; the test route needs `TEST_MANAGEMENT` WRITE
   `apps/api/src/questions/question-import.controller.ts:149`
   → run id + actor
2. `openRun` re-reads the stored file; the actor must be the previewer
   `apps/api/src/questions/question-import.service.ts:158`
   → file bytes
3. The run's target must equal where this commit is going. A null target (a run from before the column) commits anywhere
   `apps/api/src/questions/question-import.service.ts:160`
   → else `CONFLICT` "previewed for somewhere else"
4. The file is re-planned with the stored overlay (edits, left-out lines) and remaps laid over it. Only `create` rows with a draft are kept
   `apps/api/src/questions/question-import.service.ts:164`
   → `creatable[]`
5. Pictures are uploaded before any row is written
   `apps/api/src/questions/question-import.service.ts:178`
   → S3 objects
6. One interactive `$transaction` (`TX_LIMITS.BULK`). It first asserts each target job (the typist's assignment) is still open
   `apps/api/src/questions/question-import.service.ts:181`
   → open-job guard
7. It claims the run with `updateMany ... status != COMMITTED`, setting counts and `COMMITTED`. Of two concurrent commits the second finds it taken and throws `CONFLICT` "already imported"
   `apps/api/src/questions/question-import.service.ts:187`
   → `ImportLog` closed
8. Per row, three statements: `Question`, its version 1, then the pointer `currentVersionId`. Every question carries the `imported` tag. A whole-test row takes its own section's assignment id
   `apps/api/src/questions/question-import.service.ts:202`
   → `Question`, `QuestionVersion` rows
9. A throw rolls the whole transaction back (so the claim too); the catch then closes the run `FAILED` with the rows it never reached and the message, unless another commit already imported it
   `apps/api/src/questions/question-import.service.ts:221`
   → `status FAILED`, `errors.message`
10. After commit, one `RowActionLog` per created question; a failure here is logged and swallowed
    `apps/api/src/questions/question-import.service.ts:226`
    → `RowActionLog` rows
11. The response is the plan summary plus `created` and `skipped = total − created`
    `apps/api/src/questions/question-import.service.ts:170`
    → `QuestionImportResult`

A `FAILED` run can be previewed-through and committed again: `openRun` refuses only `COMMITTED`.

The question commit is all-or-nothing, the opposite of the roster commit. A bad row never gets here because the plan removed it, and the 100-row ceiling keeps the transaction at a few hundred statements.

Runs and their states

`ImportLog` (`prisma/schema.prisma:1044`) is the one record of a run; `status` is a plain string whose vocabulary is `IMPORT_LOG_STATUS` in `packages/contracts/src/imports.ts:21`.

Status	Set by	Meaning
`PREVIEWED`	a question preview; a roster commit when it opens its run	the file is stored and planned; for a question run nothing is written yet
`COMMITTED`	the end of a commit	the writes happened; `finishedAt` is set
`FAILED`	a commit that threw	`failed` counts the rows never reached; `errors.message` says why

A roster run is opened at commit, not at preview, so it passes through `PREVIEWED` only for the instant before its file is stored. The roster preview writes nothing at all, because a row for an abandoned preview is storage nothing ever resolves. A question preview does open a run, because the review window needs the stored file and the corrections.

Column	Holds
`feature`	`STUDENT` for all three rosters, `QUESTION` for questions
`source`	always `SHEET` here (`INDIVIDUAL`, `SCRIPT`, `SELF_SIGNUP` exist unused by this module)
`target`	question runs only (`bank`, `test:<id>`, `<testId>/<sectionId>`); null otherwise and before the column existed
`remaps`	a question run's `{ subject, topic, difficulty }` readings of missing values
`fileS3Key`	`imports/<feature>/<logId>.<xlsx csv>`
`total` `created` `updated` `skipped` `failed`	the run's counts; `skipped` is every row the run chose not to write, `failed` only what a dead run never reached
`errors`	`{ fileErrors?, rowErrors?: {line, error}[], message? }`

`ImportRowEdit` (`prisma/schema.prisma:1071`) is keyed `(importLogId, line)` and cascades with its run. `RowActionLog` rows carry `importLogId` and no `changed` payload.

Audit read-back: `GET /admin/audit/imports` pages runs newest first (`apps/api/src/audit/audit.service.ts:225`), scoped to the admin's own runs unless they are a super admin. `GET /admin/audit/imports/:id/file` streams the stored sheet with the same scope (`apps/api/src/audit/audit.service.ts:254`); it is streamed, never a signed link, because the file is full of student data. The summary now carries `target`.

The screens

All in `apps/admin/src/features/imports`. `useImportScreen` (`packages/app-kit/browser/use-import-screen.ts:36`) owns choose, preview, commit and template download. Choosing a file previews it at once, and a late answer from an earlier file is ignored. `canCommit` needs a plan with at least one write and no completed commit, so a committed run on screen cannot be pressed twice. `useErrorRows` offers the refused-lines download only while there are invalid rows. `PreviewRows` is the shared table body.

Route	File
`/students/import`	`apps/admin/src/features/imports/import-students.tsx`
`/cohorts/events/:id/import`	`apps/admin/src/features/imports/import-event-candidates.tsx`
`/cohorts/programs/:code/import`	`apps/admin/src/features/imports/import-program-students.tsx`
`/questions/import`, `/tests/:testId/import`, the authoring section import	`apps/admin/src/features/imports/import-questions.tsx` (one page, the target chosen from the URL)
`/audit/imports`	`apps/admin/src/features/admins/audit.tsx:285`

The question page swaps into `ImportWorkspace` (`apps/admin/src/features/authoring/import-workspace.tsx`) once a preview has rows. If cards hold unsaved edits, leaving for the preview or pressing Import asks first, because neither has them.

What is wrong with it

- `packages/contracts/src/imports.ts:204` — Stale. The comment says `IMPORT_MAX_ROWS` is "bounded by argon2" per new student; roster imports no longer hash anything.
- `apps/api/src/imports/imports.service.ts:442` — Stale. The banner says "both importers"; there are three, and event candidates may also write nothing to `Student` for an existing number.
- `docs/02-domain-rules.md` section 12 — Stale. Pictures are said to be "uploaded only on commit", while the same section later says they are stored at preview; the code stores them at preview (`apps/api/src/questions/question-import.service.ts:132`).
- `apps/api/src/questions/question-import.service.ts:115` — Correctness. An abandoned question preview leaves an `ImportLog` row, a stored sheet, its pictures and any `ImportRowEdit` rows; nothing in `apps/api/src` ever deletes them, and the run stays `PREVIEWED` in the audit list forever.
- `apps/api/src/imports/imports.service.ts:206` — Correctness. The program response and the run take `skipped` from before the write; if the guarded `UPDATE` enrols fewer students than planned (enrolled meanwhile), those rows appear in neither `enrolled` nor `skipped`.
- `apps/admin/src/features/admins/audit.tsx:96` — Correctness. Any question run whose target is not `bank` is labelled "Test section", including a whole-test run (`test:<id>`).

- `apps/api/src/questions/question-import.controller.ts:72` — Correctness. A READ grant on `QUESTION_MANAGEMENT` can open question runs (an `ImportLog` row and an S3 upload) though the comment says it "writes no questions". It writes storage; read, not tested against any quota.
- `apps/api/src/questions/question-import.service.ts:160` — Correctness. A run with a null target (previewed before the column) commits anywhere, including into a section's typist; deliberate and documented, but unbounded in time and never backfilled.

Part III — Who may do what

Signing in

Three questions get confused, and this part keeps them apart. *Who are you* is authentication: a token, a session in Redis, a one-time code. *What kind of actor are you* is a guard: student or admin, and for an admin which feature keys they hold. *What may you reach* is resolution, the next chapters. This chapter is the first.

Where it lives

`apps/api/src/auth/` holds the module; the route vocabulary is in `apps/api/src/common/security/`, the limits in `apps/api/src/common/throttling/`, the keyspace in `apps/api/src/redis/redis.keys.ts`, the wire types in `packages/contracts/src/auth.ts`.

File	Role
`apps/api/src/auth/auth.controller.ts`	Seven `/auth/*` routes. Only `logout` and `me` are not `@Public()`.
`apps/api/src/auth/auth.service.ts`	Orchestrates every flow. Owns no table; it reads `Student` and `Admin`.
`apps/api/src/auth/otp/otp.service.ts`	Mints the code, claims the cooldown, three daily caps, delivery, verify.
`apps/api/src/auth/session.service.ts`	Redis sessions, rotation with replay detection, one session per app kind, revocation.
`apps/api/src/auth/token.service.ts`	Access and refresh JWTs under two separate secrets.
`apps/api/src/auth/device.ts`	Reads `x-client` and `x-device-name` into a `DeviceContext`.
`apps/api/src/auth/guards/jwt-auth.guard.ts`	The global guard; builds `request.user`.
`apps/api/src/auth/student-sessions.listener.ts`	Revokes every session on deactivation or a changed mobile.
`apps/api/src/common/messaging/`	The sender seam: console, SMS, email, WhatsApp (Interakt), and the router in front.

What it is for

Two unrelated identity tables sign in in two unrelated ways, and both end in the same thing: a short access token, a rotating refresh token, and a session held in Redis. There is no password column anywhere.

- **A student has no password and no PIN.** Every sign-in is a code sent to their mobile, and the first time a number does it is the signup. There is no "have you an account?" fork on the API, the web portal or the phone.
- **An admin** signs in by a code sent to their email, every time. Admins are created by a super admin; there is no self-signup.
- **What keeps a code rare** is not a PIN but the session rule: one web session and one app session per student, thirty days long, refreshed in place. A device asks once and stays signed in.

Flow: a student asks for a code

POST /AUTH/STUDENT/OTP/REQUEST — ASKING FOR A CODE

1. `@Public()` skips the JWT guard; only the `otpRequest` throttler applies, counted per address
   `apps/api/src/auth/auth.controller.ts:33`
   → 120 per minute on `ip:<ip>` (`RATE_LIMIT_OTP_REQUEST_PER_MIN`); a branch shares one address, so the default is generous
2. Body validated by `requestStudentOtpSchema`: `mobile`, optional `channel`
   `apps/api/src/auth/auth.controller.ts:37`
   → 10-digit Indian mobile, normalised
3. One read: does a live student hold this number?
   `apps/api/src/auth/auth.service.ts:63`
   → `holdsAccount` true or false; it only picks which day's budget pays, never the answer
4. The resend wait is claimed first, in one `SET NX`
   `apps/api/src/auth/otp/otp.service.ts:84`
   → `otp:cooldown:student:<mobile>` = `1`, TTL `OTP_RESEND_COOLDOWN_SEC` (45s); a lost claim throws `RATE_LIMITED` with `details.retryAfterSec`
5. Three daily counters, each INCR'd and given a 24h TTL if it has none
   `apps/api/src/auth/otp/otp.service.ts:98`
   → per mobile (`OTP_MAX_PER_DAY`, 10), per address (`OTP_MAX_PER_DAY_PER_IP`, 3000), and the platform budget; any over its cap throws `RATE_LIMITED`
6. A refused day gives the cooldown claim back
   `apps/api/src/auth/otp/otp.service.ts:103`
   → `otp:cooldown:…` deleted, so the refusal leaves no 45 seconds behind it
7. A CSPRNG code is minted
   `apps/api/src/auth/otp/otp.service.ts:281`
   → `randomInt` over `OTP_LENGTH` digits (6)
8. Delivered on one channel
   `apps/api/src/auth/otp/otp.service.ts:194`
   → SMS today; a thrown send becomes `SERVICE_UNAVAILABLE`, counters and cooldown deliberately not refunded
9. Stored only after it has left, then the attempt count reset
   `apps/api/src/auth/otp/otp.service.ts:129`
   → `otp:student:<mobile>` = `{codeHash, createdAt}` (HMAC-SHA256 under `JWT_ACCESS_SECRET`), TTL `OTP_TTL_SEC` (300s); `otp:attempts:student:<mobile>` deleted

10. Answer returned
`apps/api/src/auth/otp/otp.service.ts:133`
→ `{sent, expiresInSec, resendAfterSec, codeLength, channel?, otherChannel?, devCode?}`; `devCode` only under `OTP_SENDER=console` in development

The budget split matters. A mobile that holds no account spends the **signup** budget (`OTP_SIGNUP_DAILY_BUDGET_PAISE`, 50,000p by default) and one that does spends the **student** budget (`OTP_GLOBAL_DAILY_BUDGET_PAISE`, 300,000p). Both are divided by `NOTIFICATION_COST_SMS_PAISE` (18p, floor of 1) to give a send count, about 16,666 and 2,777 a day. Every sign-in is a code now, so a stranger typing fresh numbers must not be able to spend the budget the existing students sign in on (`apps/api/src/auth/otp/otp.service.ts:56`).

> **Note** The code is stored claim-first, written-last on purpose. The cooldown is taken before anything is spent so two requests landing together send one code; the code is stored only once it has left, so a send that failed never replaces a code that arrived.

Flow: a student gives the code back

POST /AUTH/STUDENT/OTP/VERIFY — SIGNING IN, OR SIGNING UP

1. Auth throttler, per address
`apps/api/src/auth/auth.controller.ts:45`
→ 120 per minute on `ip:<ip>` (`RATE_LIMIT_AUTH_PER_MIN`)
2. Pending codes read: the sent one and the desk one, if any
`apps/api/src/auth/otp/otp.service.ts:243`
→ both absent throws `OTP_EXPIRED`
3. HMAC compared in constant time against either
`apps/api/src/auth/otp/otp.service.ts:253`
→ a miss INCRs `otp:attempts:student:<mobile>` (TTL set from the code's own), `OTP_INVALID` with `details.attemptsRemaining`
4. At `OTP_MAX_VERIFY_ATTEMPTS` (5) the challenge is burnt
`apps/api/src/auth/otp/otp.service.ts:261`
→ code, desk code, attempts and cooldown keys deleted; `RATE_LIMITED` "Request a new code"
5. Single use: one `DEL` over both keys decides
`apps/api/src/auth/otp/otp.service.ts:276`
→ a verification whose delete removed nothing reads as `OTP_EXPIRED`; two simultaneous right codes open one session, not two
6. Live student by mobile, or sign up
`apps/api/src/auth/auth.service.ts:86`
→ `findFirst({mobile, deletedAt: null})`, else a `create`
7. Signup creates the row
`apps/api/src/auth/auth.service.ts:208`
→ `Student{mobile, studentType: NON_IACE}` and `STUDENT_SIGNED_UP` emitted; a unique-violation from a raced signup returns the winner's account instead
8. Deactivated account refused
`apps/api/src/auth/auth.service.ts:87`
→ `FORBIDDEN` "This account has been deactivated"; no session
9. Identity built, tokens issued
`apps/api/src/auth/auth.service.ts:222`
→ fresh `randomUUID()` session id; access `{sub, actor, sid}` under `JWT_ACCESS_SECRET` (15m), refresh adds a `jti` under `JWT_REFRESH_SECRET` (30d)
10. Older session of the same kind retired
`apps/api/src/auth/session.service.ts:205`
→ tombstone `session-replaced:student:<id>:<sid>` = `{replacedBy}` written first, then the old session deleted
11. Session written
`apps/api/src/auth/session.service.ts:65`
→ `session:student:<id>:<sid>` = `{refreshTokenHash, deviceName, ip, userAgent, createdAt, lastSeenAt, client}`, TTL 30d; `sessions:student:<id>` SADD + EXPIRE
12. Response
`apps/api/src/auth/auth.service.ts:94`
→ `{tokens: {accessToken, refreshToken, expiresInSec}, identity}`

Uniqueness of a live mobile is a hand-written partial index in the migrations that Prisma cannot express, which is why the lookups are `findFirst` rather than `findUnique`. The new account is `NON_IACE` because a student who signed themselves up is outside the institute; an institute student already exists by roster import, and signing in finds that row.

The desk code

A student whose code does not arrive can be read one. `POST /admin/students/:id/desk-code` (`apps/api/src/students/students.controller.ts:157`, `STUDENT_MANAGEMENT` write, audited) calls `issueStudentDeskCode` (`apps/api/src/auth/auth.service.ts:98`), which refuses a missing student (`NOT_FOUND`) or a deactivated one (`CONFLICT`) and then writes `otp:desk:<mobile>` with a five-minute TTL (`apps/api/src/auth/otp/otp.service.ts:231`). It is never sent. It lives beside the sent code, so a student pressing Resend does not void it; `verify` accepts either, and one `DEL` over both keys makes it single use. Issuing resets the attempt count so the desk code gets its own five guesses. Whoever is told the code signs in as that student, which is why the admin screen confirms and the audit row records `deskCode: issued`.

Flow: an admin signs in

POST /AUTH/ADMIN/OTP/REQUEST, THEN /VERIFY

1. Auth throttler, per address
`apps/api/src/auth/auth.controller.ts:58`
→ 120 per minute on `ip:<ip>`

2. Admin looked up by email
   `apps/api/src/auth/auth.service.ts:114`
   → unknown email throws `ADMIN_NOT_REGISTERED` (404) with `fieldErrors.email`; deliberately says so, because a silent "sent" left people waiting for a code that never comes
3. Same cooldown, same code store, no daily counters
   `apps/api/src/auth/otp/otp.service.ts:96`
   → `otp:admin:<email>` TTL 300s; only a student's send is metered because email costs nothing
4. Sent by email
   `apps/api/src/auth/otp/otp.service.ts:211`
   → Gmail through nodemailer; no channel is named in the answer
5. Verify: same attempt cap, same single-use delete
   `apps/api/src/auth/auth.service.ts:130`
   → `OTP_INVALID`, `OTP_EXPIRED`, or burnt at five misses
6. Admin re-read and identity built
   `apps/api/src/auth/auth.service.ts:133`
   → deliberately not gated on `isActive`: a deactivated admin signs in and can do nothing, and the screen can say so
7. Tokens issued, session written
   `apps/api/src/auth/auth.service.ts:137`
   → `session:admin:<id>:<sid>` TTL 30d; the one-per-kind rule is a student's only, so an admin may hold many sessions

Flow: a request on a protected route

Guards run in registration order (`apps/api/src/app.module.ts:89`), inside the request-id and audit middleware.

1. `JwtAuthGuard` (`apps/api/src/auth/guards/jwt-auth.guard.ts:20`) lets `@Public()` routes through, then rejects a missing or non-Bearer header, a bad or expired signature, and claims that fail `accessTokenClaimsSchema`. It then asks Redis whether the session still exists (`:33`). A gone session reads the tombstone: `SESSION_REPLACED` if a newer sign-in of the kind replaced it, otherwise `UNAUTHENTICATED`. An admin's grants are read from Postgres on every request, never from the token, so a revoked permission bites on the next call (`:38`); a student carries none. It sets `request.user = {id, actor, sessionId, isSuperAdmin, isActive, permissions}`.
2. `ActorGuard`, `FeaturePermissionGuard`, `SuperAdminGuard` (`apps/api/src/auth/guards/`) no-op without their decorators. Feature keys are code-owned in `packages/contracts/src/admins.ts`.
3. `ThrottlerGuard` runs last, so a signed-in caller counts as `<actor>:<id>` and only anonymous traffic shares `ip:<ip>` (`apps/api/src/common/throttling/rate-limit.ts:12`). Windows are one minute in Redis; a Redis failure answers allowed rather than blocking a hall mid-exam.

Flow: refresh and revoke

POST /AUTH/REFRESH — ROTATING THE PAIR

1. Auth throttler; body is `{refreshToken}` only
   `apps/api/src/auth/auth.controller.ts:81`
   → 120 per minute per address
2. Verified under `JWT_REFRESH_SECRET`, narrowed to `{sub, actor, sid}`
   `apps/api/src/auth/token.service.ts:63`
   → expired and tampered are one answer, `UNAUTHENTICATED`
3. Account still there? One column for a student, `identityOf` for an admin
   `apps/api/src/auth/auth.service.ts:261`
   → a gone or deactivated account throws `UNAUTHENTICATED`
4. Next refresh token signed with a fresh `jti`
   `apps/api/src/auth/token.service.ts:46`
   → new 30d token
5. Compare-and-swap rotation, three tries
   `apps/api/src/auth/session.service.ts:79`
   → a missing session goes through `throwEnded`; a token the session never held, replayed outside the 60s grace window, revokes the session and throws `UNAUTHENTICATED`
6. A retry inside `REFRESH_RETRY_GRACE_SEC` (60s) is accepted
   `apps/api/src/auth/session.service.ts:244`
   → the consumed hash is kept as a sibling (capped at 8) so two tabs sharing a token both stay valid until the next rotation
7. Three lost rounds
   `apps/api/src/auth/session.service.ts:116`
   → `CONFLICT`, not a sign-out; the client waits and asks again
8. Session rewritten, index re-added
   `apps/api/src/auth/session.service.ts:107`
   → same key, TTL 30d again, `sessions:<actor>:<id>` re-SADD'd and re-EXPIRE'd
9. Access token re-signed
   `apps/api/src/auth/auth.service.ts:165`
   → `{accessToken, refreshToken, expiresInSec}`

Revocation has five callers, all in `SessionService` or its listeners:

Trigger	Call	Scope
`POST /auth/logout`	`revoke` (`apps/api/src/auth/auth.service.ts:175`)	this device only

Trigger	Call	Scope
`DELETE /me/sessions/:id`	`signOutOther` (`apps/api/src/auth/session.service.ts:173`)	one other device; refused for the asking device and for a session that is not theirs
Refresh replay	`revoke` (`apps/api/src/auth/session.service.ts:98`)	that session
A newer sign-in of the same kind	`replaceSameKind` (`apps/api/src/auth/session.service.ts:199`)	the old session, with a tombstone saying who replaced it
`STUDENT_DEACTIVATED`, `STUDENT_MOBILE_CHANGED`, `ADMIN_DEACTIVATED`	`revokeAll` through the two listeners	every session, in one script so a racing sign-in lands wholly before or after

`GET /me/sessions` lists a student's live devices, newest activity first, with the asking device marked. A mobile change revokes every session because operators recycle numbers: the old holder must not stay signed in.

Channels and senders

`OTP_SENDER` selects the provider for every message kind, not only codes (the name is historical, `apps/api/src/common/messaging/messaging.module.ts:19`). `console` is refused in production. `sms` and `whatsapp` both require `SMS_PROVIDER_URL` and, because an admin signs in by email, `MAIL_USER` and `MAIL_PASSWORD`; `whatsapp` additionally needs the Interakt key and an OTP template, checked at boot. The router sends SMS to the aggregator (DLT template id plus variables, an 8s timeout), email through Gmail, and WhatsApp through Interakt only when its key is set.

WhatsApp is switched off for sign-in. `OTP_WHATSAPP_ENABLED` defaults to false (`apps/api/src/config/env.schema.ts:100`), and `studentChannels` returns SMS alone whatever `OTP_SENDER` says and whatever channel the client asks for (`apps/api/src/auth/otp/otp.service.ts:298`). The code path is kept intact. Switched on, a `whatsapp` deployment offers WhatsApp first (it is billed on delivery) with SMS behind it; a failed WhatsApp send falls back to SMS at once (`:222`); a student who asks for the other channel gets it; and `console` mode offers both so the choice can be walked through with no provider. The answer names the channel used and the one other that may be asked for, and the code screen turns its Resend button into that choice after the countdown. A code is never sent on two channels at once. Only sign-in codes are gated; an announcement can still choose WhatsApp.

Device kind and what the client does

`deviceFrom` (`apps/api/src/auth/device.ts:48`) reads `x-client` (`WEB` or `MOBILE`) and `x-device-name`. A web session with no name gets a label parsed from the user agent ("Chrome on Windows"). The kind is what the one-session-per-kind rule keys on; a session from before kinds were kept has `client: null` and is replaced by either kind.

On the clients, `packages/contracts/src/client/core.ts:175` runs one shared refresh per burst with a timeout and a back-off schedule (`:361`), and a refusal ends the session. Only an `UNAUTHENTICATED` answer triggers it (`:233`). `packages/app-kit/src/create-auth.tsx` waits four seconds for the server on sign-out and clears the device regardless (`:57`). The browser token store tells other tabs of a sign-out or sign-in through the storage event and leaves the reason beside the emptied key (`packages/app-kit/src/token-store.ts:16`); a tab that cannot reach the server shows Retry rather than sign-in (`packages/app-kit/src/create-auth.tsx:193`). The code screen disables its three buttons while any request is in flight.

Error vocabulary

Code	Status	Meaning here
`OTP_INVALID`	401	wrong code; `details.attemptsRemaining`
`OTP_EXPIRED`	401	no code pending, expired, or already spent (one answer for all three)
`RATE_LIMITED`	429	cooldown (`details.retryAfterSec`), a daily cap, a paused budget, or five wrong guesses
`ADMIN_NOT_REGISTERED`	404	admin request for an unknown email
`SESSION_REPLACED`	401	a newer sign-in of the same kind ended this session
`UNAUTHENTICATED`	401	missing, bad or expired token; session gone; account gone
`SERVICE_UNAVAILABLE`	503	the code could not be sent

Metrics

`auth_attempts_total` counts `ok`, `bad_code` and `deactivated` (`apps/api/src/common/metrics/metrics.service.ts:22`): a rising `bad_code` across many mobiles is guessing, against one mobile it is a code that never arrived. `countOtpSend` records `sent` and each refusal (`refused_mobile_daily`, `refused_ip_daily`, `refused_budget`, `refused_signup_budget`).

What is wrong with it

- `apps/api/src/auth/auth.service.ts:30` — Stale. The "Owns no table" doc comment sits on `DEACTIVATED_MESSAGE`, not on the class it describes.
- `apps/api/src/common/throttling/rate-limits.ts:23` — Stale. `AuthRateLimit` is documented as "OTP verify, PIN and refresh"; there is no PIN route.
- `apps/api/src/common/threadpool.ts:6` — Stale. The floor is explained by "PIN hashing shared the pool"; argon2 is gone.
- `packages/contracts/src/client/core.ts:233` — Stale. The comment says refreshing "on a wrong PIN hides the real code"; it now means a wrong code.
- `apps/api/src/auth/auth.service.ts:63` — Correctness. A request for a deactivated student's number still sends a paid SMS, because `findFirst` does not check `isActive`; the refusal only comes at verify (`:87`).

- `apps/api/src/common/messaging/messaging.module.ts:19` — Stale. The env var is named `OTP_SENDER` yet selects for every kind and channel; the comment admits it, the name still misleads.

What a student may reach

Where it lives

Path	Role
`apps/api/src/access/access-resolver.service.ts`	The one reach rule. Owns the held shared catalog and reads the Redis epoch.
`apps/api/src/access/access-cache.listener.ts`	Two domain events become one `INCR`.
`apps/api/src/access/test-series.service.ts`	`TestSeries` CRUD, the branch list, the kind pairing rules. Emits the bust.
`apps/api/src/access/student-grants.service.ts`	Owns `StudentGrant`: grant, revoke, export, the admin "what this student reaches" read.
`apps/api/src/access/programs.service.ts`	`Program` CRUD (super-admin writes).
`apps/api/src/access/access.controller.ts`	Four admin controllers (programs, series, grants, a student's reached series). No student route lives here.
`apps/api/src/me/me.controller.ts:74`	`GET /me/catalog`, the only student list route.
`apps/api/src/attempts/attempts.controller.ts`	`POST /me/tests/:testId/attempt`, `GET .../brief`, `GET .../paper`: the three routes that ask the resolver about ONE test.
`packages/contracts/src/access.ts`	`testIsOpen`, `START_GRACE_MS`, `TEST_SHUT`, the catalog schemas, `STUDENT_SERIES_SOURCE`.
`prisma/schema.prisma:946-1040`	`Program`, `TestSeries`, `StudentGrant`, `Event`, `EventCandidate`, `TestProgramUnlock`.

What it is for

A test reaches a student only through a series. A series reaches a student by one automatic route, chosen by its `kind`, plus a deliberate per-student grant that adds a route on top of any kind. The resolver holds that rule in one function, `seriesSources` (`apps/api/src/access/access-resolver.service.ts:404`), and answers three questions with it:

- what a student may see: the catalog;
- what they may read about: the brief (`reachableTest`);
- what they may begin: the start guard (`assertCanStart`).

It also answers backwards, for notifications, absentee reports and the admin's reach counts: who does one series reach (`studentsReaching`, `audienceCount`). Nothing a student may do is stored. It is derived on every read.

The four kinds and the grant

Kind	Reaches a student when	Source tag
`FREE`	always, if the student is live	`FREE`
`STANDARD`	`currentBranchId` is in `branchIds` AND the stage's `exam.course` is in `enrolledCourses`	`COURSE`
`PROGRAM`	`programCode` is in `Student.programs` (branch is not asked)	`PROGRAM`
`EVENT`	the student is an `EventCandidate` of the series' event (branch is not asked)	`EVENT`
any, plus `StudentGrant`	a `(studentId, testSeriesId)` row exists	`GRANT`, appended to whatever else matched

`seriesSources` returns a list, so a student can be reached by a kind AND a grant at once; the admin screen shows every route. `Student.enrolledExams` is not consulted, only `enrolledCourses`. The mirror, `audienceOf` (`:432`), expresses the same rule as a Prisma `where` for the backwards question, and its comment tells the editor to change the two together. A `FREE` audience has no `OR` at all because Prisma reads an empty `OR` member as matching nobody.

Three more facts decide whether a reached test is usable:

- A student who is inactive or soft-deleted is absent: every student-facing read answers as if they did not exist (`isPresent`, `:318`).
- `isTestBlocked` leaves the catalog fully listed but nothing startable.
- A **test** reaches nobody unless its `status` is `ACTIVE`. The shared select filters `tests` on `TEST_STATUS.ACTIVE` (`:46`). There is no series-level switch: the column was dropped in `aaff7f5`, and a test is switched from the series' test list instead. A grant does not override test status.

The catalog resolve

GET /ME/CATALOG — WHAT THE STUDENT SEES

1. A student screen mounts and runs `catalogQuery`, key `['me','catalog']`, no `staleTime`.
   `packages/app-kit/src/student-queries.ts:144`
   → `GET /me/catalog`
2. Route matches under `@Controller('me')` with `@Actors(ActorTypes.STUDENT)`; the global guards run (JWT, actor, feature, throttler). The subject is `user.id`, so there is no id in the path.
   `apps/api/src/me/me.controller.ts:74`
   → `MeController.catalog()`
3. Pass-through to the resolver.
   `apps/api/src/me/me.service.ts:36`
   → `access.catalog(studentId)`

4. Two reads together: the student's standing and the shared series list.
   `apps/api/src/access/access-resolver.service.ts:174`
   → `Promise.all([standingOf, shared])`
5. Standing: one `student.findUnique` with `STANDING_FACTS` (reach facts, grants, event candidacies, and non-VOIDED attempts ordered `attemptNo asc`). Prisma issues one batched `IN` per relation.
   `apps/api/src/access/access-resolver.service.ts:301`
   → `Standing`
6. Absent student (unknown, inactive, soft-deleted) is `NOT_FOUND`.
   `apps/api/src/access/access-resolver.service.ts:175`
   → `AppException(NOT_FOUND)`
7. Standing folds two maps: `sittings` (last write wins, so the highest `attemptNo` per test) and `everSat` (every test with a SUBMITTED or EVALUATED sitting).
   `apps/api/src/access/access-resolver.service.ts:311`
   → `sittings`, `everSat`
8. Shared side: read the epoch with `GET access:catalog:epoch`; a Redis error is read as "no news" and the last seen value stands.
   `apps/api/src/access/access-resolver.service.ts:258`
   → `epoch`
9. The held copy serves only if `held.epoch === epoch` and it is under 15 minutes old. Otherwise one `testSeries.findMany` with `SHARED_SELECT` builds a new `Promise<SharedSeries[]>` and holds it; a rejected promise un-holds itself.
   `apps/api/src/access/access-resolver.service.ts:268`
   → shared series + ACTIVE tests
10. Reach filter: `reachedBy` keeps a series when `seriesSources` is non-empty.
    `apps/api/src/access/access-resolver.service.ts:346`
    → reached series
11. Each series is shaped per student: scoped sections/duration/question count, `opensAt` = the earliest of THIS student's matching `TestProgramUnlock` openings, else `Test.opensAt`; `attemptStatus` and `liveAttemptId` come from the latest sitting. Tests sort by `byOrderThenId`.
    `apps/api/src/access/access-resolver.service.ts:478`
    → `ResolvedSeries`
12. The in-order gate: if `sequentialTests`, `waiting` is the first test never sat (`everSat`); tests at index `<= waiting` take their turn.
    `apps/api/src/access/access-resolver.service.ts:526`
    → `itsTurn`
13. The clock is read here and never held: `shutBy` returns the one fact that shuts a test (`HOLD`, then `NOT_OPEN`, then `TURN`) or null.
    `apps/api/src/access/access-resolver.service.ts:557`
    → `canStart = shut === null`
14. Response through the interceptor.
    `apps/api/src/me/me.controller.ts:74`
    → `data = { testBlocked, series:[{ id, name, examStage, kind, sequentialTests, tests:[{ id, title, durationSec, sectionCount, totalQuestions, order, opensAt, attemptStatus, liveAttemptId, canStart, shut }] }] }`

Notes on that shape:

- A series with no active tests is still listed; the student apps do not draw an empty shelf (stated in `aaff7f5`).
- `testIsOpen` (`packages/contracts/src/access.ts:399`) is the single "has it opened" predicate. It subtracts `START_GRACE_MS`, five minutes, so a sitting may begin slightly before the opening. The catalog, the start guard and the admin's offering validation all call it.
- `shut` and `liveAttemptId` are optional in the schema, so an older phone build reads the same catalog it always did. `shut` exists because each client used to rebuild the reason from its own clock and got it wrong (`b3a9d07`).
- A sat test stays startable: a paper may be sat again, and Done is only where it sorts. A retake is unranked.

Sequential series and the retake bug

`everSat`, not the latest sitting, decides who waits. When the gate read the latest sitting's status, starting a retake of test 1 turned it back to `IN_PROGRESS`, the first unsat test became index 0, and test 2 shut again (`4c65a7d`). The card still shows `IN_PROGRESS`, because that is true. `apps/api/test-db/access-resolver.db.test.ts:648` pins it. Because the order is read over ACTIVE tests alone, making a test inactive stops the tests behind it waiting on it.

What is cached, and how it is busted

Nothing per student is cached. The only held thing is the series side: one `Promise<SharedSeries[]>` per API process (`:164`), tagged with the epoch it was built under. A student's block, grant, branch transfer, enrolment or submit counts on their next request because standing is read live.

The epoch is one integer at `access:catalog:epoch` (`apps/api/src/redis/redis.keys.ts:46`), no TTL. The 15-minute `CATALOG_MAX_AGE_MS` (`:33`) is the safety net under a write that changed a held field without bumping it. `opensAt` cannot be a reason to bust: the clock passing an opening emits nothing, which is why the clock is applied after the held copy.

The paper-row cache in the attempts module keys on the same epoch (`apps/api/src/attempts/attempt-paper.service.ts:213`, `catalogEpoch()`), so every bump also drops that.

PUT /admin/test-series/:id/branches — the cache-busting path

1. Admin write. `@Audit(TEST_SERIES, UPDATE)` and `@RequiresFeature(BRANCH_TEST_MANAGEMENT, WRITE)`, a separate permission from series editing.
   `apps/api/src/access/access.controller.ts:170`
   → request admitted
2. `setBranches`: the body is either `{ everyBranch: true }` or a `branchIds` list. The list is de-duplicated, checked against the kind pairing rules, and each branch must be live.
   `apps/api/src/access/test-series.service.ts:257`
   → validated list

3. Row written. For `everyBranch`, one raw `UPDATE ... FROM (SELECT ... FOR UPDATE)` reads every branch as it writes, so a branch made a moment ago is on the list and the screen never sends a list.
   `apps/api/src/access/test-series.service.ts:282`
   → `TestSeries.branchIds`
4. Fact published on the in-process bus.
   `apps/api/src/access/test-series.service.ts:277`
   → `ACCESS_CATALOG_CHANGED { testSeriesId }`
5. The bus is a fire-and-forget `EventEmitter2`; a throwing listener is logged and the producer is unaffected.
   `apps/api/src/common/events/domain-event-bus.ts`
   → listener invoked
6. Listener ignores the id beyond a log line; the bust is global.
   `apps/api/src/access/access-cache.listener.ts:17`
   → `bump()`
7. `INCR access:catalog:epoch`. It must be an increment, never a `DEL`: a missing key reads as 0, which equals the epoch of every fresh process, so a delete would bust nothing.
   `apps/api/src/access/access-resolver.service.ts:253`
   → counter + 1
8. A Redis error here is logged and swallowed; the write already landed and every held copy ages out within 15 minutes.
   `apps/api/src/access/access-cache.listener.ts:29`
   → log line
9. The next read anywhere sees `held.epoch !== epoch` (any difference, so a Valkey reset that sends the counter backwards still bites) and rebuilds.
   `apps/api/src/access/access-resolver.service.ts:272`
   → new held promise

Emitters of `ACCESS_CATALOG_CHANGED`:

- series create, update, delete, `setBranches`, and `toggleBranch` only when a row moved (`apps/api/src/access/test-series.service.ts:187`, `:223`, `:241`, `:277`, `:303`, `:338`);
- test title or exam-template edits (`apps/api/src/tests/tests.service.ts:322`) and test delete (`:369`), each behind `inTheCatalog(test)` (status is `ACTIVE`, `apps/api/src/tests/test-rules.ts:45`);
- the Offer step: a series move (`apps/api/src/tests/offering.service.ts:258`, same guard) and the status/opening save (`:496`).

`EXAM_STAGE_CHANGED` busts the same key through the same listener, from `apps/api/src/configs/exam-stages.service.ts:134`, `apps/api/src/configs/exams.service.ts:121` and `apps/api/src/configs/base-configs.service.ts:253`.

A grant, revoke, block, branch transfer or enrolment emits nothing, because every per-student fact is read live.

The start guard and the brief

POST /ME/TESTS/:TESTID/ATTEMPT — CAN THIS SITTING BEGIN

1. Route with `@SittingRateLimit()`, student actor, answers 200.
   `apps/api/src/attempts/attempts.controller.ts:63`
   → `start()`
2. Their sittings of this test, the test row, and the session-free check run in one wave.
   `apps/api/src/attempts/attempts.service.ts:83`
   → sittings, test
3. Resume short-circuits above the guard: a live sitting that matches (or no `resume` id given) reopens with no access check.
   `apps/api/src/attempts/attempts.service.ts:96`
   → resumed `LiveAttempt`
4. A stale `resume` id (the sitting was handed in elsewhere) is `SITTING_ENDED`, never a fresh paper.
   `apps/api/src/attempts/attempts.service.ts:103`
   → error
5. The guard, and the first refusal, so a test that does not exist reads like one the student cannot reach.
   `apps/api/src/attempts/attempts.service.ts:106`
   → `access.assertCanStart`
6. Same two reads as the catalog (standing and shared).
   `apps/api/src/access/access-resolver.service.ts:182`
   → standing, series
7. Absent student, or `isTestBlocked`: no test is resolved, so the message cannot become "not opened yet".
   `apps/api/src/access/access-resolver.service.ts:185`
   → `undefined`
8. `startableTest` finds the single series holding the id and checks reach on it alone. No holder (unknown test, inactive, not ACTIVE) or no route to it gives `undefined`.
   `apps/api/src/access/access-resolver.service.ts:356`
   → one series
9. That series is still projected whole, because `sequentialTests` decides a test by its index among its siblings and their sittings. Only the requested test is returned.
   `apps/api/src/access/access-resolver.service.ts:366`
   → `StudentCatalogTest`
10. `canStart` passes.
    `apps/api/src/access/access-resolver.service.ts:189`
    → return

11 Otherwise `FORBIDDEN`, with one of two messages: "This test has not opened yet" when the test is reached but early, and "This test is not open to you right now" for everything else (including a hold or an in-order turn).
`apps/api/src/access/access-resolver.service.ts:564`
→ `AppException(FORBIDDEN)`

Every refusal carries the same code, `FORBIDDEN`; only the message differs. The guard and the catalog cannot disagree because both run `project()`. `apps/api/test-db/access-resolver.db.test.ts:652` pins that the guard's answer for every test in a sequential series equals the catalog's `canStart`.

The other two single-test routes:

- `GET /me/tests/:testId/brief` (`apps/api/src/attempts/attempts.controller.ts:84`) calls `reachableTest` (`apps/api/src/access/access-resolver.service.ts:195`). It is gated on REACH, not on the window and not on a block, so a student can read about a test they cannot start yet. Any non-reach is `NOT_FOUND`, so an id is never confirmed. The brief carries `timerTemplate`, `testUi`, and the exam's journey with this stage marked (`apps/api/src/attempts/attempt-paper.service.ts:118`).
- `GET /me/tests/:testId/paper` (`apps/api/src/attempts/attempts.controller.ts:93`) calls `assertCanStart` (`apps/api/src/attempts/attempt-paper.service.ts:166`), the same gate as a start, so holding the paper and beginning open at the same instant. A candidate who fetches the paper and then starts pays the guard twice.

What the admin side reads

Route	Controller	What it returns
`GET /admin/test-series`, `GET /:id`	`TestSeriesController`	Summary (counts of tests, grants, live branches) and detail with `reachedCount` (`audienceCount`) and `satCount` (raw `COUNT(DISTINCT studentId)`). The list takes `notReachedBy=<studentId>`, which stands outside the All/Any `match`.
`POST/PATCH/DELETE /admin/test-series`	same	Create, edit, delete. Delete is refused while any test sits in the series.
`PUT/PATCH /admin/test-series/:id/branches`	same	Whole list, or one branch toggled by a single `UPDATE` so two toggles from one stale screen both land.
`POST /admin/test-series/:id/grants/export`	same	Starts an export job; the worker builds the workbook (`student-grants.service.ts:59`).
`POST/DELETE /admin/students/:studentId/grants`	`StudentGrantsController`	Grant and revoke.
`GET /admin/students/:studentId/series`	`StudentSeriesController`	Every series the student reaches, every source, and when each grant was made.
`/admin/programs`	`ProgramsController`	List needs `STUDENT_MANAGEMENT` READ; writes need super admin.

Behaviours worth knowing:

- A grant is refused for a test-blocked student (`BLOCKED_GRANT_MESSAGE`, `apps/api/src/access/student-grants.service.ts:114`).
- Granting twice writes one row (`createManyAndReturn` with `skipDuplicates`) and rings the bell once. The notification's `dedupeKey` names the grant row's `createdAt`, so a grant taken back and made again is news again.
- A revoke that took nothing logs nothing.
- `update` takes `expectedUpdatedAt` and is conditional on the row it judged (`apps/api/src/access/test-series.service.ts:214`), so a form opened before another admin's rename is refused with `CONFLICT` instead of overwriting.
- A series edit is judged on the shape it leaves. `kindPairingErrors` (`:483`) answers in the service so a form can mark the field; a raw CHECK violation could only leave as a 500. A branch deleted since is not counted, because `branchIds` has no foreign key.
- An in-order series refuses an edit whose test openings do not ascend (`opensOutOfOrder`, `:63`).

The data it touches

Column	Type	Holds	Written by
`Student.isActive`, `.deletedAt`	bool, timestamptz	absent when off or set	students
`Student.isTestBlocked`	bool	listed, nothing startable	students
`Student.currentBranchId`	uuid?	the `STANDARD` branch match	students
`Student.programs`	text[] (GIN)	the `PROGRAM` arm and program openings	students / import
`Student.enrolledCourses`	`ExamCourse[]` (GIN)	the `STANDARD` course match	students / import
`StudentGrant`	composite PK `(studentId, testSeriesId)`, `createdAt`, `createdById`	the grant, who made it, when	`StudentGrantsService`
`EventCandidate`	composite PK `(eventId, studentId)`	the `EVENT` arm	events
`TestSeries.kind`	enum `STANDARD / FREE / PROGRAM / EVENT`	which automatic arm applies	`TestSeriesService`
`TestSeries.branchIds`	uuid[] (GIN)	`STANDARD` branch reach	`setBranches`, `toggleBranch`
`TestSeries.programCode`, `.eventId`	text? / uuid?	`PROGRAM` / `EVENT` reach	`TestSeriesService`
`TestSeries.sequentialTests`	bool	the in-order gate	`TestSeriesService`

Column	Type	Holds	Written by
`TestProgramUnlock`	PK `(testId, programCode)`, `opensAt`	per-program earlier opening	offering
`Attempt.status/testId/attemptNo/voidedAt`		`attemptStatus` and the in-order gate	attempts
`Test.status`, `.opensAt`, `.scope`, `.scopeRef`		the ACTIVE gate, the opening, the scoped section list	tests

`TestSeries` has no `isEnabled` column. Redis: `access:catalog:epoch` is the module's entire surface. The CHECK constraints that hold a series to its kind (PROGRAM exactly when it has a program code, EVENT exactly when it has an event, only STANDARD names branches, only FREE spans no stage) are hand-written SQL in the migrations; `docs/02-domain-rules.md` section 5 states them.

What must already be true

`Test.status = ACTIVE`, the student `isActive = true` with `deletedAt = null`, and for a start `isTestBlocked = false`. `STANDARD` also needs a branch in `branchIds` and a stage whose exam has a course the student is enrolled in. Redis must be reachable for a first epoch read only in the sense that an outage reads as "no news": the guarded `catch` (`:260`) means a Valkey outage no longer 500s the catalog, the guard or paper serving while a valid held copy stands.

Why it is shaped this way

- **Derived, never stored.** A per-student catalog in Valkey was about 184 MB of a 280 MB dataset at 8,000 candidates (821f8db, per the old chapter; read, not re-measured), nearly all the same content copied per student. Deriving costs four statements.
- **One global epoch, not per series or per student.** Per-student counters never expired and needed a hand-run delete on deploy. The price is that any series write rebuilds every process's copy. A draft test's rename, delete and move do not bump at all (`inTheCatalog`).
- **Clock applied after the held copy.** `canStart` and `opensAt` are not part of what is held, so a 15-minute-old hold is still correct about time.
- **The hold is a promise, not rows.** Requests arriving during a cold read share one build, and a failed build is un-held rather than inherited.
- **The guard projects one series, not all.** Until `7b8b5c1` the guard resolved every reached series to keep one test.

What is wrong with it

- `apps/api/src/access/access.controller.ts:220` — the comment says a grant is "one of the three answers", but `STUDENT_SERIES_SOURCE` has five sources (`packages/contracts/src/access.ts:227`: course, program, free, event, grant). Stale.
- `apps/api/src/access/access-resolver.service.ts:414` — an `EVENT` series stays reachable after its `Event` is made inactive; `isActive` on `Event` only gates choosing the event for a new series (`apps/api/src/access/test-series.service.ts:377`). Correctness if the intent of "inactive" is "off"; read, not confirmed against product intent.
- `apps/api/src/attempts/attempts.service.ts:96` — a blocked or no-longer-reached student with a live sitting can still resume it, because resume is above the guard. Deliberate (a sitting already begun), but it is not stated beside the guard. Correctness, low.
- `apps/api/src/access/access-cache.listener.ts:30` — a two-line comment, grandfathered by the one-line rule until touched. Stale.

The roster, the admins and the catalog

This chapter is the people and the taxonomy every other chapter stands on: who the students are and where they sit, who the staff are and what each may touch, and the exam catalog (`Exam` then `ExamStage`) that a series, a base config and a test all hang off. Four Nest modules own it, and each is the only writer of its tables.

Module	Owns	Entry
`apps/api/src/students/`	`Student`, `StudentProfile`, `StudentMobileHistory`	`apps/api/src/students/students.controller.ts:63`
`apps/api/src/admins/`	`Admin`, `AdminFeaturePermission`	`apps/api/src/admins/admins.controller.ts:27`
`apps/api/src/branches/`	`Branch`	`apps/api/src/branches/branches.controller.ts:34`
`apps/api/src/configs/`	`Exam`, `ExamStage` (and `BaseConfig`, covered with tests)	`apps/api/src/configs/exams.controller.ts:34`, `apps/api/src/configs/exam-stages.controller.ts:34`

`Program` is written by `apps/api/src/access/programs.service.ts`, beside the series that reference it, but it is a roster concept and is read here. Events, imports, the access resolver and base configs have their own chapters.

The three ideas that shape all of it

Enrolment is arrays of codes, not join rows. `Student.enrolledExams`, `programs` and `enrolledCourses` are `String[]` / enum arrays with GIN indexes (`prisma/schema.prisma:839-841`), so a roster filter is `hasSome` against an index with no join (`apps/api/src/students/student-query.ts:65-66`). The price is that nothing links a code back to its row. `Exam.code`, `Program.code` and `ExamStage.stageKey` are therefore frozen once anything holds them, and a code whose row has gone renders as itself (`apps/api/src/students/students.service.ts:214`).

Grants are rows; roles are not. An admin holds a level on a feature key, and the key set is code, never a table (`packages/contracts/src/admins.ts:21`). There is no hierarchy and no per-route grant.

Erasure is anonymisation in place. The sittings, marks and every aggregate they were folded into are facts about a paper; a cohort's mean must not move because one person exercised a right. The row stays and the person goes out of it (`apps/api/src/students/anonymize.ts:1`).

Who may do what

Four global guards run in order (`apps/api/src/app.module.ts:89-94`): `JwtAuthGuard`, `ActorGuard`, `FeaturePermissionGuard`, `SuperAdminGuard`, then the throttler. An admin's grants are read from the database on every request, not from the token, so a revoke bites on the next call (`apps/api/src/auth/guards/jwt-auth.guard.ts:37-40`, `apps/api/src/admins/admins.service.ts:68`).

ANY ADMIN ROUTE — HOW A REQUEST IS AUTHORISED

1. The bearer token is verified and the Redis session must still exist, otherwise the session's end reason is thrown.
   `apps/api/src/auth/guards/jwt-auth.guard.ts:31-35`
   → claims `{ sub, actor, sid }`
2. For an admin, `identityOf` reads the row and its grants in one query. A deactivated admin gets `isSuperAdmin: false` and `permissions: {}`; a super admin gets `{}` because grants are dormant under the bypass.
   `apps/api/src/admins/admins.service.ts:68-90`
   → `request.user = { id, actor, isActive, isSuperAdmin, permissions }`
3. `@Actors(ADMIN)` is the hard boundary: a student's token is a valid JWT and stops here.
   `apps/api/src/students/students.controller.ts:64`
   → FORBIDDEN for a student
4. `FeaturePermissionGuard` refuses an inactive admin FIRST, before the super-admin bypass, so a deactivated super admin reaches nothing.
   `apps/api/src/auth/guards/feature-permission.guard.ts:34`
   → FORBIDDEN "Your account has been deactivated"
5. The route's `{ key keys, level }` is checked with `can()`: active, then super admin, then `satisfiesLevel`. WRITE implies READ; there is no DELETE level.`
   → apps/api/src/auth/guards/feature-permission.guard.ts:43
6. An export route also needs `DATA_EXPORT` READ, carried in the same decorator because a second `RequiresFeature` would overwrite the first.
   `apps/api/src/auth/guards/feature-permission.guard.ts:49`
   → FORBIDDEN "You do not have access to exports"
7. `@RequiresSuperAdmin()` is a separate gate, deliberately not a feature key: these routes decide who decides, so granting one must not open them.
   `apps/api/src/auth/guards/super-admin.guard.ts:26`
   → FORBIDDEN "Only a super admin can change this"

The feature keys

`FEATURE_KEYS` holds eleven keys (`packages/contracts/src/admins.ts:21-33`), each labelled for the permissions screen in `FEATURES` (`:39`). Adding one is a code change by design: `GET /admin/features` has no create and builds itself from `FEATURE_KEY_VALUES` (`apps/api/src/admins/admins.service.ts:260`). A stored key that code no longer defines is dropped on read rather than trusted (`:373-387`).

Key	Covers
`STUDENT_MANAGEMENT`	Directory, imports, branches, the exam catalog, programs, events (reads open to anyone who manages students)
`QUESTION_MANAGEMENT`, `QUESTION_AUTHORING`, `QUESTION_PROOFREAD`	The bank; typing questions in; reading and flagging
`TEST_MANAGEMENT`, `BRANCH_TEST_MANAGEMENT`	Base configs, tests, papers, series; which branches a series runs for

Key	Covers
`TEST_OPERATIONS`	Live boards and resolving broken sittings
`STUDENT_PERFORMANCE`	Any student's analytics, and the sittings picker
`NOTIFICATION_MANAGEMENT`	Announcements, including ones that cost money
`DATA_EXPORT`	The download, on top of access to the list itself
`REPORTS`	Every printable report

Roles are a preset, then forgotten

`AdminRole` is `TYPIST`, `PROOFREADER`, `ADMIN`, `SUPER_ADMIN`. A role does three things: it seeds ordinary grants at create (`ROLE_PERMISSION_PRESET`, `packages/contracts/src/admins.ts:102`), it labels the person, and it decides `isSuperAdmin` on every write. No guard reads the role itself; the guards read `isSuperAdmin` and the grants. Setting a role to `SUPER_ADMIN` is therefore a full authorisation change, not a label.

Role	Opening grants
`TYPIST`	`QUESTION_AUTHORING` WRITE
`PROOFREADER`	`QUESTION_PROOFREAD` WRITE, `QUESTION_MANAGEMENT` READ
`ADMIN`	`STUDENT_MANAGEMENT`, `QUESTION_MANAGEMENT`, `TEST_MANAGEMENT`, `BRANCH_TEST_MANAGEMENT`, `TEST_OPERATIONS` WRITE; `STUDENT_PERFORMANCE` READ
`SUPER_ADMIN`	none; the bypass makes them dormant

Note that `DATA_EXPORT`, `REPORTS` and `NOTIFICATION_MANAGEMENT` are in no preset: they are always ticked by hand.

Admins

Every `/admin/admins` and `/admin/features` route is super admin only (class-level `@RequiresSuperAdmin()`, `apps/api/src/admins/admins.controller.ts:29`). Admins cannot self-register and nothing in the application creates the first one; it is a SQL row (`prisma/seed.sql:34`). Admin sign-in is an email code, and an unknown email is told so rather than answered with a fake "sent" (`apps/api/src/auth/auth.service.ts:113`).

POST /ADMIN/ADMINS — CREATING A STAFF MEMBER

1. The four guards pass; the class is `@RequiresSuperAdmin()`.
   `apps/api/src/admins/admins.controller.ts:29`
   → `request.user` is a super admin
2. `createAdminSchema` trims and lowercases the email BEFORE `z.email()` (OTP login looks the row up by exact match), and defaults `role` to `ADMIN`.
   `packages/contracts/src/admins.ts:185–198`
   → validated body
3. `createdById` comes from the token, never the body: an audit field a caller can set is not an audit field.
   `apps/api/src/admins/admins.controller.ts:46–47`
   → `createdById`
4. A pre-check on the email gives a field error; the unique index remains the real arbiter of a race.
   `apps/api/src/admins/admins.service.ts:146–151`
   → CONFLICT with `fieldErrors.email`
5. One transaction writes the `Admin` row with `isSuperAdmin = (role === SUPER_ADMIN)`.
   `apps/api/src/admins/admins.service.ts:155–165`
   → `Admin` row
6. In the same transaction, one `AdminFeaturePermission` row per preset entry, skipped when the preset is empty.
   `apps/api/src/admins/admins.service.ts:166–171`
   → grant rows
7. The response carries the preset as `permissions`; `@Audit(ADMIN, CREATE)` records it.
   `apps/api/src/admins/admins.service.ts:176`
   → `{ id, email, role, isSuperAdmin, permissions, ... }`

PATCH /ADMIN/ADMINS/:ID — A ROLE CHANGE

1. `requireAdmin`, not an active check: renaming a deactivated admin, or promoting one before switching them back on, are both fine.
   `apps/api/src/admins/admins.service.ts:181`
   → the "before" row
2. A change to any role but `SUPER_ADMIN` computes `prunesTo`, the keys the new role's preset carries.
   `apps/api/src/admins/admins.service.ts:184–187`
   → key list or null
3. Inside the transaction, `keepASuperAdmin` locks every active super admin in id order (`FOR NO KEY UPDATE`) and refuses if the one leaving is the last.
   `apps/api/src/admins/admins.service.ts:190, :314–320`
   → CONFLICT "only active super admin"
4. The row is updated; `isSuperAdmin` is re-derived from the role.
   `apps/api/src/admins/admins.service.ts:191–199`
   → updated `Admin`
5. Grants outside `prunesTo` are deleted. It prunes and never seeds: whatever the permissions screen set on a kept key survives. A super admin's grants are left dormant so a later demotion narrows them.
   `apps/api/src/admins/admins.service.ts:201–205`
   → narrower `AdminFeaturePermission` set

6. The audit diff covers `fullName`, `role`, `isSuperAdmin`. The screen warns with the figures before sending.
   `apps/api/src/admins/admins.service.ts:209, apps/admin/src/features/admins/permissions.tsx:218`
   → `changed` JSON

PATCH /ADMIN/ADMINS/:ID/ACTIVE — SWITCHING SOMEONE OFF

1. You cannot deactivate yourself: that is undone only by another super admin, or with database access if you were the last.
   `apps/api/src/admins/admins.service.ts:218–221`
   → CONFLICT
2. Reactivation is a plain update; the grants are NOT restored, and the response reads whatever was granted meanwhile.
   `apps/api/src/admins/admins.service.ts:224–234`
   → `isActive: true`
3. Deactivation runs `keepASuperAdmin`, sets the flag and deletes every grant in one transaction.
   `apps/api/src/admins/admins.service.ts:237–245`
   → `isActive: false`, no grants
4. `ADMIN_DEACTIVATED` is emitted after commit.
   `apps/api/src/admins/admins.service.ts:251`
   → event
5. The listener revokes all the admin's Redis sessions, so the switch-off holds now rather than at token expiry; a failure is logged and never fails the request.
   `apps/api/src/auth/admin-sessions.listener.ts:14–22`
   → sessions removed

`PATCH /admin/admins/:id/permissions` sets a level per key (`null` removes, an absent key is untouched). It locks the admin row, refuses an inactive admin as "not found", then applies deletes and upserts in one transaction, so a deactivation cannot land between the check and the writes and have its pruned grants written back (`apps/api/src/admins/admins.service.ts:272–295`). The audit row is the per-key before/after (`:298`).

`holdersOf(key, level)` lists active admins at a level by id and name only, for pickers that must not leak the directory (`:99`).

Branches

`Branch` is a name, a `PHYSICAL | VIRTUAL` type and an active flag. Reading is open to `STUDENT_MANAGEMENT` READ, because an admin must see the list to pick from it; every write is super admin only, which is the whole reason the table exists (`apps/api/src/branches/branches.controller.ts:33`).

- **The online branch is a singleton.** One `VIRTUAL` branch is the home of every online student, so it cannot be renamed, deactivated or deleted, and a second one is refused (`apps/api/src/branches/branch-rules.ts:11–30`, `apps/api/src/branches/branches.service.ts:121`).
- **A branch with students cannot be deleted** (`branchDeletionBlocker`, counting `Student.currentBranchId`, `apps/api/src/branches/branches.service.ts:164–174`). `Student.currentBranchId` is `onDelete: Restrict`.
- **Student type and branch must agree** (`studentBranchBlocker`, `apps/api/src/branches/branch-rules.ts:37`): a `NON_IACE` student has no branch, an `ONLINE` student sits in the virtual branch, an `OFFLINE` student attends a physical one. The reason is that the access resolver reads the branch alone, so whoever holds one reaches its series.
- **An inactive branch takes no new student.** A student already sitting in one that has since been retired or deleted is not refused for it (`assertFitsStudent` with `live: false`, `apps/api/src/branches/branches.service.ts:49–74`).
- An `ONLINE` student naming no branch is placed in the virtual one; with none created yet the save is refused under the branch field (`:77–84`).
- The list puts the online branch first and returns a per-branch student count in one grouped query (`:94–107`, `:197`).

Programs

A `Program` is a coaching variant a student is a candidate for (`code`, `name`, `isActive`). It is a table and not an enum so that adding one is a row, never a migration. Reads are `STUDENT_MANAGEMENT` READ; create, update and delete are super admin only (`apps/api/src/access/access.controller.ts:63–100`).

A program code is held as free text by students (`Student.programs`) and by series (`TestSeries.programCode`). A code change or a delete is therefore refused while either holds it, with the counts, and the advice is to retire it (`codeChangeBlocker`, `apps/api/src/access/programs.service.ts:145–157`). `assertUsable` refuses a code that does not exist or is retired (`:108`); it is what the student save calls.

The exam catalog

`ExamCourse` is a four-value enum (`SSC`, `RRB`, `BANKING`, `AP_TS_POLICE`). `Exam` belongs to a course; `ExamStage` belongs to an exam and carries a table-wide unique `stageKey`, an `order`, a `mode` and a `disposition` (`CONDUCTED | PARTIAL | CATALOG_ONLY`). On screen the stages are the expanded row of the Exams page (`apps/admin/src/features/catalog/exams.tsx:387`), writable only by a super admin.

Rule	Where
An exam's code and a stage's key cannot change once anything carries it (students enrolled; base configs hung off the stage). A rename would detach every holder silently, with no error and no rows changed. Create a second row.	`apps/api/src/configs/exam-rules.ts:37`, `:70`
An exam cannot be deleted while it has stages or enrolled students; a stage cannot be deleted while a base config, test or series uses it. Retire instead.	`apps/api/src/configs/exam-rules.ts:26`, `:58`
Exam names are unique in any case, codes exactly.	`apps/api/src/configs/exams.service.ts:176–199`
A new stage needs an active exam; a base config needs an active stage whose disposition takes a paper. A `CATALOG_ONLY` stage is listed so the journey reads whole, and carries no paper.	`apps/api/src/configs/exam-stages.service.ts:150–170`, `:174–189`
An edit sends the diff, not the body, so re-sending a current value does not read as editing it.	`apps/api/src/configs/exam-rules.ts:9`

A course or code change on an exam, or a name change on a stage, emits `EXAM_STAGE_CHANGED` (`apps/api/src/configs/exams.service.ts:120–122`, `apps/api/src/configs/exam-stages.service.ts:133–135`). The access cache listener answers by bumping the catalog epoch, so every process rebuilds its shared catalog on its next read (`apps/api/src/access/access-cache.listener.ts:21–24`). The reason: the course decides who a `STANDARD` series reaches, and the code and stage name are on every student card.

Students

What a student row is

`Student` keeps identity, enrolment and state: `mobile`, `fullName`, `studentType` (`ONLINE | OFFLINE | NON_IACE`), the three enrolment arrays, `currentBranchId`, `isActive`, `isTestBlocked`, `deletedAt` and `anonymizedAt` (`prisma/schema.prisma:800–820`). `deletedAt` says the account is closed; `anonymizedAt` says the personal data is gone; a regulator asks about them separately.

`mobile` is unique only among live rows, by a partial index (`Student_mobile_live_key ... WHERE "deletedAt" IS NULL`, `prisma/migrations/20260820000000_schema_realignment/migration.sql:1152`). Every lookup by number is therefore a filtered read (`apps/api/src/students/students.service.ts:556`). A student has no password and no PIN: they sign in with a code sent to the mobile every time, and the first time is the signup.

`StudentProfile` holds the personal detail (parents' names, date of birth, email, address, gender, education and past-exam JSON) and two stored document keys, `photoUrl` and `tenthMarksheetUrl`, returned as 300-second signed links (`apps/api/src/students/students.service.ts:42`, `:269–272`). Aadhaar and PAN images are never stored; only `aadhaarVerified` and `panVerified` booleans exist.

Access is not read from `enrolledExams`. The resolver reaches a student by program, event, grant, or branch plus `enrolledCourses` (`apps/api/src/students/own-access.ts:4`, `apps/api/src/access/access-resolver.service.ts:463–466`). `enrolledExams` is a record of what they are enrolled in; it drives the "enrolled" notification and the name shown on the profile, not reach. The roster's "nothing of their own" filter is `HOLDS_OWN_ACCESS` negated.

The roster list

`GET /admin/students` (`STUDENT_MANAGEMENT` READ) runs rows and count in one transaction, then one more query for which of this page's students hold access of their own (`apps/api/src/students/students.service.ts:114–137`). `studentWhere` splits the filters into those the match toggle governs (status, block, readiness, branch, program, course, event, own-access) and those that always narrow: `deletedAt: null` and the search box (`apps/api/src/students/student-query.ts:44–85`). An erased student is never in the roster, and under `match=any` could not widen it, because that clause is kept out of the toggled set.

Search is trigram `ILIKE` on name and mobile, every term required. A whole ten-digit number, pasted with prefix or spacing, is normalised first; it also finds a student by a number they used to sign in with, via a read of `StudentMobileHistory` ahead of the main query, so an old number costs the search none of its own indexes (`:26–41`).

Adding a student

POST /ADMIN/STUDENTS — ADD A STUDENT BEFORE THEIR FIRST LOGIN

1. Guards: `@Actors(ADMIN)` then `STUDENT_MANAGEMENT` WRITE.
   `apps/api/src/students/students.controller.ts:115–117`
   → `request.user`
2. `createStudentSchema` requires only a mobile and a type; a blank name is "not known yet", not an error.
   `packages/contracts/src/students.ts:297–305`
   → `CreateStudentBody`
3. A live student with that mobile is refused with `details.studentId` so the screen can link to them.
   `apps/api/src/students/students.service.ts:280–281, :614`
   → CONFLICT "already exists"
4. Named exams and programs must exist and be active, in parallel; a missing or retired code is a field error under `enrolledExams` / `programs`.
   `apps/api/src/students/students.service.ts:283–292, apps/api/src/configs/exams.service.ts:140, apps/api/src/access/programs.service.ts:108`
   → VALIDATION_ERROR
5. `placementOf` checks a named branch fits the type, or puts an `ONLINE` student with no branch in the virtual one.
   `apps/api/src/students/students.service.ts:293–296, :341`
   → `currentBranchId`
6. One insert. If the number was taken between the check and the write, the live-unique index decides and the loser reads the same refusal.
   `apps/api/src/students/students.service.ts:308–312`
   → `Student` row; no `StudentProfile` yet
7. The detail is re-read and returned; `@Audit(STUDENT, CREATE)` names the new id.
   `apps/api/src/students/students.service.ts:309`
   → `StudentDetail`

A student can also appear without an admin: the first verified code for an unknown number creates a `NON_IACE` student (`apps/api/src/auth/auth.service.ts:206–211`). `NON_IACE` students hold no branch and reach only what a program, event, grant or `FREE` series gives them.

Editing a student

`PATCH /admin/students/:id` is a patch: an omitted key is left alone and an explicit null clears. Arrays replace wholesale. The form sends `expectedUpdatedAt`, the stamp it read.

PATCH /ADMIN/STUDENTS/:ID — SAVING THE DETAILS FORM

1. `requireLive` loads the row and profile, refusing an erased student's tombstone.
   `apps/api/src/students/students.service.ts:383, :546`
   → row with `deletedAt: null`

2. If `expectedUpdatedAt` differs from the row's `updatedAt`, the save is refused whole, not merged.
   `apps/api/src/students/students.service.ts:384-386`
   → CONFLICT "changed after the form was opened"
3. Only what the save ADDS is checked against the catalog, so a retired exam or program they still hold cannot block taking another off. A blocked student cannot gain an exam.
   `apps/api/src/students/students.service.ts:318-340, :566`
   → VALIDATION_ERROR under the field
4. `branchAfter` decides where the student ends up: a type change alone re-checks the branch they are in.
   `apps/api/src/students/students.service.ts:358-378`
   → branch id or undefined
5. In one transaction, a conditional `updateMany` keyed on the stamp that was judged, which also bumps `updatedAt` (a profile write alone leaves it where it was). Zero rows means someone else got there first.
   `apps/api/src/students/students.service.ts:415-419`
   → CONFLICT
6. The column and profile writes happen in the same transaction.
   `apps/api/src/students/students.service.ts:420-428`
   → updated row
7. Newly added exam codes write an `ENROLLMENT_ADDED` notification in that same transaction, with a dedupe key that includes the write's stamp (one taken off and added again is news again).
   `apps/api/src/students/students.service.ts:430-437`
   → `Notification` row
8. The audit diff names changed columns with their values, and changed profile fields by name only, as `[withheld]`, because the audit log outlives an erasure.
   `apps/api/src/students/students.service.ts:441-444, :650`
   → `changed` JSON

Deactivate, block, change number, desk code

Four small actions, each its own route so each is audited as what it is. Status and the block are separate facts: sign-in against sitting.

Action	Route	Who	What happens
Deactivate / reactivate	`PATCH /:id/active`	Super admin	Sets `isActive`; a deactivation emits `STUDENT_DEACTIVATED`, whose listener revokes all their sessions. Reactivation revokes nothing. Audited as ACTIVATE or DEACTIVATE. (`apps/api/src/students/students.service.ts:479-489`)
Block / unblock tests	`PATCH /:id/test-blocked`	`STUDENT_MANAGEMENT` WRITE	Sets `isTestBlocked`. Sign-in and history are untouched; they cannot start a test. Audited as BLOCK or UNBLOCK. (`:531`)
Change mobile	`PATCH /:id/mobile`	`STUDENT_MANAGEMENT` WRITE	See the flow below.
Desk code	`POST /:id/desk-code`	`STUDENT_MANAGEMENT` WRITE	For a student at the desk whose code did not arrive. Refused for an inactive student. A second code kept beside the sent one, valid five minutes, single use, with its own five guesses. (`apps/api/src/auth/auth.service.ts:98-106`, `apps/api/src/auth/otp/otp.service.ts:231-238`)

Blocking is enforced at the gate that matters: `assertCanStart` treats a blocked student as not open to them, never as "not opened yet" (`apps/api/src/access/access-resolver.service.ts:179-190`), a new grant is refused (`apps/api/src/access/student-grants.service.ts:114`), and the cohort a test reaches excludes them (`apps/api/src/access/access-resolver.service.ts:442`). It is read at start, not mid-sitting.

PATCH /ADMIN/STUDENTS/:ID/MOBILE — MOVING WHO A STUDENT SIGNS IN AS

1. `STUDENT_MANAGEMENT` WRITE. Only an admin changes a number; a student moving it would make a paid enrolment something to hand on.
   `apps/api/src/students/students.controller.ts:143-152`
   → `ChangeStudentMobileBody`
2. The same number is refused as "already their number"; a number held by another live student is refused with their id.
   `apps/api/src/students/students.service.ts:494-500`
   → CONFLICT
3. One transaction: `updateMany` guarded by the number that was read, so an erasure or second change that landed first moves nothing.
   `apps/api/src/students/students.service.ts:504-510`
   → `Student.mobile`
4. Same transaction: the old number is stored with who changed it.
   `apps/api/src/students/students.service.ts:511-513`
   → `StudentMobileHistory` row
5. A race for one number is decided by the live-unique index; the loser reads the normal refusal.
   `apps/api/src/students/students.service.ts:516-518`
   → CONFLICT
6. The audit diff holds masked numbers only, because the log survives an erasure.
   `apps/api/src/students/students.service.ts:521-524`
   → `changed` JSON
7. `STUDENT_MOBILE_CHANGED` is emitted; the listener revokes every session, because the device holding the old number may not be theirs.
   `apps/api/src/students/students.service.ts:526, apps/api/src/auth/student-sessions.listener.ts:23-26`
   → sessions removed

The old number is kept for search only. It is never a way to sign in: operators recycle numbers, and the next owner of an old one is a stranger who gets an account of their own.

Erasure

`POST /admin/students/:id/erasure` is super admin only and irreversible (`apps/api/src/students/students.controller.ts:165–172`). The confirm dialog says the person is no longer named against any sitting, which still counts (`apps/admin/src/features/students/student-detail-actions.tsx:349–359`).

POST /ADMIN/STUDENTS/:ID/ERASURE — ERASING A STUDENT'S PERSONAL DATA

1. Super admin gate. The student must be live; an already-erased one is refused.
   `apps/api/src/students/student-privacy.service.ts:28–37`
   → NOT_FOUND / CONFLICT
2. Their photo and marksheet are deleted from S3 BEFORE the row changes, so an erasure that answered success cannot have left files behind. S3 deletes are idempotent, so a retry after a part-way failure is clean.
   `apps/api/src/students/student-privacy.service.ts:40, :63–68`
   → objects removed
3. One transaction: a guarded `updateMany` sets `mobile` to the tombstone `0000000000`, nulls `fullName` and `externalRef`, sets `isActive` false, `deletedAt` and `anonymizedAt`. Of two erasures landing together, one writes and one is refused.
   `apps/api/src/students/student-privacy.service.ts:45–49, apps/api/src/students/anonymize.ts:13–22`
   → tombstoned `Student`
4. Same transaction: the profile is emptied (parents, dob, email, address, gender, document keys, education and past-exam JSON), and both verification booleans return to false.
   `apps/api/src/students/student-privacy.service.ts:50, apps/api/src/students/anonymize.ts:25–40`
   → blank `StudentProfile`
5. Same transaction: every `StudentMobileHistory` row is deleted, because an old number names them as surely as the current one.
   `apps/api/src/students/student-privacy.service.ts:52`
   → rows gone
6. The count of their attempts is read as the receipt's `attemptsKept`; the attempts themselves are untouched.
   `apps/api/src/students/student-privacy.service.ts:53`
   → number
7. `STUDENT_DEACTIVATED` is emitted, because this write sets `isActive` itself and `setActive` never sees it.
   `apps/api/src/students/student-privacy.service.ts:57`
   → sessions revoked
8. Receipt returned.
   `apps/api/src/students/student-privacy.service.ts:59`
   → `{ studentId, anonymizedAt, attemptsKept }`

The tombstone number is not a real one: `mobileSchema` demands a leading 6-9, so nothing live can collide with it (`apps/api/src/students/anonymize.ts:9`). Every later write to the student passes through `requireLive`, which excludes `deletedAt`, so a tombstone takes no more writes.

Exports

`POST /admin/students/export` is `RequiresExport(STUDENT_MANAGEMENT)`, so it needs that READ plus `DATA_EXPORT`; the performance view additionally needs `STUDENT_PERFORMANCE` (`apps/api/src/students/students.controller.ts:84–97`). It answers 202 and an export job; the workbook is built by a worker, not by the request. The file holds the list's own filters, minus paging, and respects `deletedAt` (`apps/api/src/students/student-export.ts`, read, not traced end to end).

What changed since the first handbook

- **The PIN is gone.** A student has no PIN and no `pinHash`; creating a student no longer mints or SMSes a starting PIN. Sign-in is a code every time, and the first verified code creates the account.
- **A desk code** covers a student whose code did not arrive.
- **An admin can change a student's mobile**, with `StudentMobileHistory` and session revocation.
- **Admins have roles** (`TYPIST`, `PROOFREADER`, `ADMIN`, `SUPER_ADMIN`) with grant presets, and a role change prunes the grants it no longer carries. Feature keys grew to eleven.
- **The last active super admin cannot be demoted or deactivated**, and nobody can deactivate themselves.
- **Stale-form saves are refused.** The student save carries `expectedUpdatedAt`.
- **Erasure is real.** It removes the files first, empties the profile, deletes number history, and the audit trail records field names, never values.
- **Access no longer reads `enrolledExams`.** It is course plus branch for `STANDARD` series; the exam list is a record.
- **The exam catalog gained stage** `disposition`, with `CATALOG_ONLY` stages carrying no paper.
- **The audit log is a super admin's alone.**

What is wrong with it

- `packages/contracts/src/admins.ts:106` — Stale: the comment says every admin reaches the audit log scoped to their own rows. The controller is `@RequiresSuperAdmin()` at `apps/api/src/audit/audit.controller.ts:47`.
- `packages/contracts/src/students.ts:322` — Stale: "an enrolment reaches every series tagged with that exam" is false; access reads courses and programs. The block therefore guards the wrong field: `assertMayEnrol` checks only `enrolledExams` (`apps/api/src/students/students.service.ts:326`), while a blocked student can still gain a program or a course.
- `packages/contracts/src/students.ts:245` and `apps/admin/src/features/students/students.tsx:242` — Stale: the "nothing of their own" filter is described as no enrolment and no program, but `enrolledExams` plays no part in `HOLDS_OWN_ACCESS`; it is programs, branch plus course, grants and events.

- `prisma/seed.sql:34` — Correctness: the seeded super admin omits `role`, so it defaults to `ADMIN` while `isSuperAdmin` is true. The migration that added roles backfilled existing rows, a fresh database does not. Saving that admin with its displayed role would strip the bypass unless it is the last one.
- `apps/api/src/imports/imports.service.ts:502` — Privacy: an import run's uploaded sheet is stored (`ImportLog.fileS3Key`) and streamed back at `apps/api/src/audit/audit.controller.ts:82`. Erasure never touches it, so a student's name and mobile outlive their erasure in the original roster file.
- `apps/api/src/branches/branches.controller.ts:66` and `apps/api/src/configs/exams.controller.ts:64` — Dead: `@RequiresFeature(STUDENT_MANAGEMENT, WRITE)` sits beside `@RequiresSuperAdmin()` on delete. A super admin passes the former by bypass and nobody else passes the latter, so the feature check can never decide anything.

Events and their candidates

The name is a trap. `apps/api/src/events` is not an event bus. An **Event** is a product entity: a named gathering of people who will sit a paper, such as an entrance exam or a scholarship round, some of whom are not IACE students yet. The repo's real bus is `DomainEventBus` in `apps/api/src/common/events`. `app.module.ts` imports it under the alias `DomainEventsModule` so that `./events` keeps the plain name (`apps/api/src/app.module.ts:9-10`).

`prisma/schema.prisma:1001` declares `Event` (`name`, `description`, `isActive`). `prisma/schema.prisma:1013` declares `EventCandidate`, the membership join with a composite key of `(eventId, studentId)`. The creating migration, `prisma/migrations/20260903080000_an_event_gathers_the_people_sitting_it/migration.sql:1-15`, explains the shape. A candidacy is a **membership**, meaning rows in the event and nothing more. It is not a **grant**, which is an exception to a rule, so it does not overload `StudentGrant`.

Where it lives

Path	Role
`apps/api/src/events/events.controller.ts`	7 routes under `admin/events`, `@Actors(ADMIN)`.
`apps/api/src/events/events.service.ts`	Every read and write of `Event` and `EventCandidate`.
`apps/api/src/events/events.module.ts:10`	Controller mounted only on the `CORE` API role. The service is exported.
`apps/api/src/imports/imports.controller.ts:87-176`	Candidate template, preview, errors and commit.
`apps/api/src/imports/imports.service.ts:118-168`	Their bodies. `planCandidates` is at `:324`.
`apps/api/src/imports/candidate-import.ts:21`	The pure planner: create, add or skip. It writes nothing.
`apps/api/src/access/access-resolver.service.ts`	The only reader of the roster for access.
`packages/contracts/src/access.ts:86-132, 498-504`	Event schemas and the `EVENT` routes.
`apps/admin/src/features/catalog/events.tsx`	The list, the create/edit dialog, and the add-candidates dialog.
`apps/admin/src/features/imports/import-event-candidates.tsx`	The bulk-import screen.
`apps/admin/src/features/students/student-detail-events.tsx`	The student's Events tab, where a candidate is removed.

Nothing in `apps/exams` or `apps/mobile` calls the events API. It is admin-only.

What it is for

A test reaches a student only through a `TestSeries`, and the series `kind` picks the automatic route. `STANDARD` reads branch plus enrolled course. `PROGRAM` reads a program code. `FREE` reaches everyone. `EVENT` has nothing on the student to read, because an entrance round reaches people with no enrolment and no branch, and often no `Student` row yet. So the Event is the roster, and an `EVENT` series names its event with `TestSeries.eventId`. The import lets that roster arrive as a sheet of mobile numbers, minting a bare `NON_IACE` account for each unknown number.

The pairing is held in two places. SQL has `CHECK ((kind = 'EVENT') = ("eventId" IS NOT NULL))` in `prisma/migrations/20260904110000_the_new_access_shape_holds_itself_together/migration.sql:59`. The app has `KIND_PAIRING_MESSAGES` at `apps/api/src/access/test-series.service.ts:39-42` and the check at `:496-499`.

Permissions

Events have no feature key of their own. Every route gates on `STUDENT_MANAGEMENT`: READ for the GETs and for preview and errors, WRITE for the rest. The `EVENT` key was removed in `prisma/migrations/20260905110000_events_are_reached_on_the_students_key/migration.sql`, which deletes any `AdminFeaturePermission` rows left under it instead of converting them. `STUDENT_MANAGEMENT` reaches far more, and a super admin should grant it deliberately.

Writes are audited under `AUDIT_FEATURE.EVENT`. That value is separate from `STUDENT` and `PROGRAM`, added in `prisma/migrations/20260915000000_programs_and_events_are_audited/migration.sql`.

Flow: create an event

POST /ADMIN/EVENTS — CREATING AN EVENT

1. "New event" opens `EventDialog`, which submits `{name, description}`
   `apps/admin/src/features/catalog/events.tsx:272-275`
   → request body
2. Global guards in order: JWT, actor type, feature permission, super-admin, throttler
   `apps/api/src/app.module.ts:89-94`
   → ADMIN token holding `STUDENT_MANAGEMENT` at WRITE
3. `ZodBody(createEventSchema)` trims the name and caps it at 120 characters. The description is capped at 500.
   `apps/api/src/events/events.controller.ts:58, packages/contracts/src/access.ts:100-103`
   → validated `CreateEventBody`
4. `event.create` inserts the row with `isActive` defaulting to true
   `apps/api/src/events/events.service.ts:89-93`
   → one `Event` row
5. The row is mapped with `NOTHING_YET` counts, since a new event cannot have a roster or a series
   `apps/api/src/events/events.service.ts:28-29, 93`
   → `Event` with `candidateCount: 0`, `seriesCount: 0`

6. `AuditInterceptor` runs after the handler and writes an `AuditLog` row for `EVENT` / `CREATE`. It cannot fail the request.
   `apps/api/src/audit/audit.interceptor.ts:45–68`
   → `AuditLog`, entityId read from the payload
7. `ResponseInterceptor` wraps the result
   `apps/api/src/app.module.ts:87`
   → `{success:true, data, meta:{requestId}}`

PATCH `:id` follows the same path. It loads the row with `requireEvent` and applies only the fields sent. It calls `setPatchDiff(fieldDiff(...))` over `name`, `description` and `isActive`, so an edit that changes nothing leaves no audit row (`apps/api/src/events/events.service.ts:96–112`). The admin client sends only the fields the admin changed (`changedValues`, commit `2cf945c`).

Flow: add candidates by student id

POST /ADMIN/EVENTS/:ID/CANDIDATES — ADDING STUDENTS TO THE ROSTER

1. Row menu "Add candidates" opens `AddCandidatesDialog`, which posts the chosen ids
   `apps/admin/src/features/catalog/events.tsx:316`
   → `{studentIds}`
2. `addEventCandidatesSchema` requires 1 to 1000 uuids. Same guards as above, `@HttpCode(OK)`.
   `packages/contracts/src/access.ts:125–130, apps/api/src/events/events.controller.ts:80–90`
   → validated body
3. `requireEvent` throws `NOT_FOUND` "No such event" if the event is gone
   `apps/api/src/events/events.service.ts:154–157`
   → refusal
4. The roster is counted before the write
   `apps/api/src/events/events.service.ts:132`
   → `before`
5. `createMany({skipDuplicates:true})`. The composite primary key makes a repeat harmless.
   `apps/api/src/events/events.service.ts:134–140`
   → new `EventCandidate` rows, `count` of them
6. `setPatchDiff(rosterDiff(before, before + count))` stores a size, not a list
   `apps/api/src/events/events.service.ts:141, 161–163`
   → `AuditLog.changed = {candidates:{from,to}}`
7. If nothing was added the diff is null, the context is marked unchanged, and the interceptor writes no audit row
   `apps/api/src/audit/audit.context.ts:31–36, apps/api/src/audit/audit.interceptor.ts:50`
   → no `AuditLog` row
8. Response is `{success:true, data:null}`. No catalog cache is touched.
   `apps/api/src/events/events.controller.ts:85–90`
   → none

Flow: bulk-import candidates from a sheet

The import has three calls: preview, an optional errors download, and commit. Preview and errors write nothing.

POST /IMPORTS/EVENTS/:EVENTID/CANDIDATES/COMMIT — IMPORTING A ROSTER

1. "Import candidates" routes to `/cohorts/events/:id/import`. The page reads the event for its title and posts the file on commit.
   `apps/admin/src/features/imports/import-event-candidates.tsx:35–42`
   → multipart file
2. `FileInterceptor(IMPORT_FILE_FIELD)` with the import body limit. A missing file is a `VALIDATION_ERROR` with `fieldErrors.file`.
   `apps/api/src/imports/imports.controller.ts:168–176`
   → `UploadedSheet`
3. Guard: `STUDENT_MANAGEMENT` at WRITE. The commit takes the actor id from `@CurrentUser()`.
   `apps/api/src/imports/imports.controller.ts:166–174`
   → `actorId`
4. The event lookup and the sheet parse run together. A missing event wins over a bad sheet.
   `apps/api/src/imports/imports.service.ts:346–353`
   → `CsvTable`
5. `planCandidates` makes one `student.findMany` on the sheet's mobiles and splits the hits into live and soft-deleted
   `apps/api/src/imports/imports.service.ts:324–343`
   → `existingByMobile`, `deletedMobiles`
6. `planCandidateImport` checks the headers and the row ceiling, then decides each row. Errors give `skip`, a known live number gives `add`, anything else gives `create`. A number held by a soft-deleted student is an error.
   `apps/api/src/imports/candidate-import.ts:21–71`
   → `{rows, summary:{total, willCreate, willAdd, invalid}, fileErrors}`
7. `withRun` opens an `ImportLog` row with status `PREVIEWED`, uploads the sheet to S3 and writes `fileS3Key` back
   `apps/api/src/imports/imports.service.ts:447–456, 479–499, 503–507`
   → `ImportLog`, one S3 object
8. Each non-skip row is processed in order. A known number yields its existing id and touches nothing on that student.
   `apps/api/src/imports/imports.service.ts:136–149`
   → id list `studentIds`
9. An unknown number is inserted as a `Student` with `studentType: NON_IACE`, no branch, course, exam or program. A unique violation, from a concurrent signup, falls back to that holder.
   `apps/api/src/imports/imports.service.ts:210–220, 286–295`
   → new `Student` row

10. In a `finally`, `events.addCandidates(eventId, studentIds)` writes the whole list through the owning service, even when a row threw, so an account minted this run is not left off the roster
`apps/api/src/imports/imports.service.ts:155–157`
→ `EventCandidate` rows

11. `closeRun` records one `RowActionLog` per student, then writes the counts, `COMMITTED` and `finishedAt`. A failure to record rows is logged, not thrown.
`apps/api/src/imports/imports.service.ts:518–549`
→ `ImportLog` closed

12. Result returned: the plan summary plus `created`, `added` and `skipped`
`apps/api/src/imports/imports.service.ts:162–167`
→ `CandidateImportResult`

How it differs from the student import:

- The sheet carries only `mobile` and `fullName`. `CANDIDATE_IMPORT_COLUMNS` is the student columns filtered (`packages/contracts/src/imports.ts:332`).
- A known number joins the roster and nothing else changes. The student import rewrites name, access and profile.
- A minted account has no PIN and no password, because a student has none. The first sign-in is by an OTP code sent to the mobile, which is the signup. Nothing in the import hands out a credential.
- A soft-deleted holder is refused here. The planner reads only mobiles, so unlike the student import it does no branch-scope check.
- The run is logged under `AUDIT_FEATURE.STUDENT`, because the rows it writes are students (`imports.service.ts:80–83`).
- `IMPORT_MAX_ROWS` is 1000 (`packages/contracts/src/imports.ts:205`), the same as `CANDIDATE_BATCH_MAX`.

> **Note** The commit is serial and has no wrapping transaction. A thrown row stops the loop, but the `finally` still adds every id collected so far, and the run is closed `FAILED`. Rows already written stay.

Flow: remove a candidate

DELETE /ADMIN/EVENTS/:ID/CANDIDATES/:STUDENTID — REMOVING ONE CANDIDATE

1. Student detail, Events tab, "Remove from event" opens a `ConfirmDialog` naming the consequence
`apps/admin/src/features/students/student-detail-events.tsx:81–120`
→ confirmation

2. `DELETE` with `@HttpCode(OK)`, because Express 5 drops a 204 body. Same guards, WRITE.
`apps/api/src/events/events.controller.ts:92–98`
→ request

3. `requireEvent`, then the roster is counted
`apps/api/src/events/events.service.ts:145–146`
→ `before`

4. `deleteMany({eventId, studentId})` removes one membership row and nothing else. The student, attempts, scores and rank stay.
`apps/api/src/events/events.service.ts:148–150`
→ one fewer `EventCandidate` row

5. `rosterDiff(before, before − count)`. Removing someone not on the roster deletes nothing, so no audit row is written.
`apps/api/src/events/events.service.ts:151`
→ `AuditLog`, or none

6. The client invalidates the STUDENTS and EVENTS query keys. No server cache is busted, since reach is read live.
`apps/admin/src/features/students/student-detail-events.tsx:86–87`
→ refreshed screen

Delete and retire

- **Delete** is refused with `CONFLICT` while any `TestSeries` names the event, and the message gives the count (`apps/api/src/events/events.service.ts:114–127`). Otherwise the roster goes with it by `ON DELETE CASCADE` on `EventCandidate.eventId`. `TestSeries.eventId` is `onDelete: Restrict` (`prisma/schema.prisma:975`). The admin list leaves the action out when `seriesCount > 0` (`apps/admin/src/features/catalog/events.tsx:124`).
- **Retire** is PATCH `isActive:false`. It only keeps the event out of the series builder, because `assertEventUsable` refuses an inactive one (`apps/api/src/access/test-series.service.ts:366–378`), and `EventPicker` lists active events only (`apps/admin/src/components/access-picker.tsx:40–45`). A series that already names a retired event keeps reaching its candidates. The admin's retire text says so (`events.tsx:155–164`).

How an event gives access

The roster is read live and never cached per student. `eventCandidacies: {select: {eventId: true}}` is one field of `REACH_FACTS` (`apps/api/src/access/access-resolver.service.ts:129`), read inside the single `Student.findUnique` the resolver already issues.

1. `reachFrom` folds it into `Reach.events`, a `Set` of event ids (`:321–330`).
2. `sourcesOf` computes `isCandidate = row.eventId !== null && reach.events.has(row.eventId)` (`:332–343`).
3. `seriesSources` has one `EVENT` arm, `series.isCandidate` (`:414`). It yields `STUDENT_SERIES_SOURCE.EVENT` (`:428`).
4. A series with no source is dropped by `reachedBy` (`:346`). The catalog, `assertCanStart` and `reachableTest` all go through it, so one resolution gates the catalog, starting a test and any single-test read.
5. `audienceOf` and `automaticAudience` (`:432–457`) are the mirror image as a `StudentWhereInput`, `{eventCandidacies: {some: {eventId}}}`. `studentsReaching` and `audienceCount` use it (`:226–250`).

A reached test looks the same as any other. `studentCatalogSeriesSchema` carries `kind` but no sources. The source list goes only to admin screens, badged "Event candidate" (`apps/admin/src/lib/constants.ts:266`). A candidate sees a normal card, and `canStart` turns on `opensAt`, sequential order and the test's own settings, not on the event.

Because it is a plain join, a roster change needs no invalidation. Adding or removing a candidate does not touch `access:catalog:epoch`. That counter only covers the shared series list and moves when series or tests are written.

Two other readers use the roster:

- `HOLDS_OWN_ACCESS` counts a candidacy as a student's "own" access (`apps/api/src/students/own-access.ts:9`), alongside programs, a branch plus course, and grants. It drives the students list and the detail flag.
- The `EVENT_CANDIDATES` report lists the roster and how many of the event's tests each candidate has sat (`apps/api/src/reports/enrolment-reports.ts:388`).

Data touched

Model.column	Holds	Written by		
`Event.name` / `description`	text, max 120 / 500	create, update		
`Event.isActive`	bool, default true. Controls the series builder and the pickers only.	update		
`EventCandidate (eventId, studentId)`	the membership. The composite key makes `skipDuplicates` idempotent.	`addCandidates`, `removeCandidate`		
`EventCandidate.createdAt`	when they joined	default		
`TestSeries.eventId`	the roster an `EVENT` series draws on	`TestSeriesService`		
`Student.studentType`	`NON_IACE` for every candidate minted by the import	`createCandidate`		
`ImportLog`, `RowActionLog`	the import run and its rows	`withRun`, `closeRun`		
`AuditLog.changed`	`{name|	description|	isActive:{from,to}} `or` {candidates:{from,to}}`	`AuditInterceptor`

No Redis key is written. S3 holds one object per import run, at the path `importFileKey` builds.

Why it is shaped this way

- **A table, not a column.** A `Student.events` array has no row to hang on until the candidate enrols, and `StudentGrant` exists for exceptions to a rule where a roster is the rule. The cost is a join on each catalog read.
- **Read live.** Nothing per student is cached, so nothing per student is invalidated on a roster change.
- **Counts through `groupBy` scoped to the page's ids** (`apps/api/src/events/events.service.ts:66-79`), not a relation `_count`, which would group the whole child table on every read.
- **The audit log stores a roster size, not a roster.** That avoids thousands of uuids in jsonb per import. The cost is that it answers how many, never who.
- **The import goes through `EventsService.addCandidates`.** `EventCandidate` is owned by the events module, and `ImportsService` injects it (`apps/api/src/imports/imports.service.ts:70`).

What is wrong with it

- Correctness: `apps/api/src/events/events.service.ts:130-140` accepts any student id, and the roster write has no existence check beyond the foreign key. A bad id fails the whole `createMany` with a database error rather than a field error (read, not run).
- Stale: `docs/02-domain-rules.md` states "`EVENT` reaches only the candidates on its `Event`" and never says a retired event keeps reaching them. The behaviour is correct, since `isActive` is read nowhere in `access-resolver.service.ts`, but it is undocumented.
- Stale: `apps/api/src/events/events.service.ts:31` and `events.module.ts:7` describe the module as "who an EVENT series draws its roster from", and omit that it also serves the pickers, the student filter and the report.

Watching a live test, and intervening

Where it lives

Piece	Path	What it is
Routes	`apps/api/src/attempts/live-ops.controller.ts:33`	Six routes under `admin/live-ops`, all on `TEST_OPERATIONS`.
Board reads	`apps/api/src/attempts/live-ops.service.ts:38`	The picker list and the board: two raw SQL panel reads, one `count(*) FILTER`, three Prisma reads, one Redis MGET.
Pure board rules	`apps/api/src/attempts/live-ops.ts:38`	Lays held Redis state over a sitting row; decides which tests the picker offers.
Support actions	`apps/api/src/attempts/attempt-resolution.service.ts:49`	Force-submit, extend, reset, void, each audited against the student.
Action rules	`apps/api/src/attempts/attempt-resolution.ts:25`	Which action a status admits, and the audit diff.
Cohort report	`apps/api/src/attempts/test-analytics.service.ts:64`	Live curve plus the three rollup tables.
Workbook	`apps/api/src/attempts/test-report.ts:95`	Five sheets, built by the export worker.
Export plumbing	`apps/api/src/common/exporting/export-jobs.ts:68`, `apps/api/src/exports/`	Queue, status and file routes, the builders, the processor.
Audit	`apps/api/src/audit/`	Interceptor, context, service, controller, the daily archive.
Admin screens	`apps/admin/src/features/live-ops/`, `apps/admin/src/features/tests/test-analytics.tsx`, `apps/admin/src/features/admins/audit.tsx`	Board, analytics, audit log.
Wire and policy constants	`packages/contracts/src/live-ops.ts`	Row cap, recent window, poll interval, the reason and minutes schemas.

What it is for

Two jobs either side of a paper. While it runs, an ops admin needs to see, per test, who is still answering, who is past their own deadline and still open, and what landed in the last half hour, and to reach into one broken sitting without touching the other two thousand nine hundred and ninety-nine. After it, a test manager needs the cohort's spread, section averages and item analysis, and a workbook for the branches. A third concern runs under both: every intervention is a write against a real student's sitting, so every one leaves an audit row.

The two jobs sit on different feature keys on purpose: an ops admin holds `TEST_OPERATIONS` and neither `TEST_MANAGEMENT` nor the paper (`apps/api/src/attempts/live-ops.controller.ts:41` against `apps/api/src/attempts/test-analytics.controller.ts:33`). The audit log is on neither: it is a super admin's alone (`apps/api/src/audit/audit.controller.ts:47`).

Flow: the live board

GET /ADMIN/LIVE-OPS/TESTS/:TESTID — THE LIVE BOARD

1. Route enters under `@Actors(ADMIN)` and `@RequiresFeature(TEST_OPERATIONS, READ)`; no `@Audit`. Super admins bypass the feature check.
   `apps/api/src/attempts/live-ops.controller.ts:49`
   → request reaches `board()`
2. Test row read for id and title; absent is `NOT_FOUND`.
   `apps/api/src/attempts/live-ops.service.ts:81`
   → `{ id, title }`
3. The "landed" window is the last `LIVE_OPS_RECENT_MINUTES` (30) minutes, matched on both `submittedAt` and `updatedAt` so the `(testId, updatedAt)` index finds voided rows too.
   `apps/api/src/attempts/live-ops.service.ts:87`
   → `since`, `landed` filter
4. Seven reads in one `Promise.all`, no transaction: the paper's question count, the active panel, the past-deadline panel, both panel sizes, the landed count, the awaiting-scoring count, the landed rows.
   `apps/api/src/attempts/live-ops.service.ts:91`
   → counts and row sets
5. Each panel is its own raw query, `LIMIT 100` (`LIVE_OPS_ROW_CAP`), ordered by deadline, joining `Student` and `Branch` by primary key. Read apart so a hall of stuck sittings cannot fill the window and empty the running list.
   `apps/api/src/attempts/live-ops.service.ts:125`
   → up to 100 rows per panel
6. Both panel sizes come from one `count(*) FILTER` pass over the live index; the deadline only splits it.
   `apps/api/src/attempts/live-ops.service.ts:139`
   → `{ active, stuck }`
7. The `IN_PROGRESS` predicate is spliced in as a literal, not bound, so the planner can prove the partial index `Attempt_live_by_test_idx`.
   `apps/api/src/attempts/live-ops.service.ts:150`
   → index scan
8. One Redis MGET of at most 200 `attempt:state:<id>` keys. A GET, never a take: the board evicts nothing.
   `apps/api/src/attempts/attempt-state.service.ts:211`
   → `Map<attemptId, HeldState>`

9. The held state is laid over each row. `endsAt` comes from Redis when the key exists and from the row when it has gone; `answeredCount` is null with no key, which is what `hasLiveState: false` means.
   `apps/api/src/attempts/live-ops.ts:53`
   → `LiveSitting[]`
10. Response `LiveOpsBoard` with `serverNow`, the four counts and three row sets, wrapped by the envelope.
    `apps/api/src/attempts/live-ops.service.ts:113`
    → `{ testId, testTitle, serverNow, counts, active, stuck, recent }`
11. The page polls every `LIVE_OPS_POLL_MS` (15 s) and stops asking once the server has refused rather than failed. A failed poll keeps the last good rows and raises one warning Alert.
    `apps/admin/src/features/live-ops/live-ops.tsx:67`
    → rows on screen

The board reads identity from the row and progress from Redis rather than keeping a counter in Postgres: an `answeredCount` column would be a write per autosave, which the scaling rules forbid. The cost is that a lost key shows "Live state lost" instead of a number (`apps/admin/src/features/live-ops/live-ops.tsx:287`), which is exactly the signal the reset action exists for. The board never reads or writes `AttemptSheet`, never takes a live key, and reads `Attempt` rows that are not written while a sitting runs. The one shared resource is Redis, and one MGET every 15 s against roughly 120 autosave operations a second at 3,000 sittings is noise (read, not measured).

The three panels

Tab	Source	Meaning
Active now	`IN_PROGRESS` and `endsAt >= now`	Still inside the deadline.
Past deadline	`IN_PROGRESS` and `endsAt < now`	Open past the deadline; the sweeper (every `ATTEMPT_SWEEP_EVERY_MS`, 2 min, `apps/api/src/queue/queues.ts:174`) ends it once the save grace has passed and the key is gone or abandoned.
Landed	submitted in the last 30 minutes, voided ones included	Newest first, capped at 100, with the true count beside it.

A panel is a sample of a hall and says so: "Showing N of M" appears whenever the count exceeds the rows (`apps/admin/src/features/live-ops/live-ops.tsx:235`). An info Alert names sittings that ended without a score yet (`:199`); the sweeper asks again for them on its own. The picker offers only `ACTIVE` tests (`apps/api/src/attempts/live-ops.ts:7`) and is searched by test title or series name.

Flow: force-submit one sitting

POST /ADMIN/LIVE-OPS/ATTEMPTS/:ATTEMPTID/FORCE-SUBMIT — ENDING A SITTING FOR THE STUDENT

1. Guards: admin, `TEST_OPERATIONS` at WRITE, then `@Audit(STUDENT, UPDATE)`. Body passes `ZodBody(forceSubmitAttemptSchema)`: `reason` is required, trimmed, 1 to 280 characters.
   `apps/api/src/attempts/live-ops.controller.ts:55`
   → validated `{ reason }`
2. The row is read for eight columns; `resolutionBlocker` refuses anything but `IN_PROGRESS` with `CONFLICT`, and a voided one with the void message whatever was asked.
   `apps/api/src/attempts/attempt-resolution.service.ts:189`
   → `ResolvableAttempt`
3. `submit.expire(attemptId)` with no deadline: the sweeper's own entry point, so scoring and the fold happen as on a real submit.
   `apps/api/src/attempts/submit.service.ts:106`
   → `end()` begins
4. Live state is READ, not taken, and written to `AttemptSheet.answers` before the claim, so a write that throws leaves the sitting open.
   `apps/api/src/attempts/submit.service.ts:113`
   → sheet holds the answers
5. The claim: `updateMany({ id, status: IN_PROGRESS }) -> SUBMITTED`, stamping `submittedAt`. The single gate; count 0 means the student's own submit won.
   `apps/api/src/attempts/submit.service.ts:150`
   → `status = SUBMITTED`
6. Behind the claim, `state.take` does GETDEL on the key and SREM on `attempt:dirty`; whatever it returned is written again. A later save is refused, not swallowed.
   `apps/api/src/attempts/submit.service.ts:121`
   → key gone, sheet final
7. A BullMQ scoring job is enqueued; a failure is only logged because the sweeper asks again.
   `apps/api/src/attempts/submit.service.ts:166`
   → scoring job queued
8. If another call ended it first, `submittedByThisCall` is false: the action answers `CONFLICT` and files nothing.
   `apps/api/src/attempts/attempt-resolution.service.ts:66`
   → no audit row
9. `record()` sets the audit entity to the student and the diff to `{ status: {from,to}, supportAction, attemptId, reason }`.
   `apps/api/src/attempts/attempt-resolution.service.ts:179`
   → `AuditContext` populated
10. After the response value, `AuditInterceptor` writes one `RowActionLog` row; a failed write is logged and never fails the request.
    `apps/api/src/audit/audit.interceptor.ts:48`
    → `RowActionLog` row

Force-submit differs from the student's own submit in what it omits: no ownership check, no tab check, no last-batch-as-newest-revision, no accepted/refused metric. `end()` and everything after it are identical, which is the point of routing through `expire`.

Flow: extend, and how the new deadline reaches a counting browser

POST /ADMIN/LIVE-OPS/ATTEMPTS/:ATTEMPTID/EXTEND — ADDING TIME

1. Body `{ minutes: 1..180, reason }`; the ceiling is `EXTEND_MINUTES_MAX`.
   `packages/contracts/src/live-ops.ts:109`
   → validated body
2. `IN_PROGRESS` only, or `CONFLICT`.
   `apps/api/src/attempts/attempt-resolution.ts:25`
   → row read
3. One statement locks the row `FOR UPDATE`, requires `IN_PROGRESS` again, and sets `endsAt = GREATEST(endsAt, now) + minutes`, bumping `updatedAt`. A stuck sitting counts from now or the extension buys nothing; an extension cannot shorten a clock.
   `apps/api/src/attempts/attempt-resolution.service.ts:82`
   → `Attempt.endsAt` moved, `{ from, to }` returned
4. No row back means the sitting left the state it was read in: `CONFLICT`.
   `apps/api/src/attempts/attempt-resolution.service.ts:94`
   → no audit row
5. `pushDeadline` patches `attempt:state:<id>.endsAt` by compare-and-swap, keeping the later of the two deadlines, TTL reset to 48 h. With no key it writes nothing, correctly, because the row now carries the deadline.
   `apps/api/src/attempts/attempt-state.service.ts:236`
   → Redis `endsAt` follows the row
6. If Redis fails, the error is logged and the extension stands. Refusing would be retried into double the minutes; the key follows the row on its next open.
   `apps/api/src/attempts/attempt-resolution.service.ts:96`
   → row is the record
7. The student's next autosave (about every 25 s, plus or minus 5 s jitter, or at 20 pending answers) returns the ack with `endsAt` and `serverNow`. Answering is the clock check.
   `packages/app-kit/src/autosave-policy.ts:9`
   → `AttemptSaveAck`
8. The client sets its clock from the ack.
   `packages/app-kit/src/exam/use-attempt-state.ts:295`
   → `ExamClock` replaced
9. `useClockCountdown` re-keys on the new clock, so the countdown jumps up on the next tick with no reload. Worst case is about 30 s plus the request.
   `packages/app-kit/src/exam/use-countdown.ts:42`
   → seconds left grows
10. The audit diff carries `endsAt {from,to}` and `minutes`.
    `apps/api/src/attempts/attempt-resolution.service.ts:101`
    → `RowActionLog` row

Reset and void

Reset (`apps/api/src/attempts/attempt-resolution.service.ts:109`) is for a lost live key. `state.reestablish` rebuilds the key from the durable state (the attempt, `AttemptSheet.answers`, the paper rows; unvisited slots are dropped) and merges whatever the key still holds over it, the key's answers winning and the later deadline standing (`apps/api/src/attempts/attempt-state.service.ts:225`, `:480`). It writes Redis only; the audit diff carries `answersRestored`. It admits `IN_PROGRESS` alone: a marked sitting is never put back, "Void it instead" is the refusal text (`apps/api/src/attempts/attempt-resolution.ts:20`). The dialog now says what is true of both branches: section timers keep what they have left, and start again from where the paper says only if the live state is gone (`apps/admin/src/features/live-ops/sitting-actions.tsx:74`).

Void (`apps/api/src/attempts/attempt-resolution.service.ts:120`) archives rather than deletes and is the only action open to every status except already void.

POST /ADMIN/LIVE-OPS/ATTEMPTS/:ATTEMPTID/VOID — STANDING A SITTING DOWN

1. Guards as above but `@Audit(STUDENT, DEACTIVATE)`; the current admin id is passed in. Body `{ reason, regrantRanked }`.
   `apps/api/src/attempts/live-ops.controller.ts:86`
   → `voidSitting()`
2. `require` allows any status but `VOIDED`.
   `apps/api/src/attempts/attempt-resolution.ts:25`
   → row read
3. The ranked slot is spent by default: `regranted = isGraded && regrantRanked`.
   `apps/api/src/attempts/attempt-resolution.service.ts:128`
   → decision
4. `updateMany` where `status != VOIDED` sets `status`, `voidedAt`, `voidReason`, `voidedById`, and `isGraded = false` only when regranting. Clearing the flag IS the regrant. Count 0 is `CONFLICT`.
   `apps/api/src/attempts/attempt-resolution.service.ts:130`
   → `Attempt` row archived
5. `state.take` drops the live key; from then on the Postgres fallback refuses every save with `SITTING_ENDED`.
   `apps/api/src/attempts/attempt-resolution.service.ts:143`
   → key gone
6. If the sitting was `IN_PROGRESS` and a stray state came back, it is written to the sheet so what was answered is kept. Behind an ended sitting it would unpick a marked sheet, so it is skipped.
   `apps/api/src/attempts/attempt-resolution.service.ts:145`
   → `AttemptSheet.answers` current
7. `rollup.rebuild(testId)` and `rollup.rebuildStudent(studentId)` are enqueued, delayed 60 s, unconditionally; failure is logged and the void stands.
   `apps/api/src/attempts/attempt-resolution.service.ts:168`
   → two BullMQ jobs

8. Audit diff: `status {from, VOIDED}`, plus `isGraded {true,false}` when regranted.
   `apps/api/src/attempts/attempt-resolution.service.ts:150`
   → `RowActionLog` row

A void is terminal: there is no un-void route, and every other action needs `IN_PROGRESS`. A mid-sitting student's next autosave is answered `SITTING_ENDED`, and the client treats that as over: `refused` calls `endHere()` and the exam shows "This test has ended" (`packages/app-kit/src/exam/use-attempt-state.ts:208`, `packages/app-kit/src/exam/exam-view.ts:95`). Scoring cannot overwrite a void: the scorer's claim is limited to `SUBMITTED` and `EVALUATED` (`apps/api/src/attempts/scoring.processor.ts:57`).

The row menu

`SittingActions` offers Submit now, Add time and Put the live sitting back only on a live row, then Void on every row; the Landed panel hides the menu on a voided row (`apps/admin/src/features/live-ops/live-ops.tsx:381`). The menu renders only with WRITE (`:65`, `:312`). One `ConfirmDialog` and one mutation serve a whole panel rather than a hundred rows each holding a form (`apps/admin/src/features/live-ops/sitting-actions.tsx:147`). The dialog title names the student by name and mobile, because a hall can hold two students with the same name and the rows re-sort under the pointer. The "Give the ranked attempt back" checkbox appears only on a ranked sitting.

Flow: the audit row

ANY @AUDIT ROUTE — HOW A WRITE BECOMES A ROWACTIONLOG ROW

1. The reflector finds `@Audit(feature, action)` on the handler or class; no metadata, no audit.
   `apps/api/src/audit/audit.interceptor.ts:37`
   → `AuditRoute`
2. A service that knows the diff sets it on the per-request `AsyncLocalStorage` store; support actions also set the entity to the student.
   `apps/api/src/audit/audit.context.ts:25`
   → `{ changed, entityId }`
3. Only after the handler succeeds does `tap` run. `setPatchDiff(null)` marks a no-op and suppresses the row.
   `apps/api/src/audit/audit.interceptor.ts:48`
   → decision to write
4. Entity resolves as the store's, else a created row's id, else `:id`, else the payload's `id`. With none, no row.
   `apps/api/src/audit/audit.interceptor.ts:55`
   → `entityId`
5. Actor is the authenticated admin or student; with no user it is `SCRIPT`, never defaulted to ADMIN.
   `apps/api/src/audit/audit.interceptor.ts:79`
   → `actorType`, `actorId`
6. `AuditService.record` creates the row, fire-and-forget: a failure is logged, never raised, because the write already happened.
   `apps/api/src/audit/audit.service.ts:137`
   → `RowActionLog` row

What the screen shows

The log lists `RowActionLog` rows newest first, 30 days of them (`AUDIT_WINDOW_DAYS`, `packages/contracts/src/audit.ts:124`). Student-actor rows are excluded for every viewer (`apps/api/src/audit/audit.service.ts:104`): the log is what admins did. The screen shows no entity id; the export keeps a Record column. The whole controller is behind `@RequiresSuperAdmin()`, the route gate refuses non-super-admins before mounting (`apps/admin/src/lib/constants.ts:727`), and the service still pins a non-super viewer to their own rows as a second lock (`apps/api/src/audit/audit.service.ts:102`).

A daily job moves older rows to S3. The scheduler is registered by the worker container only (`apps/api/src/audit/audit.module.ts:30`) at `15 21 * * *` UTC (`apps/api/src/queue/queues.ts:177`), which is 02:45 IST. It takes the oldest pending institute day, holds a Redis lock for it, writes gzip NDJSON to `audit/row-actions/YYYY/MM/DD.ndjson.gz` with actor names resolved in, checks the stored size equals the bytes written, and only then deletes the day's rows (`apps/api/src/audit/audit-archive.processor.ts:141`). A run catches up at most 14 days.

Flow: analytics, re-sync and export

GET /ADMIN/TESTS/:ID/ANALYTICS — THE COHORT REPORT

1. `@RequiresFeature(TEST_MANAGEMENT, READ)`.
   `apps/api/src/attempts/test-analytics.controller.ts:33`
   → request reaches `forTest`
2. Test read for id, title and series; absent is `NOT_FOUND`.
   `apps/api/src/attempts/test-analytics.service.ts:103`
   → `{ id, title, testSeriesId }`
3. Seven reads in one `Promise.all`: `TestStat` by primary key, the live curve, the topper, `TestSectionStat`, `TestQuestionStat`, the count of every sitting (voided included), the audience size for the series.
   `apps/api/src/attempts/test-analytics.service.ts:74`
   → raw ingredients
4. The curve is a raw `GROUP BY score` over the ranked cohort (`IN_COHORT`: graded, evaluated, score not null), then mean, median, highest, lowest and bands are derived in Node.
   `apps/api/src/attempts/cohort-curve.ts:7`
   → `CohortShape`
5. The topper is one probe of `Attempt_ranking_idx`, then one `findUnique` for name, score and sheet time.
   `apps/api/src/attempts/test-analytics.service.ts:148`
   → `TestTopper`
6. Stored sums become averages here. `isSettling` is `live.size !== evaluatedCount`, the only freshness signal; `computedAt` is the last recount.
   `apps/api/src/attempts/test-analytics.ts:52`
   → `TestAnalyticsSummary`

7. Sections are ordered by paper order; items carry the p-value and three derived signals (below chance, mostly left blank, twice the paper average).
   `apps/api/src/attempts/test-analytics.ts:75`
   → `TestAnalytics`

POST /ADMIN/TESTS/:ID/ANALYTICS/RESYNC — RECOUNT ON DEMAND

1. `@RequiresFeature(TEST_MANAGEMENT, WRITE)`, answers 202 with an empty body.
   `apps/api/src/attempts/test-analytics.controller.ts:40`
   → accepted
2. Test checked, then `rollups.rebuildNow(testId)`.
   `apps/api/src/attempts/test-analytics.service.ts:98`
   → queue call
3. Job `REBUILD_TEST` is added under its own id `rollup-rebuild-<testId>-now`, delay 0, apart from the debounced id so a waiting rebuild cannot swallow it.
   `apps/api/src/attempts/rollup-queue.ts:42`
   → BullMQ job
4. The rollup worker runs `rebuildTest`: `recountTest` (upsert `TestStat`, delete and insert `TestSectionStat`, one transaction) then `recountTestItems` (replay every first sitting's sheet, delete and insert `TestQuestionStat`).
   `apps/api/src/attempts/rollup.service.ts:271`
   → three rollup tables rewritten
5. The screen marks `computedAt` and `wasSettling` at the click and polls every 3 s until either moves, giving up at 150 s with a toast.
   `apps/admin/src/features/tests/test-analytics.tsx:97`
   → figures refresh

A resync exists because nothing folds the cohort on a first evaluation. The sweep picks tests where a sitting that moves the cohort (voided, or graded and evaluated) was updated after `TestStat.computedAt` minus a 2 minute lag (`apps/api/src/attempts/rollup.service.ts:97`, `:146`), and item statistics wait on their own clock as well (`:161`). A void, re-score or late submission can therefore leave the tables behind for minutes, and `isSettling` is how the screen says so.

The export is a job, not a request

The workbook no longer builds inside the HTTP request. The route is a POST that queues a job and answers 202 with an id.

POST /ADMIN/TESTS/:ID/REPORT/EXPORT — THE TEST WORKBOOK

1. `@RequiresExport(TEST_MANAGEMENT)`: that key's READ and `DATA_EXPORT` READ. Answers 202 `{ id }`.
   `apps/api/src/attempts/test-analytics.controller.ts:47`
   → request accepted
2. `ExportJobsService.start` adds a BullMQ job `build` with a random UUID as its id, carrying the request and the asker's id, super-admin and active flags, never the session.
   `apps/api/src/common/exporting/export-jobs.ts:74`
   → job queued
3. The exports processor routes the request to the owning module's builder; an `AppException` it raises completes the job as that error instead of retrying.
   `apps/api/src/exports/exports.processor.ts:44`
   → `BuiltExport` or error outcome
4. `buildTestReport` runs the analytics read again, then `assertExportable` caps at 50,000 rows for results and again for absentees, and writes five sheets: Results, Absent, Sections, Questions, Summary.
   `apps/api/src/attempts/test-report.ts:95`
   → workbook Buffer
5. The file is uploaded to `exports/<jobId>.xlsx` and a delayed `discard` job is queued for one hour later.
   `apps/api/src/exports/exports.processor.ts:51`
   → S3 object, 1 h life
6. The EXPORT audit row is written with the test as entity and `results`, `absent` counts; a failure there never costs the asker the file.
   `apps/api/src/exports/exports.processor.ts:56`
   → `RowActionLog` row
7. The client polls `GET /admin/exports/:id` and collects `GET /admin/exports/:id/file`. Only the asker can read either; another admin's id is `NOT_FOUND`.
   `apps/api/src/exports/exports.controller.ts:14`
   → `ExportJob`, then the file

Seven exports share this path: the question bank, the student roster and performance view, every report, a test's results, a series' grants, an announcement's deliveries and the audit log (`apps/api/src/exports/export-builders.ts:50`). The client still hands every screen one `Promise<Blob>` (`requestExport`), so the button (`apps/admin/src/features/tests/test-analytics.tsx:180`) is unchanged. The analytics screen is reached from the test list, the builder and the series screen; it has no nav entry. The nav entry for the board is "Live operations > Sittings" (`apps/admin/src/lib/constants.ts:518`).

The data it touches

Column / key	Holds	Written by
`Attempt.status`	`IN_PROGRESS`, `SUBMITTED`, `EVALUATED`, `EXPIRED`, `VOIDED`	the claim `apps/api/src/attempts/submit.service.ts:153`; void `apps/api/src/attempts/attempt-resolution.service.ts:130`
`Attempt.endsAt`	server-owned deadline	extend `apps/api/src/attempts/attempt-resolution.service.ts:82`
`Attempt.isGraded`	the one ranked slot	cleared by a void with regrant
`Attempt.voidedAt`, `voidReason`, `voidedById`	the archive stamp	void
`AttemptSheet.answers`	one slot per paper row	the submit path and a void of a live sitting

Column / key	Holds	Written by
`TestStat`, `TestSectionStat`, `TestQuestionStat`	sums and counts for the ranked cohort	`apps/api/src/attempts/rollup.service.ts:194`, `:243`
`RowActionLog.changed`	`{ supportAction, attemptId, reason, ...moved }`	`AuditInterceptor`
Redis `attempt:state:<attemptId>`	held sitting state, TTL 48 h	autosave, extend, reset
Redis `attempt:dirty`	set of keys owed a flush; SREM'd by `take`	autosave, `take`
S3 `exports/<jobId>.xlsx`, `audit/row-actions/...`	export files for an hour; archived audit days	export worker; archive processor

What must already be true

An admin JWT; `TEST_OPERATIONS` at READ to watch and WRITE to act, or super admin; `TEST_MANAGEMENT` READ for analytics, WRITE for resync, plus `DATA_EXPORT` READ for the workbook. The picker offers only `ACTIVE` tests. The board shows a denominator only for a test with `PaperQuestion` rows. Analytics needs a `TestStat` row to show the recount; without one it still shows the live curve and counts, and the body says "No ranked results yet" at zero. The partial index `Attempt_live_by_test_idx` (`prisma/migrations/20260918100000_the_live_sittings_are_indexed_apart_from_the_finished_ones/migration.sql:49`) and `Attempt_unscored_idx` must exist. Resync and export need the BullMQ rollup and exports workers running.

Why it is shaped this way

- **Panels are capped and read apart from the counts.** One unbounded query over 3,000 sittings would be the same read; a hall of stuck sittings would fill one window and empty the running list.
- **Void archives and asks for a recount.** A decrement would be a second place the truth lives and was wrong after every void. `TestStat` stores counts and sums only; mean, median, highest, lowest and topper are counted live off `Attempt_ranking_idx` on every read.
- **Actions go through the engine's own gates.** Force-submit is `expire`, extend moves the row first and Redis second, void claims by status. Two calls racing on one sitting land exactly once, because each write matches the status it read.
- **A failed queue never hands a sitting back.** The void and the extension are committed; the recount and the Redis push are logged on failure.

Is it optimal

Board: 8 Postgres statements and one Redis MGET per poll, 15 s apart, about 32 statements a minute per watching admin, independent of how many students sit. The two panel reads and the count ride `Attempt_live_by_test_idx`, a partial index that holds a few thousand entries whatever history grows to (the migration measured 129 pages and 0.9 ms down to 3 to 7 pages and 0.07 to 0.25 ms). The awaiting-scoring count has its own partial index, measured at 513 buffers and 0.82 ms (`docs/03-conventions.md`). Analytics is about ten statements, none per row; its payload is one object per paper question. Export no longer holds a request open: the build and the 50,000-row workbook sit on a worker. Read, not measured: board cost at a 3,000-sitting landing spike.

What changed

Commit	Effect
`043c6f2`	A void keeps what was answered (stray state written to the sheet); an extend cannot shorten a clock.
`a09d0d5`	Extend is one locked statement; force-submit, void and reset each win or lose once against a racing call.
`a952f22`	A refused board stops polling and says so; each dialog names its student.
`ed468ed`, `39a8541`	Spread and section columns show before the first recount lands.
`dc1f1db`	Every admin export is a queued job; the test workbook is POST and answers 202.
`d68dcb5`	The audit log is a super admin's alone; the screen is refused at the route before it mounts.
`5998b7e`	The log drops the entity id column and student-actor rows.
`f472a4d` (earlier)	Board reads became two raw panel reads plus one `count(*) FILTER`, with `status` as a literal.

What is wrong with it

- **Correctness** `apps/api/src/audit/audit-archive.ts:9` — `AUDIT_RETENTION_DAYS` is its own `30`, while `packages/contracts/src/audit.ts:124` calls `AUDIT_WINDOW_DAYS` "one number, two consumers"; the screen's "last 30 days" copy and the archive cut-off can drift apart, and the archive has no reader in the app, so an older row is reachable only from S3 by hand.
- **Stale** `apps/api/src/attempts/live-ops.ts:18` — an orphan doc comment ("What the ops board shows...") sits above `SittingRow` and describes nothing in particular.

Reports

A report is a page somebody carries out of the building: a result sheet pinned to a notice board, a letter to a parent, the one page a superior reads on Monday. The reports module is the read model behind all of them. It writes no table, owns no table, and answers 53 reports from one route. Whatever a report holds is shaped once, then read three ways: by the admin screen, by the printer, and by the spreadsheet download.

The code is `apps/api/src/reports/`. The catalogue, the query rules and the document shape are in `packages/contracts/src/reports.ts`. The admin screens are `apps/admin/src/features/reports/`. A student's slice of it is five reports behind `/me/reports`.

The shape of a report

A report has two halves, and a third piece joins them.

- **A catalogue row** (`REPORTS`, `packages/contracts/src/reports.ts:146`) is code-owned. It holds a title, a group, `needs` (what it cannot be built without), `takes` (what narrows it), the named period its picker opens on, and an optional `superAdminOnly`. Both SPAs and the API read the same row.
- **A builder** (`ReportBuilder`, `apps/api/src/reports/report.ts`) is a function `(sources, query, viewer) => Report`. A `Report` is `about` facts (what it covers), `figures` (headline numbers), `sheets`, and optional `preface` / `closing` paragraphs. A sheet is an `ExportSheet`, the same type every other export writes.
- **A registry** (`BUILDERS`, `apps/api/src/reports/reports.service.ts:35`) spreads eight per-group maps into one `ReportBuilders` object. Its type is `{ [K in ReportKey]: ReportBuilder<ReportQueryOf<K>> }`, so a key without a builder is a compile error.

The query type each builder receives is derived from its catalogue row (`ReportQueryOf`, `packages/contracts/src/reports.ts:378`). A builder whose row `needs` a test gets `testId` as a required field, and it cannot read a parameter its row does not name. The service checks the row at run time (`reportFieldsMissing`) before it calls the builder, so the type's promise is kept.

Because a sheet is an export sheet, the printed table and the workbook sheet are written off one column list and cannot disagree. A column marked `fileOnly` is the only thing a file holds that a page does not (`tableOf`, `apps/api/src/reports/report.ts:101`).

Report inventory

53 keys, in seven groups. "Needs" is what the report cannot be built without; "takes" is optional. A period is two civil dates (`from`, `to`). Sheet names are the tab names of the workbook and the tabs of the screen.

Key	Title	Needs	Takes	Sheets	Builder file
Tests					
`test-results`	Result sheet	test		Results	`test-reports.ts`
`test-summary`	Test summary	test		Score spread, Sections	`test-reports.ts`
`test-sections`	Section analysis	test		Sections	`test-reports.ts`
`test-items`	Item analysis	test		Questions	`test-reports.ts`
`test-absentees`	Absentees	test		Absent	`test-reports.ts`
`test-merit`	Merit list	test	top	Merit list, Branch toppers	`test-reports.ts`
`test-branches`	Branch comparison	test		Branches	`test-reports.ts`
`test-cutoffs`	Sectional cutoffs	test		Sectional cutoffs	`test-reports.ts`
`test-typing`	Typing results	test		Typing results	`test-reports.ts`
`test-voided`	Void sittings	test		Void sittings	`test-reports.ts`
`test-answer-key`	Answer key (super admin)	test		Answer key	`test-reports.ts`
`test-activity-weekly`	Weekly test activity	period (last week)		Tests	`period-reports.ts`
`test-activity-monthly`	Monthly test summary	period (last month)		Tests	`period-reports.ts`
`series-progress`	Series progress	series		Tests, Students	`period-reports.ts`
`test-schedule`	Test schedule	period (this week)		Schedule	`period-reports.ts`
`participation-trend`	Participation trend	period (this month)		Days	`period-reports.ts`
Students					
`student-score-card`	Score card	student, attempt		Sections	`student-reports.ts`
`student-weekly`	Weekly student report	student, period (last week)		Sittings, Subjects	`student-reports.ts`
`student-monthly`	Monthly student report	student, period (last month)		Sittings, Subjects	`student-reports.ts`
`student-cumulative`	Cumulative student report	student		Subjects, Sittings	`student-reports.ts`
`student-parent-letter`	Progress letter to parents	student, period (last month)		the weekly sheets, plus a printed letter	`student-reports.ts`

Key	Title	Needs	Takes	Sheets	Builder file
`student-topics`	Topic-wise accuracy	student		Topics	`student-reports.ts`
`student-missed`	Tests missed	student		Tests missed	`student-reports.ts`
Performance					
`performance-by-branch`	Branch performance	period (last month)		one comparison sheet	`cohort-reports.ts`
`performance-by-program`	Program performance	period (last month)		one comparison sheet	`cohort-reports.ts`
`performance-by-exam`	Exam performance	period (last month)		one comparison sheet	`cohort-reports.ts`
`top-performers`	Top performers	period (last month)	branch, top	Top performers	`cohort-reports.ts`
`most-improved`	Most improved	period (last month)	branch, top	Most improved, Fallen back	`cohort-reports.ts`
`weak-subjects`	Subject-wise accuracy	period (last month)	branch	Subjects	`cohort-reports.ts`
`weak-topics`	Topic-wise difficulty	period (last month)		Topics	`cohort-reports.ts`
`absentees`	Absentees across tests	period (last month)		Absentees	`cohort-reports.ts`
`inactive-students`	Inactive students	none	days, branch	Inactive students	`cohort-reports.ts`
`retakes`	Retakes	period (last month)		Retakes	`cohort-reports.ts`
Enrolment					
`student-roster`	Student roster	none	branch	Students	`enrolment-reports.ts`
`new-enrolments`	New enrolments	period (this month)		By branch, Students	`enrolment-reports.ts`
`strength`	Strength	none		headcount sheets	`enrolment-reports.ts`
`profile-completeness`	Unfinished profiles	none	branch	Unfinished profiles	`enrolment-reports.ts`
`student-status`	Suspended, blocked and deleted students	none		Students	`enrolment-reports.ts`
`manual-grants`	Access granted by hand	period (this month)		Grants	`enrolment-reports.ts`
`event-candidates`	Event candidates	event		Candidates	`enrolment-reports.ts`
`mobile-changes`	Mobile number changes	period (this month)		Mobile changes	`enrolment-reports.ts`
Content					
`authoring-progress`	Authoring progress	test		Sections	`content-reports.ts`
`overdue-assignments`	Overdue sections	none		Overdue	`content-reports.ts`
`staff-output`	Staff output	period (last week)		Staff output	`content-reports.ts`
`send-backs`	Questions sent back	period (last week)		Send-backs	`content-reports.ts`
`bank-inventory`	Question bank inventory	none		Subjects, Topics	`content-reports.ts`
`question-usage`	Question usage	none	top	Subjects, Most used	`content-reports.ts`
Operations					
`audit-trail`	Audit trail (super admin)	period (last week)		the audit log's own sheet	`operations-reports.ts`
`admin-activity`	Admin activity (super admin)	period (last week)		Admin activity	`operations-reports.ts`
`announcements`	Announcements and their cost	period (last month)		Announcements, Deliveries	`operations-reports.ts`
`permissions-matrix`	Admins and permissions (super admin)	none		Permissions	`operations-reports.ts`
Institute					
`digest-weekly`	Weekly institute report	period (last week)		the activity, branch and top-performer sheets	`digest-report.ts`
`digest-monthly`	Monthly institute report	period (last month)		the same	`digest-report.ts`

Three builders are worth knowing by shape.

- **The digest** composes other reports. It runs the weekly activity, branch performance and top-performers builders, adds institute headline counts, and returns their sheets. A figure on the digest is the figure on those reports because it is their figure (`apps/api/src/reports/digest-report.ts:16`). It finds the attendance headline by its label string, `'Students who sat'`.
- **The parent letter** is the weekly student report with words around it. `preface` opens with "Dear Parent," and `closing` ends in a signature line (`apps/api/src/reports/student-reports.ts:254`). The screen shows these in a Letter tab, the printer sets them around the tables, and a workbook ignores them.
- **Test reports lend, not recount.** The test group does not re-query results. It asks `TestReportSheets` and the figure owners that `attempts` exports (`SittingFigures`, `apps/api/src/reports/report.ts`: analytics, leaderboard, performance, student overview, attempt report, paper sheet). Where an owner already defines a figure, the report reads it rather than counting again.

Periods, choices and the query

One query schema covers every report (`reportQuerySchema`, `packages/contracts/src/reports.ts:337`): `testId`, `studentId`, `attemptId`, `seriesId`, `eventId`, `branchId`, `from`, `to`, `top` (1 to 500), `days` (1 to 365). Two refinements guard the period. It cannot end before it starts, and it is at most 366 days long, so no query can be asked to walk centuries.

- **A period is civil dates at the institute.** `periodOf` turns two `YYYY-MM-DD` strings into a pair of bounds with `startOfInstituteDay` / `endOfInstituteDay` (`apps/api/src/reports/period.ts:14`), so a sitting handed in at 00:30 IST on Monday is Monday's sitting. `reportPeriodOf` (`packages/contracts/src/reports.ts:399`) turns a name (this week, last week, this month, last month) into dates; a week runs Monday to Sunday.
- **A report opens on its own period.** A report whose row `needs` a period opens on the row's `period`, so a screen is never blank. A weekly report opens on last week, a roll report on this month.
- **Pickers have their own route.** A test, student, attempt, series, event or branch is chosen from a list too long to hand over whole. `GET /admin/reports/choices/:param` (`apps/api/src/reports/reports.controller.ts:44`) pages and searches each, behind `REPORTS` READ alone. Borrowing each list's own route would make asking for a report require every other feature's grant. The attempt picker lists marked sittings only and refuses to answer until a student is named (`apps/api/src/reports/report-choices.ts:72`).

Reading a report

GET /ADMIN/REPORTS/:KEY — READING A REPORT ON THE SCREEN

1. Route is declared after `choices/:param`, which would otherwise be taken as a key. The `:key` param is parsed by `reportKeySchema`; the query by `reportQuerySchema`
   `apps/api/src/reports/reports.controller.ts:53`
   → a typed `ReportKey` and `ReportQuery`
2. `FeaturePermissionGuard` reads `RequiresFeature(REPORTS, READ)`; a deactivated admin is refused before the super-admin bypass
   `apps/api/src/auth/guards/feature-permission.guard.ts:34`
   → 403 or on
3. `ReportsService.document` calls `build`, which refuses a `superAdminOnly` row to anyone who is not a super admin
   `apps/api/src/reports/reports.service.ts:119`
   → `FORBIDDEN` "Only a super admin may open this report"
4. `build` lists the fields the row needs and the query lacks, and refuses with field errors
   `apps/api/src/reports/reports.service.ts:122`
   → `VALIDATION_ERROR` with `fieldErrors` per missing field
5. The builder for the key runs against Postgres and the figure owners, with the viewer
   `apps/api/src/reports/reports.service.ts:129`
   → a `Report`
6. `toDocument` turns each sheet into a table: hidden `fileOnly` columns dropped, dates formatted, rows cut at `REPORT_MAX_ROWS` (5,000), `total` keeps the full count
   `apps/api/src/reports/report.ts:113`
   → `ReportDocument` with `asOf`
7. The screen draws `about` and `figures` in a side card, one tab per table
   `apps/admin/src/features/reports/report.tsx:155`
   → the rows on screen

A read writes no audit row. Rank and percentile are counted live from Postgres, so `asOf` is the moment of building, and the screen and the printed page both say so.

The admin screens are two.

- **The catalogue** (`apps/admin/src/features/reports/reports.tsx`) groups the 53 reports as cards under seven headings. A search covers titles and one-line summaries (`REPORT_SUMMARIES`, `apps/admin/src/lib/constants.ts:567`), with apostrophes folded so a typed `'` matches the catalogue's `’`. A card names what the report needs, what may narrow it, and the period it opens on. A report only a super admin may open is left out for everyone else, and shown with a "Super admin only" badge to a super admin (`reports.tsx:81`).
- **A report page** (`report.tsx`) is a split frame with a filter bar, a side card of facts, and the rows. Its filters are drawn from the catalogue row (`reportFilters`, `report-filters.tsx`): each param has one control, and a needed one cannot be cleared. Choosing another student lets go of the sitting (`report-filters.tsx:85`). The filters live in the URL, so a link lands on the same report with the same filters, down to the open table (`?table=`).

The page asks only when it is ready. The query is built from the URL, parsed by the same schema the API uses, and checked against the row's `needs`. Until it is ready the body names what is lacking ("No student chosen"); a refusal from the schema ("A period is at most a year long") shows as itself.

Printing

Print is client side. `reportHtml(document)` (`packages/app-kit/src/report-html.ts`) builds one standalone page of HTML, and `printHtml` sets it in a hidden frame. The browser paginates, repeats column headings and breaks between rows. On a phone the same HTML goes to the print service.

- Every cell is escaped, since each value is a student's name, a test title or a reason somebody typed.
- A table of more than seven columns is turned on its side.
- A column whose every filled cell is a number is set right-aligned (`holdsFigures`, `packages/contracts/src/reports.ts:430`).
- The page carries the IACE logo as a drawing at its head and behind its content, and a footer of its own with the page count, so the browser adds no date or address line.
- Print carries at most the 5,000 rows the document holds. The table tells the reader: "Print carries 5,000, and Export every one" (`report.tsx:321`).

Exporting

The workbook is built off the request path, on the exports queue, like every other admin export.

POST /ADMIN/REPORTS/:KEY/EXPORT — DOWNLOADING A REPORT AS A SPREADSHEET

1. Guard: `RequiresExport(REPORTS)` sets the feature to `REPORTS` READ with `export: true`; the guard then also demands `DATA_EXPORT`
   `apps/api/src/reports/reports.controller.ts:62`
   → 403 "You do not have access to exports" without it
2. Key and query are parsed as for a read; nothing is built yet
   `apps/api/src/reports/reports.controller.ts:66`
   → typed `key`, `query`
3. `ExportJobsService.start` enqueues a `build` job carrying the request and a snapshot of the asker (`id`, `isSuperAdmin`, `isActive`), never the session
   `apps/api/src/common/exporting/export-jobs.ts:74`
   → BullMQ job whose id is a new UUID; 202 `{ id }`
4. The admin client polls `GET /admin/exports/:id` every 1.5 s for up to 10 minutes
   `packages/contracts/src/exports.ts:23`
   → `running`, `done` or `failed`
5. `ExportsProcessor` runs the job and routes `REPORT_EXPORT` to `ReportsService.workbook`
   `apps/api/src/exports/export-builders.ts:72`
   → `Report`
6. `workbook` writes an About sheet (report title, `about`, `figures`, "As of") ahead of the report's sheets, every row, uncut
   `apps/api/src/reports/reports.service.ts:107`
   → `Buffer` and a row count
7. An `AppException` raised by a builder (a missing student, an oversize roll) completes the job as the asker's answer, not a retry
   `apps/api/src/exports/exports.processor.ts:47`
   → job `failed` with the error's code and message
8. The workbook is uploaded to storage under `exports/<jobId>.xlsx` and a `discard` job is scheduled for one hour later
   `apps/api/src/exports/exports.processor.ts:52`
   → S3 object, delayed BullMQ job
9. An `EXPORT` audit row is recorded against the admin, feature `REPORT`, with the report key, the chosen filters and the row count; a failed audit write is logged and the file is still served
   `apps/api/src/exports/exports.processor.ts:63`
   → `RowActionLog` row
10. `GET /admin/exports/:id/file` streams the workbook to the asker alone; another admin's id reads as no file
    `apps/api/src/exports/exports.controller.ts:19`
    → `.xlsx` named `<key>-<date>.xlsx`

Each report downloads under its own key (`ExportKind` is the old kinds plus `ReportKey`), so ten reports do not land as ten files of one name. The button (`apps/admin/src/components/export-button.tsx`) is left out for an admin without `DATA_EXPORT`.

Reports that read a roll of people count first and refuse a read that is too large, before any row is read (`assertExportable`, `EXPORT_MAX_ROWS` is 50,000): the roster, unfinished profiles, student status, manual grants, and the cohort reports' sittings. A spreadsheet is never cut at 5,000. The screen and the print are.

A student's own reports

A student reads five reports of themselves, never anyone else's: score card, weekly, monthly, cumulative (every test) and topic-wise accuracy (`STUDENT_REPORT_KEYS`). They come from `GET /me/reports/:key` and `GET /me/reports/:key/export` (`apps/api/src/reports/reports.controller.ts:75`).

- The caller is the student. `own` overwrites `studentId` in the query with the token's id (`reports.service.ts:98`) and passes a viewer that is nobody's admin, so no super-admin report is theirs to ask for.
- Their spreadsheet is built in the request, not on the queue, and returned directly (`sendWorkbook`). It asks for no `DATA_EXPORT` and writes no audit row, because those are about an admin carrying other people's records out.
- A student's report reads their newest 200 marked sittings (`REPORT_SITTINGS_MAX`), since each sitting is ranked against its whole cohort on a route a student can call. A long career loses its oldest rows (`sittingsOf`, `apps/api/src/reports/student-reports.ts:91`).
- The Downloads tab of `apps/exams` and the Downloads tab of the mobile app share the pick logic in `packages/app-kit/src/report-downloads.ts`: Weekly, Monthly, Test, Every test and Topic-wise accuracy, with twelve weeks or months to go back. The student reads the report on the page first, then prints, saves as PDF or takes the spreadsheet.

Who may read what

Thing	Needs
The catalogue page, any report, any picker	`REPORTS` READ
The spreadsheet of any report	`REPORTS` READ and `DATA_EXPORT` READ
Answer key, audit trail, admin activity, permissions matrix	super admin (checked in `build`; hidden from the catalogue for others)
Student status "Since" and "By" columns	an admin sees only the status changes they made; a super admin sees all (`apps/api/src/reports/enrolment-reports.ts:296`)
Audit trail rows	the audit log's own scoping, handed the viewer (`apps/api/src/reports/operations-reports.ts:43`)
A student's five reports	the student's token, and only their own record

The question paper is not a report. It prints from the test's own paper screen behind `TEST_MANAGEMENT`, because an unsat paper is the most sensitive thing on the platform and `REPORTS` is a grant a branch office holds.

How it was built

The module landed on 2026-10-06 in a run of commits and was refined for a week.

- `0c77901` set the idea: a report is one shape, read by the screen, the printer and the file. `2d2962a` added the admin catalogue, a page each, Print and Export.
- `2cdfb8e`, `7e68e77`, `4497d88`, `85ef1b1`, `863bf33`, `ad32fa2` added the report families in turn: tests across periods, one student, branches and cohorts, the roll, the paper before anyone sits it, the office's own record and the weekly/monthly digest.
- `6ab1918` added the answer key and the rules a printed number is held to. `fee3547` capped a period at a year.
- `da4d249` and `4b3e81e` gave students a Downloads tab; `fabc5f8` added their own spreadsheet.
- `c2b0b64`, `429a8c2`, `7fdc801` gave a printed page the IACE mark and its own footer. `754a408` and `ca5f170` rebuilt the catalogue as cards with a search.
- `d20e62e` made the student-status report read the suspension itself rather than a sitting voided since. `3944c7c` made a student's report number options as served and list only the test's sections. `ee10363` made an invalid report address say what is wrong.
- `da2c7f8` (2026-10-13) added the Typing results report for a test with a typing section.

What is wrong with it

- `apps/api/src/reports/student-reports.ts:91` (the 200-sitting cap): a student's cumulative report silently drops sittings older than the newest 200 and does not say so on the page. Correctness, minor.
- `apps/api/src/reports/digest-report.ts:30`: the digest finds its attendance figure by the label string `'Students who sat'`, which `period-reports.ts:78` writes. Rename the label and participation goes blank with no error. Correctness, latent.
- `apps/api/src/reports/test-reports.ts:419`: the void-sittings builder reads every voided attempt with no count guard, unlike the roll and cohort reports. Correctness, minor, since the set is small.
- Reports are institute-wide: no builder scopes students by the reader's branch, so any `REPORTS` holder reads every branch's roster, mobile numbers and results (read, not exercised against a branch-office account). The domain doc calls `REPORTS` "a grant a branch office holds". Privacy, if a branch office is meant to see its own branch only.
- A report read writes no audit row, though the roster, student-status and mobile-change reports show phone numbers; only the export is logged. Privacy, by design per `docs/03-conventions.md`, and a call for the product owner.
- `apps/admin/src/features/reports/report.tsx:307`: the Letter tab keys paragraphs by their text, so two identical lines collide. Correctness, cosmetic.

Part IV — The clients

The student portal

`apps/exams` is the student web app: a Vite SPA, one of three student clients (with the Expo app) over one exam engine in `packages/app-kit/src/exam/`. The engine is DOM-free on purpose, so the rules of a sitting (clock, autosave, hand-in) cannot diverge between a phone app and a browser. This chapter covers the web client only: its routes, the shell, how it fetches, the four exam-screen skins, and the result screens. Server internals belong to the attempts, auth and catalog chapters.

Where it lives

Path	What it is
`apps/exams/src/App.tsx`	The whole route table. Every authed screen is `React.lazy`, so first paint pays for login and the shell alone.
`apps/exams/src/lib/constants.ts`	`ROUTES`, `NAV_ITEMS`, `STORAGE_KEYS`, the shared query keys re-exported from app-kit.
`apps/exams/src/lib/api.ts`	The one API client and token store (`iace.test.auth`), `CLIENT_KINDS.WEB`.
`apps/exams/src/lib/queries.ts`	Every shared student read, bound from app-kit's `createStudentQueries`.
`apps/exams/src/features/exam/`	The exam hall, instructions, and the skins under `templates/`.
`apps/exams/src/features/performance/`	`ReportShell` and its five tabs, plus the performance overview.
`apps/exams/src/features/typing/`	Typing practice, and the typing/written panes the sitting reuses.
`apps/exams/src/components/ui/index.ts`	This app's own composition layer (feed and shelf, not admin's table).
`apps/exams/public/sw.js`	The service worker: shell offline, push display.

The app was `apps/test` until `7912c96`; the storage keys still say `iace.test.*` and are named for the app, not the audience.

Boot and the shell

`apps/exams/src/main.tsx:19` builds one query client with `createAppQueryClient({ notify: toast })`, so every mutation failure is announced in one place. `captureInstallPrompt` runs before mount (the browser offers `beforeinstallprompt` once); `registerServiceWorker` waits for `load`.

`AuthProvider` comes from app-kit's `createAuth` with `actor: ActorTypes.STUDENT` (`apps/exams/src/providers/auth.ts:9`): an admin's JWT is valid but is not a session here. On sign-in it re-binds the browser's push subscription, because the server binds a subscription to one session.

`ProtectedRoute` (`packages/app-kit/browser/protected-route.tsx:7`) waits while identity loads, shows a Retry screen when a session is held but the server did not answer (an outage is not a sign-out), and otherwise sends the visitor to `/login` with `state.from`, so a deep link survives the sign-in (`apps/exams/src/features/auth/login.tsx:44`).

Sign-in is a code every time. A student has no password and no PIN. `MobileStep` (`login.tsx:92`) asks for the mobile and requests a code; `CodeStep` (`:130`) draws `challenge.codeLength` boxes, shows the dev code in an info Alert only when the sender returns one, and offers resend (`useResendCode`) and "Use a different number". The first sign-in is the signup. Codes go by SMS alone since `6afdf06`; the WhatsApp channel is behind `OTP_WHATSAPP_ENABLED`, server-side.

The shell (`apps/exams/src/components/app-shell.tsx:26`) wraps `AppShell` from app-kit with the nav, a wide frame, the avatar from the profile query, and a `TourProvider` keyed `iace.test.tours`. It runs one cheap read for the bell (`pageSize: 1`, because `meta.total` is the answer wanted, `:32`). Nothing polls: a push message from the worker invalidates it (`:40`), and refocusing the tab catches the rest. A rising unread count invalidates the notification lists (`:50`). If `student.isTestBlocked` a warning Alert is drawn above the outlet (`:82`); it is a banner, not a gate, since the server refuses the attempt anyway. There is no default-PIN gate any more.

Two screens sit outside the shell, inside `ProtectedRoute` but above `<Route element={<AppShell />}>` (`App.tsx:160–177`): `/tests/:testId/instructions` and `/tests/:testId/exam`, plus `/typing/:presetKey`. An exam hall has no navigation out of it, and the walk up to a paper is not a page to browse. So they get no nav rail, no bell, no tours, and no blocked banner.

Route map

Path	Screen	Frame
`/login`	`features/auth/login.tsx`	own
`/tests/:testId/instructions`	`features/exam/test-instructions.tsx`	own, outside shell
`/tests/:testId/exam`	`features/exam/exam.tsx`	own, outside shell
`/typing/:presetKey`	`features/typing/typing-session.tsx`	own, outside shell
`/`	`features/dashboard/dashboard.tsx`	`PageFrame`
`/tests`, `/series/:seriesId`	shelf per series; the series' own screen	`PageFrame`
`/tests/:testId/about`	brief, journey dialog, when it opens	`PageFrame`
`/attempts/:attemptId/submitted`	handed-in page	`PageFrame`
`/attempts/:attemptId/report` + `subjects`, `solutions`, `questions`, `compare`	`ReportShell`, five tabs as routes	`PageFrame` with tabs
`/attempts/:id/{score-card,review,questions}`	`ReportRedirect`, legacy links	redirect

Path	Screen	Frame
`/performance`, `/leaderboard`	figures across sittings; the ranking	`PageFrame`, `PanelFrame`
`/typing`	pick a standard, history	`PageFrame`
`/saved`	starred and wrong questions	`ListView`
`/downloads`	own reports, to print or keep	`PageFrame`
`/notifications`, `/notifications/settings`	the bell, as a feed	`PanelFrame`
`/profile`, `/account`	details; where the account is signed in	`PageFrame`
`*`	`not-found.tsx`	own

In DEV only, `/railway-preview`, `/banking-preview` and `/ssc-preview` (`App.tsx:124-148`) render a skin on a canned paper from `features/exam/templates/railway/preview-data.ts`; the page is lazy, so the canned paper never reaches a student's payload.

Data fetching

All reads go through TanStack Query. Keys live in app-kit (`student-queries.ts`) so web and mobile cache under the same names. The conventions worth knowing:

- **Reads that wait on marking** (`scoreCard`, `solutions`, `questionReport`, the typing-in-attempt read) share `AWAITS_MARKING` (`packages/app-kit/src/student-queries.ts:139`): a refusal that means "not marked yet" is retried a bounded number of times on the client's jittered backoff and is silent (no toast). There is no polling loop. If the retries run out the screen says "No marks yet" with a Retry (`score-card-gate.tsx:23`).
- **The brief** is silent on refusal too (`meta.silent: isBriefRefused`); the instructions page draws "This test is not open to you" instead of a toast.
- **The sitting's start and paper** are `staleTime: Infinity, gcTime: 0` (`packages/app-kit/src/exam/start-sitting.ts:87-90`): a second start is a resume on the server, so a refetch must never happen, and a cached start must not outlive its screen.
- **The paper** query stamps `arrivedAt: Date.now()` where the payload lands (`start-sitting.ts:101`), because that instant anchors the clock.

Flow: from "Start test" to the exam hall

CLICK START TEST ON A TILE — WALKING UP TO A PAPER

1. `StartSitting` is a `Link` whose click also calls `fullscreen.enter()` (a user gesture is what full screen needs; an effect cannot ask)
   `apps/exams/src/features/exam/start-sitting.tsx:31`
   → navigation to `/tests/:id/instructions` (with `?resume=<attemptId>` for a Resume)
2. `TestInstructionsPage` reads the brief; no question or answer is in it
   `apps/exams/src/features/exam/test-instructions.tsx:28`
   → `GET /me/tests/:testId/brief` cached under `briefQueryKey`
3. Meanwhile it warms the `./exam` and `./submitted` chunks, so beginning and handing in on a bad network never wait on a download
   `apps/exams/src/features/exam/test-instructions.tsx:23`
   → two lazy chunks in the browser cache
4. Once the brief lands it prefetches the shared paper for the languages the brief offers
   `apps/exams/src/features/exam/test-instructions.tsx:35`
   → `GET /me/tests/:testId/paper?languages=` held under `testPaperQueryKey`, `staleTime: Infinity`
5. `InstructionsShell` picks the skin from `brief.examTemplate`; `useInstructions` holds step, language, declaration
   `apps/exams/src/features/exam/instructions-shell.tsx:22`
   → `InstructionsView` (`ready` once declared and a language chosen unless the paper is DUAL)
6. Begin: ask for full screen again (the walk up may have been refused, this click is a gesture), then navigate with `state.languages` and the resume search
   `apps/exams/src/features/exam/test-instructions.tsx:83`
   → `/tests/:id/exam?resume=…`

/TESTS/:TESTID/EXAM MOUNT — THE SITTING BEGINS

1. `ExamPage` calls `useAddressedSitting`, which starts the sitting once with `tab: tabId()`
   `apps/exams/src/features/exam/exam.tsx:36`
   → start query keyed `startedAttemptQueryKey(testId)`
2. The start asks `holdsPaper` = whether the shared paper is already cached; a retry only on a hang or fault, three tries
   `packages/app-kit/src/exam/start-sitting.ts:83`
   → `POST /me/tests/:testId/attempt`
3. Server resumes a live sitting or starts one, stamps `startedAt` / `endsAt`, and omits its copy of the paper when `holdsPaper`
   `apps/api/src/attempts/attempts.controller.ts:63`
   → `StartedAttempt { paper
4. Once the start answers, the page rewrites its own address to carry `?resume=<attemptId>` (replace, not push), so a reload is a reclaim, never a second start
   `apps/exams/src/features/exam/use-sitting-address.ts:20`
   → address now names the sitting (`17c1a20`)
5. The paper resolves cheapest first: the held shared paper reshuffled with `shuffleSeed`, else the start's own paper, else a fetch
   `packages/app-kit/src/exam/served-paper.ts:16`
   → `ExamPaper` with `arrivedAt` stamped on landing
6. Branches on the start's failure: `SITTING_ENDED` on a resume goes to the result; `SITTING_HELD_ELSEWHERE` shows a refused EmptyState with Try again; anything else offers Retry / Go to your tests
   `apps/exams/src/features/exam/exam.tsx:41`
   → no sitting drawn

7. `ExamHall` mounts `useFullscreen`, `useExamView` and `useLeaveGuard`
   `apps/exams/src/features/exam/exam.tsx:129`
   → the engine's `ExamView`
8. Only a resumed sitting GETs its saved state; a sitting this call started skips it and is `sectionsSettled` at once
   `packages/app-kit/src/exam/use-attempt-state.ts:216`
   → answers merged under local ones, section clocks, `revision` seeded never backwards
9. `ExamShell` picks the skin and the clock is `state.clock ?? paperClock`
   `apps/exams/src/features/exam/exam-shell.tsx:37`
   → the paper on screen; the timer counts to the server's `endsAt`

The paper is fetched before Begin and reshuffled client-side from the start's seed, not fetched per sitting at the bell: a second copy per student at a synchronised open is the most expensive answer the server gives (`start-sitting.ts:82`).

The clock, autosave and hand-in (client side)

The timer is client-side and the server owns `startedAt` / `endsAt`. The countdown is `secondsLeft({endsAt, serverNow, arrivedAt}, Date.now())`: it subtracts elapsed device time from a server-granted span and never treats the device clock as the deadline (`packages/app-kit/src/exam/use-countdown.ts:42`). A save's reply carries a fresh `endsAt` / `serverNow` and re-anchors the clock; that is how an extension reaches a running screen (`use-attempt-state.ts:295`). Expiry fires once per arrival at zero (`use-countdown.ts:28`). Sectional papers use `SectionTimer` with `useAnchoredCountdown` (`apps/exams/src/features/exam/section-timer.tsx`); entering a section tells the server, which stamps when its clock started, so a reload cannot restart it.

MARKING AN ANSWER — TAP TO DURABLE

1. Option, Save & next, Mark for review or Clear response call the view's stable handlers; each reads the current render through a ref, so no handler has a dep list to get wrong
   `packages/app-kit/src/exam/use-exam-view.ts:366`
   → handler invocation
2. `record` refuses while locked (waiting for state, hand-in in the air, out of time); on a forward-only paper it strips `marked`
   `packages/app-kit/src/exam/use-exam-view.ts:307`
   → `AnswerIntent`
3. `answer` banks the question's seconds (capped at `QUESTION_TIME_MAX_SEC`) and builds an `AnswerChange`
   `packages/app-kit/src/exam/use-attempt-state.ts:430`
   → `AnswerChange`
4. `queue.put` keeps the change by `questionId` and mirrors the list synchronously to sessionStorage; local state is what the screen draws, so a failed save never takes an answer off screen
   `packages/app-kit/src/exam/pending-answers.ts:60`
   → `iace.test.queued.<attemptId>`
5. An early flush at 20 unsent, only if nothing is in flight and the last save did not fail
   `packages/app-kit/src/exam/use-attempt-state.ts:437`
   → maybe an immediate save
6. Otherwise a rescheduling timer at 25s ± 5s jitter fires `flush`; an idle tick sends an empty heartbeat while the screen is in touch
   `packages/app-kit/src/exam/use-attempt-state.ts:366`
   → next save
7. `flush` waits out any in-flight save, then saves; a backlog past `SAVE_BATCH_MAX` goes up as consecutive whole batches
   `packages/app-kit/src/exam/use-attempt-state.ts:311`
   → `PATCH /me/attempts/:id/state` `{revision, answers, sections, tab}` under a 15s abort
8. Server writes Redis only (the hot path) and drops any change to a seat behind a forward-only section's furthest
   `apps/api/src/attempts/attempts.controller.ts:121`
   → `AttemptSaveAck { applied, revision, endsAt, serverNow }`
9. `applied !== false` → `queue.acknowledge(batch)` deletes only the identical change objects, so an answer rewritten mid-flight survives; a failure sets `hasUnsaved`, which draws the "answers have not reached the server" warning
   `packages/app-kit/src/exam/pending-answers.ts:71`
   → queue shrinks; sessionStorage key removed when empty

A typing or written answer takes a different path to the same queue: the box keeps its own copy and hands it over after one second of quiet, on blur, and on unmount (`apps/exams/src/features/typing/use-draft.ts:5`), through `state.type`, which refuses until the saved text has arrived so an empty box never overwrites it (`use-attempt-state.ts:444`). Clear response is disabled for those kinds (`use-exam-view.ts:317`): one click must not empty minutes of typing.

SUBMIT — HANDING IN

1. Submit asks: the default skin shows a `ConfirmDialog` naming what is unanswered; the three copied skins ask over their own paper (`asksItself`)
   `apps/exams/src/features/exam/exam-shell.tsx:71`
   → `submit.asking`
2. Confirm (or the clock reaching zero, which locks the paper first) calls `end`, guarded by an `ending` ref because the clock and a tap can both fire before the next render
   `packages/app-kit/src/exam/use-exam-view.ts:245`
   → one `submit.mutate()`
3. The mutation retries itself inside the server's grace (1s, 2s, 4s) and never waits on the browser's offline flag
   `packages/app-kit/src/exam/use-exam-view.ts:92`
   → up to four tries
4. `handIn` banks the open question, then `finish` waits at most 5s on an in-flight save, stops autosave, drains any backlog
   `packages/app-kit/src/exam/use-attempt-state.ts:326`
   → at most one batch left unsent

5. The last batch rides the submit under a 10s abort
   `packages/app-kit/src/exam/use-exam-view.ts:207`
   → `POST /me/attempts/:id/submit` `{tab, last}`
6. Server ends the sitting (second call reports the first's outcome) and enqueues scoring
   `apps/api/src/attempts/attempts.controller.ts:109`
   → `SubmittedAttempt { answeredCount, status, … }`
7. On success: clear the queue, mark the catalog stale without refetching, leave full screen, hand `EndedSitting` to the page
   `packages/app-kit/src/exam/use-exam-view.ts:227`
   → `onEnded`
8. The page navigates with `replace`, so Back cannot re-enter a paper handed in
   `apps/exams/src/features/exam/exam.tsx:112`
   → `/attempts/:id/submitted` with the device's own tally in router state
9. After retries fail: `ending` clears, saving resumes, the footer shows a danger Alert with Try again
   `packages/app-kit/src/exam/use-exam-view.ts:223`
   → paper still open

Stand-down. A refusal on a save or the state read decides what this tab becomes (`use-attempt-state.ts:206`): `SITTING_ENDED` clears the queue and the page shows "This test has ended" with a button to the result (`exam.tsx:140`); `SITTING_TAKEN_OVER` or `SITTING_SET_ASIDE` stops saving but keeps the queue, and the page shows a refused EmptyState with Continue here (`exam.tsx:165`), which releases the leave guard and reloads with `?resume=<attemptId>` (`exam.tsx:209`), resending the queue as it stands. A sitting stays with the sign-in answering it (`19f35d0`); a start from the other sign-in is refused as held elsewhere until the first has been quiet for a minute.

Leaving. `useLeaveGuard` prevents `beforeunload` while answers are unsent and on `pagehide` calls `leave`, one keepalive save (`packages/app-kit/browser/use-leave-guard.ts:4`, `use-attempt-state.ts:475`). **Pause** is a header button on every skin: `onPause` flushes with `saveNow`, and only if that succeeds releases the guard, exits full screen and goes to `/tests` (`exam.tsx:197`). Silence is what pauses a sitting server-side; the client leaves only once its answers are saved.

Forward-only papers are enforced on both halves: the client's `canOpen` uses `mayOpenQuestion`, a reload lands on `furthestSeat`, `record` strips `marked`, and the legend filters through `isStateShown` (`use-exam-view.ts:135,175,279`); the API's `withinReach` drops writes behind a section's furthest seat (`apps/api/src/attempts/attempt-state.ts:193`), and the rest of the batch still lands.

The exam screen and its skins

`ExamShell` (`apps/exams/src/features/exam/exam-shell.tsx:21`) is the frame every skin sits in. It owns what a skin must not be able to omit: the offline/unsaved Alert (`:47`), the full-screen warning (`:68`) with the paper made `inert` underneath (`:56`, so Tab and Space cannot answer behind it), the calculator (mounted here, not in a skin, so a config that offers one cannot be drawn without it), the default-skin submit confirm (`:71`), `useLockedZoom` (pinch, ctrl-wheel and zoom keys refused, `lock-zoom.ts:10`), and `data-exam-template` on the root, which scopes the skin's tokens. A template with no config falls back to `DEFAULT` rather than a blank page (`:37`).

The `SKINS` table (`exam-shell.tsx:87`) has one row per `ExamTemplate`, each with `Skin`, `Nag` and `asksItself`, so a new skin cannot ship without its warning and its way of asking. `EXAM_TEMPLATE_CONFIG` (`packages/contracts/src/configs.ts:70`) is the other half: timer position and format, palette side, section switch, watermark. The admin's skin preview reads the same constant.

Skin (`ExamTemplate`)	Look	Notes
`DEFAULT` (`templates/shared/layout.tsx`)	The design system's own exam tokens; light and dark. Header, section tabs, palette rail or drawer, bottom bar.	Composes the shared slots (`templates/shared/slots.tsx`). Paper and Instructions panels. Palette is a drawer below the desktop breakpoint.
`RAILWAYS` (`templates/railway/`)	The Railway CBT: IACE title bar, utility bar, button-style section tabs, labelled clock in the section bar, screen watermark.	Pinned light. Asks Submit by taking the question area over (`submit-summary.tsx`). Renamed from `SSC_RAILWAYS` in `3b3bd1f` by renaming the enum value, so no row changed.
`BANKING` (`templates/banking/`)	IBPS-style: brand title bar with font-size steps, section tabs with a blue (i) tally, timer beside the tabs, right palette.	Asks with a "Test Summary" dialog per section, or for the whole test (`submit-dialog.tsx`). Pinned light.
`SSC` (`templates/ssc/`)	SSC-style: clock in the header, a band of four buttons, small coloured chips, a photo column, an Exam Summary replacing the paper.	Font steps 0 to 10. "Please Wait" while the paper goes in. Pinned light.

The three copied skins pin their chrome light (`railway.css`, `banking.css`, `ssc.css`) and share one token block keyed on `[data-exam-template]` in `packages/ui/src/tokens.css:622`; the default has its own pair (`:566`, `:673`). They share `Countdown` (`templates/shared/countdown.tsx`), `PHONE_QUERY` (`templates/shared/phone-query.ts`) and the question markup in `templates/railway/question.tsx`. Every value and callback comes off the view, so only markup and class names differ; none of them decides what is saved. Each skin has its own instructions screen, and `InstructionsShell` chooses by the same field (`instructions-shell.tsx:22`), so a candidate reads the paper's own screens before it opens.

Behaviours every skin shares:

- **A section can be handed in before its clock.** `view.submitSection` is null when no section follows (the last one's hand-in is the paper's); Banking and SSC ask first (`banking/layout.tsx:261`).
- **Panes move by scrollbar and arrow keys.** `ScrollPane` refuses the wheel unless a skin asks, answers Up/Down on the pane under the pointer, and draws its own bar that shows only while moving (`features/exam/scroll-pane.tsx`). The default skin keeps its wheel.
- **A passage is a second pane** beside its question (`shared/layout.tsx:100`); a typing question takes the whole answer pane and a written one draws its prompt above a box (`shared/exam-typing.tsx`).
- **Own modal boxes** (`use-modal-box.ts:9`) make a skin's dialogs modal in fact: everything beside the overlay goes `inert` while it is up, and only what it silenced is woken.

- **OMR** papers (`testUi === OMR`) draw `FillBubble`s; a full bubble is Save & next, a part fill flags for review (`option-list.tsx`, `use-exam-view.ts:375`).
- **Phones.** A palette folds away below 768px and re-opens as a drawer or panel; the header keeps only glyphs (`slots.tsx:61`).
- **The watermark** is the student's mobile number: the one thing on a paper that leads back to a person (`exam.tsx:110`).

Typing practice

`/typing` (`features/typing/typing.tsx`) picks one of the code-owned standards (`TYPING_PRESETS`), states what it is held to and lists earlier sessions through `ListView`. A session (`typing-session.tsx`) is full screen outside the shell. It fetches one passage (`GET /me/typing/passage`, never refetched mid-session), runs a device clock (nothing is ranked), and scores in the browser with `scoreTyping` the moment the clock stops or Submit is confirmed. It then saves with `POST /me/typing/practices`; the server re-runs the same evaluator for the history, and a failed save still shows the result and says it is not in the history. `TypingPane` refuses paste, drop, context menu and copy of the passage, and where a standard allows no backspace only text that extends what is typed is kept (`typing-pane.tsx:65-72`).

The same pane serves a typing section inside a test (`shared/exam-typing.tsx`). The score card reads the typing back from `GET /me/attempts/:id/typing` once the card is marked (`score-card.tsx:41`); a paper whose `maxMarks` is 0 is a typing test alone and shows only the typing, with the other four report tabs removed (`report.tsx:161`).

Results screens

HAND-IN TO RESULT

1. `SubmittedPage` reads the device's tally from router state; with none (an old link, or a count the server kept differently) it redirects straight to the report
   `apps/exams/src/features/exam/submitted.tsx:31`
   → `Navigate` to `/attempts/:id/report`
2. It reads the field's effort for the same sections and prefetches the score card silently behind itself
   `apps/exams/src/features/exam/submitted.tsx:26`
   → `GET /me/attempts/:id/field`, `GET /me/attempts/:id/scorecard`
3. It draws attempted and time per section beside the topper and the field. Nothing on it is a mark, so nothing waits on marking
   `apps/exams/src/features/exam/submitted.tsx:43`
   → the page, then a "See your result" button
4. `ReportShell` routes five tabs; its picker lists every sitting newest first and the figure strip shows marks
   `apps/exams/src/features/performance/report.tsx:37`
   → `GET /me/performance` (the `sittings` list)
5. `ScoreCardGate` turns a refusal into the right absence: set aside, "No marks yet" with Retry, or a failure with Retry
   `apps/exams/src/features/performance/score-card-gate.tsx:20`
   → children drawn only with a `ScoreCard`

Tab	File	Reads
Score card (index)	`features/performance/score-card.tsx`	`GET /me/attempts/:id/scorecard`, and `/typing` once marked
Sections	`features/performance/subject-report.tsx`	the same score card query
Solutions	`features/review/review.tsx` → `review-paper.tsx`	`GET /me/attempts/:id/solutions?sectionId=` per section, plus bookmarks in the attempt
Questions	`features/performance/question-report.tsx`	`GET /me/attempts/:id/question-report`, `ListView`
Compare	`features/performance/compare.tsx`	score card, `GET /me/performance`, and `GET /me/leaderboard?testId=&page=` only when the cohort is larger than zero

The tabs are routes, so a tab is a place a student can be sent; `ROUTES.REPORT_TAB` builds them and `ReportRedirect` sends the legacy `/score-card`, `/review` and `/questions` links to their tab (`App.tsx:228-239`). A written answer in the solutions tab shows what they wrote, each check's share and a notice that the mock evaluates by key points (`features/typing/written-review.tsx`). The leaderboard (`features/leaderboard/leaderboard.tsx`) lists the whole ranking a page at a time with the reader's own seat pinned. Downloads (`features/downloads/downloads.tsx`) lets a student choose a report and its week, month or sitting, read it on the page, then print it (`printHtml`), save it as PDF, or export a spreadsheet.

Design language

`apps/exams/src/components/ui/` is this app's composition layer over `packages/ui` primitives: `PageBody` (the gap rhythm), `Hero` / `HeroFigure`, `Section`, `SurfaceCard`, `DividedList` / `DividedRow` (rows in one card parted by hairlines), `Shelf`, `StatBand`, `TileGrid` / `StatTile`, and skeletons. No colour or spacing value is chosen here. The default is a feed or a shelf, not admin's `PageHeader` + `ListView` + `DataTable`: `tests.tsx` and `notifications.tsx` each carry a written justification for the deviation, and only `saved.tsx`, the typing history and the question report use `ListView`.

Browser storage and the worker

Key	Store	Holds
`iace.test.auth`	localStorage	access and refresh token (`lib/api.ts:7`)
`iace.test.tab`	sessionStorage	this tab's UUID, sent as `tab` on start, save and submit (`lib/tab-id.ts:6`)
`iace.test.queued.<attemptId>`	sessionStorage	unsent `AnswerChange` list (`pending-answers.ts`)
`iace.test.tours`	localStorage	page tours this device has seen

Section progress is not mirrored to storage (`pending-answers.ts:64`): a reload loses an unsent section stamp, which the server's own stamp then supplies.

`public/sw.js` does two jobs. It serves the document network-first with a cached shell as fallback, and content-hashed `/assets/` cache-first, but never an API response and never the exam path (`/tests/:id/exam`), because offline an exam must fail rather than look as if it opened (`sw.js:17`). It has no `skipWaiting`, so a new bundle is not swapped under a sitting in progress. It renders pushes and tells open tabs, which move the bell. `vite.config.ts` stamps the cache name per build.

What must be true

A student session; a test visible in `GET /me/catalog` and open; `isTestBlocked` false for a new attempt; `VITE_API_URL` (default `http://localhost:3000`, `lib/api.ts:10`); `crypto.randomUUID` for the tab id. The DEV preview routes need `import.meta.env.DEV`.

Why it is shaped this way

- The paper is held before Begin and reshuffled client-side, so beginning is a small request, not a paper download for every student at once.
- The answer queue is a plain keyed list in sessionStorage, not a per-question version vector. The version attempt was built and reverted (`29269a8`); a multi-device overwrite remains an accepted limit.
- Skins are tokens plus an `ExamTemplateConfig` and one slot set, not one screen per skin, so they cannot drift in behaviour.
- Every shared component lives in `packages/ui` or `packages/app-kit`, never copied, so the mobile client reads the same keys, the same copy (`SITTING_ENDED_SAYS`, `HELD_ELSEWHERE_SAYS`) and the same engine.

What is wrong with it

- `apps/exams/src/App.tsx:117` — Stale: the comment "Phase 0 routing: a login screen and one authed shell" describes a table with some thirty routes.
- `apps/exams/src/features/exam/exam-shell.tsx:44` — Stale: "neither skin lets the paper be selected" is written for two skins; the root now serves four.
- `apps/exams/src/features/auth/login.tsx:126` — Dead: empty banner comments stand above `MobileField` with nothing between them.
- `apps/exams/src/features/exam/templates/railway/preview-data.ts:2` — Stale: the header says the data is for "the railway skin" and "the DEV-only preview route alone", but the Banking and SSC previews import it too (`apps/exams/src/features/exam/railway-preview.tsx:6`).

The admin portal

`apps/admin` is a Vite + React SPA that holds everything staff do that is not sitting a test: students and their imports, the exam catalogue, the question bank with its authoring and proof-reading pipeline, base configurations, tests and series, live sitting operations, reports, announcements, and the audit log. It is a client only. It touches no database row; every screen's gate is mirrored by `@RequiresFeature` or `@RequiresSuperAdmin` on the controller behind it, and the client's gate is courtesy, not the security boundary.

Where it lives

Piece	Path
Mount	`apps/admin/src/main.tsx:18` builds one query client, then `mountApp`
Route table	`apps/admin/src/App.tsx` (36 `React.lazy` chunks; login and not-found are static)
Constants	`apps/admin/src/lib/constants.ts` (1015 lines): `ROUTES`, `NAV_ITEMS`, `ROUTE_ACCESS`, `QUERY_KEYS`, ~70 query-key builders, `STORAGE_KEYS`, label maps
API client	`apps/admin/src/lib/api.ts` (16 lines)
Session	`apps/admin/src/providers/auth.ts`
Shell	`apps/admin/src/components/app-shell.tsx` over `packages/app-kit/browser/app-shell.tsx`
Feature folders	`apps/admin/src/features/<area>/`: admins, announcements, auth, authoring, catalog, dashboard, imports, live-ops, questions, reports, students, test-series, tests
Shared pieces	`apps/admin/src/components/` keeps only what has two or more consumers (pickers, `section-thread`, `question-link`, `export-button`, badges)

Feature folders hold their screens and the components nothing else imports. There are no per-feature barrels: an SPA ships as one bundle, so nothing is gained by a public surface, and a few cross-feature imports remain on purpose. `authoring` is the largest folder (35 files): the card-per-question workspace shared by the bank question page and the section page, the typist's one-box editor, the assignment queues, the candidate view.

The route map

Every route except `/login` and `*` sits inside `ProtectedRoute` then `AppShell`; the candidate view is protected but outside the shell, because the whole window is the question (`apps/admin/src/App.tsx:272`). Paths are `ROUTES` values (`apps/admin/src/lib/constants.ts:83`, 69 entries). Order in the table matters only where a static segment must beat `:id` (`new`, `import`, `taxonomy`, `history`, `configs`, `series`); each such place has a one-line comment.

Area	Routes	Screen files (under `features/`)
Sign-in	`/login`	`auth/login.tsx` (emailed code, no sign-up)
Home	`/`	`dashboard/dashboard.tsx`
Students	`/students`, `/students/import`, `/students/:id`, `/students/:id/performance`	`students/students.tsx`, `imports/import-students.tsx`, `students/student-detail.tsx`; the last route is a `Navigate` to the detail's Performance tab
Catalogue	`/branches`, `/exams`, `/cohorts`, `/cohorts/events/:id/import`, `/cohorts/programs/:code/import`	`catalog/branches.tsx`, `catalog/exams.tsx`, `students/cohorts.tsx`, `imports/import-event-candidates.tsx`, `imports/import-program-students.tsx`
Question bank	`/questions`, `/questions/new`, `/questions/import`, `/questions/taxonomy`, `/questions/:id`, `/questions/:id/edit`	`questions/questions.tsx`, `questions/bank-question.tsx` (read and edit are one component, `readOnly` on the read route), `imports/import-questions.tsx`, `questions/taxonomy.tsx`
Authoring	`/authoring`, `/authoring/:id`, `/authoring/history`, `/authoring/assignments`, `…/assignments/tests/:testId/sections/:sectionId` (+ `/import`)	`authoring/authoring-editor.tsx`, `authoring/authoring-history.tsx`, `authoring/assignment-queue.tsx`, `authoring/section-authoring.tsx`
Proof-reading	`/proofreading/assignments`, `/proofreading/assignments/tests/:testId/sections/:sectionId`	`authoring/assignment-queue.tsx` (role PROOFREADER), `authoring/section-authoring.tsx`
Tests	`/tests`, `/tests/new`, `/tests/:id`, `/tests/:id/paper`, `/tests/:id/paper/print`, `/tests/:id/analytics`, `/tests/:id/import`, `/tests/:testId/sections/:sectionId`, `/tests/section-progress`	`tests/tests-and-series.tsx`, `tests/test-builder.tsx`, `tests/test-paper.tsx`, `tests/test-paper-print.tsx`, `tests/test-analytics.tsx`, `authoring/section-progress.tsx`
Blueprints and series	`/tests/configs`, `/tests/configs/new`, `/tests/configs/:id`, `/tests/series/new`, `/tests/series/:id`	`catalog/base-configs.tsx`, `catalog/base-config-form.tsx`, `test-series/test-series-form.tsx`
Live ops	`/live-ops`	`live-ops/live-ops.tsx`
Reports	`/reports`, `/reports/:key`	`reports/reports.tsx`, `reports/report.tsx`
Announcements	`/announcements`	`announcements/announcements.tsx`

Area	Routes	Screen files (under `features/`)
Super admin	`/admins`, `/permissions`, `/audit`, `/audit/imports`, `/tests/section-progress`	`admins/admins.tsx`, `admins/permissions.tsx`, `admins/audit.tsx`, `authoring/section-progress.tsx`
Fallback	`*`	`components/not-found.tsx`

Three route patterns (`TYPING_SECTION_PATTERN`, `READING_SECTION_PATTERN`, `TEST_SECTION_PATTERN`) mount the same `SectionAuthoringPage`: a section answers to its typist, its proof-reader and the test's owner alike (`apps/admin/src/App.tsx:225`). `AUTHORING_IMPORT_PATTERN` and `TEST_IMPORT_PATTERN` both mount `ImportQuestionsPage`, the first for one section, the second for a whole typed test from one sheet, each row into its section's typist's drafts.

Session, shell and navigation

`createAuth` (`apps/admin/src/providers/auth.ts:14`) builds the provider from the typed client. `ActorTypes.ADMIN` is load-bearing: a student JWT is not a session here. Tokens live in `localStorage` under `iace.admin.auth` (`STORAGE_KEYS.AUTH`), a per-SPA key because both SPAs share an origin. Token refresh, the single-flight retry and `SESSION_REPLACED` handling are in the API client core (`packages/contracts/src/client/core.ts:175`, `:227`); when a refresh fails, `onUnauthorized` clears the store and emits the sign-out signal (`packages/app-kit/src/api-client.ts:40`), and `createAuth` answers by clearing the whole query cache so the next person on a shared machine sees none of it (`packages/app-kit/src/create-auth.tsx:116`). The identity read (`/auth/me`) has `staleTime` 5 minutes and retries once with jitter; a permission change lands on reload. A held token the server never answered for gives `isUnreachable`, which `ProtectedRoute` draws as "Could not reach the server" with Retry instead of bouncing to sign-in (`packages/app-kit/browser/protected-route.tsx:31`).

`createAuth` extends the identity with `can(key, level = READ)`, the contract's own `can`, so a screen cannot offer a button the API's guard refuses (`apps/admin/src/providers/auth.ts:29`). It also requires `isActive`.

The nav is `NAV_ITEMS` (`apps/admin/src/lib/constants.ts:463`): ten sections of leaves (Students, Question bank, Proof-reading, Authoring, Tests, Live operations, Reports, Administration, Announcements, Audit log). Home is the brand mark, not a row. Three filters run in sequence:

1. `filterAdminNav` (`:748`) drops `superAdminOnly` rows and any leaf whose route `opensRoute` refuses, so a read-only typist loses Authoring's Editor.
2. The shell runs `filterNavByPermission(nav, can)` over each section's `featureKey` (`packages/app-kit/browser/app-shell.tsx:87`).
3. `collapseLoneSections` turns a section left with one leaf into that leaf (`packages/app-kit/src/nav.ts:54`).

Nav is hidden, not disabled: a disabled row advertises what the viewer cannot get. A deactivated admin keeps the session but gets an empty nav and an "Access removed" notice in place of the outlet (`apps/admin/src/components/app-shell.tsx:23`, `:41`); login is not refused because that would answer a real account as invalid.

Permission gating of a typed address

Hiding a nav row is not enough, since a typed address skips it. `ROUTE_ACCESS` (`apps/admin/src/lib/constants.ts:682`) maps a route pattern to `{ keys, level }`: the key at READ to open a screen, at WRITE to make or import one, any of several where the server reads `RequiresAnyFeature` (`SECTION_KEYS`, the typist, reader and test-owner keys). `SUPER_ADMIN_ROUTES` (`:727`) holds the five screens no key opens. A pattern in neither is open to every admin (home, the performance redirect, the candidate view). `App.tsx` mounts every page through `screen()`, so no route can be added without its gate.

ADMIN TYPES /TESTS/NEW INTO THE ADDRESS BAR

1. `ProtectedRoute` holds a spinner while `isLoading`, shows retry on `isUnreachable`, else `<Navigate to="/login" state={{from}}>` when there is no identity
   `packages/app-kit/browser/protected-route.tsx:23`
   → identity known, or a redirect that keeps the address to come back to
2. Router matches `ROUTES.TEST_NEW` and `screen()` wraps the page in `PageErrorBoundary` + `Suspense` with the shared skeleton
   `apps/admin/src/App.tsx:161`
   → a held frame, so a route swap never shifts layout
3. `Gated` reads `identity.isSuperAdmin` and `can` from `useAuth`
   `apps/admin/src/App.tsx:168`
   → the viewer
4. `opensRoute('/tests/new', viewer)`: not a super-admin route; `ROUTE_ACCESS` says `writes(TEST_MANAGEMENT)`; passes if `can(TEST_MANAGEMENT, WRITE)`
   `apps/admin/src/lib/constants.ts:741`
   → true or false
5. False: `EmptyState` REFUSED "This screen is not open to you" replaces the page. The chunk is never fetched because the lazy element is only rendered inside the true branch
   `apps/admin/src/App.tsx:175`
   → a refusal in place of the builder
6. True: the page mounts; its own endpoints are still guarded server-side and any refusal arrives as `AppException`
   `apps/api (controller guards)`
   → the page, not a security grant

Pinned by `apps/admin/test/route-access.test.ts` (nav and address agree for each key and level) and `apps/admin/test/route-gate.dom.test.tsx` (refused at the door without loading the page). Inside screens the level is read again: a typist holding only READ opens their section read-only, My sections offers Mark done at WRITE on the role's own key, History offers Edit and Delete at WRITE, and the bank offers Edit question only at WRITE.

Data fetching

One client from `createAppQueryClient({ notify: toast })` (`packages/app-kit/src/query-client.ts:40`): `staleTime` 30s, no refetch on focus, one retry only when `isWorthAskingAgain` (status 0, 429 or 5xx), with doubling delay and a full base of jitter. `gcTime` is TanStack's default. Overrides: exam and branch pickers 5 minutes (`features/students/use-exams.ts:25`, `lib/use-branches.ts:34`); import drafts and subject languages `Infinity`; live ops polls every `LIVE_OPS_POLL_MS` (15s, `packages/contracts`) and stops on a refusal (`features/live-ops/live-ops.tsx:74`); analytics polls every 3s, capped at 150s, only while a re-sync it asked for has not landed (`features/tests/test-analytics.tsx:91`); a section page polls every 10s only while another admin holds the editing claim (`features/authoring/section-authoring.tsx:64`).

Keys: `QUERY_KEYS` (`:766`) is 31 roots, all prefixed `'admin'` except `ME`. Every key is built by a named helper below it. `QUERY_SCOPES` (`:813`, module-private) separates a picker (offers only live rows) from a filter (must reach retired ones): they cannot share a cache entry. A record's key nests under its list's root, which is the invalidation strategy: invalidating the list root also reaches the details.

Errors: no screen composes a message. `MutationCache.onError` and `QueryCache.onError` toast once, honouring `meta.silent`, `meta.fields` and `meta.success`. `bannerMessage(error, fields)` returns null when every message already sits on a form field (`packages/app-kit/src/form-errors.ts:42`), and `applyFieldErrors` feeds react-hook-form (`:18`). A replaced session is never toasted per request; it is said once on the sign-in screen.

The list screen

Sixteen screens drive `useListScreen` (`packages/app-kit/browser/use-list-screen.ts:9`). The filter spec is declared once and does three jobs: it names the URL keys, it types `values`, and it is the same array handed to `ListView`, so the controls and the query cannot disagree.

ADMIN OPENS /STUDENTS AND NARROWS THE LIST

1. The screen declares `filterSpec` (search, choice, multi, date, custom kinds) and calls `useListScreen({ queryKey: QUERY_KEYS.STUDENTS, filters, toQuery, fetchPage })`
   `apps/admin/src/features/students/students.tsx:247`
   → list state: rows, pagination, values, setFilter
2. `useFilterSpec` reads values from the URL search params; a set filter is CSV (`CSV_SEPARATOR`); the Any/All toggle is `?match=any`
   `packages/app-kit/browser/use-filter-spec.ts:30`
   → a link carries the whole filter state
3. `toQuery` maps spec values to the endpoint's params (one filter may set several, e.g. `status`)
   `apps/admin/src/features/students/students.tsx:247`
   → `{ q, branchId, …, match }`
4. `useListQuery` keys the query `[...queryKey, { filters, page, pageSize }]`; a changed filter resets the page during render, not in an effect
   `packages/app-kit/src/use-list-query.ts:69`
   → one request per `{filters, page, pageSize}`
5. `queryFn` passes the caller's `AbortSignal`, so a superseded search is aborted; `placeholderData: keepPreviousData` holds old rows while the new page loads
   `packages/app-kit/src/use-list-query.ts:71`
   → no flicker, no stale-search race
6. `api.admin.students.list(params)` issues the GET and parses with `studentSummarySchema`
   `packages/contracts/src/client/admin.ts:343`
   → `Paginated<StudentSummary>` (items, total)
7. A page past the end after a delete snaps to the last page; a read that never ran offline is not shown as the previous filter's rows
   `packages/app-kit/src/use-list-query.ts:88`
   → rows or an error state
8. `ListView` draws the filter row (primary search, the rest behind "Filters"), `DataTable` with a "Could not load this list" retry on failure, `Pagination` in the footer; `emptyFiltered` replaces `empty` when any filter is set
   `packages/ui/src/components/ui/list-view.tsx:346`
   → the screen

The spec's kinds are closed in the `ListFilter` union (`packages/ui/src/components/ui/list-view.tsx:57`): `search`, `choice`, `multi`, `date`, `custom`, `customMulti`. A dialog that lists something passes `useLocalFilters()` as `store` so its filters stay out of the page's URL. Breadcrumbs come from `PageCrumbs nav={NAV_ITEMS}`, which also remembers each section's last URL in memory (`packages/app-kit/browser/nav-memory.ts`) so a crumb returns to the filtered list the reader came from.

Rules the screens follow (from `ui-conventions`, read here, not re-derived): cells wrap in `TruncatedText`; row actions sit behind one `RowActions` menu; a destructive item opens `ConfirmDialog` naming the consequence and count.

FormDialog

`FormDialog` (`packages/ui/src/components/ui/form-dialog.tsx:32`) is the modal for creating or editing one entity. It takes the whole react-hook-form `form` because closing must reset it; it stops submit bubbling through the portal so the page's own form does not fire; while `loading` neither Esc nor the close button dismisses it. Twelve files use it.

ADMIN ADDS A STUDENT FROM THE LIST

1. "New student" opens `NewStudentDialog`; `useForm` holds defaults and `useWatch` feeds the dependent pickers (branch locks by student type)
   `apps/admin/src/features/students/students.tsx:379`
   → form state
2. Submit: `FormDialog` calls `form.handleSubmit`; Zod resolver validates
   `packages/ui/src/components/ui/form-dialog.tsx:63`
   → validated values
3. `create.mutate(values)`; the `mutationFn` trims the name, sends optional arrays only when non-empty, and omits a locked branch
   `apps/admin/src/features/students/students.tsx:402`
   → `CreateStudentInput`

4. `api.admin.students.create` issues `POST` with the Bearer token and `x-client-kind: WEB`
   `packages/contracts/src/client/admin.ts:349`
   → `StudentDetail` response
5. Failure: `applyFieldErrors` places each `fieldErrors` entry on its field; `meta.fields` makes the central toast skip a failure fully on fields
   `apps/admin/src/features/students/students.tsx:422`
   → messages on fields, or one toast
6. Success: `meta.success` toasts "Student added."; `invalidateQueries(QUERY_KEYS.STUDENTS)` refreshes every list variant; navigate to `ROUTES.STUDENT(id)`
   `apps/admin/src/features/students/students.tsx:419`
   → the new student's page, the list refetched on next view

A save of an existing record sends only the fields the admin changed (student, exam, stage, program, event edits), so two people editing different fields do not overwrite each other. Authoring sends a stamp (`expectedUpdatedAt`) and the builder an `expectedVersion`; a row that moved since the page read it is refused.

Local storage, tours, candidate view

`STORAGE_KEYS` (`apps/admin/src/lib/constants.ts:1007`): `iace.admin.auth` (token pair), `iace.admin.authoring` (the one-box editor's draft, restored only if something was typed), `iace.admin.candidate-view` (one entry per opened view under a random id; only the newest is kept), `iace.admin.tours` (which page tours this device has seen, one key holding the set). The candidate view hands the question's text to a new tab through storage rather than the address, and `readCandidateView` treats a stored string as untrusted (`apps/admin/src/features/authoring/candidate-view-store.ts`). Tours are declared in `apps/admin/src/lib/tours.ts` (eight tour ids); renaming an id shows that tour again.

Why it is shaped this way

- **One `constants.ts`.** Cross-feature invalidation is why the keys are central; splitting them reintroduces two names for one cache entry.
- **Read and edit are one component.** The bank question, with a key swap between reading and writing, replaced three drawings of a question that had drifted apart.
- **The gate is one table.** Before `ROUTE_ACCESS`, a typed address bypassed the nav filter; a read-only Tests holder got the new-test builder and learned only when the save came back refused. The router and the rail now read the same rule.
- **Audit is a super admin's.** `/audit`, `/audit/imports`, the Audit trail report and the dashboard Activity card are super-admin only; the service still pins any other viewer to their own rows as a second lock.

Tests

Admin tests are in `apps/admin/test/`: pure logic (`route-access`, `assignment-filters`, `duration`, `nav-authoring`, `nav-cohorts`, `question-scaffold`, `student-detail-tabs`, `test-offer-draft`, `test-paper-view`, `test-schedule-draft`, `base-config-sessions`, `work-clock`) and DOM tests for five screens (`route-gate`, `live-ops`, `test-series-form`, `student-detail`, `base-config-form`). DOM tests import `test/support/offline.ts` first so no real network is reachable. Read, not run.

What is wrong with it

- `apps/admin/src/lib/constants.ts:173` — Stale: the comment "Every admin reaches these — the service, not the route, scopes what they see" sits on `AUDIT`, but the audit log is super-admin only (`SUPER_ADMIN_ROUTES`, `:727`).
- `apps/admin/src/features/admins/super-admin-only.tsx:6` — Dead: `Gated` already refuses `/admins`, `/permissions` and `/tests/section-progress` to a non-super-admin, so the inner `SuperAdminOnly` wrappers (`admins.tsx:160`, `permissions.tsx:107`, `section-progress.tsx:245`) can never render their refusal; its comment says "three super-admin screens" and the router now covers five.
- `apps/admin/src/lib/constants.ts:682` — Correctness: `ROUTES.STUDENT_PERFORMANCE_PATTERN` has no entry, so it is open to any admin; it only redirects to a gated page, so there is no exposure, but the table's "unlisted is every admin's" rule hides the omission.

The mobile client

Where it lives

`apps/mobile` is an Expo SDK 57 app, Android first, in its own workspace package. Its pieces:

Path	What it is
`apps/mobile/app/`	expo-router routes: the root stack, a `(tabs)` group, and the pushed screens
`apps/mobile/src/lib/`	the API client and token store, push, theme, link guard, tours, nav constants
`apps/mobile/src/components/`	56 files: a hand-built `ui/` tier (22) styled with NativeWind, plus exam, report, tests, performance, saved and account components
`apps/mobile/webview/`	the build step and source of the one bundled question page
`apps/mobile/app.json`, `apps/mobile/eas.json`	native config and the three build profiles
`apps/mobile/google-services.json`	the Firebase client config, tracked on purpose (below)

Mobile imports two workspace packages, `@iace/app-kit` and `@iace/contracts`. It never imports `@iace/ui` or `@iace/app-kit/browser`; `apps/mobile/eslint.config.js` enforces that. It shares the exam engine, the session factory, the query keys, the filter specs and the report copy with the web, and owns everything visual: tokens come from `packages/ui/src/tokens.css` through `apps/mobile/tailwind.preset.native.js`, which reads the same token map as the web preset (`e5bbaaa`).

It is a second composition over one engine, not a second implementation. Its reasons to exist are that IACE students are phone-first and that a native client can keep the unsent answer queue in SQLite across a process kill.

Launch state

Not launched. No store listing exists and `main` deploys nothing. What a tester can have today:

Profile (`apps/mobile/eas.json`)	Result
`development`	dev client, JS from Metro; the API URL is derived from Metro's host (`apps/mobile/src/lib/dev-api-url.ts`), with `localhost` rewritten to `10.0.2.2` on Android
`preview`	internal APK on channel `preview`, with `EXPO_PUBLIC_API_URL` set to `https://api.staging.examprep.iace.co.in` in the profile (`537e9b3`)
`production`	app bundle, `autoIncrement`, channel `production`, **no API URL**

A production build throws at launch naming `EXPO_PUBLIC_API_URL` (`apps/mobile/src/lib/api.ts:24`), by design: `api.examprep.iace.co.in` does not resolve yet, and an APK that installs and silently fails every request is worse than one that refuses to start. Expo inlines `EXPO_PUBLIC_*` at build time, so the URL cannot be a runtime switch.

Three things changed how a build is made:

- **Over-the-air JS.** `app.json` sets `runtimeVersion` to the `fingerprint` policy and points `updates.url` at the EAS project. `eas update --branch preview` reaches a tester at the next launch with no reinstall. The fingerprint hashes the native layer, so an update is only offered to a build whose native code matches; a client that lacks a new native module stops being offered updates instead of crashing on the screen that needs it (`ebaf86c`).
- **Cloud builder.** EAS builds what git tracks. `apps/mobile/package.json:8` (`eas-build-post-install`) regenerates the two artefacts git ignores: the contracts `dist` and `webview/dist/question-page.ts` (KaTeX and the Indic fonts inlined, about 1.1 MB) (`49a2389`).
- **Firebase config tracked.** `google-services.json` was ignored, and the first cloud build after `googleServicesFile` returned to `app.json` failed on its absence. It carries a project number, the package name `in.co.iace.mobile` and a package-restricted key, and ships inside every APK, so it is tracked (`bf2f66f`). One Firebase project, `iace-exams-mobile`, serves every environment.

Still open before a store build: icons are 512 px placeholders (a wordmark on flat red, `adaptiveIcon.backgroundColor: "#bf0d10"`), the production API URL, and a first signed bundle. The EAS project is owned by the organisation `indian-academy-of-competitive-exams`, so the upload keystore is org-owned.

Navigation

`apps/mobile/app/_layout.tsx` renders one root `<Stack>` whose every real screen sits under `Stack.Protected` (`:142`, signed in) and `login` under the opposite guard (`:162`). Signing in or out flips `identity`, and the router swaps the stack; nothing navigates by hand.

Route	File	What it is
`/login`	`app/login.tsx`	mobile number, then code
`/`, `/tests`, `/performance`, `/saved`, `/downloads`, `/account`	`app/(tabs)/*.tsx`	the six tabs, icon-only (`222af38`), listed in `MOBILE_NAV_ITEMS` (`apps/mobile/src/lib/nav.ts:33`)
`/series/[id]`, `/test/[id]`, `/test/[id]/instructions`	`app/series`, `app/test`	one series, test details with past sittings, the rules and language choice
`/exam/[testId]`	`app/exam/[testId].tsx`	the sitting; `gestureEnabled: false`, `dangerouslySingular` so a second push cannot stack another paper

Route	File	What it is
`/attempts/[id]/submitted`, `/attempts/[id]/report`	`app/attempts/`	"Handed in" (back hidden), and the tabbed report
`/profile`, `/notifications`, `/notification-settings`	`app/*.tsx`	the Account tab's pushed screens

The old Change PIN screen is gone with the PIN. Downloads and Notification settings are new.

Boot is deliberate about failure. The splash is held until the session is known (`_layout.tsx:91`). The identity read is abandoned after 15 seconds (`apps/mobile/src/providers/auth.ts:8`) and shown as "Could not reach the server" with a retry (`_layout.tsx:120`): a student holding a token whose server did not answer is neither signed in nor out, and the sign-in screen would be a lie. A render error draws a fallback with a retry (`_layout.tsx:39`), and `unstable_settings.anchor = '(tabs)'` puts the tabs beneath a cold deep link so Back never leaves the app.

The link guard. `apps/mobile/src/lib/link-guard.ts` lives outside React because the system hands a link to `app/+native-intent.ts` before any screen mounts. While a paper is on screen (`_layout.tsx:95` sets it from the pathname) `admit` returns `null`, so a link or a notification tap cannot take the phone off a running sitting. A link that arrives signed out is remembered and opened after sign-in, once.

Appearance. Account offers System, Light or Dark, saved in SQLite under `iace.mobile.theme` and applied through NativeWind's `colorScheme.set` before the first screen draws (`apps/mobile/src/lib/theme.ts:17`), so `dark:` classes, the status bar and the question WebView all follow one choice (`5a6bb58`).

Sign-in

A student has no PIN. Every sign-in is a code sent to the mobile; the first one for a number is the signup (`29f3f88`). The login screen has two steps, a number and a code. The code field takes its length from the server's `codeLength` and shows the development code in an Alert only when the API is on the console sender. Android Back on the code step returns to the number, and is held while a request is out.

SIGNING IN ON THE PHONE

1. The student types a number; `typedMobile` strips a pasted prefix and the form validates with `requestStudentOtpSchema`
   `apps/mobile/app/login.tsx:88`
   → a validated `{ mobile }`
2. `api.auth.requestStudentOtp` posts with `x-client: MOBILE` and a device name from `Device.modelName`
   `apps/mobile/src/lib/api.ts:34`
   → the request carries the client kind
3. The route is public and rate-limited; the service looks up a live student only to choose which daily budget pays, then asks the OTP service
   `apps/api/src/auth/auth.controller.ts:34`
   → an `OtpRequestResponse`: channel, `codeLength`, resend wait
4. The code goes by SMS alone; the WhatsApp channel is closed by `OTP_WHATSAPP_ENABLED`, default false
   `apps/api/src/auth/otp/otp.service.ts:299`
   → the code in Redis, one SMS sent
5. The code step posts `{ mobile, code }`; a wrong code returns `OTP_INVALID` with `fieldErrors.code`, which lands under the input
   `apps/mobile/app/login.tsx:145`
   → field error or success
6. Verify consumes the code, takes the live student or makes one, refuses a deactivated account
   `apps/api/src/auth/auth.service.ts:74`
   → an identity row
7. Access and refresh tokens are signed and a session id is minted
   `apps/api/src/auth/auth.service.ts:222`
   → tokens with `sid`
8. A student keeps one session per client kind, so the previous MOBILE session is tombstoned, then the new one is written
   `apps/api/src/auth/session.service.ts:52`
   → Redis `session:student:<id>:<sid>` with the refresh TTL, id added to the session index
9. `signIn` stores the tokens, flips `hasToken`, seeds the identity into the query cache, and fires `registerPushDevice`
   `packages/app-kit/src/create-auth.tsx:165`
   → the protected stack mounts
10. The token store writes the in-memory mirror at once and SecureStore in the background
    `apps/mobile/src/lib/secure-storage.ts:22`
    → key `iace.mobile.auth` on the keystore

A restart restores the session without a sign-in: `hydrate()` fills the mirror from SecureStore before any token read (`apps/mobile/app/_layout.tsx:60`, then `AuthProvider` seeds `hasToken`), and `GET /auth/me` confirms it. The mirror exists because app-kit's `KeyValueStorage` is synchronous and SecureStore is not; the cost is that a write that does not land before a process kill is lost, and a failed write only logs a warning (`secure-storage.ts:10`).

Because sign-in costs an SMS, what keeps it rare is the one-session-per-kind rule: a phone asks once and stays signed in for the 30-day refresh TTL. A student whose code never arrives is read one by an admin (`issueStudentDeskCode`, `apps/api/src/auth/auth.service.ts:98`); the phone needs nothing for that, it is the ordinary code step.

Sign-out drops the push registration first, waiting seconds at most, then ends the session, so the server stops pushing the bell to a phone that has left (`apps/mobile/app/(tabs)/account.tsx:107`).

The exam sitting on the device

`apps/mobile/app/exam/[testId].tsx` is the hall. It does not run sitting logic itself; it starts the sitting, hands app-kit's engine the paper, and lets the skin draw the engine's view.

Three things about the screen:

- **The skin mounts before the attempt exists** (`:69`), so the question WebView loads alongside the network instead of on the student's clock.
- **A paper that cannot be sat on a phone is refused before a clock starts.** `needsKeyboard(brief)` is true when any section has a typing preset or is descriptive (`packages/app-kit/src/keyboard.ts:7`); the screen shows "This test is not sat on a phone" (`:56`). A brief that failed to load does not block an ordinary paper.
- **Back cannot leave.** Android Back is intercepted and asks to submit (`:129`); the only exits are submit, Pause and the ended or taken-over screens.

SITTING A TEST ON THE PHONE

1. Begin on Instructions navigates to the hall with the chosen languages and, for a Resume, the sitting's id
   `apps/mobile/src/lib/nav.ts:63`
   → route `/exam/<testId>?languages=…&resume=…`
2. The brief is read first; a typing or descriptive section stops the screen here
   `apps/mobile/app/exam/[testId].tsx:56`
   → refusal EmptyState, no attempt created
3. `useStartedSitting` posts the start with this install's tab id and whether the screen already holds the paper
   `apps/mobile/app/exam/[testId].tsx:103`
   → start request
4. The start is idempotent: a second start resumes the running one, clock and all. A different sign-in that opens a sitting held by one recently heard from is refused with `SITTING_HELD_ELSEWHERE`
   `apps/api/src/attempts/attempts.controller.ts:65`
   → `StartedAttempt` with `endsAt` and `serverNow`
5. A held-elsewhere refusal draws "Try again"; an ended sitting redirects to its Handed in screen; any other error offers Retry or Go to your tests
   `apps/mobile/app/exam/[testId].tsx:141`
   → one of three screens
6. The paper is read for the attempt (or taken from the start), and the instant it lands is stamped as the clock's anchor
   `packages/app-kit/src/exam/start-sitting.ts:99`
   → `{ paper, arrivedAt }` in cache
7. `SittingEngine` runs `useExamView` with the SQLite queue as its answer store; it renders nothing and reports each view upward before paint
   `apps/mobile/app/exam/[testId].tsx:270`
   → the skin draws from `view`
8. An answer is put in the queue synchronously, so the last answer before a kill is still there on relaunch
   `apps/mobile/src/lib/sitting-store.ts:7`
   → SQLite key `iace.mobile.queued`
9. The autosave tick sends the batch about every 25 seconds (jitter 5 s, or at 20 pending); with nothing to send it sends an empty heartbeat so silence means absence
   `packages/app-kit/src/autosave-policy.ts:9`
   → `PATCH /me/attempts/:id/state`
10. The server writes Redis only and answers with `endsAt` and `serverNow`; the client re-anchors its clock on every answer, and only `applied` removes the batch from the queue
    `packages/app-kit/src/exam/use-attempt-state.ts:287`
    → queue acknowledged, clock re-synced
11. Going to the background sends what is unsent and counts a leave
    `apps/mobile/app/exam/[testId].tsx:280`
    → one flush
12. Submit asks to confirm, then banks the open question's seconds and posts the last batch with the submit, retried inside the server's grace
    `packages/app-kit/src/exam/use-exam-view.ts:201`
    → `POST /me/attempts/:id/submit`, a scoring job enqueued
13. On success the ended sitting is written to the cache and the screen is replaced, so Back never re-enters a handed-in paper
    `apps/mobile/app/exam/[testId].tsx:110`
    → route `/attempts/<id>/submitted`

What the student sees on the paper. A header with the clock, a section timer where one applies, and Calculator (only on a paper that enables it), Pause, and Submit. The header buttons wrap under the clock rather than pushing Submit off the right edge (`c810b4d`). Sheets cover the paper rules, the question paper with per-section tallies, and the calculator (`apps/mobile/src/components/exam/exam-sheets.tsx`). Pause saves first and leaves only if that landed (`app/exam/[testId].tsx:203`). A passage draws above the question, capped at 40% of the page so the options stay reachable (`6180ea4`).

Held and taken over. One sitting has one holder. If another sign-in takes it, the phone shows an overlay (not a replacement, so the WebView stays warm) with "Continue here", which bumps the engine key and re-reads the queue. An ended or set-aside sitting says so and offers "See your result".

The question page is a WebView. Authored stems carry HTML, KaTeX, tables and Indic script, so one persistent `react-native-webview` loads a page built by `apps/mobile/webview/build.mjs` from the web's own `richHtml` and stylesheets, with a CSP whose only script source is the page's own hash. The page is swapped, never remounted, between questions. It is a view: answer, clock and autosave stay native. The bridge (`apps/mobile/src/components/exam/question-bridge.ts`, `question-protocol.ts`) posts the whole screen in, and refuses any message from the page that is malformed or names an action the current screen would not allow. `apps/mobile/src/components/exam/question-page-view.tsx` refuses every navigation but `about:blank`, opens no windows, scales text with the phone's font scale, and re-sends the screen if the render process dies.

Push

The client is built, the server is built, and no push is confirmed to have reached a phone. Read, not measured: no commit says a notification arrived on a device.

Half	State
App: registration	`apps/mobile/src/lib/push-device.ts:22` asks permission and registers the phone's own FCM token with `POST /me/push-device`. It returns early in Expo Go, on a simulator, and on iOS, where Expo hands back an APNs token FCM cannot address, so the platform ever sent is ANDROID (`:25`)
App: Firebase	`google-services.json` is tracked and named in `app.json`, so a build made after `1415ecb` has FCM compiled in. An older dev build must be rebuilt
Server	`FcmSender.isConfigured` needs `FCM_PROJECT_ID`, `FCM_CLIENT_EMAIL` and `FCM_PRIVATE_KEY` (`apps/api/src/notifications/fcm.sender.ts:51`). Unconfigured, `targetsOf` reads no `PushDevice` rows (`apps/api/src/notifications/push.service.ts:116`). The three are empty in both `.env.example` files
Delivery	a push carries the bell's own line, folded to one line and cut at 140 characters, and no score or answer (`ab5d593`); a broadcast is one row the bell resolves, not a copy per student (`b73c9dd`)

The in-app bell is the source of truth and does not depend on any of this. Registration is best effort: a refused permission or a missing Firebase config leaves a phone with no push, never a phone that cannot sign in.

The server keys a device on its token, so the same phone re-registering is one row moved to whoever is signed in, with the `sessionId` from the JWT (`push.service.ts:23`). A row whose session has ended is dropped as read. `POST` and `DELETE /me/push-device` are `apps/api/src/me/me.controller.ts:121` and `:130`.

On the phone, `usePushDevice` (`apps/mobile/src/lib/use-push-device.ts:26`) registers once per process for a restored session, shows a notification while the app is open, and routes a tap to `/notifications`. While a paper is on screen the banner is suppressed, a tap goes nowhere, and the unread query is not invalidated, so one broadcast is not a read per sitting. `expo-notifications` is loaded dynamically and never in Expo Go, where the module throws at load (`apps/mobile/src/lib/notifications.ts:16`).

The shape of a screen

- **Dropdowns are bottom sheets.** `Picker` and `MultiPicker` (`apps/mobile/src/components/ui/picker.tsx`) open a slide-up `Modal` with the field's name, a search box on every list, rows scrolling in up to 80% of the screen, Done, and Clear on a multi choice. The sheet sits in a `KeyboardAvoidingView` so the search stays above the keyboard (`22c47b7`, `b7af173`). Filters, the Downloads report and period, the leaderboard's test, the performance scope and the solutions' section use them. Chips remain where a few fixed views act as tabs. The tests spec makes Series a multi filter on both clients.
- **Type scale.** The body steps (11 to 14 px) are the web's. The display steps are compressed for a phone: `md` to `3xl` are 15, 16, 18, 22 and 28 px and the lockup is 20 (`apps/mobile/tailwind.preset.native.js:53`). `inlineRem: 16` in `apps/mobile/metro.config.js:18` makes NativeWind read the tokens at the 16 px root they were authored at, not its default of 14.
- **Parity with the web.** `d407f4c` closed an audit that found mobile a simplified port: it gained the paper rules card, calculator, test title, question-paper sheet, section tallies, the opening bar, the web's home hierarchy, shut reasons on tiles, the exam journey, time and cohort figures on the score card, notification settings and the profile's footer. Shared logic moved into app-kit rather than being copied.
- **One failure, one alert.** The query client's error notifier raises a native `Alert`, deduplicated by message for up to 30 seconds so Home offline does not stack four of the same (`apps/mobile/src/lib/alert-once.ts`). A failed reload keeps what was on screen.
- **Page tours** (`apps/mobile/src/lib/tours.ts`, `page-tour.tsx`) are keyed by id; the set seen is one SQLite key.
- **Downloads.** The student picks a report and a period (week, month, or one sitting), reads its figures first, and prints it through the phone's own print service with `printAsync`, which is also where it is saved as a PDF (`apps/mobile/app/(tabs)/downloads.tsx:51`).
- **Documents.** Photo and tenth marksheet upload from `/profile`; the file is passed as an `expo-file-system` `File` because Expo's fetch rejects React Native's `{ uri, name, type }` descriptor. A picked photo needs no library permission and `cameraPermission` and `microphonePermission` are `false` in `app.json`.

Device-verified, and not

From commit bodies, not from running anything.

Verified on hardware	Source
Document upload (a photo and a PDF marksheet)	`7bee05e`
Sign-in against an unreachable API	`1a8cf21`
Android text rendering (the first run on a device)	`05437cd`

Everything since is explicitly unverified by its own body. `a9c1268` ("None of this has been tried on a phone"), `1008dc8` (fifteen faults, "none of them seen on a device"), `820629d`, `45714fd` and `6180ea4` ("Nothing here has been seen on a device") all say so. The 14 October work (the dropdown sheet, the type scale, the theme choice, the tab bar, icons on dialogs, the wrapped exam header) carries no such claim either way. The exam sitting, the question WebView, the link guard, the Downloads print, and push are written in full and device-verified nowhere in the history. Treat the first preview APK run as the real test of all of it; the repo has no record that one has been built and installed.

The unit tests in `apps/mobile/test/` cover pure logic only: the link guard, the secure-storage mirror, the filters, the question bridge, the alert dedupe, the tour placement and the dev API URL. No screen is rendered under test.

What is wrong with it

- `apps/mobile/.env.example:8` says no API is deployed and that the preview and production profiles carry no URL; the preview profile names staging (`apps/mobile/eas.json`). Stale.
- `docs/01-architecture.md` still says of mobile that "nothing has shipped" while the preview profile can build an installable APK against staging. Not false, but it undersells the state. Stale.
- `apps/mobile/src/components/ui/pin-field.tsx:1` is named and described as the web's PinInput and is now the one-time-code field; the PIN it was named for is gone. Stale.
- The production profile names no API URL (`apps/mobile/eas.json`), so a production build throws at launch by design. Blocking for a store release only.
- Real FCM service-account values sit in git-ignored `deploy/.env.staging.bak-20261008` and `deploy/.env.local.bak-20261008`. They are not tracked (`.gitignore:33` covers `.env.*`), but they are plaintext private keys left in backup files beside the tree. Privacy.

What the three clients share

One API, three clients (two Vite SPAs and one Expo app), one wire format. Four directories carry what they have in common.

Package	What it is	Who imports it
`packages/contracts`	The wire format: Zod schemas, route strings, the error vocabulary, the typed API client, the date helpers, and the exam rules both sides run	the API, both SPAs, mobile
`packages/app-kit`	Client plumbing that is identical on web and phone, plus the exam engine. DOM-free in `src`; web adapters in `browser/`	both SPAs, mobile (`src` only)
`packages/ui`	The web design system and `tokens.css`	both SPAs, `app-kit/browser`; mobile imports none of it except the token file by relative path
`packages/config`	tsconfigs and the ESLint configs that make the division above mechanical	every workspace

`contracts` means a field renamed on the server is a compile error in all three clients, not `undefined` on a screen. `app-kit` is the piece that must behave the same in a browser and in React Native. `ui` cannot run in React Native, so the phone has its own components over the same tokens.

How each package is built

The build output differs, and the difference matters.

- `contracts` compiles to CommonJS plus `.d.ts` (`packages/contracts/package.json:6`, `packages/contracts/tsconfig.json:4`) because NestJS consumes it. Both SPAs bypass `dist` and alias the package to its TypeScript source (`apps/exams/vite.config.ts:39`, `apps/admin/vite.config.ts:10`), with `optimizeDeps.exclude` for it and `@iace/ui`, so an edit hot-reloads with no rebuild.
- `app-kit` and `ui` have no build script. Their `exports` point at `.ts`, `.tsx` and `.css` (`packages/app-kit/package.json:9`, `packages/ui/package.json:9`), and Vite or Metro compile them inside the app.
- `config` ships raw JSON and `.js`.
- `turbo.json:11` makes `build` depend on `^build`, which is why `contracts/dist` exists before the API typechecks or runs. `prisma generate` runs first too, for the enums `contracts` mirrors.

Each SPA passes its own token-store key, because the two SPAs share one origin (`apps/exams/src/lib/constants.ts:125` holds `iace.test.auth`). A shared key would hand an admin's token to a student's requests.

contracts

The envelope and the error vocabulary

`packages/contracts/src/envelope.ts` defines the only response shape: `{ success: true, data, meta }` or `{ success: false, error: { code, message, fieldErrors?, details? }, meta }`. Controllers return data or throw; an interceptor wraps and one exception filter maps. A lint rule on `*.controller.ts` forbids building the envelope by hand (`apps/api/eslint.config.mjs:22`).

`ErrorCodes` (`envelope.ts:12`) has 19 members. Each one has a status in `ERROR_CODE_STATUS` (`:39`) and a fallback sentence in `DEFAULT_MESSAGES` (`:66`). Both are `Record<ErrorCode, …>`, so a new code that leaves one out is a type error.

Group	Codes
Request	`VALIDATION_ERROR` 400, `NOT_FOUND` 404, `CONFLICT` 409, `RATE_LIMITED` 429, `FORBIDDEN` 403
Session	`UNAUTHENTICATED` 401, `SESSION_REPLACED` 401
Sitting	`SITTING_TAKEN_OVER`, `SITTING_SET_ASIDE`, `SITTING_HELD_ELSEWHERE`, `SITTING_ENDED`, `SITTING_VOIDED` (all 409)
Sign-in	`OTP_INVALID`, `OTP_EXPIRED` (401), `ADMIN_NOT_REGISTERED` 404
Business	`DRAW_SHORTFALL`, `EXPORT_TOO_LARGE` (422)
Platform	`SERVICE_UNAVAILABLE` 503, `INTERNAL` 500

The five sitting codes are how a screen learns that its paper is no longer its own. The two PIN codes the old vocabulary carried are gone; a student has no PIN. `AppException` (`:205`) is thrown by the API and rebuilt by the client from a failure body (`fromFailure`, `:224`), and `AppException.is` tests a brand symbol rather than `instanceof`, so it survives duplicated module copies. `errorCodeForStatus` (`:264`) maps a bare status with no envelope: 400, 401, 403, 404, 409, 429 and 503 each map to their own code, any other 5xx to `INTERNAL`, anything else to `VALIDATION_ERROR`.

Lists carry their counts in `meta`. A handler returns `Paginated<T>` (`:169`), the interceptor splits it, and the client reassembles it. `paginationQuerySchema` caps `pageSize` at `PAGE_SIZE_MAX` (100) so no list can pull a whole table.

Dates

The institute runs on IST and storage is UTC. `packages/contracts/src/common.ts` holds the one Intl-only set of helpers: `INSTITUTE_TIME_ZONE` (`:12`), `civilDate` (`:23`), `instituteDayLabel` (`:36`), `instituteDateTimeLabel` (`:49`), `instituteWallTime` (`:99`) and `fromInstituteWallTime` (`:106`). `todayISO` lives in `packages/contracts/src/students.ts:53` and is just `civilDate()`. The offset is derived per instant from Intl rather than hard-coded. The server-side set (`date-fns` and `@date-fns/tz` in `apps/api/src/common/time/institute-day.ts`) stays out of `contracts` on purpose: a date library there would ship to every student's browser.

The typed client

`packages/contracts/src/client.ts` builds two clients from one transport (`createApiCore`, `client/core.ts:79`): `createApiClient` returns `{ auth, me }` and `createAdminApiClient` returns `{ auth, admin }`. They are two factories, not one lazy namespace, so the 1,100-line admin group and its

schemas never enter a student bundle. The groups are `client/auth.ts`, `client/me.ts` and `client/admin.ts`. Every method returns `data` or throws an `AppException`, and every response is validated against its schema before a caller sees it.

The transport (`client/core.ts`):

- `send` (`:97`) adds `Authorization`, a fresh `x-request-id` (so Caddy's access log joins the API's line) and, from the app, `x-client` and `x-device-name`. FormData is sent without a `Content-Type` so the browser sets the boundary. A thrown `fetch` becomes `AppException(INTERNAL, httpStatus: 0)` ("Cannot reach the server"), so callers need no second code path.
- `parse` (`:134`) validates both halves. A shape the schema refuses becomes `INTERNAL` / "Unexpected response shape from API", never silently typed data.
- `queryString` (`:24`) drops empty values, and an empty array is dropped rather than sent, so an unset filter never reaches the server as `in: []`.
- `list` (`:329`) recombines `data` and `meta` into one `Paginated` with no second request. A list query may carry an `AbortSignal`; it rides the fetch and never the query string.
- `requestExport` (`:279`) is how an admin export is fetched: it POSTs to start a worker-built file, polls the job until it is done, then downloads it, and gives up with `SERVICE_UNAVAILABLE` after `EXPORT_WAIT_MS`. The caller still gets one `Promise<Blob>`.
- `requestBlob` / `requestUploadBlob` (`:274`, `:298`) go through `blobOf` (`:302`), which refreshes like any call and turns a failure into an envelope error.

ANY AUTHENTICATED CALL THAT MEETS AN EXPIRED ACCESS TOKEN

1. `envelopeOf` sends with the current access token
   `packages/contracts/src/client/core.ts:207`
   → a 401 comes back
2. `tokenAfter` peeks the failure body without consuming it
   `packages/contracts/src/client/core.ts:223`
   → the envelope's code, or null for a bare 401
3. `SESSION_REPLACED` ends it for good: `onUnauthorized(replaced)` is called and it throws without refreshing
   `packages/contracts/src/client/core.ts:227`
   → the app is told why it was signed out
4. any code other than `UNAUTHENTICATED` throws as the real error
   `packages/contracts/src/client/core.ts:234`
   → the typed `AppException`
5. if another request already rotated the token, reuse it; a second refresh would spend the new refresh token and be refused as a reuse
   `packages/contracts/src/client/core.ts:240`
   → the newer access token
6. otherwise `refreshInFlight ??= refreshTokens()`: every parallel 401 shares one refresh
   `packages/contracts/src/client/core.ts:242`
   → one POST to the refresh route
7. `refreshTokens` asks with a 10s abort; only a 401 is a refusal, anything else is asked again after a halved, jittered 1s, 2s, 4s wait
   `packages/contracts/src/client/core.ts:175`
   → rotated `AuthTokens`
8. `onTokensRefreshed` stores the pair, but only if a session still exists here
   `packages/app-kit/src/api-client.ts:37`
   → token store written
9. the original request is sent again with the new token; a second 401 calls `onUnauthorized`
   `packages/contracts/src/client/core.ts:218`
   → the caller's data, or a sign-out

The retry schedule is the reason for the jitter: a hall whose tokens expired in the same minute must not come back in step. Four tries of 10s plus the backoff end near 50s, inside the 60s for which the server still answers a replaced token.

Rules both sides run

`contracts` also holds the exam rules that must give one answer on the server and in the browser: `secondsLeft`, `paletteCounts`, `mayOpenQuestion`, `openSections` and `servedQuestions` (`attempts.ts`, `exam-order.ts`). `servedQuestions` runs in the browser for a prefetched paper (`packages/app-kit/src/exam/served-paper.ts:32`) and on the server for a reload or a report. Two copies would be two answers to "same seed, same order". The typing standards and their one evaluator also live here, as do the report shapes and the `ClientKind` / `CLIENT_HEADERS` pair the API reads in `apps/api/src/auth/device.ts`.

ADDING AN ENDPOINT, END TO END

1. the response schema and `z.infer` type go in the domain file; a list query extends `paginationQuerySchema`
   `packages/contracts/src/saved.ts:1`
   → one source for both sides
2. the file's routes go in a `…_ROUTES as const` object and the file is added to the barrel
   `packages/contracts/src/index.ts:2`
   → importable as `@iace/contracts`
3. a new failure mode gets one line in each of the three code tables
   `packages/contracts/src/envelope.ts:12`
   → a type error until all three are filled
4. the server binds the same schema as a pipe and the same type as the return
   `apps/api/src/saved/saved.controller.ts:1`
   → `ZodBody` parses or throws `VALIDATION_ERROR` with `fieldErrors`

5. the client adds one line to its group (`list`, `get` or `write` with the schema)
   `packages/contracts/src/client/me.ts:1`
   → validated on arrival

A service returning the wrong shape, a controller whose return type drifted, a bad query argument or an unhandled `ErrorCodes` member is caught while writing. A field the server sends but the schema omits is stripped by Zod and surfaces as a missing column on screen.

app-kit

`packages/app-kit/src/index.ts` is the barrel (208 value exports and 59 types); `browser/index.ts` is the web tier (40 values and 11 types). `src` may not touch the DOM or import a web-only package (below), so each browser capability is an injected adapter.

Piece	File	What it does
App API client	`packages/app-kit/src/api-client.ts`	`createAppApiClient` / `createAdminAppApiClient` wrap the contracts factories with a token store, a sign-out signal and the `x-client` / `x-device-name` headers. Header values are cut to printable ASCII, 128 characters
Token store	`packages/app-kit/src/token-store.ts`	`createTokenStore(storageKey, storage)`; an unreadable entry reads as signed out
Sign-out signal	`packages/app-kit/src/sign-out-signal.ts`, `signed-out-message.ts`	A one-way "the session is gone" channel, and the sentence a replaced device shows
Auth	`packages/app-kit/src/create-auth.tsx`	One `createAuth` for every app; the identity query, `signIn`, `signOut`, cross-tab following
Query policy	`packages/app-kit/src/query-client.ts`	`createAppQueryClient`: one retry for a read, no refetch on focus, every mutation failure announced once
Form errors	`packages/app-kit/src/form-errors.ts`	`fieldErrors` into react-hook-form; `bannerMessage` for what no field owns
Lists	`use-list-query.ts`, `use-infinite-pages.ts`, `list-filters.ts`	Paging, keyed filters, the bell's pages
Student reads	`student-queries.ts`, `catalog.ts`, `student-figures.ts`, `form-changes.ts`	Every read both student clients cache, keyed once; the lines and figures about a test brief, one copy
Exam engine	`packages/app-kit/src/exam/`	The sitting: below

Session and sign-out

`createTokenStore` is synchronous so the API client can read a token from any call site. The web store (`createBrowserTokenStore`, `packages/app-kit/browser/index.ts:41`) adds three things on top: it leaves the reason for a sign-out in a second key (`<key>.ended`) before it clears the token, it reads that reason back with `endedBy`, and it `subscribe`s to the `storage` event so a tab hears a sign-in or sign-out made in another. The phone has one holder and omits all three.

`createAuth` (`create-auth.tsx:79`) is parameterised by the identity type and the one actor an app accepts. `identity` is null for no token, a dead token or a token for the wrong actor. `isUnreachable` is a third state: a token is held but the server never said whose, so the screen offers a retry rather than a sign-in. `/auth/me` runs once per launch at a five-minute `staleTime`, with one jittered retry.

A SESSION ENDS IN ONE TAB OF THE WEB APP

1. a refresh is refused, or a 401 meets `SESSION_REPLACED`
   `packages/contracts/src/client/core.ts:227`
   → `onUnauthorized(cause)`
2. the app turns the cause into a reason (`replacedBy` kind) and clears the store with it
   `packages/app-kit/src/api-client.ts:40`
   → the token key removed, `<key>.ended` written first
3. the signal fires as a window event, since the client holds no reference to the React tree
   `packages/app-kit/browser/index.ts:30`
   → `iace:signed-out`
4. `createAuth` hears it, keeps the reason if there is one and clears the session: `queryClient.clear()` drops every cached read so the next person on a shared machine sees nothing of this one
   `packages/app-kit/src/create-auth.tsx:155`
   → `hasToken` false, sign-in screen with the sentence
5. the other open tabs get the `storage` event; they read `endedBy`, show the same sentence and forget their state without writing back
   `packages/app-kit/src/create-auth.tsx:142`
   → every tab signed out

A tab that hears a sign-in rather than a sign-out calls `adopt` (`:125`). It refetches the identity; if the person is a different one, it clears the cache and remounts every screen by changing the provider's `key`. `signOut` (`:176`) waits at most `SIGN_OUT_WAIT_MS` (4s) on the logout request, aborts it if it is still pending, and clears this device whatever the answer.

The query client

`isWorthAskingAgain` (`query-client.ts:10`) is the rule for retries everywhere: a request that never landed (`httpStatus` 0), a 429 or a 5xx is worth asking again; a refusal is the server's answer and is not. Reads retry once with a doubling, jittered delay (`retryDelayMs`, `:25`). `queryCache.onError` and `mutationCache.onError` toast the failure centrally, unless the query or mutation is marked `silent`, the failure is a replaced session (said once on the sign-in screen) or every message already sits on a form field (`isFullyFieldMapped`).

The exam engine

`apps/exams/src/features/exam/exam.tsx:130` and `apps/mobile/app/exam/[testId].tsx:271` call the same hook, `useExamView` (`packages/app-kit/src/exam/use-exam-view.ts:109`). They differ only in the focus adapter, the answer-queue storage and the tab id. The engine takes four injected dependencies (`api`, `focus`, `tab`, `answerQueue`) and a mounted `QueryClientProvider`.

It is two hooks. `useAttemptState` (`use-attempt-state.ts:134`) owns the mutable sitting: answers, sections, the clock, whether a save is in the air, whether the tab was taken over. State is named callbacks on `AttemptStateHandle`, not a reducer. `useExamView` owns navigation and ending, and returns one `ExamView` (`exam-view.ts:126`): every value a skin may draw and every move it may make. A skin holds no state, owns no clock and reaches for no mutation.

The view's identity must not change, so about 14 handlers are built once. They read the current render through a ref published in `useInsertionEffect` (`use-exam-view.ts:361`), which is never stale and needs no dependency list.

A CANDIDATE TAPS AN OPTION AND IT BECOMES A DURABLE SAVE

1. the skin calls `view.chooseOption(id)` on the handlers built once
   `packages/app-kit/src/exam/use-exam-view.ts:373`
   → `live.current.record({ selectedOptionId })`
2. a forward-only paper forces `marked: false`; the paper must be unlocked or the tap is dropped
   `packages/app-kit/src/exam/use-exam-view.ts:309`
   → an `AnswerIntent`
3. the seconds on the open question are banked and the state is derived: `changeFor` accumulates `timeSpentSec` (capped at `QUESTION_TIME_MAX_SEC`) and keeps the earliest `firstActionAt`
   `packages/app-kit/src/exam/use-attempt-state.ts:514`
   → an `AnswerChange`
4. `record` queues it and draws it at once; nothing waits on the server, so a failed save never takes an answer off the screen
   `packages/app-kit/src/exam/use-attempt-state.ts:391`
   → the pending map holds it
5. the queue mirrors itself to storage under `<prefix>.<attemptId>` (sessionStorage on web, the device store on the phone)
   `packages/app-kit/src/exam/pending-answers.ts:46`
   → undelivered answers survive a reload
6. with 20 unsent answers and nothing in the air or failed, flush now; otherwise the jittered timer (25s plus or minus 5s) does it
   `packages/app-kit/src/exam/use-attempt-state.ts:437`
   → `flush()`
7. `flush` waits behind any save in the air and sends whole batches; a backlog past `SAVE_BATCH_MAX` (200) goes as consecutive saves
   `packages/app-kit/src/exam/use-attempt-state.ts:311`
   → one batch per call
8. `save` bumps the revision, arms a 15s abort and calls `api.me.saveAttemptState(attemptId, { ...batch, tab })`
   `packages/app-kit/src/exam/use-attempt-state.ts:277`
   → `PATCH` on the attempt's state route
9. the server validates with `ZodBody(saveAttemptStateSchema)` and merges into the Redis state in a compare-and-swap loop of 5 tries (`CONFLICT` once they are spent)
   `apps/api/src/attempts/attempts.controller.ts:121`
   → `attempt:state:<attemptId>`, TTL 48 hours
10. a non-heartbeat batch adds the attempt id to the dirty set; a flusher, not this request, writes Postgres
    `apps/api/src/attempts/attempt-state.service.ts:185`
    → `attempt:dirty` set member
11. the ack carries `applied`, `revision`, `endsAt` and `serverNow`; the client keeps the greatest revision and re-anchors the clock
    `packages/app-kit/src/exam/use-attempt-state.ts:295`
    → a clock an admin extension can reach with no reload
12. only `applied !== false` acknowledges; `acknowledge` removes the exact copies the batch carried, so an answer rewritten in the meantime stays queued
    `packages/app-kit/src/exam/pending-answers.ts:71`
    → queue shrinks

Four rules are worth stating on their own.

- **A refusal ends the screen's right to answer.** `refused` (`use-attempt-state.ts:206`) maps `SITTING_ENDED` to `endHere` (queue cleared, nothing more sent), and `SITTING_TAKEN_OVER` / `SITTING_SET_ASIDE` to `standDown`, which stops saving but keeps the queue; continuing here sends it as it stands.
- **The idle tick is a heartbeat.** With nothing unsent and the server heard from within `PRESENT_GRACE_SEC` (60s), the tick still saves an empty batch. Only this device's silence hands the sitting on.
- **The hand-in rides its last batch.** `finish` (`:326`) waits at most `FINISH_WAIT_MS` (5s) behind a save in the air, stops all saving, sends any backlog past one batch as saves, and passes the final batch to the submit call. The submit is retried at 1s, 2s and 4s with a 10s abort each, so the last try lands inside the server's 30s grace (`autosave-policy.ts:21`). The mutation sets `networkMode: 'always'` so the browser's word that it is offline never holds it.
- **One clock, one ending.** `ending` is a ref (`use-exam-view.ts:122`), so a clock reaching zero and a tap on Submit cannot both submit. `useCountdown` fires expiry once per arrival at zero and does not treat the callback's identity as a trigger, since the engine rebuilds it each render and a failed paper would resubmit in a storm.

`createPendingAnswers` is plain TypeScript with no React. On load it parses the stored copy as a save would read it: any other shape is an empty queue, and a time held from before the cap is clamped to it (`pending-answers.ts:95`, `:110`). Section progress is held in memory only; it is not mirrored to storage.

The start of a sitting is in `start-sitting.ts`: `useStartedSitting` starts once (`staleTime: Infinity`, `gcTime: 0`, so a later entry never reuses a stale clock), tries three times at a synchronised open, and takes the paper from the cache the instructions screen filled, then from the start's answer, then from a fetch (`served-paper.ts`). `SITTING_HELD_ELSEWHERE` on a start is the hold screen: another sign-in is still answering.

Web adapters, `app-kit/browser`

`browser/` is outside `src` because the DOM ban is scoped to `src/**`.

Export	What it adapts
`browserStorage`, `browserSessionStorage`	Web Storage narrowed to `KeyValueStorage`
`browserSignOutSignal`, `createBrowserTokenStore`	A window event for `SignOutSignal`; the token store with sign-out reason and cross-tab `subscribe`
`mountApp`	`createRoot` plus the fixed provider order: query client, theme, tooltip, router, toaster
`AppShell`, `PageCrumbs`, `ProtectedRoute`, `useWorkspace`	the shell, built from `react-router-dom` and `@iace/ui`
`useFullscreen`, `useLeaveGuard`	the Fullscreen API as a `FullscreenHandle`; `beforeunload` and `pagehide`
`saveBlob`, `shrunkForUpload`, `printHtml`	an authenticated download, canvas/WebP shrinking, printing a report
`useFilters`, `useLocalFilters`, `useFilterSpec`, `useListScreen`, `useScrollList`, `useImportScreen`	URL and list plumbing
the figure modules	overview, performance and streak figures (`CohortFigure`, `MarksFigure`, `TrajectoryFigure`, `StreakFigure`) over `@iace/ui`'s Recharts wrappers

ui

`packages/ui/src/index.ts` is the inventory: 126 values and 33 types. 68 `.tsx` files sit in `components/ui` and 11 in `components/charts` (read, not run). The tokens are in `packages/ui/src/tokens.css` (692 lines, 17 numbered sections: primitives, type, spacing, control sizes, semantic roles, charts, exam CBT, watermark, skeleton, overlays, empty state, app shell, elevation, radii, motion, focus, z-index), plus a dark theme (`:408`) and the exam block.

One token set, two compositions. Admin is dense and data-first; the student apps (`apps/exams`) are lighter. The exam screen is deliberately not brand-coloured. Its palette lives per template under `[data-exam-template='default']` (`:566`) and a second block for `railways`, `banking` and `ssc` (`:622`), so swapping the attribute re-skins the whole screen with no component change. Dark has its own default-template block (`:673`).

Tailwind reads the tokens through a map shared by both presets: `packages/ui/tailwind.tokens.cjs` is called with each preset's token function (`packages/ui/tailwind.preset.js:29`, `apps/mobile/tailwind.preset.native.js:10`), so a token added on the web can no longer be forgotten on the phone. What differs is kept in each preset: shadows, font sizes (native has no line heights, since the tokens are unitless multipliers that React Native reads as points) and the dark block. The native preset reads the dark custom properties out of `tokens.css` itself (`apps/mobile/tailwind.preset.native.js:22`) because native has no document root for `data-theme`, and it throws if the selector it looks for has gone.

Mobile imports `packages/ui/src/tokens.css` by relative path (`apps/mobile/global.css:2`), with a comment that the `@iace/ui` specifier hangs Metro's CSS transform. `apps/mobile/package.json` declares no dependency on `@iace/ui`, so moving the file breaks the phone with nothing in the dependency graph to warn.

Each SPA imports the stylesheets once at its entry and extends the preset in `tailwind.config.js`. A missing `content` glob emits no CSS and no error.

Boundaries made mechanical

Rule	Where	Effect
`contracts/src` and `app-kit/src` may not name `window`, `document`, `localStorage`, `sessionStorage` (nor via `globalThis`), nor import `@iace/ui`, `react-dom` or `react-router-dom`	`packages/config/eslint.no-dom.js:10`, `:13`, `:35`; wired at `packages/contracts/eslint.config.mjs:5`, `packages/app-kit/eslint.config.js:5`	a web-only import belongs in `app-kit/browser`
`apps/mobile` may not import `@iace/ui` or `@iace/app-kit/browser`	`packages/config/eslint.no-web-ui.js:7`, wired at `apps/mobile/eslint.config.js:8`	the failure would otherwise read as a bundler problem
API modules reach a sibling only through its public entry	`packages/config/eslint-rules/api-module-boundaries.js`, wired in `eslint.api-boundaries.js`	
No Postgres write on the live answer path	`packages/config/eslint-rules/no-hot-path-db-write.js`, scoped to three files (`eslint.api-boundaries.js:19`)	the flusher, submit and the sweeper write by design
SPAs may not use raw `fetch` or `axios`	`packages/config/eslint.react.config.js:46`	use the typed client
A controller may not build the envelope	`apps/api/eslint.config.mjs:22`	

`eslint.react.config.js` also carries the design-system bans a type cannot state and four repo-local rules: `confirm-destructive` and `no-narration` (error), `no-hardcoded-route` and `no-manual-focus` (warn).

Why it is shaped this way

- The date helpers are Intl-only so `contracts`' sole runtime dependency is `zod` (`packages/contracts/package.json:26`).
- The engine is DOM-free so mobile reuses it verbatim. The cost is an adapter per capability (`FullscreenHandle`, `KeyValueStorage`, `SignOutSignal`) and a second web tier. The alternative, a web implementation and a native copy, is two answers to "when does the clock expire".
- Two client factories rather than one lazy namespace measurably took admin routes out of the student bundle (`608e15c`).
- `plural` exists twice, in `packages/ui/src/lib/utils.ts:13` and `packages/app-kit/src/home-lines.ts:13`: `ui` cannot be imported by the phone, and the phone takes the `app-kit` copy.

What is wrong with it

- Stale: `packages/contracts/src/client/core.ts:233` says refreshing "on a wrong PIN hides the real code"; no PIN exists any more, and the line guards every non-`UNAUTHENTICATED` 401.
- Stale: `packages/contracts/src/client/core.ts:354` has a doc comment ("Reads a failure body…") stacked above `NO_CONTENT`; it belongs to `peekFailure` at `:410`.
- Stale: `packages/contracts/src/envelope.ts:52` puts "A wrong or stale credential is an authentication failure" above `ADMIN_NOT_REGISTERED: 404`; it was written for the OTP codes below it.
- Dead: the 204 branch (`packages/contracts/src/client/core.ts:135`, `:377`) cannot be reached against this API; no route answers 204, and `apps/api/src/me/me.controller.ts:171` says why.
- Stale: `packages/ui/src/tokens.css:3` names the file `packages/ui/tokens.css`; it is `packages/ui/src/tokens.css`.
- Correctness: `packages/app-kit/eslint.config.js:5` spreads `noDom` after the React config, and both set `no-restricted-globals` and `no-restricted-imports`; the later entry replaces the earlier, so inside `app-kit/src` the raw-`fetch` and `axios` bans no longer apply. No file there uses either today (grep), so nothing is exposed yet.
- Stale: `packages/ui/src/index.ts:5` still exports `PinField`, a name from the PIN era; it is now the sign-in code field.

Part V — The platform underneath

The machinery every module leans on

Every feature module answers HTTP, throws a refusal, pages a list, counts a rate, enqueues work, writes to S3 and records who changed what. This chapter is the one copy of each, so a feature module is a controller plus a service. The payoff is uniformity the clients rely on: one failure shape, one page shape, one request id on every response. The second payoff is that the live-test scaling rules are enforced here (client timer, Redis for the sitting, BullMQ for scoring) rather than remembered per feature.

Where it lives: `apps/api/src/common/` (envelope, filter, request id and budget, throttling, events, metrics, exporting, messaging, time), `apps/api/src/{config,queue,redis,storage,prisma,health,audit}/`, `apps/api/src/main.ts`, `apps/api/src/app.module.ts`. The client half is `packages/contracts/src/envelope.ts`.

One image, three roles

`API_ROLE` decides what a container registers (`apps/api/src/config/api-role.ts:33`). It is read off `process.env` at file load, because a module's decorator is evaluated long before Nest has a container to ask. An unknown value logs a warning and serves everything (`:24`); the env schema separately refuses it at boot (`apps/api/src/config/env.schema.ts:72`).

Role	Registers	Does not
`exam`	`AttemptsController`, `MeLeaderboardController` (`apps/api/src/attempts/attempts.module.ts:49`)	any other controller, any processor
`core`	every other controller: auth, `me`, admin, imports, reports, exports, audit, post-submit reads (`attempts.module.ts:51`)	processors, schedulers
`worker`	all nine processors and every scheduler; no routes but `/health*` and `/metrics`	controllers
`all` (unset)	everything; local development and every test	

`onRole(roles, list)` (`api-role.ts:42`) filters a module's `controllers` or `providers` array; `servesRole` (`:39`) gates imperative work such as scheduler registration. Every module is still imported in every container (`apps/api/src/app.module.ts:49`), so the guards, services and Prisma are present everywhere; only the routes and the queue consumers differ. Caddy sends a leaf-path list to the exam container and everything else to core (`deploy/Caddyfile:11`), so that list must match what `AttemptsController` registers.

Two behaviours key on the role being exactly `exam`, never on `servesRole`, because under `all` the scoring sweeps share the process for minutes: the 8 s request budget and the Postgres `statement_timeout` (below).

`deploy/compose.yml` runs exam, core and exactly one worker from one image. The worker is a singleton because `concurrency: 1` on the flush and sweep queues only holds inside one process (`docs/04-infrastructure.md` §3). Compose probes `/health/ready` (`deploy/compose.yml:68`).

Boot

`main.ts` imports `load-env.ts` first (`apps/api/src/main.ts:1`). That calls Node's own `process.loadEnvFile` on `./.env` or `../../.env` if either exists (`apps/api/src/load-env.ts:5`); in production compose has already put the environment in the process, so neither exists.

Then, in order: `NestFactory.create` with `bodyParser: false` and `appLogger()` (`main.ts:21`); two advisory boot checks that only warn in production, `UV_THREADPOOL_SIZE` under 16 (`apps/api/src/common/threadpool.ts:10`) and a V8 heap outside 60–80% of the cgroup memory limit (`apps/api/src/common/heap.ts:51`); `helmet` with a deny-everything CSP, because this API answers JSON and nothing else (`apps/api/src/common/security-headers.ts:18`); `compression`; JSON and urlencoded parsers at `BODY_LIMIT_DEFAULT`, 256 kb (`main.ts:40`); `trust proxy` set to `TRUST_PROXY_HOPS` (`:44`); CORS with `x-request-id` exposed so a SPA can report the id of a response it could not parse (`:47`); shutdown hooks (`:57`); keep-alive 65 s and headers timeout 66 s, above any load balancer's idle timeout (`:60`).

`AppConfigService` validates the whole environment in a field initializer (`apps/api/src/config/app-config.service.ts:8`), so a bad variable fails the process at construction with every problem listed at once. `NODE_ENV=production` additionally refuses (`env.schema.ts:221`): a `DATABASE_URL` with no `connection_limit`, an S3 key half-set or an own `S3_ENDPOINT` without a key pair, an empty `CORS_ORIGINS`, no `METRICS_TOKEN`, `TRUST_PROXY_HOPS` of 0, and either JWT secret still prefixed `dev_only_`. Types: `Env`, read through `config.get('KEY')`. There is no `@nestjs/config`.

The request path

ANY HTTP REQUEST — FROM SOCKET TO ENVELOPE

1. `RequestIdMiddleware` accepts an inbound `x-request-id` only if it matches `^[A-Za-z0-9._-]{8,128}$`, else mints a UUID; stamps `startedAt`; sets the response header
   `apps/api/src/common/request-id.ts:19`
   → `request.requestId`, `x-request-id` header
2. `RequestObserverMiddleware` hooks the response's `finish` event, so it sees every request, including a 401, a 429 and a body-parser 413 that no interceptor ever sees
   `apps/api/src/common/metrics/request-observer.middleware.ts:25`
   → histogram sample and one log line when the response ends
3. `AuditContextMiddleware` opens an `AsyncLocalStorage` store; `next()` runs the whole request inside it
   `apps/api/src/audit/audit.middleware.ts:10`
   → `{ changed, entityId, unchanged }` store
4. Guards in order: `JwtAuthGuard`, `ActorGuard`, `FeaturePermissionGuard`, `SuperAdminGuard`, `ThrottlerGuard`
   `apps/api/src/app.module.ts:89`
   → `request.user`, or a thrown refusal

5. `RequestBudgetInterceptor` (exam role only) wraps the stream in `timeout(8000)`; on expiry it throws `SERVICE_UNAVAILABLE`
   `apps/api/src/common/request-budget.ts:57`
   → a retryable 503 instead of a request nobody ends
6. `ResponseInterceptor` wraps the handler's return; the exact `{items,page,pageSize,total}` shape is split into `data` and `meta`
   `apps/api/src/common/response.interceptor.ts:26`
   → `{ success: true, data, meta: { requestId, … } }`
7. `AuditInterceptor` runs innermost, so it taps the handler's own value, not the envelope; it reads `@Audit` metadata and writes a `RowActionLog` row without awaiting it
   `apps/api/src/audit/audit.interceptor.ts:45`
   → one row, or nothing for a no-op save
8. The controller method runs
   `the feature module`
   → its return value
9. Any throw anywhere above lands in `AllExceptionsFilter`, the single exit
   `apps/api/src/common/all-exceptions.filter.ts:31`
   → `{ success: false, error, meta: { requestId } }`

Interceptors are listed in registration order (`app.module.ts:86`): budget first, so it bounds the interceptors below it as well as the handler. Nest applies their return paths in reverse, which is why the audit tap sees the raw value. `apps/api/test/audit-interceptor-order.e2e.test.ts` pins that order.

Guards and decorators

Guard	Does	Line
`JwtAuthGuard`	skips `@Public()`; verifies the bearer token; checks the Redis session still exists (else `throwEnded`); for an admin re-reads grants from Postgres through `identityOf`, not from the token, so a revoke bites on the next request; attaches `AuthenticatedUser`	`apps/api/src/auth/guards/jwt-auth.guard.ts:20`
`ActorGuard`	`@Actors(...)`: a student token is a valid JWT and must not reach an admin route	`apps/api/src/auth/guards/actor.guard.ts:11`
`FeaturePermissionGuard`	admin only; `isActive` is checked before the super-admin bypass, so a deactivated super admin has no access; any one key at the level via `can()`; `@RequiresExport` adds a `DATA_EXPORT` check	`apps/api/src/auth/guards/feature-permission.guard.ts:34`
`SuperAdminGuard`	`@RequiresSuperAdmin()`; a feature grant never opens these routes	`apps/api/src/auth/guards/super-admin.guard.ts:11`
`ThrottlerGuard`	last on purpose, so `request.user` exists and a caller is counted as themselves	`apps/api/src/app.module.ts:94`

The decorators are in `apps/api/src/common/security/decorators.ts`: `@Public` (`:21`), `@Actors` (`:24`), `@RequiresFeature` (`:34`), `@RequiresAnyFeature` (`:38`), `@RequiresExport` (`:42`, one decorator because a second `RequiresFeature` would overwrite the first), `@RequiresSuperAdmin` (`:51`), `@CurrentUser` (`:54`). Also `@Audit` (`apps/api/src/audit/audit.decorator.ts:14`), three rate-limit marks, and `ZodBody` / `ZodQuery` / `ZodParam` (`apps/api/src/common/zod-validation.pipe.ts`). Every route is authenticated unless it opts out.

The exception filter

`translate` (`apps/api/src/common/all-exceptions.filter.ts:100`) maps every thrown value to one envelope error, in this order:

1. `AppException` carries its own code and status.
2. `ZodError` becomes `VALIDATION_ERROR` with `fieldErrors` keyed by the full dotted path; a path-less issue goes under `FORM_LEVEL_FIELD` (`:212`).
3. A deadlock becomes `CONFLICT` ("somebody else was editing"); Prisma `P2002` becomes `CONFLICT` with the colliding columns in `details`; `P2003` becomes `CONFLICT` naming the relation; `P2025` and a malformed uuid become `NOT_FOUND`; any other known Prisma error is `INTERNAL`.
4. Nest `HttpException` below 500 maps through `errorCodeForStatus`; 5xx becomes `INTERNAL`.
5. An Express middleware's 4xx (`:187`) with wording the filter chooses; 5xx deliberately excluded.
6. Anything else is `INTERNAL` with a generic message, because an unhandled error's text may be a connection string.

A 5xx logs one `error` line with a `bug=` fingerprint (SHA-1 of the first four non-`node_modules` frames with line and column stripped, so a refactor does not look like a new bug, `:78`), the stack, and the request body through `redact` (2,000 characters, 6 levels, 20 items, word-level key matching: `apps/api/src/common/redact.ts`). A 4xx is one `debug` line. There is no error-tracking vendor; the fingerprint is what survived Sentry's removal.

`ErrorCodes` has 19 members, each with a status and default message beside it (`packages/contracts/src/envelope.ts:12`, `:39`, `:66`). `RATE_LIMITED` is 429; `DRAW_SHORTFALL` and `EXPORT_TOO_LARGE` are 422; `SERVICE_UNAVAILABLE` is 503. `meta.requestId` is the same value as the header and the log line.

The 8-second budget

The exam role gives up before its clients do. A submit times out client-side at 10 s and a save at 15 s; without a server limit a retry arrived at a container still doing the first copy of the work. Two layers, both exam-only: `statement_timeout=6s` appended to the Prisma URL unless the URL already sets `options=` (`apps/api/src/common/request-budget.ts:26`, applied at `apps/api/src/prisma/prisma.service.ts:18`), and an 8 s `timeout` on the handler stream (`request-budget.ts:20`). The query limit is lower so the error a caller sees is the database's, not a bare timeout. `PrismaService` also logs any query over 500 ms without its parameters, and omits `Question.searchText` from every read (`prisma.service.ts:36`).

Transaction limits are named in `TX_LIMITS` (`prisma.service.ts:7`): `SHORT` 5 s wait / 10 s run, `MEDIUM` 10 s / 15 s, `BULK` 10 s / 120 s.

Rate limiting

Four throttlers, all one-minute fixed windows (`apps/api/src/common/throttling/throttling.module.ts:22`). The unmarked default applies to every route; each named one carries `skipIf: notMarkedWith(name)` (`apps/api/src/common/throttling/rate-limits.ts:31`), so it runs only where its decorator sits. A marked route is therefore counted by both its named throttler and the default.

Throttler	Env, default per minute	Routes
default	`RATE_LIMIT_DEFAULT_PER_MIN` 300	every route
`otpRequest`	`RATE_LIMIT_OTP_REQUEST_PER_MIN` 120	`POST /auth/student/otp/request` (`apps/api/src/auth/auth.controller.ts:33`)
`auth`	`RATE_LIMIT_AUTH_PER_MIN` 120	student and admin OTP verify, admin OTP request, refresh (`auth.controller.ts:45,58,68,81`)
`sitting`	`RATE_LIMIT_SITTING_PER_MIN` 60	start attempt, submit, state autosave (`apps/api/src/attempts/attempts.controller.ts:63,109,121`)

The tracker is `actor:id` once authenticated, `ip:{ip}` only as a fallback, because a branch of two hundred students shares one address (`apps/api/src/common/throttling/rate-limit.ts:12`). Counting is in Redis, one `MULTI` of `INCR` + `PTTL` per throttler per request (`redis-throttler.storage.ts:22`), so containers share one limit. It fails open on a Redis error (`:31`): a blip must not turn every request into a 429 mid-test. `/health*` and `/metrics` are `@SkipThrottle()` at the class.

Domain events

`DomainEventBus.emit` wraps `EventEmitter2.emit` in try/catch and logs, so a failing listener cannot fail the producer's request (`apps/api/src/common/events/domain-event-bus.ts:12`). It is in-process, with `wildcard: false` so a dotted name is never read as a namespace (`events.module.ts:9`). Delivery is not durable: a crash between the write and the listener drops the event, and each listener says why that is acceptable.

Six events are declared (`apps/api/src/common/events/event-catalog.ts:3`), all wired:

Event	Emitted by	Handled by
`student.signed_up`	`auth.service.ts:212`	`NotificationListener.onSignedUp` writes the welcome row (`notification.listener.ts:22`)
`student.deactivated`	`students.service.ts:487`, `student-privacy.service.ts:57`	`StudentSessionsListener` revokes all sessions (`auth/student-sessions.listener.ts:18`)
`student.mobile_changed`	`students.service.ts:526`	same listener (`:23`): the device holding the old number may not be theirs
`admin.deactivated`	`admins.service.ts:251`	`AdminSessionsListener` (`auth/admin-sessions.listener.ts:14`)
`access.catalog_changed`	`test-series.service.ts` (six sites), `tests.service.ts:322,369`, `offering.service.ts:258,496`	`AccessCacheListener.onCatalogChanged` (`access/access-cache.listener.ts:16`)
`exam_stage.changed`	`exam-stages.service.ts:134`, `exams.service.ts:121`, `base-configs.service.ts:253`	same listener (`:21`)

The catalog invalidation is one `INCR` of `access:catalog:epoch` (`access-resolver.service.ts:254`); every process rebuilds its held copy on its next read. Scoring, rollups and notifications do not use the bus: they are queue jobs or rows written inside their producer's own transaction (`docs/03-conventions.md` §6). `apps/api/src/events/` is the Event/EventCandidate domain feature, not the bus; `app.module.ts:10` aliases the bus as `DomainEventsModule`.

Queues

Nine queues, each registered with its own `jobOptionsFor(name)` (`apps/api/src/queue/queue.module.ts:18`) on a separate Redis connection with `maxRetriesPerRequest: null` (`:14`), because workers issue blocking commands that would stall the application client.

Queue	Concurrency	Attempts, backoff	Trigger
`scoring`	8	5, exp 5 s	submit, re-score, sweeper
`rollup`	2	5, exp 5 s	debounced cohort sweep (5 s), rebuilds (60 s)
`attempt-flush`	1	3, exp 2 s	scheduler every 60 s
`attempt-sweep`	1	3, exp 2 s	scheduler every 120 s
`audit-archive`	1	3, exp 2 s	cron `15 21 * * *` UTC (02:45 IST)
`notification-prune`	1	3, exp 2 s	cron `0 21 * * *` UTC (02:30 IST)
`notifications`	4	5, exp 5 s	schedulers: relay sweep every 60 s, tests-opened sweep every 300 s; the `announce` job
`notification-delivery`	2	4, exp 15 s	keyed per delivery row; limiter 10 per second
`exports`	2	2, exp 5 s	an admin's download request

Policy is `QUEUE_POLICY` (`apps/api/src/queue/queues.ts:21`); names are `QUEUE_NAMES` (`:4`). Retention: a success is kept one hour or 1,000 jobs (`:49`), a failure seven days or 500 (`:43`). `keyedJob(jobId)` (`:63`) sets `removeOnFail: true`, because BullMQ silently drops an `add` whose id still has a retained failure. Every keyed id is built by a function in `queues.ts` (`scoringJobId`, `rescoreJobId`, `rollupRebuildJobId`, `notificationDeliveryJobId`, `announceJobId`), so asking twice queues once.

Where workers run. Processors are listed under `onRole([API_ROLES.WORKER], …)` in `attempts.module.ts:84`, `notifications.module.ts:44`, `audit.module.ts:19` and `exports.module.ts:30`. Scheduler registration is behind `servesRole(WORKER)` in each module's `onModuleInit`, using `upsertJobScheduler` with a fixed scheduler id so a redeploy cannot stack a second one (`attempts.module.ts:106`, `notifications.module.ts:61`, `audit.module.ts:25`). `keepScheduled` (`apps/api/src/queue/keep-scheduled.ts:7`) registers immediately and again on every `ready` event of the queue's connection, so a Redis that comes back empty gets its schedulers back. A `FLUSHALL` with the connection held is not seen (`:12`). Stem rehash on boot is worker-only too (`apps/api/src/questions/stem-rehash.service.ts:20`).

`ReportingWorkerHost` (`apps/api/src/queue/reporting-worker-host.ts:9`) is subclassed by every processor; Nest's metadata scan walks the prototype chain, so one base class gives each its `failed` and `error` listeners. `QueueFailures.record` (`apps/api/src/common/metrics/queue-failures.ts:17`) separates a retry (`warn`) from a spent job (`error` with stack) and counts both.

Exports are the ninth queue. `ExportJobsService.start` enqueues a `build` job under a fresh UUID (`apps/api/src/common/exporting/export-jobs.ts:74`); the worker builds the workbook, uploads it to `exports/{jobId}.xlsx`, enqueues a `discard` job delayed one hour, and writes the audit row (`apps/api/src/exports/exports.processor.ts:36`). Status and file reads check the asker owns the job and answer a stranger's id as not found (`export-jobs.ts:106`). A builder's own `AppException` is returned as the job's result rather than thrown, so it is shown, not retried.

Redis

One `RedisService` (`apps/api/src/redis/redis.service.ts:68`), global, lazy-connect with a capped retry backoff of 200 ms steps to 5 s (`:63`). `onModuleInit` refuses to boot production against a Redis whose `maxmemory-policy` is not `noeviction` (`:91`, `apps/api/src/redis/eviction-policy.ts:11`), because an eviction drops a live sitting; a policy it cannot read is a warning. Shutdown is `onApplicationShutdown`, so the workers close first and still have the connection (`:109`).

Five Lua scripts give compare-and-set: replace-if-unchanged (`:7`, `replaceJson`), delete-if-held (`:14`, `deleteIfUnchanged` and `releaseLock`), remove-set-members-if-unchanged (`:20`), delete-an-index-and-its-members (`:32`), and take-over-lock (`:43`, `holdLock`).

Every key is minted in `apps/api/src/redis/redis.keys.ts`:

Key	TTL
`otp:{actor}:{id}`, `otp:cooldown:…`	`OTP_TTL_SEC` 300 s, `OTP_RESEND_COOLDOWN_SEC` 45 s
`otp:attempts:…`	inherits the code's TTL (`otp.service.ts:258`)
`otp:daily:{mobile}`, `otp:daily:ip:{ip}`, `otp:daily:global`, `otp:daily:signup`	86,400 s, set on the first INCR and re-checked each call (`otp.service.ts:147`)
`otp:desk:{mobile}`	300 s (`otp.service.ts:30`)
`session:{actor}:{sub}:{sid}`, `sessions:…` index, `session-replaced:…`	refresh lifetime, `JWT_REFRESH_TTL` 30 d
`attempt:state:{id}`	48 h (`PAUSE_LIMIT_SEC`); the key is the pause
`attempt:dirty`	none; a SET drained by the flush worker
`sitting:claim:{studentId}`	48 h; opening a sitting stands down the previous one (`attempt-state.service.ts:148`)
`access:catalog:epoch`	none; a counter
`ratelimit:{name}:{tracker}`	the window, 60 s
`test:edit:{id}`, `base-config:edit:{id}`, `section:edit:{testId}:{sectionId}`	15 min from the last edit (`redis.keys.ts:72`)
`audit:archive:{YYYY-MM-DD}`	900 s lock

Storage

One `StorageService` (`apps/api/src/storage/storage.service.ts:24`), global, never branched on environment: no `NODE_ENV` or `isProduction` appears in it. MinIO is reached by configuration alone (`S3_ENDPOINT` plus `S3_FORCE_PATH_STYLE=true` locally); production leaves `S3_ENDPOINT` empty for the regional AWS endpoint. `signedWith` returns `undefined` when either half of the key pair is missing so the SDK uses its own credential chain, the EC2 instance role (`:14`); the env schema refuses a half-set pair.

Surface: `upload` (`:50`); `publicUrl(key)` is `MEDIA_BASE_URL/key`, unsigned, so the CDN caches one copy rather than one per student (`:68`); `createDownloadUrl` is a presigned GET, 900 s default (`:72`); `read` is for a file the API itself must re-read, such as an import committing the sheet it previewed (`:79`); `remove`; `objectSize` returns `null` on any HEAD failure (`:91`); `ping` is a `HeadBucket` (`:102`).

Audit

The table is `RowActionLog` (`prisma/schema.prisma:1084`): feature, entity id, action, actor type, actor id, `changed` jsonb, nullable `importLogId` (`onDelete: Restrict`), `createdAt`. `AuditFeature` has 18 members and `AuditAction` 9 (`packages/contracts/src/audit.ts:14,40`). `@Audit(feature, action)` appears 78 times across 18 files.

`AuditInterceptor` resolves the action (static, or `TOGGLE_ACTIONS.signIn` / `.tests` off the body: `audit.decorator.ts:18`), then the entity id from, in order, the context store, the created row's `id` for a CREATE, `params.id`, the payload's `id` (`audit.interceptor.ts:55`). The write is not awaited and its failure is logged and swallowed (`:67`), since the change already committed. A request with no token is recorded as `SCRIPT`, never defaulted to `ADMIN` (`:81`). A service calls `AuditContext.setChanged` or `setPatchDiff`; `setPatchDiff(null)` marks the save a no-op and nothing is written (`audit.context.ts:31`).

Reading is scoped in `rowActionWhere` (`apps/api/src/audit/audit.service.ts:81`): a non-super admin's `actorId` filter is ignored and `where.actorId = viewer.id` is applied outside the `AND`, so an ANY match cannot widen past the viewer; student-authored rows are excluded for

everyone (`:100`). Date bounds go through `startOfInstituteDay` / `endOfInstituteDay` (`:51`). The screen is a super admin's alone (`d68dcb5`).

Archive. Thirty days stay hot in Postgres (`apps/api/src/audit/audit-archive.ts:9`). The daily job archives the oldest institute day still owed, repeating up to 14 days per run so a missed night is a delay, not a stranded day (`audit-archive.processor.ts:22,63`). Per day: take a Redis lock keyed on the institute day (`:126`), page 1,000 rows by id, gzip each page into one concatenated NDJSON object at `audit/row-actions/YYYY/MM/DD.ndjson.gz` (the date is the institute day, `audit-archive.ts:13`), upload, verify the stored size equals the body length, and only then delete the same `[gte, lt)` window.

Logging and metrics

`appLogger()` is the one place stdout's shape and volume are chosen (`apps/api/src/common/logging.ts:15`), read off `process.env` because it runs before the config module exists. Production is JSON; the default level is `>=log` in production and `>=debug` elsewhere; `LOG_LEVEL` overrides in `filterLogLevels` ' vocabulary and a malformed value is refused at boot.

The per-request line comes from `RequestObserverMiddleware.record` (`request-observer.middleware.ts:30`):

```
GET /me/attempts/:id/paper → 200 42ms STUDENT:0192…ab [req-id]
```

It carries the route pattern, never the URL (`routeOf` , `request-id.ts:38`), so 5,000 ids are one series. A status of 400 or more logs `debug` with `REFUSED` ; a success at 1,000 ms or more logs `warn` with `SLOW` ; the rest are `debug` , which production's `>=log` drops. The actor is inside the string because this Nest emits each extra parameter as its own line.

`MetricsService` (`apps/api/src/common/metrics/metrics.service.ts`) uses a private `Registry` , prefix `iace_` , plus default process metrics. Counters and histograms: `http_request_duration_seconds` (method, route, status), `attempt_submits_total` , `auth_attempts_total` , `scoring_duration_seconds` , `otp_sends_total` , `queue_job_failures_total` , `queue_connection_errors_total` . Gauges read on the scrape itself: `queue_depth` and `queue_oldest_waiting_seconds` per queue (the queues are found by name off `QUEUE_NAMES` , so a new one is measured without being listed), `redis_memory_bytes` , `redis_memory_ratio` , `redis_evicted_keys_total` , `live_attempts` (`SCARD attempt:dirty`), `db_connections` (`pg_stat_activity`), and `scoring_backlog` , pushed by the sweeper.

`GET /metrics` is `@Public()` , `@SkipThrottle()` , answers `text/plain; version=0.0.4` through `@Res` , and requires `Authorization: Bearer ${METRICS_TOKEN}` whenever one is set (`metrics.controller.ts:35`); production requires one.

`GET /health` is liveness: it always answers 200 with `status: 'ok' | 'degraded'` (`apps/api/src/health/health.controller.ts:35`). `GET /health/ready` throws `SERVICE_UNAVAILABLE` naming the down dependencies (`:50`). Both probe database, Redis, storage and the scoring queue in parallel, each under a 2 s race (`:94`), but readiness turns only on database, Redis and queue (`packages/contracts/src/health.ts:25`): a storage outage is reported, not fatal.

Shutdown

On SIGTERM `HttpDrain.beforeApplicationShutdown` starts draining before Nest closes the listener: every response gets `Connection: close` , idle connections are swept every 250 ms, and whatever is left is cut after 10 s (`apps/api/src/common/http-drain.ts:12,28`). Workers close in their own shutdown hook; Prisma and Redis disconnect afterwards, in `onApplicationShutdown` , so a job mid-body still has its pool (`prisma.service.ts:51` , `redis.service.ts:109`). Compose gives each container 30 s.

WORKER BOOT — HOW THE SCHEDULERS COME TO EXIST

1. Container starts with `API_ROLE=worker` ; `apiRole` is fixed at file load
   `apps/api/src/config/api-role.ts:33`
   → role `worker`
2. `onRole([WORKER], …)` leaves the processors in the module's providers; controllers are filtered out
   `apps/api/src/attempts/attempts.module.ts:84`
   → nine `@Processor` classes live
3. `RedisService.onModuleInit` connects and refuses a non- `noeviction` policy in production
   `apps/api/src/redis/redis.service.ts:84`
   → connected, or boot aborts
4. Each module's `onModuleInit` passes `servesRole(WORKER)` and calls `keepScheduled`
   `apps/api/src/queue/keep-scheduled.ts:7`
   → register once now
5. `upsertJobScheduler` under a fixed scheduler id (`attempt-flush` every 60 s, `attempt-sweep` every 120 s, `relay-sweep` every 60 s, `tests-opened-sweep` every 300 s, `notification-prune` and `audit-archive` by cron)
   `apps/api/src/attempts/attempts.module.ts:111`
   → repeatable jobs in Redis, one per id however many boots
6. The queue's connection emits `ready` after a reconnect and `register()` runs again
   `apps/api/src/queue/keep-scheduled.ts:13`
   → schedulers restored on a Redis that came back empty

Time

Storage is UTC; every use is `Asia/Kolkata` . There is one helper set for each side, and a second is a bug: `packages/contracts/src/common.ts` (`INSTITUTE_TIME_ZONE` , `civilDate` , `instituteWallTime` , `fromInstituteWallTime` , Intl-based and dependency-free because contracts ships to the browsers); `apps/api/src/common/time/institute-day.ts` (`startOfInstituteDay` , `endOfInstituteDay` , `instituteDayOf` , `shiftInstituteDay` , `toDateColumn` , `fromDateColumn` , on `date-fns` and `@date-fns/tz` , server-only); `apps/api/src/common/importing/date-cell.ts` (`toIsoDate` for a spreadsheet cell of unknown format). Deriving a civil date from a stored instant with `getUTC*` is wrong every night from 00:00 to 05:30 IST. A `@db.Date` column is a civil date at UTC midnight by convention, and `toDateColumn` deliberately uses UTC.

What a caller must already have

`DATABASE_URL` (with `connection_limit` in production), `REDIS_URL`, both JWT secrets of at least 24 characters, `S3_BUCKET`, `MEDIA_BASE_URL`; in production also `CORS_ORIGINS`, `METRICS_TOKEN`, a counted `TRUST_PROXY_HOPS`, and a `noeviction` Redis. At least one container must run as `worker` or `all`, or no job and no scheduler exists anywhere. `OTP_SENDER=console` is refused in production (`apps/api/src/common/messaging/messaging.module.ts`), and a real sender also needs `SMS_PROVIDER_URL` and the mail credentials.

Why it is shaped this way

One envelope and one filter, rather than per-endpoint shapes: a wrapper on every response and a duck-type check on lists, against a client that parses one failure shape and one page shape platform-wide. Distinct codes stay distinguishable although several share a status (`RATE_LIMITED` and an OTP lockout are both 429-class).

In-process events, no broker and no outbox: the cost is at-least-once delivery, the gain is zero infrastructure. The outbox was removed after its two non-re-derivable consumers moved onto their own durable paths (notification rows in the producer's transaction, re-scores onto `Attempt.scoredRevision`).

One image with a role rather than separate services: a per-module `onRole` call, against duplicating the whole bootstrap in a second entry point. The point of the split is that the sitting is the only load that scales with a hall, so it runs alone and an import or a rollup cannot slow it.

Postgres off the hot path: Redis holds the live sitting, and the price is the `noeviction` requirement and the 48-hour ceiling on resuming one.

What is wrong with it

- `apps/api/src/events/../common/events/events.module.ts:10` Stale: the comment's example `student.pin_reset` is not in the catalog; students have no PIN.
- `apps/api/src/common/logging.ts:4` Stale: the header says the per-request line is in `MetricsInterceptor`; it is `RequestObserverMiddleware` (`request-observer.middleware.ts:30`).
- `apps/api/src/common/throttling/rate-limits.ts:21` Stale: the `AuthRateLimit` comment says "OTP verify, PIN and refresh"; there is no PIN route.
- `apps/api/src/common/threadpool.ts:6` Stale: "Set when PIN hashing shared the pool"; no hashing of a PIN exists.
- `apps/api/src/redis/redis.keys.ts:42` Stale: the archive lock is described as one UTC day, but the key is the institute day (`audit-archive.processor.ts:125`).
- `apps/api/src/common/metrics/metrics.service.ts:113` Stale: "A spent one is gone for good — nothing retains it" is false for every non-keyed queue, whose failures are retained seven days (`queues.ts:43`).
- `docs/04-infrastructure.md` §3 table Stale: the worker row says "all eight processors and schedulers"; there are nine queues and nine processors (`QUEUE_NAMES`, `queues.ts:4`).

Telling a student something happened

Where it lives

`apps/api/src/notifications/` (17 files) and `apps/api/src/common/messaging/`.

File	Owns
`apps/api/src/notifications/notifications.service.ts`	`Notification` writes. `tell()` is the only writer of a per-student row.
`apps/api/src/notifications/bell.service.ts`	The bell read: a student's own rows merged with the broadcasts that reach them; mark-read.
`apps/api/src/notifications/announcements.service.ts`	The admin broadcast: preview, cap, record, delivery job, stats, export.
`apps/api/src/notifications/announcement-cohort.ts`	`cohortWhere` (live students matching an audience) and `reachedBy` (those who existed at send time).
`apps/api/src/notifications/test-opening.service.ts`	The clock-driven sweep that stamps `Test.announcedAt` and pushes the opening.
`apps/api/src/notifications/notifications.processor.ts`	The worker: push sweep, `announce` job, `tests-opened-sweep`.
`apps/api/src/notifications/push.service.ts`	`PushSubscription` and `PushDevice`, and the free fan-out to both.
`apps/api/src/notifications/web-push.sender.ts`, `apps/api/src/notifications/fcm.sender.ts`	The two transports; each exposes `isConfigured`.
`apps/api/src/notifications/notification-delivery.processor.ts`	The only place money is spent: one paid send per ledger row, plus `repairStalled`.
`apps/api/src/notifications/notification-policy.ts`	Skip reasons, the paid-channel map, the grace window.
`apps/api/src/notifications/notification-prune.processor.ts`	Nightly prune of settled `NotificationDelivery` rows.

What it is for

Every fact the platform learns about a student has to reach them: a result scored, a grant added, an enrolment, a welcome, and whatever an admin chooses to say to a cohort. There are two shapes, and the split is the point of the module.

- **A per-student row** (`Notification` with a `studentId`): written by the producer inside its own transaction through `tell()`. Producers are `apps/api/src/attempts/scoring.processor.ts:192` and `:208` (result ready, result updated), `apps/api/src/access/student-grants.service.ts:141`, `apps/api/src/students/students.service.ts:430` (enrolment added), the welcome listener at `apps/api/src/notifications/notification.listener.ts:36`, and a paid announcement's copies at `apps/api/src/notifications/announcements.service.ts:132`.
- **A broadcast** (an in-app announcement, or a test that has opened): ONE row for everybody, the `Announcement` or the `Test` with `announcedAt` set. Nothing is written per student. `BellService` works out who a broadcast reaches at the moment a student reads the bell, and only the act of reading is stored per student, in `BroadcastRead` (`prisma/schema.prisma`, `model BroadcastRead`).

The reason is in commit `b73c9dd`: an in-app send to a cohort used to write one row per student inside the request (42K students, 8.2 s), and every test that opened wrote one per student its series reached. Nothing pruned them, so a few tests a week grew the table by hundreds of thousands of rows that said the same thing.

A paid announcement (one that chose SMS or WhatsApp) is the exception. Each copy is a billed message and the paid chain runs off the copy, so the worker still writes one `Notification` per student for it.

Free channels (web push, FCM) go at once. A paid channel is bought only for a paid announcement, and only after a 10-minute window the student can close by reading the bell. `NotificationDelivery` is the per-channel ledger that answers "what did this send cost, who did it reach".

Flow: an announcement goes out

POST /ADMIN/ANNOUNCEMENTS — AN ADMIN SENDS A BROADCAST

1. Admin submits the compose dialog; the client posts the title, body, audience (the Students-screen filter) and `paidChannels`
   `apps/admin/src/features/announcements/compose-announcement-dialog.tsx:109`
   → request body
2. Class guard `@Actors(ActorTypes.ADMIN)`; method guard `@RequiresFeature(NOTIFICATION_MANAGEMENT, WRITE)`; body parsed by `ZodBody(createAnnouncementSchema)`
   `apps/api/src/notifications/announcements.controller.ts:33`
   → a typed `CreateAnnouncementBody`, or a 4xx with `fieldErrors`
3. `send()` counts the cohort with `cohortWhere(audience)`, which is live students only (`deletedAt: null`)
   `apps/api/src/notifications/announcements.service.ts:83`
   → `recipientCount`
4. Over `NOTIFICATION_MAX_RECIPIENTS` (50000, `.env.example:197`) is refused; zero is refused as "That reaches nobody"
   `apps/api/src/notifications/announcements.service.ts:85`
   → `VALIDATION_ERROR`, nothing written
5. One `Announcement` row is created with the audience, `paidChannels`, `recipientCount`, and `estimatedCostPaise` priced off the FIRST paid channel only
   `apps/api/src/notifications/announcements.service.ts:95`
   → `Announcement` row. For an in-app send this row IS the broadcast.

6. A BullMQ job `announce` is queued under the job id `announce-<id>`, so asking twice is one delivery
   `apps/api/src/notifications/announcements.service.ts:108`
   → job on the `notifications` queue
7. `@Audit(ANNOUNCEMENT, CREATE)` files the send with the recipient count and channels once the handler returns
   `apps/api/src/notifications/announcements.controller.ts:67`
   → audit row; the response is `detail(id)`
8. The worker picks the job up by name
   `apps/api/src/notifications/notifications.processor.ts:72`
   → `announcements.deliver(id)`
9. `deliver()` pages the cohort by id, 1000 at a time, over `reachedBy(audience, sentAt)`: the filter among students who existed when it was sent
   `apps/api/src/notifications/announcements.service.ts:122`
   → ordered id pages
10. In-app only: `push.broadcast(ids, …)` pushes to those students' browsers and phones with no ledger and no retry. The bell already holds the broadcast
    `apps/api/src/notifications/announcements.service.ts:137`
    → pushes sent; nothing durable per student
11. Paid: `tell()` writes one `Notification` per student with `announcementId` and `dedupeKey announcement:<id>`, `skipDuplicates`, so a replayed page lands once
    `apps/api/src/notifications/announcements.service.ts:132`
    → `Notification` rows with `pushedAt` NULL; the push sweep takes them from here

The response of step 7 comes back before steps 8 to 11 run. Nothing on the request grows with the cohort.

Flow: the bell is read

GET /ME/NOTIFICATIONS — A STUDENT OPENS THE BELL

1. Student guards: JWT with a live Redis session, then actor must be STUDENT; the student id is the token's, never the request's
   `apps/api/src/me/me.controller.ts:79`
   → `user.id`
2. Query parsed: `unreadOnly`, `cursor`, `unreadAsOf`, `q`, `type`, `from` / `to` (civil days in Asia/Kolkata), plus paging
   `packages/contracts/src/access.ts:285`
   → `NotificationListQuery`
3. `BellService.list` first resolves the student's broadcasts
   `apps/api/src/notifications/bell.service.ts:72`
   → in-memory `Entry[]`
4. The window starts at the later of 90 days ago (`BELL_WINDOW_DAYS`) and the student's own `createdAt`, so nobody receives what went out before they joined
   `apps/api/src/notifications/bell.service.ts:127`
   → `since`
5. Announcements: up to 50 in the window with no paid channel, each tested with one `student.count` of `cohortWhere(audience)` AND the student id, all in one `$transaction`
   `apps/api/src/notifications/bell.service.ts:167`
   → announcements that match this student
6. Openings: the student's catalog series, then tests in them with `announcedAt >= since`, newest 200
   `apps/api/src/notifications/bell.service.ts:205`
   → opening entries typed `TEST_ASSIGNED`
7. Reads come from `BroadcastRead`; a broadcast the student already holds a per-student copy of (rows written before `b73c9dd`) is skipped so it does not show twice
   `apps/api/src/notifications/bell.service.ts:139`
   → each entry gets `isRead`, `readAt`
8. The student's own rows come from one `findMany` plus one `count`; a cursor is turned into an anchor and the cursor row is excluded by position, not `skip: 1`, because once read it leaves the unread filter
   `apps/api/src/notifications/bell.service.ts:77`
   → rows older than the anchor
9. The two lists are filtered the same way (database `where` for rows, `shownBy` for broadcasts), merged, sorted newest first with an id tie-break, and sliced
   `apps/api/src/notifications/bell.service.ts:90`
   → `Paginated<Notification>`; `meta.total` is `total + shown.length`

PATCH /ME/NOTIFICATIONS/:ID/READ — MARKING ONE READ

1. Same guards; the id comes from the path
   `apps/api/src/me/me.controller.ts:87`
   → `bell.markRead(user.id, id)`
2. A row scoped by `{ id, studentId }`: found and unread means `isRead: true, readAt: now`
   `apps/api/src/notifications/bell.service.ts:98`
   → `Notification` updated; already read returns as is
3. Not found as a row: the id is looked up among the student's broadcasts. Not there either is `NOT_FOUND`
   `apps/api/src/notifications/bell.service.ts:108`
   → someone else's id finds nothing
4. A broadcast read is `createMany({ skipDuplicates })` into `BroadcastRead` keyed `(studentId, broadcastId)`
   `apps/api/src/notifications/bell.service.ts:112`
   → `BroadcastRead` row

The unread badge is not a separate endpoint. The web shell asks the same list for `unreadOnly` with `pageSize 1` and reads `meta.total` (`apps/exams/src/components/app-shell.tsx:34`).

Flow: the push sweep

RELAY-SWEEP (EVERY 60 S) — PUSHING WHAT WAS WRITTEN

1. The scheduler is upserted at boot, on the worker role only, under a fixed id so a redeploy cannot stack a second sweep
   `apps/api/src/notifications/notifications.module.ts:67`
   → repeat job `relay-sweep` , `every: NOTIFICATION_SWEEP_EVERY_MS`
2. The job runs on the `notifications` queue (concurrency 4, 5 attempts); anything not named `announce` or `tests-opened-sweep` is the sweep
   `apps/api/src/notifications/notifications.processor.ts:67`
   → `pushPending()`
3. `pushPending` loops up to 25 pages, stopping early on a short page
   `apps/api/src/notifications/notifications.processor.ts:86`
   → pages of `RELAY_BATCH` (200)
4. One transaction claims a page: a raw `UPDATE … SET pushedAt = now()` over unpushed rows whose `nextPushAt` is null or past, oldest first, `FOR UPDATE SKIP LOCKED` , joined to the `Announcement` for its `paidChannels`
   `apps/api/src/notifications/notifications.processor.ts:119`
   → rows stamped `pushedAt`
5. In the same transaction the FIRST paid channel (only) is booked as a `PENDING` `NotificationDelivery` row
   `apps/api/src/notifications/notifications.processor.ts:139`
   → ledger row per paid notification
6. `push.deliverAll` takes the page; a row with no student is dropped
   `apps/api/src/notifications/notifications.processor.ts:100`
   → calls into `push.service.ts`
7. One read for the page: existing `WEB_PUSH` and `MOBILE_PUSH` ledger rows, and the students' subscriptions (only if `WebPushSender.isConfigured`) and devices (only if `FcmSender.isConfigured`)
   `apps/api/src/notifications/push.service.ts:140`
   → `Reach`
8. Every target's session is checked with one Redis `MGET` of `session:student:<id>:<sessionId>` . A missing key means signed out; that target is deleted, matched on `(target, studentId, sessionId)` so a sign-in that re-bound it meanwhile survives. A subscription whose host fails the allow-list is deleted too
   `apps/api/src/notifications/push.service.ts:161`
   → dead targets removed
9. The page is pushed in lanes of 8. Each channel is decided on its own: closed when a ledger row is `SENT` , or `FAILED` at 3 attempts; no target is an absence and books nothing
   `apps/api/src/notifications/push.service.ts:98`
   → HTTP calls to the push service / FCM
10. Payload is the title, one line of the bell's body (whitespace folded, cut at 140 characters), `/notifications` , and the id. No score, no answer
    `apps/api/src/notifications/web-push.sender.ts:31`
    → JSON payload
11. Browser leg: `sendNotification` with TTL 6 h and a 5 s timeout; 404 or 410 is `GONE` . Phone leg: a service-account JWT is traded for a token and posted to FCM v1; 404, or 400/403 with an unregistered-style error code, is `GONE`
    `apps/api/src/notifications/web-push.sender.ts:68`
    → `SENT` , `FAILED` or `GONE` per target
12. `settle` deletes dead endpoints and tokens, `createMany` s the first-attempt ledger rows (`SENT` , or `FAILED` with `lastError`) and updates retried ones
    `apps/api/src/notifications/push.service.ts:303`
    → `NotificationDelivery` rows
13. `reopen` hands back the claim of a row whose pushes all failed, under the cap: `pushedAt = NULL` and `nextPushAt = now + 1 min` after the first failure, `+ 5 min` after the second
    `apps/api/src/notifications/push.service.ts:337`
    → the row is claimable again, but not within this pass
14. The paid rows of the page are re-read as `PENDING` and added in one `addBulk` to the `notification-delivery` queue, job id keyed on the delivery row, delay 600 s (0 when `actBy` leaves under 2400 s)
    `apps/api/src/notifications/notifications.processor.ts:145`
    → delayed delivery jobs
15. After the pushes, whatever the pass did: `repairStalled()` takes up to 25 `PENDING` paid rows older than 20 minutes and forces a decision on each, one at a time
    `apps/api/src/notifications/notification-delivery.processor.ts:157`
    → rows moved to `SENT` , `SKIPPED` or the next channel

NOTIFICATION-DELIVERY JOB — BUYING ONE PAID MESSAGE

1. Job `{ deliveryId }` arrives after the grace delay; the worker rate-limits (`DELIVERY_RATE_LIMIT`) and runs 2 at a time
   `apps/api/src/notifications/notification-delivery.processor.ts:83`
   → `deliver(id, attempt)`
2. The row is re-read with its notification and announcement. Already `FAILED` means booking the next channel threw earlier, so book it again
   `apps/api/src/notifications/notification-delivery.processor.ts:95`
   → repair
3. The notification already read: `SKIPPED` with `ALREADY_READ` , no money spent. This is what the grace window is for
   `apps/api/src/notifications/notification-delivery.processor.ts:102`
   → `skipReason ALREADY_READ`
4. No message kind for the type is `NO_TEMPLATE` ; no mobile on the student is `NO_CONTACT` ; each of those falls back to the next channel
   `apps/api/src/notifications/notification-delivery.processor.ts:108`
   → `SKIPPED` plus a booking of the next channel

5. The row is claimed `SENT` BEFORE the provider call, with `updateMany … where status PENDING`; a retry that finds it taken stops
   `apps/api/src/notifications/notification-delivery.processor.ts:123`
   → `status SENT`, `sentAt`, `attempts`
6. `sender.send` with the channel, the kind, the mobile, and the announcement body in `data.message`
   `apps/api/src/notifications/notification-delivery.processor.ts:130`
   → provider call
7. `MessageNotConfiguredError` skips and falls back. Any other error is recorded (`PENDING` again, or `FAILED` at the 4th attempt); under the cap it rethrows so BullMQ backs off 15 s on the same channel, at the cap it falls back
   `apps/api/src/notifications/notification-delivery.processor.ts:141`
   → `lastError`, `failedAt`
8. `fallBack` upserts the next channel in the announcement's `paidChannels` and queues it keyed on its row. No next channel is a warning and the end
   `apps/api/src/notifications/notification-delivery.processor.ts:204`
   → the next `NotificationDelivery` row and job

TESTS-OPENED-SWEEP (EVERY 5 MIN) — A TEST OPENS BY THE CLOCK

1. Scheduled beside the push sweep, same worker-only rule
   `apps/api/src/notifications/notifications.module.ts:73`
   → repeat job `tests-opened-sweep`
2. Up to 20 `ACTIVE` tests with `announcedAt` null and `opensAt` null or past
   `apps/api/src/notifications/test-opening.service.ts:43`
   → the tests to speak about
3. `Test.announcedAt = now` is written FIRST: a crash before the push costs the push, never a second one
   `apps/api/src/notifications/test-opening.service.ts:71`
   → `announcedAt` set. This stamp IS the bell entry.
4. `access.studentsReaching(seriesId)` names the audience, and `push.broadcast` pushes to it in chunks of 1000, no ledger
   `apps/api/src/notifications/test-opening.service.ts:72`
   → pushes sent
5. One test failing is caught and logged; the others go on
   `apps/api/src/notifications/test-opening.service.ts:58`
   → a test whose stamp failed is retried by the next sweep

Flow: a device registers for push

POST /ME/PUSH-SUBSCRIPTION AND POST /ME/PUSH-DEVICE — A BROWSER OR PHONE OPTS IN

1. Web: the settings screen reads the VAPID public key from `GET /me/push-subscription`, which returns `{ publicKey }` or null when the keypair is unset
   `apps/api/src/me/me.controller.ts:96`
   → key, or a channel shown as unavailable
2. Web: the page re-binds an existing subscription once per load and on each sign-in, because the server ties a subscription to one session
   `apps/exams/src/lib/pwa.ts:118`
   → `POST /me/push-subscription`
3. Body parsed by `pushSubscriptionSchema`, whose endpoint host must be https on a known push service (FCM, Mozilla, Windows, Apple)
   `packages/contracts/src/access.ts:347`
   → `PushSubscriptionBody`
4. Upsert keyed on `endpoint`: the same device resubscribing is the same row, moved to the student and session now signed in
   `apps/api/src/notifications/push.service.ts:64`
   → `PushSubscription` with `sessionId`
5. Phone: on sign-in, Android on a real device only (not Expo Go, not iOS, because FCM cannot address an APNs token), permission is asked and the FCM token read
   `apps/mobile/src/lib/push-device.ts:22`
   → token, or nothing
6. `POST /me/push-device` with token (up to 4096 characters), platform, device name; upsert keyed on `token`, since the token is the identity and the phone is not
   `apps/api/src/notifications/push.service.ts:80`
   → `PushDevice` with `sessionId`
7. Teardown: the phone calls `DELETE /me/push-device` BEFORE ending its session, waiting a bounded time; the web does the same for `DELETE /me/push-subscription`. Both `deleteMany` scoped by `studentId`
   `apps/mobile/src/lib/push-device.ts:48`
   → rows removed; if refused, the next sweep drops them as signed-out

Two tables, not one: a browser subscription carries the keys that encrypt for it and dies with a 404 or 410; a device token is one opaque string FCM owns and rotates and dies with an error code. One table would mean nullable keys and a discriminator.

The data it touches

Column	Holds	Written by
`Notification.studentId` / `adminId`	exactly one recipient (a CHECK); nothing writes `adminId`	`tell()`
`Notification.type`	`TEST_ASSIGNED`, `RESULT_READY`, `ENROLLMENT_ADDED`, `GRANT_ADDED`, `GENERIC`, `RESULT_UPDATED`, `PIN_CHANGED`, `WELCOME`	`tell()`
`Notification.title`, `body`, `data`	what the bell shows; `data` carries template variables (an announcement copy writes `{ message }`)	`tell()`

Column	Holds	Written by
`Notification.dedupeKey`	the natural key of the fact (`announcement:<id>`, `welcome:<studentId>`, `result:<attemptId>`); two partial uniques, one per recipient column	`tell()`
`Notification.isRead`, `readAt`	read receipt; the paid send checks it	`markRead`
`Notification.pushedAt`	the sweep's claim stamp	the claim query
`Notification.nextPushAt`	floor a retry waits for; NULL means now	`reopen` only
`Notification.actBy`	when the notification stops being actionable; no producer sets it	nothing
`Announcement`	title, body, `audience` (the student filter, verbatim), `paidChannels` in fallback order, `recipientCount`, `estimatedCostPaise`	`send()`
`BroadcastRead`	`(studentId, broadcastId)` primary key, `readAt`; cascades with the student	`markRead`
`Test.announcedAt`	the opening sweep's watermark and the broadcast's timestamp	`announce()`
`NotificationDelivery`	one row per `(notificationId, channel)`; status, `skipReason`, `attempts`, `lastError`; updated on retry, never appended	push service, delivery processor
`PushSubscription`, `PushDevice`	endpoint or token, the keys, the registering `sessionId`	`subscribe`, `registerDevice`

Redis: this module writes no key. It reads `session:student:<id>:<sessionId>` (set at sign-in with the refresh lifetime as TTL) to decide whether a push target is still live.

What must already be true

An admin with `NOTIFICATION_MANAGEMENT` at READ (list, detail), WRITE (preview, send) or export. The controller mounts only under `API_ROLES.CORE`; the processors and schedulers only under `API_ROLES.WORKER` (`apps/api/src/notifications/notifications.module.ts:39`). All three `VAPID_*` for web push and all three `FCM_*` for the phone, else `isConfigured` is false and that channel is skipped without a row. A paid send also needs a configured SMS or WhatsApp provider, or `RoutedMessageSender` throws "Nothing delivers ... messages" (`apps/api/src/common/messaging/routed-message-sender.ts:14`). `OTP_WHATSAPP_ENABLED=false` (`.env.example:103`) only gates a student's sign-in code; a paid announcement can still choose WhatsApp.

Whether a given deployment can push is an environment fact, not a code fact. `.env.example` ships the VAPID and FCM values blank (read, not measured on any running environment).

Why it is shaped this way

- **The row is written by its producer.** `tell()` takes the caller's transaction, so the fact and the bell row commit together. An outbox row and a relay cost a minute of lag and a second durable write per fact; they are gone. The cost is `pushedAt` as a column on a hot table.
- `dedupeKey` **plus** `skipDuplicates` instead of a "have I told them" read. A replay lands once, with no extra query. Two partial uniques exist because the recipient lives in two columns, and a keyless notification can double up.
- **A broadcast is resolved at read time**, the way the catalog resolves access. The write cost of a send no longer depends on the cohort; the read cost is one audience `count` per announcement in the window (capped at 50), paid by the student who opens the bell.
- **The ledger is UPDATEd on retry.** `@@unique([notificationId, channel])` keeps it at notifications times channels. An attempt log would turn per-announcement stats into a window query and make the prune larger.
- **Paid channels are a fallback chain, not a fan-out.** Only the first is booked. Fanning out would be simpler and double the invoice.
- **Push retry is spaced.** Three attempts over about six minutes. Without `nextPushAt` a reopened row was the oldest unpushed row, so it was reclaimed on the next page of the same pass and burned all attempts in under a second against a device that was merely offline (`2cd235c`).
- **A push carries one line.** Titles alone were all phones showed; the bell's own body is fixed copy or text an admin wrote to be read, so it is sent, folded to one line and cut at 140 characters (`ab5d593`). The rule that holds: a push names no score and no answer.

Which screen calls it

Endpoint	Caller	Screen
`GET /admin/announcements`	`apps/admin/src/features/announcements/announcements.tsx`	admin Sent list
`POST /admin/announcements/preview`, `POST /admin/announcements`	`apps/admin/src/features/announcements/compose-announcement-dialog.tsx:100`, `:109`	compose dialog
`GET /admin/announcements/:id`	the same screen's row detail	detail panel, reads and delivery counts
`POST /admin/announcements/:id/deliveries/export`	the detail panel	async workbook export
`GET /me/notifications`	`apps/exams/src/features/notifications/notifications.tsx:54`, `apps/exams/src/components/app-shell.tsx:34`, `apps/mobile/app/notifications.tsx:38`	the bell, its badge
`PATCH /me/notifications/:id/read`	`apps/exams/src/features/notifications/notifications.tsx:187`, `apps/mobile/app/notifications.tsx:45`	bell

Endpoint	Caller	Screen
`GET`, `POST`, `DELETE /me/push-subscription`	`apps/exams/src/features/notifications/notification-settings.tsx`, `apps/exams/src/lib/pwa.ts`	web settings, every signed-in load
`POST`, `DELETE /me/push-device`	`apps/mobile/src/lib/push-device.ts:31`, `:54`	mobile, sign-in and sign-out

The admin send screen shows an in-app announcement by its reads alone; it has no delivery ledger to count. A paid one shows sent, failed, skipped and "saved by read" counts, and `DELIVERED` is not shown because no provider reports back.

Is it optimal

Read from code and migration headers; no `EXPLAIN` was run.

- **Send:** one `student.count`, one create, one queue add. Constant in the cohort size.
- **Delivery job (in-app):** one id-ordered page read per 1000 recipients, plus the push reads. No writes per student.
- **Delivery job (paid):** one `createMany` per 1000. A 50,000 cohort is 50 statements, now off the request.
- **Bell read:** two queries for the student's own rows, then per-broadcast cost: one `student.count` for each announcement in the 90-day window (up to 50), a catalog resolve for openings, and two small reads for reads and legacy copies. Not measured. A student with a long announcement history pays this on every open; a comment at `apps/api/src/notifications/bell.service.ts:29` already names the upgrade (cache per student).
- **Sweep:** per 200-row page, one claim transaction and three parallel reads plus one `MGET`; the page's pushes are the real cost, 8 in flight.

What changed recently

- `b73c9dd`: broadcasts resolved instead of copied; `BroadcastRead`; the `announce` job; `BellService`.
- `ab5d593`: a push carries one line under its title.
- `6afdf06`: a student's sign-in code goes by SMS alone. Announcements are unaffected.
- `8e166fa`, `ccd2bfa`: the bell takes a `cursor` and `unreadAsOf`, so reading rows between page loads cannot skip rows or make the badge fall twice.
- `be562e4`: the bell and the announcements list filter by words, type and day, and page.
- `2cd235c`: `nextPushAt`; a failed push waits before it is tried again.
- `cc92db1`: an opening that dies part-way finishes on the next sweep. Its per-student fan-out has since gone with `b73c9dd`; the stamp-first order remains.
- `677cdd5`: sending an announcement is audited.
- `b1d57e7`: the phone does not show a banner over a paper in progress; the notification still lands in the shade.

What is wrong with it

- `apps/mobile/src/lib/use-push-device.ts:3` — Stale: the header says a push "carries a title and a way in and nothing else"; since `ab5d593` it carries a line of body too.
- `.env.example:203` — Stale: "served to the SPA by `GET /me/notification-preferences`"; no such route exists, it is `GET /me/push-subscription` (`apps/api/src/me/me.controller.ts:96`).
- `apps/api/src/notifications/notifications.service.ts:18` and `apps/api/src/notifications/notification-policy.ts:51` — Dead: `actBy` is never set by any producer, so `escalationFor` always waits the full 600 s and the pay-immediately branch is reachable only from a test.
- `apps/api/src/notifications/notification-delivery.processor.ts:41` — Dead: `KIND_OF` lists `RESULT_READY` and `TEST_ASSIGNED`, but only an announcement books a paid channel and it is always `GENERIC`. The comment above it says so; the entries are inert.
- `apps/api/src/queue/queues.ts:140` — Stale: `'relay-sweep'` is the name of the push sweep, a leftover of the removed outbox relay. The comment explains why it cannot be renamed (the live scheduler id), so it stays a trap rather than a bug.
- `apps/api/src/notifications/bell.service.ts:170` — Correctness: an announcement is matched against the student's attributes at read time, not at send time. A student who changes branch or programme later gains or loses old announcements in the bell, while the push went to whoever matched when the job ran. Read from code, not reproduced.

A student's own account

Where it lives

`apps/api/src/me/` and `apps/api/src/saved/`, with the writes landing in `apps/api/src/students/` (the admin-owned module, reused) and sessions in `apps/api/src/auth/session.service.ts`.

File	Owns
`apps/api/src/me/me.controller.ts`	The student-facing routes under `/me`, class-level `@Actors(ActorTypes.STUDENT)`. The only ids in a path are the session or notification being acted on.
`apps/api/src/me/me.service.ts`	Profile read, profile update, document save; all delegate to `StudentsService`.
`apps/api/src/me/documents.ts`	Kind to column map, S3 key shape, byte sniffing.
`apps/api/src/saved/saved-questions.service.ts`	The only writer of `SavedQuestion`.
`apps/api/src/saved/saved.controller.ts`	Five routes under `/me/saved`.
`apps/api/src/students/students.service.ts`	`Student` and `StudentProfile`: `detail`, `update`, `saveDocumentKey`, `enrolmentOf`.
`apps/api/src/students/student-privacy.service.ts`	Erasure.
`packages/contracts/src/me.ts`, `packages/contracts/src/saved.ts`	Schemas and routes shared by both SPAs and the phone.

Both controllers register only under `API_ROLES.CORE` (`apps/api/src/me/me.module.ts:24`, `apps/api/src/saved/saved.module.ts:11`), so an `exam` container serves none of this.

What it is for

Everything a signed-in student may read or change about themselves, with the subject taken from the token. It answers: who am I and what do I stand on (`/me`), what can I sit (`/me/catalog`, which belongs to access, `apps/api/src/me/me.controller.ts:74`), what did I star (`/me/saved`), what has the institute told me (`/me/notifications`, above), and the chores: the profile, a photo and a marksheet, and where I am signed in.

A student has no password and no PIN. The routes that changed a PIN left `/me` with `29f3f88`; the only credential is the code sent to their mobile, and the mobile number is editable by an admin only.

Flow: reading the profile

GET /ME — WHO AM I

1. Bearer token verified and the Redis session checked; a student gets `permissions: {}`
   `apps/api/src/auth/guards/jwt-auth.guard.ts:20`
   → `request.user = { id, actor, sessionId }`
2. `ActorGuard` refuses an admin token
   `apps/api/src/auth/guards/actor.guard.ts:11`
   → `FORBIDDEN` for the wrong actor
3. `MeService.profile` delegates to `StudentsService.detail`
   `apps/api/src/me/me.service.ts:27`
   → one call
4. `student.findFirst` with the profile, event candidacies and mobile history, in parallel with a `student.count` for "holds own access"; a missing row is `NOT_FOUND`
   `apps/api/src/students/students.service.ts:179`
   → the row and a boolean
5. `toProfileView` presigns `photoUrl` and `tenthMarksheetUrl` (stored as S3 keys) for 300 s; `dob` goes through `fromDateColumn`; malformed JSON in the education arrays reads as null
   `apps/api/src/students/students.service.ts:233`
   → two GET URLs, `StudentDetail['profile']`
6. `enrolmentOf` resolves programme, exam and branch codes to names; a code with no catalog row keeps its own code
   `apps/api/src/students/students.service.ts:214`
   → `enrolment`
7. The interceptor wraps it
   `apps/api/src/me/me.controller.ts:68`
   → `{ success: true, data: Me, meta: { requestId } }`

`preTestReady` and `profileCompleted` are derived on every read from the profile columns (`READINESS_FIELDS`), not stored. They were two columns once; four writers applied three different rules (`e66ab91`).

Flow: editing the profile

PATCH /ME — SAVING OWN DETAILS

1. The form saves through `saveOwnDetails`, which sends `expectedUpdatedAt` from the newest record while none of the student's details moved since the edit began, else the stamp the edit began on
   `packages/app-kit/src/student-queries.ts:37`
   → request with a stamp

2. `@Audit(STUDENT_PROFILE, UPDATE)`; body parsed by `ZodBody(updateMeSchema)`, an allowlist of `fullName`, `profile` and `expectedUpdatedAt`, so enrolment, branch, type and the verified flags cannot arrive
   `apps/api/src/me/me.controller.ts:139`
   → `UpdateMeBody`; blank fields become null
3. `MeService.update` calls `StudentsService.update`
   `apps/api/src/me/me.service.ts:41`
   → one call
4. A stamp that differs from the row's `updatedAt` is refused before anything is judged
   `apps/api/src/students/students.service.ts:384`
   → `CONFLICT` "These details changed after the form was opened. Nothing was saved."
5. One short transaction: `updateMany … where updatedAt = <the stamp read>` stamps a new `updatedAt` and claims the row (a second writer gets the same `CONFLICT`), then `student.update` with a profile `upsert`; `dob` through `toDateColumn`
   `apps/api/src/students/students.service.ts:419`
   → `Student.updatedAt` moved, `StudentProfile` upserted
6. The enrolment notification branch inside the transaction cannot fire on this route, since `enrolledExams` is not patchable here
   `apps/api/src/students/students.service.ts:430`
   → none
7. The audit diff is filed from the columns that changed; it names a personal field and withholds the value it held. An unchanged save records nothing
   `apps/api/src/students/students.service.ts:442`
   → audit row
8. `MeService` names the entity (the path has no `:id`) and returns the profile with enrolment
   `apps/api/src/me/me.service.ts:45`
   → `Me`
9. On `CONFLICT` the client re-reads the profile; if none of the student's own fields moved it retries once on the new stamp, otherwise it surfaces the refusal
   `packages/app-kit/src/student-queries.ts:55`
   → saved, or a message naming what moved

A document upload while the form is open used to wipe the fields being typed; the form now follows the record only while nobody is editing it (`3c0f749`).

Flow: uploading a photo or a marksheet

POST /ME/DOCUMENTS/:KIND — PHOTO OR TENTH-MARKSHEET

1. `kind` is in the path and parsed by `ZodParam(documentKindSchema)`; the file field is read by `FileInterceptor` with a 5 MB limit (`DOCUMENT_MAX_BYTES`), also set on the module
   `apps/api/src/me/me.controller.ts:150`
   → `kind`, `file`
2. `checkDocument`: no file, size 0 or over 5 MB, or a type the kind does not accept is `VALIDATION_ERROR` with `fieldErrors.file`. The BYTES decide the type (`sniffImage`, plus a `%PDF-` check for the marksheet), never the browser's mime type
   `apps/api/src/me/documents.ts:54`
   → a sniffed content type
3. `assertExists` runs before the upload, so no object is pushed to S3 for a student who is gone
   `apps/api/src/students/students.service.ts:450`
   → `NOT_FOUND` otherwise
4. The key is `students/<studentId>/<kind>-<Date.now()>.<ext>`; a replacement is a NEW key
   `apps/api/src/me/documents.ts:30`
   → key
5. One `PutObject` through `StorageService`, the single upload path
   `apps/api/src/storage/storage.service.ts:50`
   → object in the bucket
6. `saveDocumentKey` upserts `StudentProfile.photoUrl` or `tenthMarksheetUrl` with the key; a student erased meanwhile is refused by the write itself
   `apps/api/src/students/students.service.ts:464`
   → column set
7. The refreshed `Me` is returned and the client writes it straight into the cache
   `apps/api/src/me/me.service.ts:71`
   → `Me` with fresh presigned URLs

Aadhaar and PAN images are not stored. `DOCUMENT_KINDS` has two members (`packages/contracts/src/me.ts:11`) and `StudentProfile` has `aadhaarVerified` and `panVerified` booleans only.

Flow: starring a question

POST /ME/SAVED/BOOKMARKS — BOOKMARK FROM THE REVIEW SCREEN

1. The star on the review surface posts `{ attemptId, questionId }` and removes by id when one is held
   `packages/app-kit/src/review.ts:46`
   → request
2. Same student guards; body parsed by `ZodBody(bookmarkQuestionSchema)`
   `apps/api/src/saved/saved.controller.ts:48`
   → `BookmarkQuestionBody`
3. The sitting is read with `{ id, studentId }`, so another student's attempt id reads as null, and the paper row is proved by `(testId, questionId)`
   `apps/api/src/saved/saved-questions.service.ts:229`
   → `NOT_FOUND` "You did not sit this question" otherwise

4. A sitting not yet `EVALUATED` is refused
   `apps/api/src/saved/saved-questions.service.ts:241`
   → `CONFLICT` "This sitting has not been marked yet"
5. `savedQuestion.create` with the student, question, attempt and paper row; a unique violation on `(studentId, questionId)` is swallowed, so a second star is the first
   `apps/api/src/saved/saved-questions.service.ts:174`
   → `SavedQuestion` row
6. The row is re-read with subject, topic and stem so a duplicate star returns the row already held; options and the answer key are absent by design
   `apps/api/src/saved/saved-questions.service.ts:186`
   → `SavedQuestion` without a test or time spent on this path

`GET /me/saved` pages the student's rows newest first (`createdAt desc, id desc`), narrowed by subject or test, then adds each row's test title and time spent from the sitting (`apps/api/src/saved/saved-questions.service.ts:53`, `:91`). `GET /me/saved/facets` returns the filter options across all pages. `DELETE /me/saved/:id` deletes with the owner in the `where`, so another student's row reads as missing (`:194`). `attemptId` and `paperQuestionId` are nullable foreign keys with `SetNull`: a voided or removed sitting leaves the saved question standing but drops its provenance. A saved question whose sitting was voided now opens as "This sitting is no longer available" rather than a failed load with a Retry that cannot succeed (`95fcd4b`).

Flow: where am I signed in

GET /ME/SESSIONS AND DELETE /ME/SESSIONS/:ID — DEVICES

1. A session is a Redis key `session:student:<id>:<sessionId>` with the refresh lifetime as TTL, plus a set index of ids
   `apps/api/src/auth/session.service.ts:42`
   → created at sign-in
2. A student has at most one session per app kind (web, phone): a new sign-in of a kind replaces the last, writing a "replaced by" marker first so the old device learns why
   `apps/api/src/auth/session.service.ts:199`
   → the old session revoked
3. `GET /me/sessions` lists them newest activity first with the asking device marked `current`
   `apps/api/src/me/me.controller.ts:164`
   → `DeviceSession[]`
4. `DELETE /me/sessions/:id` refuses the asking device ("use Sign out") and an id that is not the student's, then revokes the other
   `apps/api/src/auth/session.service.ts:173`
   → session key deleted and removed from the index; answers 200, not 204

A sitting held on a revoked session does not move with it: the held sitting records the sign-in its tab belongs to, and another sign-in is refused until the holder has been silent for 60 s (`19f35d0`). That belongs to the sitting chapter.

Flow: erasure

POST /ADMIN/STUDENTS/:ID/ERASURE — RIGHT TO ERASURE, ACTIONED BY A SUPER ADMIN

1. The route is under `@RequiresSuperAdmin()` and audited; there is no student-facing erasure route
   `apps/api/src/students/students.controller.ts:165`
   → `ErasureReceipt`
2. The live student is read with their two document keys; absent is `NOT_FOUND`, already anonymised is `CONFLICT`
   `apps/api/src/students/student-privacy.service.ts:28`
   → `photoUrl`, `tenthMarksheetUrl`
3. The files are removed from S3 BEFORE the row changes: an erasure that answered success while the files stayed is the outcome worth refusing, and S3's delete is idempotent so a retry is clean
   `apps/api/src/students/student-privacy.service.ts:40`
   → objects deleted
4. One transaction: `student.updateMany where anonymizedAt null` writes the tombstone (`mobile 0000000000`, name and external ref null, `isActive false`, `deletedAt`, `anonymizedAt`); the profile is emptied; mobile history is deleted
   `apps/api/src/students/student-privacy.service.ts:43`
   → anonymised rows. Of two erasures racing, one writes and one gets `CONFLICT`
5. `STUDENT_DEACTIVATED` is emitted and the sessions listener revokes every session
   `apps/api/src/students/student-privacy.service.ts:57`
   → sessions gone; push targets die on the next sweep
6. The receipt counts the sittings that were kept: marks are facts about a paper and a cohort's mean must not move
   `apps/api/src/students/student-privacy.service.ts:59`
   → `{ studentId, anonymizedAt, attemptsKept }`

The data it touches

Column	Holds	Written by
`Student.fullName`	the name	`PATCH /me`, admin, import
`Student.programs`, `enrolledExams`, `enrolledCourses`, `currentBranchId`, `isTestBlocked`, `isActive`, `deletedAt`	reach facts read by the catalog	admin and import only, never `/me`
`StudentProfile.motherName`, `fatherName`, `email`, `address`, `gender`	contact and parentage	`PATCH /me`
`StudentProfile.dob`	a civil date at UTC midnight (`@db.Date`)	`PATCH /me` through `toDateColumn`

Column	Holds	Written by
`StudentProfile.photoUrl`, `tenthMarksheetUrl`	an S3 key, not a URL; presigned for 300 s on read	`POST /me/documents/:kind`
`StudentProfile.educationDetails`, `pastExamHistory`	JSON arrays, parsed on read	`PATCH /me`
`StudentProfile.aadhaarVerified`, `panVerified`	verification booleans; images are never stored	nothing sets them true
`SavedQuestion`	`studentId`, `questionId` (unique together), `attemptId?`, `paperQuestionId?`	`bookmark`

Redis: session keys as above, read by `GET /me/sessions` and written only at sign-in, refresh and sign-out.

Why it is shaped this way

- **The token is the subject.** No `/me` route takes a student id, so a student cannot address another's record. Writes to `Student` go through the module that owns the table.
- **A stamp on the save.** `expectedUpdatedAt` and the claiming `updateMany` mean a form opened before an admin changed the record is refused, not merged silently (`f18d01a`). The audit log withholds values.
- **Documents hold keys, presigned on read.** A stored public URL would make the bucket public and give an identity document a permanent link. The cost is that `GET /me` does two local presigns and a replaced file is not deleted.
- **Readiness is derived**, not stored; the cost is a profile join on every identity read.
- **Erasure anonymises, never deletes**, so scored aggregates keep their meaning.

Which screen calls it

Route	Caller	Screen
`GET /me`, `PATCH /me`	`apps/exams/src/features/account/profile.tsx:98`, `apps/mobile/app/profile.tsx:85`, the app shell on every screen	Profile
`POST /me/documents/:kind`	`apps/exams/src/features/account/document-card.tsx`, `apps/mobile/src/components/account/document-card.tsx`	Profile, both kinds
`GET /me/sessions`, `DELETE /me/sessions/:id`	`apps/exams/src/features/account/active-devices.tsx:23`, `apps/mobile/src/components/account/active-devices.tsx`	Account, devices
`GET /me/catalog`	`packages/app-kit/src/student-queries.ts`	tests, series, dashboard
`GET /me/saved`, `/saved/facets`, `POST /me/saved/bookmarks`, `GET /me/saved/bookmarks/attempts/:id`, `DELETE /me/saved/:id`	`apps/exams/src/features/review/saved.tsx`, `packages/app-kit/src/review.ts`	Saved, and the report's solutions tab

Is it optimal

Read, not measured.

- `GET /me`: about five round trips (`findFirst`, `count`, then three small catalog reads, skipped when the arrays are empty) and two local presigns. No N+1.
- `PATCH /me`: `StudentsService.update` ends with `detail(id)`, so the save pays the read of `GET /me` once, plus the stamp, update and audit. There is no second read before it.
- `GET /me/saved`: three round trips. `satContext` loads every `PaperQuestion` of every test the page's sittings belong to, then scans them per sitting inside `attempts.map` (`apps/api/src/saved/saved-questions.service.ts:113`). Correct, and the heaviest read in this area.
- `GET /me/saved/facets`: unpaged over the student's whole saved set, bounded only by how much one student stars.
- `GET /me/sessions`: Redis only.

What changed recently

- `29f3f88`: a student signs in with a code every time and has no PIN. The PIN routes are gone from `/me`.
- `f18d01a`: a details save from a form read before another write is refused and logs no values.
- `c97da47`: erasure removes the photo and marksheet from S3; the uncalled `GET /me/data-export` is deleted.
- `3c0f749`: an upload mid-edit no longer wipes the fields being edited.
- `0d2f12e`: the web and phone save a profile one way (`saveOwnDetails`).
- `19f35d0`: a sitting stays with the sign-in answering it.
- `95fcd4b`: a saved question from a voided sitting says so.

What is wrong with it

- `apps/api/src/me/me.service.ts:62` and `apps/api/src/students/student-privacy.service.ts:64` — Privacy: a replacement upload gets a new key and the old object is never deleted, but erasure removes only the two keys currently in the profile. Earlier photos and marksheets stay in S3 under `students/<studentId>/…` after an erasure. Read from code; not reproduced against a bucket.
- `apps/api/src/me/me.service.ts:57` — Dead: `if (!file)` cannot be reached, because `checkDocument` on the line above already throws for a missing file; it exists only to narrow the type.
- `apps/api/src/me/me.controller.ts:148` — Stale: the comment says "The student's photo." over a route that also takes the marksheet.

- `apps/exams/src/features/account/profile.tsx:326` — Correctness: Aadhaar and PAN show "Verified" or "Not verified" from `aadhaarVerified` and `panVerified`, but nothing outside tests and erasure ever writes them, so every student reads "Not verified" with no route to change it.
- `apps/api/src/saved/saved-questions.service.ts:113` — Correctness: the per-sitting rescan of all paper rows is quadratic in tests times questions for a saved page; right result, avoidable cost.

Part VI — The data model

Every table, and what the database refuses

`prisma/schema.prisma` is the target of record, and the hand-written SQL under `prisma/migrations` is the record of everything Prisma cannot say. Both are read here directly. The part that deserves the reader's attention is the last one: the triggers, functions, view, partial uniques and CHECKs. A rule that lives in a service method holds only for the paths that call it. A rule that lives in a trigger holds for every path, including the one somebody writes next year and the one somebody types into psql at midnight.

Facts marked *(catalog)* were read from the running dev Postgres (`iace-postgres` , all 128 migrations applied) with read-only `pg_catalog` queries; the migration file is cited for the wording. Nothing was measured by this chapter: every figure quoted from a migration is that migration's own measurement.

Where it lives

- `prisma/schema.prisma` (1380 lines): 45 models and 36 enums. Banner comments divide it into taxonomy (`:290`), base configs, the question bank, tests and papers, attempts, people, access, audit, notifications and rollups.
- `prisma/migrations/**` : 128 revisions. The baseline is `20260820000000_schema_realignment` ; the head is `20261015090000_a_broadcast_is_resolved_not_copied` . 23 revisions landed after the previous edition of this chapter (head then `20260930170000_the_outbox_and_its_relay_are_gone`).
- The dev database holds 46 tables: the 45 models plus Prisma's `_prisma_migrations` *(catalog)*.

What it is for

It holds an institute's exam catalog, a question bank with an authoring workflow, the papers built from both, and the sittings taken against them. Two guarantees are pushed into Postgres instead of a service. A blueprint (`BaseConfig`) stops moving once locked. A paper stops moving once anybody has sat it. The second exists because an answer sheet stores *positions*: a position into the paper's row order and a position into each row's option list. A paper that shifted would silently re-interpret every stored answer.

Everything on the live-test path is shaped for page count. One row per sitting instead of one per answer, packed arrays instead of keyed objects, 16-byte ids instead of 26-byte ids.

UPDATE "QUESTIONVERSION" SET OPTIONS = ... — AN ADMIN REWRITES A VERSION THAT UNOPENED PAPERS PIN (THE ONLY FLOW THAT LIVES ENTIRELY IN SQL)

1. The UPDATE starts. `question_version_sat_guard` is a BEFORE UPDATE trigger gated by `WHEN (options, answerKey or content IS DISTINCT)` , so an edit to anything else never enters it
   `prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:417`
   → trigger armed, nothing written
2. The guard locks every Test that pins this version, `ORDER BY "id" FOR UPDATE` , then refuses if any of them has an Attempt. The fixed lock order is what stops two version edits deadlocking
   `prisma/migrations/20260918160000_the_version_guard_locks_its_tests_in_one_order/migration.sql:20`
   → exception "A question version on a paper somebody has sat cannot change its options, key or content" (SQLSTATE 23000 class), or the UPDATE proceeds
3. The row is written. `question_version_option_ids_fanout` fires AFTER UPDATE OF options, `WHEN options IS DISTINCT`
   `prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql:25`
   → one statement queued
4. The fan-out re-assigns `"optionIds" = "optionIds"` on every pinning `PaperQuestion` , only to re-fire that table's BEFORE triggers
   `prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql:25`
   → N pinning rows touched
5. `paper_question_option_ids` (BEFORE INSERT OR UPDATE OF "questionVersionId", "optionIds") rebuilds `NEW."optionIds"` from the version's JSON options, in stored order
   `prisma/migrations/20260917100000_a_sitting_keeps_its_answers_on_one_sheet/migration.sql:44`
   → `PaperQuestion.optionIds` matches the new option order
6. `paper_question_sat_guard` fires next. The migration comment says BEFORE triggers run alphabetically, so `o` precedes `s` . It sees a changed row, locks the Test row `FOR UPDATE` , finds no Attempt and lets it through
   `prisma/migrations/20260917100000_a_sitting_keeps_its_answers_on_one_sheet/migration.sql:123`
   → row committed
7. Durable result: every pinning paper's `optionIds` matches the version, so no stored position decodes to the wrong option
   `prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql:17`
   → `PaperQuestion.optionIds` rewritten

> **Watch** The fan-out leans on trigger *name* order. The migration (`prisma/migrations/20260918140000_an_edited_version_resyncs_every_paper_that_pins_it/migration.sql:17`) warns that renaming either `paper_question_option_ids` or `paper_question_sat_guard` reverses it, and the sat guard then sees an unchanged row and takes its fast path without checking anything.

1. Model inventory, by owner

Thirty-three models use `String @id @default(uuid(7)) @db.Uuid` . Nine have a composite `@@id` (`AdminFeaturePermission` , `StudentGrant` , `EventCandidate` , `TestProgramUnlock` , `ImportRowEdit` , `BroadcastRead` , `StudentSubjectStat` , `TestSectionStat` , `TestQuestionStat`). Three borrow a parent's key as `@id` (`AttemptSheet.attemptId` , `StudentStat.studentId` , `TestStat.testId`). UUID v7 was chosen over v4 because its leading timestamp keeps inserts at the index's right edge: the id migration measured 90% leaf density and 5.11 MB WAL per 20,000 inserts, against 68% and 18.93 MB for v4 (`prisma/migrations/20260918120000_every_id_is_a_uuid/migration.sql:14`).

Owner	Model	Holds
Catalog	`Exam` (`:299`)	An exam under an `ExamCourse`; `code` is what `Student.enrolledExams` stores, so a code is never reused
Catalog	`ExamStage` (`:316`)	The stage/paper level that carries configs, series and tests
Configs	`BaseConfig` (`:343`)	A stage's reusable blueprint; locks at first finalize
Configs	`BaseConfigModule` (`:388`)	Session layer above sections (runtime engine deferred)
Configs	`BaseConfigSection` (`:402`)	A section: counts, marks, timing, merit/qualifying, `typingPreset`, `descriptive`, own `languages`
Bank	`Subject` (`:444`)	Taxonomy; optional `language` makes the subject itself a language
Bank	`Topic` (`:457`)	A pool within a subject, not a scope
Bank	`Question` (`:468`)	Stable identity, tags, `stemHash` (unique), trigger-maintained `searchText`
Bank	`QuestionVersion` (`:518`)	Content, options and answer key per edit
Papers	`Test` (`:541`)	A paper's identity: scope, source, version, `paperRevision`, `opensAt`, language-waiver fields
Papers	`PaperQuestion` (`:601`)	One question on one paper with marks and `optionIds`
Authoring	`QuestionAssignment` (`:634`)	Who types or proof-reads one section of one test
Authoring	`QuestionReview` (`:664`)	A proof-reader's per-question verdict
Authoring	`QuestionWorkTime` (`:690`)	Seconds one admin spent on one question, per seat; a running total
Authoring	`SectionComment` (`:712`)	The thread on one section, with edit revisions
Sittings	`Attempt` (`:735`)	Live state and scored result in one row
Sittings	`AttemptSheet` (`:783`)	One sitting's answers and verdicts as two arrays
People	`Student` (`:800`)	Identity, enrolment arrays, soft delete, `anonymizedAt`; no credential column
People	`StudentMobileHistory` (`:857`)	A number a student moved off; finds them in the roster, never signs anyone in
People	`StudentProfile` (`:872`)	PII and verification booleans (no Aadhaar/PAN images)
People	`Admin` (`:893`)	Identity plus `AdminRole`
People	`AdminFeaturePermission` (`:916`)	One feature key at one level per admin
Access	`Branch` (`:934`), `Program` (`:946`), `Event` (`:1001`)	Vocabularies
Access	`TestSeries` (`:960`)	The unit of offering; `kind` decides reach
Access	`StudentGrant` (`:987`)	Per-student override of series access
Access	`EventCandidate` (`:1013`)	Who an EVENT series reaches
Access	`TestProgramUnlock` (`:1027`)	Per-program stagger on one test's opening
Audit	`ImportLog` (`:1044`)	One import run: counts, target, `remaps`
Audit	`ImportRowEdit` (`:1071`)	A previewed row corrected or left out before commit
Audit	`RowActionLog` (`:1084`)	Central CRUD audit; actor ids carry no FK
Notifications	`Notification` (`:1107`)	One in-app message to exactly one recipient
Notifications	`BroadcastRead` (`:1149`)	A student's read of a broadcast the bell resolves
Notifications	`Announcement` (`:1163`)	One send, with audience filter and priced cost
Notifications	`NotificationDelivery` (`:1185`)	One row per channel that can fail
Notifications	`PushSubscription` (`:1294`), `PushDevice` (`:1319`)	A browser endpoint with its keys; a phone's FCM token
Rollups	`StudentStat` (`:1213`), `StudentSubjectStat` (`:1231`)	Per-student sums, per-subject per-scope buckets
Rollups	`TestStat` (`:1247`), `TestSectionStat` (`:1257`), `TestQuestionStat` (`:1273`)	Cohort count and time sum; per-section; item analysis
Student tools	`SavedQuestion` (`:1337`)	Bookmarks and mistakes keyed by bank `questionId`
Student tools	`TypingPractice` (`:1358`)	One finished typing-practice session; the typed text is not kept

Four models are new since the previous edition: `QuestionWorkTime`, `StudentMobileHistory`, `BroadcastRead` and `TypingPractice`. `OutboxEvent` is gone (dropped by `20260930170000`).

2. Enums (36)

Enum	Values and notes
`Gender`, `DifficultyLevel`, `SupportedLanguage`	MALE/FEMALE/OTHER; LOW/MEDIUM/HIGH; EN/HI/TE (lowercase inside content JSON)
`QuestionType`	SINGLE_MCQ, TEXT_FIELD, TYPING (a passage to type: no options, no key), DESCRIPTIVE (free writing marked by rule against key points)
`AnswerMode`	EXACT, NUMERIC. No column uses it; the mode lives in `answerKey` JSON (`prisma/schema.prisma:50`)
`QuestionStatus`	ACTIVE, ARCHIVED
`TestUi`	CBT, OMR, GENERIC, TYPING (reserved; anything not OMR renders the CBT screen)
`NavigationPolicy`, `LanguageMode`	FREE/FORWARD_ONLY; SINGLE/DUAL
`ExamTemplate`	DEFAULT, RAILWAYS (renamed from SSC_RAILWAYS), BANKING, SSC. Skin only; a locked config and a test freeze their copy
`TimerTemplate`	COMPOSITE_FREE, SECTIONAL_LOCKED, SESSION_MODULE_LOCKED, PER_ITEM_TIMED
`ExamCourse`, `ExamMode`	SSC/RRB/BANKING/AP_TS_POLICE; CBT/OMR/PSYCHOMETRIC/SKILL/PHYSICAL/INTERVIEW/DESCRIPTIVE
`MeritType`, `StageDisposition`	MERIT/QUALIFYING; CONDUCTED/PARTIAL/CATALOG_ONLY
`TestStatus`, `TestScope`, `PaperSource`	DRAFT/ACTIVE/INACTIVE; FULL/MODULE/SECTIONAL; FRAMED (typed) / PICKED (drawn)
`PaperQuestionStatus`	ACTIVE, DROPPED, BONUS
`AttemptStatus`	IN_PROGRESS, SUBMITTED, EVALUATED, EXPIRED (reserved, never written), VOIDED
`AnswerState`	NOT_VISITED, NOT_ANSWERED, ANSWERED, MARKED_REVIEW, ANSWERED_MARKED
`NotificationType`	TEST_ASSIGNED, RESULT_READY, ENROLLMENT_ADDED, GRANT_ADDED, GENERIC, RESULT_UPDATED, PIN_CHANGED, WELCOME. Seven are written; PIN_CHANGED has no producer
`DeliveryChannel`	IN_APP, WEB_PUSH, MOBILE_PUSH, EMAIL, SMS, WHATSAPP
`DeliveryStatus`	PENDING, SENT, DELIVERED, FAILED, SKIPPED (a decision not to spend)
`StudentType`, `BranchType`, `ImportSource`	ONLINE/OFFLINE/NON_IACE; PHYSICAL/VIRTUAL; INDIVIDUAL/SHEET/SCRIPT/SELF_SIGNUP
`TestSeriesKind`	STANDARD, FREE, PROGRAM, EVENT
`PermissionLevel`	READ, WRITE (WRITE implies READ)
`AuditFeature`	18 values naming a changeable record kind, REPORT among them
`AuditAction`, `ActorType`	CREATE/UPDATE/DELETE/ACTIVATE/DEACTIVATE/BLOCK/UNBLOCK/IMPORT/EXPORT; ADMIN/STUDENT/SCRIPT/SYSTEM
`AssignmentRole`, `SendBackReason`, `AdminRole`	TYPIST/PROOFREADER; SPELLING/DATA_CORRECTION/ANSWER_OPTION; TYPIST/PROOFREADER/ADMIN/SUPER_ADMIN
`DevicePlatform`	ANDROID, IOS (declared mid-file, `:1311`)

3. Every jsonb column

Nineteen jsonb or jsonb-array columns *(catalog)*. The ones whose shape the database cannot state:

- `AttemptSheet.answers`: NOT NULL, CHECK `jsonb_typeof = 'array'`. Slot N is paper row N in `PaperQuestion.order`, whatever order the screen showed. A slot is `null` for a question never touched, otherwise a 5- or 6-element array (`apps/api/src/attempts/answer-sheet.ts:32`): `[state index into SLOT_STATES, option, timeSpentSec, firstActionSec, answeredSec, typedAnswer?]`. `option` is an integer position into `PaperQuestion.optionIds`, or the raw id string when the row does not hold it (which scores wrong), or `null`. Seeded by `blankSheet` in the start transaction (`apps/api/src/attempts/attempt-sheet.service.ts:27`) and written whole by `AttemptSheetService.write` (`apps/api/src/attempts/attempt-sheet.service.ts:31`).
- `AttemptSheet.verdicts`: a parallel array of `[isCorrect: boolean|null, marksAwarded: number]` (`apps/api/src/attempts/answer-sheet.ts:46`), written only by the scorer in the same statement that claims the sitting (`apps/api/src/attempts/scoring.processor.ts:175`).
- `Attempt.sectionScores`: packed arrays `[baseConfigSectionId, score, correct, wrong, unattempted, timeSpentSec]`. `packedSections` writes (`apps/api/src/attempts/score-paper.ts:52`) and `sectionScoresIn` reads (`apps/api/src/attempts/score-paper.ts:64`); a row still in the old object shape reads as no sections, never half-decoded.
- `Attempt.sectionState`: `{ [sectionId]: { remainingSec, closed, openedAt } }`, the durable copy of clocks Redis holds hot. Written with the sheet (`apps/api/src/attempts/attempt-sheet.service.ts:52`) and read back on resume (`apps/api/src/attempts/attempt-state.service.ts:416`).
- `QuestionVersion.content`: `{ <lang>: { stem, passage?, solution? } }`, each a `RichContent` array of text nodes holding escaped HTML (`packages/contracts/src/questions.ts:192`). `passage` is what a question is asked *about*.
- `QuestionVersion.options`: `[{ id, position, isCorrect, text: { <lang>: RichContent } }]` (`packages/contracts/src/questions.ts:525`).
- `QuestionVersion.answerKey`: `{ mode, answers: { <lang>: string }, tolerance?, rubric? }` (`packages/contracts/src/questions.ts:533`). `rubric` (word limits, format) is what a DESCRIPTIVE question is marked against.
- `Test.scopeRef`: `{ moduleId?, sectionId? }`, read through `scopeRefOf` (`apps/api/src/common/prisma-json.ts:32`).

- `Test.questionPoolFilter`: the draw spec (`packages/contracts/src/tests.ts:344`).
- `SectionComment.revisions` (`Json[]`): `{ body, at }` per earlier wording, oldest first, the current one never included (`packages/contracts/src/assignments.ts:305`).
- `StudentProfile.educationDetails`, `.pastExamHistory`: arrays of entries (`packages/contracts/src/students.ts:190`).
- `RowActionLog.changed`: `Record<field, { from, to }>`.
- `ImportLog.errors`: `{ fileErrors?, message? }` (`apps/api/src/imports/imports.service.ts:519`). `ImportLog.remaps` (new): `{ subject, topic, difficulty }` maps the preview chose for sheet values the bank lacks; held on the run, never on the sheet or the bank.
- `ImportRowEdit.draft`: a whole question draft; null when the row was only left out.
- `Notification.data`: template variables; the broadcast writer puts `{ message }` (`apps/api/src/notifications/announcements.service.ts:157`).
- `Announcement.audience`: the student-list filter minus paging and sort (`packages/contracts/src/announcements.ts:35`).
- `TestQuestionStat.optionCounts`: `Record<optionId, count>`.

4. What Prisma cannot express

All of this is hand-written in migrations. Graft cannot see it; the counts below come from the migration SQL, accounting for later DROPs, and agree with the catalog.

Triggers (14)

The id migration dropped and recreated every trigger and the view around the type change; the definitions in `20260918120000_every_id_is_a_uuid/migration.sql:407–418` are the live ones.

Trigger	Table, event	Does
`base_config_guard`	`BaseConfig`, BEFORE DELETE OR UPDATE	Refuses deleting a locked config, and any change to its 18 shape columns: "base config % is locked; clone it to change its shape". `name`, `isDefault`, `isActive` stay editable
`base_config_section_guard`, `base_config_module_guard`	children, BEFORE INSERT OR DELETE OR UPDATE	Run `base_config_child_guard`, which calls `base_config_assert_unlocked` on both the old and new parent, lowest id first, so two re-parenting transactions queue instead of deadlocking
`base_config_shape_guard`	`BaseConfig`, deferred constraint trigger, AFTER UPDATE OF `timerTemplate`	Re-checks every section against the new template
`base_config_section_shape_guard`	`BaseConfigSection`, deferred constraint trigger, AFTER INSERT OR UPDATE	`moduleId` only under SESSION_MODULE_LOCKED and required there; `durationSec` required for SECTIONAL_LOCKED; `perQuestionSec` for PER_ITEM_TIMED
`base_config_truncate_guard` ×3	each of the three config tables, BEFORE TRUNCATE	Raises unconditionally, because TRUNCATE fires no row trigger
`paper_question_sat_guard`	`PaperQuestion`, BEFORE INSERT OR DELETE OR UPDATE	Status-only updates take a fast path (`to_jsonb(NEW) - 'status'` equals OLD's), so a drop or bonus never blocks a start. Anything else takes `FOR UPDATE` on the Test row and refuses if an Attempt exists: "A paper somebody has sat cannot change: only a question's status may move"
`question_version_sat_guard`	`QuestionVersion`, BEFORE UPDATE, WHEN options/answerKey/content distinct	The same refusal for a pinned version
`paper_question_option_ids`	`PaperQuestion`, BEFORE INSERT OR UPDATE OF `questionVersionId`, `optionIds`	Copies the version's option ids in stored order onto the paper row
`question_version_option_ids_fanout`	`QuestionVersion`, AFTER UPDATE OF `options`	Re-fires the row above on every pinning paper
`question_search_text`	`Question`, BEFORE INSERT OR UPDATE OF `currentVersionId`, `questionCode`, `tags`, `searchText`	Recomputes `searchText` from the current version's content, the code and the tags. `searchText` is in its own list on purpose: any hand-write is recomputed, and `SET "searchText" = "searchText"` is the backfill idiom
`question_version_search_text_fanout`	`QuestionVersion`, AFTER UPDATE OF `content`, WHEN content distinct	Re-touches `Question.searchText` where this version is current

`apps/api/src/configs/base-configs.service.ts` is the code path for locking; without these triggers, raw SQL, a worker or a psql session could rewrite a locked paper shape. `base_config_guard` was last replaced by `20261001110000_the_schema_drops_what_nothing_writes/migration.sql:34` (minus the dropped `featureFlags`), after `20261001100000_a_locked_config_freezes_its_skin` added `examTemplate`.

SQL functions (16, written by the project)

- Trigger bodies (11): `base_config_guard`, `base_config_child_guard`, `base_config_truncate_guard`, `base_config_shape_guard`, `base_config_section_shape_guard`, `paper_question_sat_guard`, `question_version_sat_guard`, `paper_question_option_ids`, `question_version_option_ids_fanout`, `question_search_text`, `question_version_search_text_fanout`.
- Helpers called by those bodies: `base_config_assert_unlocked(uuid)` and `base_config_assert_section_shape(uuid, "TimerTemplate", uuid, int, int)`.
- `sitting_percentile(outscored bigint, tied bigint, cohort bigint) RETURNS numeric`, IMMUTABLE PARALLEL SAFE (*catalog*: `i`, `s`). 100 for a cohort of 1 or fewer, else `ROUND(LEAST((outscored + tied/2) / cohort, 1) * 100, 2)` with every input clamped. Created by

`20260911190000_rank_is_counted_in_postgres/migration.sql:20`, made parallel-safe by `20260912010000_percentile_is_parallel_safe/migration.sql:5`. Read at `apps/api/src/attempts/ranking-sql.ts:73`, `:167` and `:207`.

- `"tagsText"(text[])`: `array_to_string($1, ',')`, IMMUTABLE PARALLEL SAFE STRICT (`prisma/migrations/20260925120000_the_bank_is_searched_through_an_index/migration.sql:33`). It joins on a comma because no tag can hold one.
- `"questionSearchText"(jsonb, text, text[])`: the code, `tagsText(tags)`, and the `text` of every node of every field of every language (`jsonpath $.*.*[*].text`), markup stripped and `< > &` decoded. Created by `20261001140000_the_bank_is_searched_on_one_column/migration.sql:63`; the body was replaced by `20261008120000_the_bank_search_reads_the_words_not_the_json_around_them/migration.sql:59` because the first version serialised the JSON's own keys (`stem`, `text`, `type`) into the searchable string, so a search for those words matched every question.

The three outbox helpers (`notification_request_written`, `notification_from_request`, `uuid_or_null`) went with the table in `20260930170000`. The only extension is `pg_trgm`, created by `20260925120000_the_bank_is_searched_through_an_index/migration.sql:31` and not declared in `schema.prisma`; the migrating role must be able to create it. The server holds exactly `plpgsql` and `pg_trgm` *(catalog)*.

View (1)

`"AttemptSheetAnswer"` decodes one sheet slot per row: state name, `selectedOptionId` (position into the paper's `optionIds`, or the raw string), `typedAnswer`, `timeSpentSec`, `firstActionAt` and `answeredAt` as `startedAt + make_interval(...)`, and `isCorrect` and `marksAwarded` from `verdicts`. The migration labels it support and ad-hoc SQL. No application code reads it: the references are `docs/02-domain-rules.md:305` and the doc-check test. It was dropped and recreated by `20260918120000_every_id_is_a_uuid/migration.sql:418`, which is why its name is pinned.

Partial uniques (8)

Index	Rule
`Attempt_graded_per_test_key` (testId, studentId) WHERE isGraded	One ranked sitting per student per test
`BaseConfig_default_per_stage_key` (examStageId) WHERE isDefault	One default per stage
`BaseConfigSection_flat_order_key` (baseConfigId, order) WHERE moduleId IS NULL	Orders module-less sections; the Prisma `@@unique` cannot, because NULLs are distinct
`Student_mobile_live_key`, `Student_externalRef_live_key` WHERE deletedAt IS NULL	A soft-deleted row releases its number
`QuestionAssignment_active_role_key` (testId, baseConfigSectionId, role) WHERE replacedAt IS NULL	One active typist and one active proof-reader per section
`Notification_student_dedupe_key`, `Notification_admin_dedupe_key` (recipient, dedupeKey) WHERE both NOT NULL	What makes a replayed `tell()` land once

Two expression uniques: `TestSeries_name_key` on `lower(name)` and `Test_testSeriesId_title_key` on `(testSeriesId, lower(title))` (`prisma/migrations/20260912170000_a_name_is_taken_once/migration.sql:65-67`). Untitled drafts stay unconstrained because NULLs are distinct. `Question_stemHash_key` is a plain unique since `20261001120000_a_stem_is_unique_and_a_section_is_indexed`: the duplicate pre-check reads outside the write transaction, so two commits could race past it.

CHECK constraints (8) *(catalog)*

Table	Check	Says
`AttemptSheet`	`answers_is_array`	A non-array would make every positional read silently empty
`Notification`	`one_recipient_check`	`(studentId IS NOT NULL)::int + (adminId IS NOT NULL)::int = 1`
`QuestionWorkTime`	`seconds_check`	`seconds >= 0`; a total only grows
`Test`	`language_waiver_whole`	`languageWaivedById` and `languageWaivedAt` are both null or both set
`TestSeries`	`branches_are_standard_only`	Branches only on a STANDARD series
`TestSeries`	`event_kind_names_its_event`	`(kind = EVENT) = (eventId IS NOT NULL)`
`TestSeries`	`program_kind_names_its_program`	`(kind = PROGRAM) = (programCode IS NOT NULL)`
`TestSeries`	`only_free_spans_no_stage`	Only FREE may have no `examStageId`

The four on `TestSeries` are what `access-resolver.service.ts` assumes instead of re-checking.

Composite foreign keys (8, ON UPDATE NoAction) *(catalog)*

`Test → BaseConfig(id, examStageId)`, `PaperQuestion → Test(id, baseConfigId)`, `PaperQuestion → BaseConfigSection(baseConfigId, id)`, `PaperQuestion → QuestionVersion(questionId, id)`, `Question → QuestionVersion(questionId, id)`, `BaseConfigSection → BaseConfigModule(baseConfigId, id)`, `QuestionAssignment → Test(id, baseConfigId)`, `QuestionAssignment → BaseConfigSection(baseConfigId, id)`. Each stops a child pointing at a parent that belongs to a different config, stage or question, which a single-column FK cannot. The composite uniques that back them (`Test_id_baseConfigId_key`, `BaseConfig_id_examStageId_key`, `BaseConfigSection_baseConfigId_id_key`, `BaseConfigModule_baseConfigId_id_key`, `QuestionVersion_questionId_id_key`) must not be pruned as redundant. The database holds 74 foreign keys in all.

GIN indexes (9)

Array containment: `Question_tags_idx`, `Student_enrolledCourses_idx`, `Student_enrolledExams_idx`, `Student_programs_idx`, `TestSeries_branchIds_idx`. Trigram, for leading-wildcard ILIKE: `Question_searchText_trgm_idx`, `Student_fullName_trgm_idx`, `Student_mobile_trgm_idx`, `Test_title_trgm_idx`. The three bank trigram indexes (on `questionCode`, `tagsText(tags)` and `content::text`) were

dropped in `20261001140000` with the UNION that was their only reader. The new one is on the bare column because Prisma 6.19 emits `"searchText" ILIKE $1`, not `lower(col) LIKE`.

Storage parameters *(catalog)*

- `TestStat`, `TestSectionStat`, `TestQuestionStat`: fillfactor 60, autovacuum threshold 50, scale factor 0, cost delay 0 (`20260908090000_the_rows_a_sitting_rewrites_leave_room_in_the_page`), for heap-only updates of unindexed counters.
- `AttemptSheet`: fillfactor 70 (`prisma/migrations/20260918100000_the_live_sittings_are_indexed_apart_from_the_finished_ones/migration.sql:54`). A sheet is rewritten about 25 times and then never again.
- `Attempt`: vacuum threshold 5000 and scale factor 0, insert threshold 5000. Scoring a 5K exam leaves about 15,000 dead rows, and until vacuum runs a standing read fetches thousands of pages.

5. Index inventory

The database holds 173 indexes: 45 primary keys and 128 others. 17 are partial.

- **Ranking.** `Attempt_ranking_idx (testId, score DESC, timeTakenSec, id) INCLUDE (studentId, submittedAt) WHERE isGraded AND status = 'EVALUATED' AND score IS NOT NULL`. Rebuilt with the INCLUDE by `20260929090000_the_bank_boards_and_ledger_read_through_indexes/migration.sql:31` because every board read those two columns and went back to the heap per ranked row. It serves rank, percentile, standings and the topper (`apps/api/src/attempts/ranking-sql.ts`). `Attempt_testId_score_idx` is gone as redundant beside it.
- **Other partial Attempt indexes**, each named for its sweep: `Attempt_live_by_test_idx` (testId, endsAt WHERE IN_PROGRESS) for live-ops counts; `Attempt_expiring_idx` (endsAt WHERE IN_PROGRESS) for the expiry sweeper; `Attempt_unscored_idx` (submittedAt WHERE SUBMITTED AND score IS NULL) for the unscored backlog; `Attempt_rescore_idx` (testId, scoredRevision, id WHERE EVALUATED) for the re-score sweep; `Attempt_student_drift_idx` (studentId, updatedAt WHERE status IN (EVALUATED, VOIDED) AND updatedAt > evaluatedAt) for per-student drift, narrowed to moved sittings by `20260928203000`.
- **Plain Attempt indexes**: `(testId, status)` for cohort reads (kept because the ranking index covers only graded scored rows); `(testId, updatedAt)` for the cohort sweep watermark (`apps/api/src/attempts/rollup.service.ts:156`); `(studentId, createdAt)` for a student's list; `(submittedAt)` for a report's period, added by `20261006190000_a_report_reads_sittings_by_the_day_they_were_handed_in`.
- **Notifications.** `Notification_unpushed_idx (createdAt) WHERE pushedAt IS NULL` is the push sweep's claim. `Notification.nextPushAt` (`20261001130000`) deliberately has no index: deferred rows are a handful. `NotificationDelivery_pending_idx` and `_settled_idx` serve the sender and the nightly prune.
- **Added for a named query:** `Question_status_idx` (dashboard whole-bank count), `Question_createdAt_id_idx`, `Student_fullName_id_idx` and `Student_createdAt_id_idx` (list sorts), `PaperQuestion_testId_baseConfigSectionId_idx` (authoring queue count), `SavedQuestion_questionId_idx` (the RESTRICT FK).
- **Dropped as redundant or readerless:** four by `20260922100000` (`Notification_actBy_idx`, `NotificationDelivery_providerMessageId_idx`, `NotificationDelivery_status_queuedAt_idx`, `StudentSubjectStat_subjectId_idx`; the nightly prune later got `_settled_idx` back); three prefix duplicates by `20260926150000` (`ExamStage_examId_idx`, `QuestionAssignment_testId_idx`, `Test_testSeriesId_idx`).
- **Present with a likely-duplicate or thin reader, read not measured:** `Student_mobile_idx` and `Student_externalRef_idx` repeat the leading column of their live partial uniques (adding only soft-deleted rows); `BaseConfig_isDefault_idx` is covered by `BaseConfig_default_per_stage_key` for the only predicate the service uses; `AdminFeaturePermission_featureKey_idx` is not led by any query this chapter found. `20260926150000` says the real answer needs `pg_stat_user_indexes.idx_scan` from production traffic (`scripts/index-usage.mjs`); the dev database's counters are not evidence.

6. Migrations that move data

No data move is proved by a from-scratch apply. The ones that moved data:

- `20260917100000`: `AttemptQuestion` (a row per answer) became `AttemptSheet` (a row per sitting).
- `20260918110000`: `sectionScores` objects became packed arrays; same numbers, same order.
- `20260918120000`: every id became a native uuid. A DO block scans every table and refuses to run against a database holding rows, so it aborts on any non-empty database.
- `20260912170000`: series and test names made unique case-insensitively, renaming collisions first.
- `20260903100000` then `20260904130000`: every test moved onto the series that holds it before the old access shape was dropped.
- `20260930140000`: re-scores moved from `OutboxEvent` rows to `Attempt.scoredRevision`.
- `20260930170000`: `DROP TABLE "OutboxEvent"` plus its three functions.
- `20261006150000` (new): dropping `TestSeries.isEnabled` would have opened every ACTIVE test in a switched-off series, so first each such test became INACTIVE, `announcedAt` was cleared (nobody had been told) and `version` bumped, then the column was dropped.
- `20261008120000` (new): replaced `questionSearchText`'s body, then rewrote every `Question.searchText` with `SET "searchText" = "searchText"`. The migration's own proof is that the 14 triggers enabled before are the 14 enabled after; it measured 10.0s for 50,005 rows.
- Smaller: `20260829100000` canonicalised exam codes and rewrote `Student.enrolledExams`. `20261013100000` renamed an enum value, which touches no rows.
- `20261007110000` dropped `Student.pinHash` and `pinIsDefault`; there is nothing to move because nothing replaces them.

7. Relationship map (condensed)

ON UPDATE is CASCADE except on the eight composite FKs (NoAction). Deletes:

- **Cascade from** `Student`: `Notification`, `EventCandidate`, `PushDevice`, `PushSubscription`, `SavedQuestion`, `StudentGrant`, `StudentProfile`, `StudentStat`, `StudentSubjectStat`, `StudentMobileHistory`, `BroadcastRead`, `TypingPractice`. `Attempt.studentId` is Restrict, as is `Student.currentBranchId → Branch`.
- **From** `Test`: `Attempt.testId` is Restrict, `Notification.testId` SetNull. Cascade to `QuestionReview`, `QuestionWorkTime`, `SectionComment`, `TestProgramUnlock`, `TestStat`, `TestSectionStat`, `TestQuestionStat`, and (through the composite) `PaperQuestion` and `QuestionAssignment`. `Test → BaseConfig`, `→ ExamStage`, `→ TestSeries` and `→ Admin` (language waiver) are Restrict.
- **Attempt:** `AttemptSheet.attemptId` Cascade; `SavedQuestion.attemptId` SetNull.
- **Question side:** `QuestionVersion.questionId`, `PaperQuestion.questionId`, `SavedQuestion.questionId`, `TestQuestionStat.questionId` Restrict; `QuestionReview`, `QuestionWorkTime`, `TypingPractice` cascade from `Question`; `Question.assignmentId` SetNull.
- **Config side:** `BaseConfigModule` and `BaseConfigSection` cascade from `BaseConfig`; `BaseConfig.clonedFromId` is a Restrict self-reference; `BaseConfigSection.subjectId`, `TestSectionStat.baseConfigSectionId` Restrict.
- **Admin:** `AdminFeaturePermission` and `Notification.adminId` Cascade; `Announcement.createdById`, `Question.createdById`, `QuestionAssignment.assigneeId`, `SectionComment.authorId` Restrict.
- **Access and audit:** `StudentGrant.testSeriesId`, `EventCandidate.eventId`, `Notification.announcementId`, `NotificationDelivery.notificationId`, `ImportRowEdit.importLogId` Cascade; `TestSeries → ExamStage/Program/Event`, `TestProgramUnlock.programCode`, `RowActionLog.importLogId` Restrict. `QuestionReview`, `QuestionWorkTime`, `StudentMobileHistory` and `RowActionLog` actor ids carry no FK so the record outlives the person.

Why it is shaped this way

- **One positional sheet per sitting.** The alternative, a row per answer, was the schema until `20260917100000`. Positions beat ids because an id costs bytes on every slot of the hottest write.
- **Rank and percentile are counted live** off `Attempt_ranking_idx`. A stored rank column existed and was removed (`20260911220000_no_saved_rank`).
- **UUID v7**, not cuid or v4, for index density and sort cost under `en_US.utf8`.
- **The database as the second line.** The service holds policy; triggers and CHECKs are the backstop for every other path.

What changed recently

`OutboxEvent` and its relay are gone, and notifications are written inside the producer's own transaction. Re-scoring is found by `Attempt.scoredRevision` against `Test.paperRevision`. Rollup columns that could be derived were removed. A student has no PIN. A series has no `isEnabled`. A broadcast is one row the bell resolves, with only the read written per student (`BroadcastRead`). Questions grew two types (TYPING, DESCRIPTIVE) and sections grew `typingPreset`, `descriptive` and their own `languages`. Subjects can be languages, and a test records who waived a missing language and which languages it skipped. The bank is searched through one trigger-maintained column.

What is wrong with it

- Stale: `prisma/schema.prisma:176` documents `PIN_CHANGED` as "Their PIN changed". Students have no PIN since `20261007110000`, and no code writes the value; it survives only as a filter in `packages/app-kit/src/list-filters.ts:63`.
- Stale: `prisma/schema.prisma:1104` banner reads "in-app now, push behind the same sender seam later", but `PushSubscription`, `PushDevice` and `NotificationDelivery` are built and live in the same section.
- Stale: `docs/schema.dbml` (header: generated from the schema) has no `BroadcastRead` and 44 `Table` entries against 45 models; last regenerated in `be62aa6`, before `20261015090000`.
- Dead: `prisma/schema.prisma:51` `AnswerMode` is declared and no column uses it (the file says so itself); the mode lives in `answerKey` JSON.
- Dead: `prisma/schema.prisma:155` `AttemptStatus.EXPIRED` is never written (only the label maps `apps/api/src/attempts/test-report.ts:37` and the admin/live-ops tables reference it).
- Dead: `prisma/schema.prisma:67` `TestUi.TYPING` is selectable in the contract and it has an admin label but nothing branches on it; typing now arrives as a question type and a section preset instead.
- Dead: `prisma/schema.prisma:244` `AuditFeature.BRANCH_TEST_CONFIG` has no writer in `apps/api/src` or `packages`; only the contract constant `packages/contracts/src/audit.ts:25` and an admin label `apps/admin/src/lib/constants.ts:295` name it.
- Stale: `prisma/migrations/20261001140000_the_bank_is_searched_on_one_column/migration.sql:46` and the `20261008120000` header say the search reaches "stem and solution", but the `$.*.*[*].text` path also reads `passage`, which `questionContentSchema` now carries (`packages/contracts/src/questions.ts:195`). Behaviour is right; the comments are narrower than it.

Part VII — What it runs on

The boxes, the pipeline, and where the docs drift

Staging exists on AWS; production does not. The shape is the one `docs/04-infrastructure.md` §2 describes: one API box per environment running Docker Compose behind Caddy, one shared Valkey box, one shared RDS Postgres 17, and the two SPAs on S3 behind CloudFront. Nothing here is Fargate, and no ALB sits in front. The repo side of it is `deploy/`, `apps/api/Dockerfile`, `.github/workflows/` and the env templates; `docs/04-infrastructure.md` (1,019 lines, 16 sections) is the why and the money, and is checked against them at the end of the chapter. Prices, sizing arithmetic and AWS console state were not re-checked: read, not measured.

The pieces

Piece	Where it is defined	What it is
Box A (API)	`deploy/compose.yml`	`migrate` (runs to completion), `exam`, `core`, `worker`, `alloy`, `caddy`. Only Caddy publishes ports (`deploy/compose.yml:153`).
Box B (Valkey)	`deploy/valkey/compose.yml`	One Valkey 8 process per environment: production `:6379` at 1200 MB, staging `:6380` at 256 MB. AOF on, `noeviction`, `requirepass` each, published on `PRIVATE_IP` only.
Image	`apps/api/Dockerfile`	One file, two targets: `runtime` (`apps/api/Dockerfile:50`) and `migrate` (`apps/api/Dockerfile:60`). `tini` is PID 1 on both.
Front door	`deploy/Caddyfile`	One hostname (`{$API_HOST}`), Let's Encrypt over HTTP, path split between exam and core.
SPAs	`deploy/publish-spas.sh`	Builds exams and admin, syncs to a private bucket, invalidates four paths.
Telemetry	`deploy/alloy/config.alloy`	Container stdout to Grafana Cloud Loki; `/metrics` of exam, core and worker to Prometheus.
Disk	`deploy/prune-disk.sh`	Keeps the newest two builds of each app image; never touches volumes.
Staging switch	`deploy/staging-power/`	One Lambda behind a function URL that starts and stops the staging API box, the Valkey box and the RDS instance.
Boot update	`deploy/boot-update.sh`, `deploy/examprep-boot-update.service`	A systemd unit on the staging box only: on every start, fast-forward the checkout, pull the `staging` tag, bring the stack up.

One image, three roles

`API_ROLE` is read once from `process.env` (`apps/api/src/config/api-role.ts:33`) and decides which controllers and processors a container registers; unset means all, which is what a laptop runs. Compose sets it per service, with the settings that differ by role: `UV_THREADPOOL_SIZE`, `NODE_OPTIONS` heap, `cpu_shares` and `mem_limit`.

Service	`cpu_shares`	Heap default	Notes
`exam`	2048	384 MB	`AttemptsController`, `MeLeaderboardController` (`apps/api/src/attempts/attempts.module.ts:49`). Runs under a request budget.
`core`	1536	384 MB	Everything else.
`worker`	1024	384 MB	Exactly one. No routes but health and metrics.

CPU is a weight, not a quota (`deploy/compose.yml:11`). A hard `cpus` ceiling below one core makes CFS spend the quota early in each 100 ms window and freeze the process, which shows as a low CPU average beside a bad p99. Memory stays a hard limit. The heap defaults are staging's; production overrides them in its env file (`EXAM_MEM=2g`, `EXAM_HEAP_MB=1536`, as `deploy/.env.example:24` lists). The API warns at boot when the heap and the limit disagree.

The worker is a singleton because `concurrency: 1` in `QUEUE_POLICY` is the only thing stopping the 60 s flush and the 120 s sweep from overlapping, and it only works inside one process. `scale.yml` has no `worker` input and always passes `--scale worker=1` (`.github/workflows/scale.yml:131`).

Every container shares one `deploy/.env` (`env_file: .env`, `deploy/compose.yml:54`). One `DATABASE_URL` therefore carries one `connection_limit=25`, so the budget is 25 per container including the worker, not a per-role split. On `db.t4g.micro` (about 112 connections) one exam, one core and one worker is 75 and a third API container is over.

Routing

`deploy/Caddyfile:11` lists the exam role's leaf paths; everything else falls to core:

```
@exam path /me/tests/*/attempt /me/tests/*/brief /me/tests/*/paper
        /me/attempts/*/paper /me/attempts/*/submit /me/attempts/*/state /me/leaderboard
```

This matches the routes `AttemptsController` and `MeLeaderboardController` register (`apps/api/src/attempts/attempts.controller.ts:64`, `apps/api/src/attempts/leaderboard.controller.ts:17`). The split is not prefix-clean: `/me/attempts/:id/scorecard`, `/solutions` and `/question-report` belong to core under the same prefix, so a blanket `/me/attempts/*` would send them to a container that does not register them. Both upstreams are `dynamic a` with a 10 s refresh (`deploy/Caddyfile:13`), so `docker compose up -d --scale exam=4` needs no edit; failure is passive only (3 fails, out for 30 s). Caddy's access log goes to stdout as JSON (`deploy/Caddyfile:36`), where Alloy ships it.

> **Watch** A new exam-role route must be added to the matcher by hand. Nothing checks the list against the controllers. A missing entry routes to core, which does not register it, and the caller sees a 404 from a healthy box.

Environment files

The env file is the unit, not the variable. Four files exist per environment pair:

File	Tracked	Role
`.env.example`	yes	Every schema variable, `dev_only_` secrets; local development.
`deploy/.env.example`	yes	The deployed subset plus compose/Caddy/Alloy keys.
`deploy/.env`	no (`.gitignore:33` is `.env.*`)	The real file on the box, root-owned, `chmod 600`.
`deploy/valkey/.env`	no	Box B: `PRIVATE_IP` and a password per process.

`pnpm env:check` (`scripts/check-env.mjs`) reads `envSchema` in `apps/api/src/config/env.schema.ts` as the list of keys, splits them into `LOCAL_ONLY` and `DEPLOY_ONLY`, and fails when either template, or a real file beside it (`.env`, `deploy/.env.local`, `deploy/.env`, `deploy/.env.staging`, `deploy/.env.production`), names different keys. It reads names only, never values. CI runs it in the `verify` job (`.github/workflows/ci.yml:80`).

The master copy lives in SSM Parameter Store at `/examprep/<env>/env` (and `/examprep/valkey/env`); nothing reads it at runtime, so the API boots whether or not AWS answers. `deploy/README.md` pushes it to the Advanced tier (8 KB) because a filled-in file passes the Standard 4 KB cap. The instance profile carries `ssm:GetParameter` and deliberately not `PutParameter`, so a box cannot overwrite what every replacement pulls. S3 keys stay unset on a real box and the SDK uses the instance role.

The pipeline

Seven workflows. Two are triggers, three are shared, one is CI, one is manual scaling.

Workflow	Trigger	Does
`ci.yml`	push to `staging` or `main`, PRs, weekly cron	`verify`, `database`; `docs-check` only on cron or dispatch.
`staging.yml`	push to `staging`, or dispatch with an image tag	What changed, then build, API, SPAs as one chain.
`production.yml`	cron 20:30 UTC (02:00 IST), or dispatch	API then SPAs. **Disabled in the Actions settings** (`.github/workflows/production.yml:8`).
`build-api.yml`	called by staging; push to `main` touching API paths; dispatch	Builds `runtime` and `migrate` natively on `ubuntu-24.04-arm`, pushes `<sha>` and the moving tag (`staging` or `prod`). Does not deploy.
`_deploy-api.yml`	called	SSM into Box A, fast-forward, pull, recreate.
`_publish-spas.yml`	called	Builds and publishes the SPAs for one environment.
`scale.yml`	dispatch only	Resizes the box and sets container counts, refusing what the database cannot hold.

`ci.yml` `verify` runs install, Prisma generate, format, `deps:check`, `env:check`, lint, typecheck, test, the staging-power unit test and build; each step is `if: '!cancelled()'` so one failure does not hide the next. `database` runs against `postgres:17-alpine` (`.github/workflows/ci.yml:111`): migrations from scratch, `db:check`, then `pnpm test:db`.

PUSH TO STAGING — A RELEASE REACHES THE STAGING BOX

1. GitHub starts `Staging`; one run at a time, never cancelled
   `.github/workflows/staging.yml:19`
   → group `staging-release`
2. `what-changed` finds the last GREEN run of this workflow on `staging` and diffs it against the pushed sha (two dots, so a force-push still compares trees)
   `.github/workflows/staging.yml:54`
   → `api` and `spa` outputs; a missing or unreachable base means both
3. API half is true for `apps/api/`, `packages/contracts|config`, `prisma/`, lockfile, root `package.json`, and the deploy files the box reads; SPA half for `apps/exams|admin`, any `packages`, `publish-spas.sh`
   `.github/workflows/staging.yml:69`
   → a run that deploys nothing says so in the step summary
4. `build` (only if API changed) calls `build-api.yml`: OIDC into the AWS role, build both targets for arm64
   `.github/workflows/build-api.yml:67`
   → ECR `examprep/api` and `examprep/api-migrate` tagged `<sha>` and `staging`
5. `api` finds the instance by its `Name` tag; if it is stopped and `skip_when_stopped` is set, reports "ECR updated" and ends green
   `.github/workflows/_deploy-api.yml:48`
   → `off=true`, nothing else runs
6. SSM `send-command` as root on the box: fast-forward the checkout to this sha, ECR login, set `API_IMAGE`, `MIGRATE_IMAGE`, `API_TAG`
   `.github/workflows/_deploy-api.yml:89`
   → box checkout and image tags match the release
7. `compose pull`, then `compose up -d`: `migrate` runs `prisma migrate deploy` and exits 0 before exam, core, worker start
   `deploy/compose.yml:64`
   → new containers; there is a gap, no draining

8. Restart Caddy and Alloy only if their mounted files changed between the old and new HEAD
`.github/workflows/_deploy-api.yml:102`
→ mounted config reloaded (a recreate would not do it)

9. `prune-disk.sh` keeps the newest two builds, then the job polls the command until it ends and fails unless SSM says `Success`
`.github/workflows/_deploy-api.yml:104`
→ disk bounded; green only on a real success

10. `spas` runs strictly after `api`, never beside it, and only if no earlier job failed or was cancelled
`.github/workflows/staging.yml:121`
→ SPAs published with `VITE_API_URL` baked in

The order in step 10 is the point: an old tab keeps its old bundle until every tab closes, so the API must accept the previous client, but a new bundle against the old API can send a field the old API strips (a submit's last batch was exactly that).

PUBLISH-SPAS.SH — A FRONT-END RELEASE

1. Read `version.json` from both prefixes in S3 and compare to `HEAD`; if both match and `FORCE` is not 1, exit
`deploy/publish-spas.sh:26`
→ no-op on an unchanged commit

2. Build exams and admin with `VITE_API_URL` set to the environment's API origin
`deploy/publish-spas.sh:31`
→ `apps/exams/dist`, `apps/admin/dist`; an environment is a build, a staging artifact cannot be promoted

3. Sync `assets/` first with `max-age=31536000, immutable`
`deploy/publish-spas.sh:37`
→ hashed chunks live before any shell points at them

4. Write `version.json` (commit, time, API origin) beside the build
`deploy/publish-spas.sh:41`
→ the site can say which commit it is

5. Sync the rest with `no-cache`; old assets are never deleted
`deploy/publish-spas.sh:46`
→ shell and `sw.js` revalidate

6. Invalidate `/index.html /sw.js /manifest.webmanifest /version.json`, never `/*`
`deploy/publish-spas.sh:51`
→ CloudFront serves the new shell

A new service worker installs but does not activate until every tab of the old one closes (no `skipWaiting`). Deploying and students seeing the build are different moments.

Rollback and scaling

Rollback is a tag: run **Staging** or **Production** with a previous sha as `image_tag`. Dispatch builds nothing (`.github/workflows/staging.yml:93`). The checkout still moves to the sha of the run itself (`.github/workflows/_deploy-api.yml:96`), so an old image runs under the current `compose.yml`.

`scale.yml` computes `(exam + core + 1) * 25` against the live RDS class (112, 225 or 450 for micro, small, medium and larger) and refuses anything over 90% (`.github/workflows/scale.yml:80`). It runs `compose down` before a resize so containers get their grace period, then `up -d --no-recreate --scale exam=N --scale core=N --scale worker=1`. It shares the `<environment>-release` concurrency group with the release, so a scale and a deploy never recreate containers together.

Staging is off between sessions

The instance hours are most of the staging bill, so the Valkey box, the API box and the RDS instance are stopped when nobody is testing. `deploy/staging-power/index.mjs` serves a page and a small JSON API behind a shared passcode (`x-passcode`, compared by hash with `timingSafeEqual`, `deploy/staging-power/logic.mjs:90`):

Route	Effect
`POST /on`	Starts the stopped resources, checks the caller in (an EC2 tag on the API box), saves the auto-off setting.
`POST /off`	Refuses with 409 when others are checked in, unless `force`. Stops everything and clears check-ins.
`POST /checkin`, `/checkout`	Tag bookkeeping; a check-in lapses after 12 hours.
`POST /auto-off`	Sets none, nightly (`HH:MM` IST) or timed (1 to 24 h) via an EventBridge Scheduler schedule.

A second schedule, `keep-off`, fires daily at 03:00 IST: AWS restarts a stopped RDS instance after seven days, and this stops it again if the API box is off. With `MANAGE_SHARED=false` (the day production exists) it touches only the staging API box. `policy.json` is the whole of what the function may touch. A deploy never starts anything; `boot-update.sh` pulls the `staging` tag on every box start and retries `compose up -d` for 20 minutes (80 attempts of 15 s) because `migrate` cannot finish until RDS is up. The Lambda is not part of any pipeline; `deploy/README.md` has the zip and `update-function-code` commands. CI runs its unit test (`.github/workflows/ci.yml:96`).

Telemetry

Alloy discovers containers off the Docker socket and labels log lines `service_name` (the compose service), `level` and `env`; anything per-request stays in the line and is extracted at query time. It skips its own container so one shipping error does not become a loop. It scrapes `:3000/metrics` on exam, core and worker every 15 s with `METRICS_TOKEN`. Docker's `json-file` cap (10 MB x 3, `deploy/compose.yml:24`) stays as the local safety net, so Alloy going down costs visibility and never the disk. `PROM_USERNAME` is not `LOKI_USERNAME`.

> **Note** Alloy mounts `/var/run/docker.sock`, and `:ro` protects the file, not the API behind it: the container is root on the box. That is accepted so `service_name` is a name rather than a 64-character id (`deploy/compose.yml:137`).

Where `docs/04-infrastructure.md` disagrees with the repo

The document was rewritten since the old handbook, and most of that chapter's ledger is closed (the Dockerfile builds `apps/exams`, the pool is stated as 25 per container, the service worker cache is stamped with the build id). What is still wrong:

Doc says	Repo does
§2: Box A security group has "22 from one address" (`docs/04-infrastructure.md:143`)	§8 of the same doc, `deploy/README.md:18` and the SSM-only runbook all say no inbound SSH.
§2: "the worker takes no HTTP at all" (`docs/04-infrastructure.md:142`)	The worker listens on `API_PORT` and answers `/health/ready` (`deploy/compose.yml:104`, `deploy/compose.yml:112`); §3 of the doc says so itself.
§1: "`build-api.yml` builds on CodeBuild" (`docs/04-infrastructure.md:81`)	It builds on GitHub's `ubuntu-24.04-arm` runner (`.github/workflows/build-api.yml:40`). CodeBuild is the deferred replacement in §15.
§9: "Failures never depend on the level … a 4xx as one line" (`docs/04-infrastructure.md:583`)	A 4xx logs at `debug` (`apps/api/src/common/all-exceptions.filter.ts:65`); production's `>=log` drops it. Only the 5xx branch survives.
§9: a slow call "is a `warn` instead" (`docs/04-infrastructure.md:571`)	Only a slow call that was not refused. A 4xx, slow or not, is `debug` REFUSED (`apps/api/src/common/metrics/request-observer.middleware.ts:39`).
§11: `deploy/.env` is "`.gitignore` line 32" (`docs/04-infrastructure.md:707`)	The rule is `.env.*` at `.gitignore:33`.
§11: "Three files, and only one of them is real" above a table of four rows	Four files are listed (`docs/04-infrastructure.md:700`).
§11: Standard tier works if comments are stripped, Advanced is optional	`deploy/README.md:64` says a filled-in file is past 4 KB even stripped and pushes with `--tier Advanced`.
§12: staging role has `CONNECTION LIMIT 85`	Nothing in the repo sets it; it is a manual DBA step, not verified.
§15: "The service worker never prunes its cache (`iace-shell-v1` is a fixed name)" (`docs/04-infrastructure.md:964`)	`apps/exams/vite.config.ts:29` rewrites `iace-shell-dev` to `iace-shell-<build>` at build time, so the `activate` handler does fire.
§9 states "Nothing logs a successful response body" twice (`docs/04-infrastructure.md:572`, `docs/04-infrastructure.md:590`)	Editorial repeat.
§5: page routes reach `index.html` through a CloudFront Function	Nothing in the repo defines one; console-side, not verified.

What is wrong with it

- Stale — `deploy/.env.example:78` still budgets "exam 8 + core 25 + worker 25 = 58"; every container reads one URL with `connection_limit=25`, so it is 75 (`.github/workflows/scale.yml:71` and doc §6 say so).
- Stale — `deploy/compose.yml:22` says "Box A has 30 GB"; `docs/04-infrastructure.md:81` and `deploy/README.md:149` size it at 20 GB.
- Stale — `deploy/compose.yml:1` says Box A runs "Caddy and the three roles, and nothing else"; Alloy is a sixth service.
- Stale — `deploy/README.md:308` claims "all three images build"; there are two (`runtime` and `migrate`), and the claim is dated 26 September.
- Stale — `deploy/README.md:296` tells you to read `logs api`; no service by that name exists (`exam`, `core`, `worker`, `caddy`, `alloy`, `migrate`).
- Stale — `apps/api/src/common/logging.ts:4` names `MetricsInterceptor` as the per-request line; it is `RequestObserverMiddleware`.
- Stale — `.github/workflows/scale.yml:4` cites `docs/04 §16` for scaling (§16 is the un-remeasured figures) and `.github/workflows/scale.yml:71` says `docs/04 §6 still describes` a per-role split that §6 now disclaims.
- Correctness — a 4xx never reaches a production log line (`apps/api/src/common/all-exceptions.filter.ts:65`, `apps/api/src/common/metrics/request-observer.middleware.ts:39`); a rejected submit leaves no trace beyond a metric.
- Correctness — a rollback keeps the checkout at the run's own sha (`.github/workflows/_deploy-api.yml:96`), so an older image runs under a newer `compose.yml`; a compose change that the old image does not tolerate is not rolled back.
- Correctness — `.github/workflows/staging.yml:70` ends its SPA pattern with `/?`, so it matches any path that merely begins with `packages`, `apps/exams` or `apps/admin`; harmless today, but the pattern is looser than it reads.
- Correctness — `build-api.yml` runs on every API-touching push to `main` with `environment: production` (`.github/workflows/build-api.yml:42`) while `production.yml` is disabled; whether the `production` environment and `AWS_ROLE_ARN` exist is GitHub-side and not verified here.
- Privacy — `deploy/compose.yml:139` gives Alloy the Docker socket (documented, root-equivalent on Box A).

Part VIII — What is wrong with it

The ledger

Every chapter ends with what its reader found wrong at HEAD, with the path and line that proves it. This part ranks them. Comment-only items stay in their chapters. Only the findings that change what a user gets, what is exposed, or what an operator can trust are raised here.

The previous edition's 141 findings were closed on 1 October 2026, except the super-admin role control in the admin UI, which was waved off as fine. They are not carried forward.

Docs corrected in this pass

The readers found these documents stating something false. Each was fixed in the same commit as this handbook.

Document	Was	Is
`docs/01-architecture.md`	points boards, a 128 ms series board, a 60-second per-scope cache	one test board, held in process per `TestStat.computedAt` in 8 slots, paged 3 + 20
`docs/01-architecture.md`	workers run "imports"	workers run scoring, exports and notifications; there is no import queue
`docs/02-domain-rules.md` §7	the server takes forward-only answers either side of the furthest seat	the server drops them (`withinReach`)
`docs/02-domain-rules.md` §3	"nothing else gates the draw"	the section's answer kind and work open under another assignment also gate it
`docs/02-domain-rules.md` §12	pictures "uploaded only on commit"	stored at preview
`docs/02-domain-rules.md`	a typing passage is the English stem; English is required	the lead language, which a language subject replaces
`docs/03-conventions.md` §5, §7	no `BroadcastRead`; worker does "import processing"	both corrected
`docs/04-infrastructure.md`	eight processors; CodeBuild builds; port 22 open; a 4xx logged in production; the service worker never prunes	nine; GitHub's arm64 runner; no inbound SSH; a 4xx is `debug`; the cache name is stamped per build
`docs/schema.dbml`	44 tables, missing `BroadcastRead`	45, regenerated

Privacy

Finding	Where
Erasure leaves the student's name and mobile in the uploaded roster sheet, which the audit screen still streams back	`apps/api/src/imports/imports.service.ts:502`
Erasure removes only the current photo and marksheet keys; earlier uploads stay in S3	`apps/api/src/me/me.service.ts:62`, `apps/api/src/students/student-privacy.service.ts:64`
Reports are institute-wide: a `REPORTS` holder reads every branch's roster, numbers and results. Whether a branch office should see only its own branch is a product call	`apps/api/src/reports`
Reading a report writes no audit row, though several show phone numbers; only the export is logged	`apps/api/src/reports`
Plaintext FCM service-account keys sit in git-ignored `deploy/.env.*.bak-20261008` backups beside the tree	untracked files
The first student marked can read the key while the batch is still sitting. Accepted by design	`apps/api/src/attempts/attempt-report.service.ts:139`

Correctness — a user gets a wrong answer

Finding	Where
Section clocks and closes are enforced by the screen only; the server never refuses an answer to a closed section	`apps/api/src/attempts/attempt-state.service.ts:463`
Two devices on one sitting: a continued stand-down resends its stale queue over the other's answers. Known; the fix is a per-question version on the server's save	`packages/app-kit/src/exam/use-attempt-state.ts:184`
A sign-in code request for a deactivated student still sends a paid SMS; the refusal comes only at verify	`apps/api/src/auth/auth.service.ts:63`
`prisma/seed.sql` seeds the first super admin with role `ADMIN`; saving it with its displayed role strips the bypass	`prisma/seed.sql:34`
An `EVENT` series keeps reaching its candidates after the event is retired. Intended or not is a product call	`apps/api/src/access/access-resolver.service.ts:414`
A blocked student can still be given a program or a course; the block guards only `enrolledExams`	`apps/api/src/students/students.service.ts:326`

Finding	Where
Aadhaar and PAN always read "Not verified"; nothing outside tests writes them	`apps/exams/src/features/account/profile.tsx:326`
A saved question previews its current version but opens the version that was sat	`apps/api/src/saved/saved-questions.service.ts:272`
The bell matches an announcement's audience at read time, so a student who changes branch gains or loses old ones	`apps/api/src/notifications/bell.service.ts:170`
An abandoned question preview leaves its log row, sheet and pictures for ever	`apps/api/src/questions/question-import.service.ts:115`
A program import can lose rows from both `enrolled` and `skipped` when a student enrols mid-run	`apps/api/src/imports/imports.service.ts:206`
A typed-answer error for a Hindi subject lands on the English field the form does not show	`packages/contracts/src/question-rules.ts:344`
Per-question work time is judged only by the client; a modified client can add time to any card, capped at an hour per report	`apps/api/src/questions/section-work.service.ts:150`
Assigning and removing a section assignment carries no `@Audit`	`apps/api/src/assignments/assignments.controller.ts:89`
The question report re-reads the paper and its key from Postgres on every open, bypassing the solutions cache	`apps/api/src/attempts/question-report.service.ts:110`
The audit archive keeps its own 30 apart from the screen's window, and nothing in the app reads the archive	`apps/api/src/audit/audit-archive.ts:9`
A 4xx never reaches a production log line, so a rejected submit leaves no trace beyond a metric	`apps/api/src/common/all-exceptions.filter.ts:65`
A rollback runs an older image under the newer `compose.yml`	`.github/workflows/_deploy-api.yml:96`
The app-kit lint config's `noDom` block replaces the raw-`fetch` and `axios` bans instead of adding to them	`packages/app-kit/eslint.config.js:5`

Release blockers that are not bugs

- The mobile production profile names no API URL, so a production build throws at launch by design (`apps/mobile/eas.json`).
- Push is built at both ends and has never been seen to reach a device.
- The 6K load test has not been run against staging.

Stale and dead

About sixty comments, labels and lines across the chapters describe the PIN era, the outbox, the points boards, or a skin count that has since grown. Each is listed at the end of its chapter with its path. None changes behaviour; each misleads the next reader.